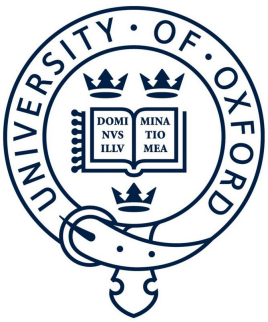
Noise and Uncertainty in Molecular Property Prediction:

Robust Models and Adaptive Representations



Adelaide Punt

St Catherine's College

University of Oxford

A thesis submitted for the degree of

Doctor of Philosophy

Trinity Term 2026

In memory of my grandfather, Frederick Rebarber.

Statement of Contribution

Chapter 3 is adapted from a jointly authored manuscript, Punt, A., Hanser, T., Werner, S., and Morris, G.M., *NoiseInject: A Benchmarking Framework to Study Noise Robustness in QSAR Models*, which will be released as a preprint and submitted to the *Journal of Cheminformatics*. Most of the figures, tables and text in that chapter are identical or similar to the manuscript. I implemented the NoiseInject framework, conducted all experiments, and drafted the manuscript. T.H., S.W., and G.M. supervised the project, contributed to the experimental design, and revised the manuscript. NoiseInject is available at `https://github.com/adpunt/noise_inject`.

Chapters 5 to 7 describe the adaptive hybrid representation and its use under label noise. We intend to submit this work as two manuscripts with G.M., S.W., and T.H. as co-authors: one on the adaptive hybrid representation (Chapters 5 and 6) and one on detecting and handling label noise (Chapter 7). I conceived the adaptive hybrid search and designed the system, conducted all experiments, and wrote these chapters. The code was implemented with Claude (Anthropic) within that design, as described in the Statement on the Use of Generative AI. G.M., S.W., and T.H. supervised the work. The code for the adaptive hybrid search is available at `https://github.com/adpunt/KIRBy`.

Acknowledgments

First and foremost, I would like to thank my supervisors, Prof. Garrett M. Morris at the University of Oxford, and Dr. Thierry Hanser and Stéphane Werner at Lhasa Limited, for their guidance and support throughout my doctoral research.

I would like to extend my deepest gratitude to Garrett, who has supported me in every aspect of my DPhil. I am grateful for his patience, his belief in me, and the encouragement he has always given my ideas. His mentorship has been instrumental in shaping this work and my growth as a researcher, and I have become a much better scientist because of his guidance. I feel incredibly lucky to have been his student.

I would also like to thank Thierry for his valuable input into my research and for always sharing his technical expertise so generously. I am especially grateful that he continued to attend our meetings and to help me with this thesis after he left Lhasa and stepped down from his formal role as my supervisor. That commitment went far beyond what was asked of him. I would also like to thank Stéphane, who likewise came back to help with my work after leaving his role at Lhasa. I am grateful to Lhasa Limited for supporting this project, and to Lhasa Limited and the UK EPSRC Centre for Doctoral Training in Sustainable Approaches to Biomedical Science: Responsible and Reproducible Research (EP/S024093/1) for their generous financial support.

I also thank everyone at Lhasa Limited, and Adrian Fowkes, my current supervisor at Lhasa.

I would like to thank Prof. Charlotte M. Deane and the members of the Oxford Protein Informatics Group, past and present, and the Department of Statistics. I would also like to thank my former undergraduate professors, Prof. Elizabeth Orwin and Prof. Ran Libeskind-Hadas.

I thank the Oxford Protein Informatics Group for pointing me to the antibody and protein datasets used in Chapters 5 to 7.

I would also like to thank my parents, Beth Rebarber and Douglas Punt, for their unending love, encouragement, and belief in my abilities. I could not have done this without you. My grandmother, Annette Rebarber, gave me a love of mathematics and

computer science from a young age, and her work as a punch-card coder at Bell Labs and her continued pursuit of both inspired me. Thank you to my friends, especially Mariam Ahmed, Nico Naar, Crystal Tang, Alycia Perkins, Angela Xiaoyun and Daniel Lobaton, for supporting me throughout my DPhil. Thank you also to my cat, Rhea, for staying by my side throughout the writing of this thesis (not always willingly) and for keeping me entertained.

I am grateful to Rabbi Mendy for his support and guidance through these years. And above all, I thank God, without whom none of this would have been possible.

Contents

Statement of Contribution ii

Acknowledgments iii

Abstract ix

Statement on the Use of Generative AI x

List of Abbreviations xviii

1 Introduction 1

1.1 Outline of the thesis and contributions 4

1.2 Collaborations 6

2 Molecular representations, models and label noise 7

2.1 Overview 7

2.2 Molecular representations in QSAR 7

2.3 Models for molecular property prediction 11

2.3.1 Tree ensembles 14

2.3.2 Support vector machines 14

2.3.3 Gaussian processes 14

2.3.4 Bayesian neural networks 15

2.4 Molecular similarity and activity cliffs 16

2.5 Noise in molecular data 17

2.5.1 Artificial noise injection 19

2.5.2 Learning with noisy labels 22

2.6 Implementation of the noise conditions 24

2.6.1 The NoiseInject package 24

2.7 Evaluation methodology 27

2.8 Conclusions 27

3 Noise robustness across molecular representations and model architectures 28
3.1 Introduction . . . 28
3.2 Methods . . . 29
3.2.1 Datasets . . . 29
3.2.2 Representations . . . 30
3.2.3 Models . . . 31
3.2.4 Noise conditions and levels . . . 33
3.2.5 QM9 data pipeline . . . 34
3.2.6 Uncertainty estimation . . . 34
3.2.7 Evaluation . . . 35
3.3 Results . . . 38
3.3.1 Variance decomposition . . . 38
3.3.2 Predictive performance and noise robustness . . . 40
3.3.3 Noise conditions on the HOMO–LUMO gap . . . 42
3.3.4 Robustness on the three assay datasets . . . 46
3.3.5 Probabilistic and deterministic counterparts . . . 52
3.3.6 Uncertainty under increasing label noise . . . 55
3.4 Conclusions . . . 60

4 Limitations of the similarity assumption 64
4.1 Introduction . . . 64
4.2 Methods . . . 66
4.2.1 Distance metrics for continuous molecular descriptors . . . 66
4.3 Results . . . 67
4.3.1 Flaws in standard representations . . . 67
4.3.2 Flaws in similarity and other pairwise computations . . . 71
4.3.3 Consequences for similarity-based methods . . . 74
4.4 Conclusions . . . 79

5 Building adaptive hybrid representations 81
5.1 Introduction . . . 81
5.1.1 Combining representations . . . 82
5.1.2 The adaptive hybrid representation . . . 84
5.1.3 Blueprints and their chemical interpretation . . . 85
5.1.4 Applications beyond small molecules . . . 86
5.2 Methods . . . 87

5.2.1 From base representations to a hybrid . . . 87
5.2.2 The search space . . . 90
5.2.3 Augmentation . . . 96
5.2.4 Search grid per dataset . . . 96
5.2.5 Evaluation protocol . . . 97
5.2.6 Datasets and splitting . . . 99
5.2.7 Representations by domain . . . 102
5.2.8 Models and metrics . . . 109
5.2.9 Comparators, controls and ablations . . . 110
5.2.10 Transfer, collaboration studies and separate sites . . . 115
5.3 Results . . . 117
5.3.1 Performance of the hybrid . . . 118
5.3.2 Sources of the gain . . . 126
5.3.3 Blueprints on specific datasets . . . 134
5.3.4 Collaboration studies . . . 135
5.3.5 Features chosen across separate sites . . . 138
5.4 Conclusions . . . 138

6 Similarity in an adaptive hybrid representation 141
6.1 Introduction . . . 141
6.2 Methods . . . 142
6.2.1 Feature spaces compared . . . 142
6.2.2 Similarity in a hybrid . . . 143
6.2.3 Nearest-neighbor settings . . . 145
6.2.4 Experiments . . . 146
6.3 Results . . . 150
6.3.1 Distance against label difference . . . 150
6.3.2 Prediction from nearest neighbors . . . 152
6.3.3 Activity cliffs . . . 158
6.3.4 Prediction error, retrieval and clustering . . . 159
6.4 Conclusions . . . 162

7 Detecting and handling label noise 164
7.1 Introduction . . . 164
7.2 Methods . . . 164
7.2.1 Noise conditions . . . 165
7.2.2 Noise detection . . . 168

7.2.3 Noise mitigation . . . 170
7.2.4 Noise-robust training . . . 175
7.2.5 Evaluation and controls . . . 178
7.3 Results . . . 179
7.3.1 Overall effect of noise handling . . . 179
7.3.2 The clean-label test: where noise is removed . . . 180
7.3.3 Changing a label against deleting a molecule . . . 181
7.3.4 Detection of the suspect labels . . . 190
7.3.5 Effect by noise type . . . 194
7.3.6 Effect by dataset . . . 201
7.3.7 Cleaning inside the feature search . . . 203
7.3.8 Changing the model instead of the labels . . . 205
7.3.9 Effect by model and representation . . . 210
7.4 Conclusions . . . 212

8 Conclusions and Further Thoughts 213
8.1 Summary of findings . . . 213
8.2 Choosing a noise-handling method . . . 215
8.3 Contributions . . . 215
8.4 Limitations . . . 216
8.5 Future work . . . 216
8.6 Concluding remarks . . . 217

Summary of Research Contributions 218

Abstract

This thesis investigates label noise in quantitative structure–activity relationship (QSAR) models, whose labels carry measurement error, differences between laboratories and assay limits. Most studies of label noise in QSAR assume Gaussian noise and study representations or models in isolation. We show that the kind of noise matters, and that noise can be handled once the features suit the data.

First, we introduced NoiseInject, an open-source package for benchmarking noise robustness. Across 13 base models, six representations and seven types of label noise, model family decided robustness more than the representation did; random forests were the most consistent model. Systematic noise, such as labels clipped at an assay limit, did the most damage. Predicted uncertainty rose with noise but, outside censoring, did not identify the corrupted labels.

Next, we traced why published methods for learning with noisy labels did poorly on molecular data. Within one dataset, most features of a standard representation barely vary or do not relate to the label, yet those that vary still add to the distance between molecules. Distance-based methods therefore inherit the problem.

We then built the adaptive hybrid representation, which selects features from several representations for each dataset and model. On every small-molecule dataset, though not on the reactions, it predicted better than the best single representation, and its distances tracked activity more closely. A newly published embedding can be added as one more representation.

Last, we compared single representations and hybrids under label noise. Noise was handled, mostly by methods built or adapted for molecular data. Which method worked depended on the task, the data, the model and the noise, but on small molecules the hybrid stayed ahead in almost every condition.

The noise in a molecular dataset should therefore be modeled before a method is chosen, and the features adapted to each dataset.

Statement on the Use of Generative AI

I declare that my use of generative AI in this thesis complies with the University of Oxford's policy on generative AI in research and summative assessment. It also complies with the guidance of the Mathematical, Physical and Life Sciences Division for postgraduate research students and of the Department of Statistics.

The research questions, methods, experimental design, interpretation of results and conclusions are my own, except where a citation or the Statement of Contribution says otherwise. All written text in this thesis is my own.

I used Claude (Anthropic), through the Claude web application and Claude Code, from February 2026 to October 2026. The models were Claude Opus 4.5, 4.6, 4.7, 4.8, 5 and 5.5, Claude Sonnet 5 and 5.5, and Claude Fable 5 and 5.1. Before that, I used ChatGPT (OpenAI), with GPT-5.1 and GPT-5.2, from December 2025 to January 2026.

I used Claude and ChatGPT to discuss research ideas I had already formed, for example by asking how a proposed method might fail or what related methods existed. The contributions of this thesis are my own. They include the adaptive hybrid search and the design of its search space (Chapter 5), and the cliff-aware gap, a score for finding noisy labels (Chapter 7). I did not adopt any AI output as a research contribution.

I wrote the original Python and Rust data pipelines by hand. I then used Claude, mostly through Claude Code, to extend them and to implement methods from my own specifications, including the adaptive hybrid search and the noise mitigation experiments. I also used it to write unit tests and to debug. I reviewed and tested all AI-assisted code, and checked it against the methods described in this thesis. I take full responsibility for the correctness of all code. Before February 2026 I used ChatGPT for the same purposes.

I used Claude to help write plotting scripts. Every plot in this thesis was made by code run on the recorded experimental outputs, and that code is available to the

examiners. I did not use generative AI to make a figure from a prompt, or to generate or change any data or numerical result. I drew all schematic diagrams by hand in Lucidchart. For some of them I asked generative AI for feedback on layout and clarity.

I used generative AI to help structure my writing, for example to propose outlines for chapters and sections and to point out gaps in an argument. In some cases I asked for a draft passage as a guide to structure, and then rewrote it in my own words. No AI-generated text appears verbatim or closely paraphrased in this thesis. I also asked for comments on the clarity of passages I had written.

I used Claude and ChatGPT to suggest search terms and candidate references. I checked every cited reference against its original source, and discarded any suggested reference I could not verify.

Generative AI did not produce text, data or figures in this thesis, so it is not cited as a source in the main text. All code used to produce the results and figures is available at `https://github.com/adpunt/noise_inject` and `https://github.com/adpunt/KIRBy`, with the full development history of each codebase.

List of Figures

2.1 Nodes of a deterministic and a Bayesian neural network. 15
2.2 Label distributions under the seven noise conditions. 21
2.3 Evaluation of a noise mitigation method. 24

3.1 Variance decomposition of robustness and predictive performance on QM9. 39
3.2 Variance shares of clean performance, noisy performance and robustness on all four datasets. 41
3.3 Predictive performance against Gaussian label noise on QM9. 43
3.4 Clean predictive performance against robustness on QM9. 43
3.5 Robustness of each base model on each representation on QM9. 44
3.6 Robustness under every noise condition on a subset of models on QM9. 46
3.7 Random forest under each noise condition on QM9. 47
3.8 Rank of each base model at each noise level on QM9. 48
3.9 Predictive performance against noise level on every dataset. 53
3.10 Robustness on the three assay datasets. 54
3.11 Robustness rank of each model on QM9 and the three assay datasets. . 55
3.12 Change from replacing a model with its probabilistic counterpart. . . . 56
3.13 Aleatoric and epistemic uncertainty against label noise on QM9. 58
3.14 Aleatoric uncertainty against the added noise. 59

4.1 Shares of constant, near-constant and varying features per representation. 68
4.2 Two hERG K_i stereoisomers with identical ECFP4 vectors. 70
4.3 Absolute pK_i difference against pair distance on hERG K_i. 75

5.1 Schematic overview of the hybrid search. 87
5.2 The three levels of splitting. 88
5.3 Held-out score against the number of features: small molecules and reactions. 129
5.4 Held-out score against the number of features: antibodies. 130
5.5 Features taken from each representation. 135

5.6 Features of the LogD hybrid picked on every fold. 136

7.1 How a noise method is run and scored. 166
7.2 Held-out score against the share flagged and the share clipped. 183
7.3 Did the cleaning work: molecules selected by a detector. 185
7.4 Did the cleaning work: molecules selected by a known flag. 187
7.5 Did the cleaning work: noise-robust training. 188
7.6 Precision of the deleted molecules against the score after cleaning. . . . 195
7.7 Correcting clipped training labels. 197

List of Tables

2.1 Sources of label noise in bioactivity data. 19
2.2 The seven noise conditions and their parameters. 22
2.3 Terms for methods that handle noisy labels. 23

3.1 Variance decomposition by noise condition on QM9. 40
3.2 Robustness under censoring and under Gaussian noise on QM9. 45
3.3 Change in robustness from Gaussian to grouped-shifted noise on QM9. 47
3.4 Robustness under the noise distributions on the three assay datasets. . 50
3.5 Rise in mean predicted uncertainty under Gaussian noise. 57

4.1 Molecules that share a feature vector with another molecule. 71
4.2 All-pairs correlation, five-nearest-neighbor prediction and activity cliffs per dataset. 75
4.3 Agreement between representations per dataset. 76
4.4 Recovery with neighbor-based sample weights by representation. 79
4.5 Flaws of standard representations and similarity, and the requirement each sets. 80

5.1 Methods for combining or reducing representations. 84
5.2 Settings of the importance methods. 93
5.3 The five allocation methods searched. 94
5.4 Worked example of the four fixed-share allocation methods. 94
5.5 Values the hybrid search can take for each setting. 97
5.6 Search grid of each dataset. 98
5.7 Datasets of the hybrid experiments. 100
5.8 Short names of the datasets, used in the figures. 101
5.9 Representations used for small molecules. 103
5.10 Representations used for chemical reactions. 104
5.11 Representations used for antibodies. 105
5.12 Base and augmentation representations per dataset. 106

5.13 Experiment register for the hybrid experiments. 111
5.14 Held-out score of the best single base representation and the hybrid, per dataset and model, mean over outer folds, in each dataset's own metric. Bold: the four rows where the hybrid gains most over the best single base representation, as a fraction of the single score. Best in hindsight is the single representation best on the held-out samples, with its score. 119
5.15 Hybrid against ensembles of one model per representation. 121
5.16 Search's hybrid against the top representation alone and an even split. 122
5.17 Hybrid on molecules from frequent and rare scaffolds. 124
5.18 The search's own score of its pick against the held-out score. 125
5.19 Search with one fold fewer, and stability of the selected features. 127
5.20 Share of the variation in score from the downstream model and the budget. 131
5.21 Importance scores used again during training. 131
5.22 Smaller hybrid against a wider hybrid with the model's setting tuned. . 133
5.23 Hybrid on three activity-cliff targets. 137
5.24 Graph kernel on hERG K_i with and without added features. 137
5.25 Search settings found on LogD reused on hERG K_i. 137
5.26 Feature selection when each site holds one family of representations. . . 138

6.1 Nearest-neighbor settings 145
6.2 Similarity experiments 147
6.3 Distance against label difference in single representations 151
6.4 Spearman correlation between the distance between two samples and the difference in their labels, per dataset, mean over outer folds. 152
6.5 Distance against label difference in the predictive hybrids 153
6.6 Distance against label difference in each feature space 154
6.7 Prediction score and distance against label difference by number of features 155
6.8 Held-out score of predicting each molecule from its five nearest training molecules, per dataset, mean over outer folds, in each dataset's own metric. Bold: a dataset where the hybrid is worse. 156
6.9 Prediction from nearest neighbors with the setting picked again for each fold 157
6.10 Importance weighting in an SVM 157
6.11 Importance weighting on all representations joined 158
6.12 Importance weighting in prediction from five nearest neighbors 159
6.13 Prediction on activity-cliff molecules 160

6.14 Distance to the nearest training molecules against prediction error . . . 161
6.15 Label gap to the nearest neighbors with the setting picked again 162
6.16 Label coherence of k-means clusters 163

7.1 Damage kinds on a regression dataset. 167
7.2 Damage levels. 167
7.3 Detectors. 168
7.4 Noise mitigation methods. 171
7.5 The expanded published sweep. 177
7.6 Actions and detectors under hERG label flips. 182
7.7 Deleting flagged molecules against random deletion. 182
7.8 Actions and detectors on LogD with outliers. 184
7.9 Down-weighting flags of known content. 184
7.10 Weighting flagged molecules. 186
7.11 Deleting flagged molecules on QM9 with no noise added. 186
7.12 Cleaning the labels as delivered against shuffled detector scores. 188
7.13 Label smoothing on hERG K_i by fold. 190
7.14 Putting cliff molecules back on hERG K_i. 190
7.15 Detection by the forest residual and the cliff-aware gap. 192
7.16 Detection by distance family. 193
7.17 Scaling features by importance before finding neighbors. 194
7.18 Detector precision against break-even precision. 196
7.19 Censored EM on further outer folds. 198
7.20 Three ways to raise clipped labels. 199
7.21 Per-molecule lift against shuffled lifts. 199
7.22 Removing a shared offset per scaffold family. 200
7.23 Removing family offsets from the labels as delivered. 201
7.24 Moving every label toward a Huber-fitted prediction. 202
7.25 Neighbor-agreement weights on SMAC colloidal stability. 202
7.26 Repeat measurements on hERG IC_{50}. 204
7.27 Cleaning before the feature search. 205
7.28 Searching on clean against damaged labels. 206
7.29 Changing the model instead of the labels: damaged labels. 207
7.30 Changing the model instead of the labels: labels as delivered. 208
7.31 The expanded published sweep, by domain. 209
7.32 Two results of the expanded sweep. 209

7.33 The expanded sweep on QM9 with no noise added. 210
7.34 Neighbors found in the single representation or in the hybrid. 211

List of Abbreviations

AC Activity cliff

AC-SINS Affinity-capture self-interaction nanoparticle spectroscopy

ANOVA Analysis of variance

AUROC Area under the receiver operating characteristic curve

BLOSUM Blocks substitution matrix

BNN Bayesian neural network

CDR Complementarity-determining region

CGR Condensed graph of reaction

CHO Chinese hamster ovary

CL_{int} Intrinsic clearance

CV Cross-validation

DFT Density functional theory

DNN Deep neural network

EC_{50} Half-maximal effective concentration

ECE Expected calibration error

ECFP Extended-connectivity fingerprint

ECFP4 Extended-connectivity fingerprint with radius 2, diameter 4

EGFR Epidermal growth factor receptor

EM Expectation maximization

F1 F1 score (harmonic mean of precision and recall)

FR Framework region

GCN Graph convolutional network

GIN Graph isomorphism network

GNN Graph neural network

GP Gaussian process

hERG Human ether-à-go-go-related gene (a cardiac potassium channel)

HIC Hydrophobic interaction chromatography

HLM Human liver microsome

HOMO Highest occupied molecular orbital

IC_{50} Half-maximal inhibitory concentration

IMGT The international ImMunoGeneTics information system

KL Kullback–Leibler

kNN k-nearest neighbors

LGBM LightGBM (short form used in tables)

LightGBM Light Gradient Boosting Machine

LM Language model

LNL Learning with noisy labels (the problem of training on labels that are partly wrong, not a group of methods)

LUMO Lowest unoccupied molecular orbital

MACCS Molecular ACCess System

MAE Mean absolute error

MCC Matthews correlation coefficient

MCS Maximum common substructure

MDCK Madin–Darby canine kidney

MDR1 Multidrug resistance protein 1 (P-glycoprotein)

MHG-GNN Molecular Hypergraph Grammar Graph Neural Network

ML Machine learning

MLP Multi-layer perceptron

mRMR Minimum redundancy, maximum relevance

NGBoost Natural gradient boosting

NLL Negative log-likelihood

NLP Natural language processing

NN Neural network

NN-α Neural network architecture α ([128, 64], dropout after each layer)

NN-β Neural network architecture β ([128, 128], dropout before output)

OPIG Oxford Protein Informatics Group

PCA Principal component analysis

PDV Physicochemical descriptor vector

PLS Partial least squares

QRF Quantile regression forest

QSAR Quantitative structure–activity relationship

QSPR Quantitative structure–property relationship

RBF Radial basis function

RF Random forest

RLM Rat liver microsome

RMSE Root mean squared error

SALI Structure–activity landscape index

SD Standard deviation

SHAP SHapley Additive exPlanations

SMAC Standup monolayer adsorption chromatography

SMILES Simplified molecular input line entry system

SNS Sort & Slice

SPP Similar property principle

SVM Support vector machine

USR Ultrafast shape recognition

USRCAT Ultrafast shape recognition with CREDO atom types

VAE Variational autoencoder

VBLL Variational Bayesian Last Layer

VEGF Vascular endothelial growth factor

WL Weisfeiler–Lehman

XGB XGBoost (short form used in tables)

XGBoost eXtreme Gradient Boosting

Chapter 1

Introduction

New avenues in the drug discovery pipeline have opened up thanks to advances in large dataset collection, analysis, and computation. Therapeutic targets are identified, potential drug candidates are screened and optimized, and the most promising of these go on to pre-clinical and clinical trials [1]. However, attrition is high, and roughly 90% of drugs that enter clinical trials are never approved, often due to poor efficacy or toxicity [2]. Computational methods can remove poor candidates early on in the drug discovery pipeline, long before clinical trials. This is done through virtual screening, which allows for a large-scale analysis of libraries of compounds [3]. One standard screening method is the prediction of bioactivities and pharmacological properties of such candidates. There is a long history of using Quantitative Structure-Activity Relationship (QSAR) models to make these predictions, to act as a filter for compounds with undesirable properties, and thus reduce the time and cost associated with experimental screening [4]. QSAR models aim to predict the relationship between a molecule's structural and chemical features as encoded by a particular molecular representation and a desired experimental molecular property [4] of interest. A QSAR model requires a molecular representation and some form of machine learning (ML) model. An ideal molecular representation will allow a model to learn only the relevant information about each molecule with respect to the target property. Representations are either 'static' handcrafted descriptors or circular fingerprints that encode structural and physicochemical features [5–8]; or learned representations, where the encoding is derived from a predictive model, often taken from the activations of the final layer of a neural network, or a latent space embedding [4, 9].

QSAR models are of course, sensitive to the quality of the input data on which they are trained. Experimental noise can arise from measurement errors, variability under experimental conditions, varying provenance of the data, intrinsic biological, chemical

variability, and human error. Even the very measures of model performance may be flawed if test and validation sets are laden with noise. Kolmar and Grulke [10] showed that error in the test labels can make a model look worse than it is: a model trained on noisy data can predict the noise-free values more accurately than an evaluation on the noisy test set indicates. There is also a limit to how much noise the model can tolerate; Jorner et al. [11] argued that once their model's error approached the experimental error, more training data would not be expected to improve accuracy much. In the context of activity prediction, the inherent variability within experimental setups and assays, batch effects, measurement errors, and biological variability introduce additional layers of complexity [10, 12, 13]. All of this increases the likelihood of encountering noise in the data. Even the molecular representations in the training data themselves may be incorrect or misleading [14, 15], and chemical datasets contain too few repeat measurements of each compound to characterize the distribution the label is drawn from. Thus, models treat a single label as the population mean [10]. Here pIC_{50} and $\mathrm{p}K_i$ are the negative base-10 logarithms of the IC_{50} and K_i in molar units, so a difference of one log unit is a tenfold difference in concentration. Repeat measurements of the same compound-target pair have a standard deviation of about 0.68 log units for pIC_{50} and 0.54 log units for $\mathrm{p}K_i$ [12, 13], which is about a 4.8-fold and a 3.5-fold difference in concentration. Restricting to measurements from a single assay lowers the mean disagreement from 0.50 to 0.27 log units [16], or from about a 3.2-fold to a 1.9-fold difference.

This thesis addresses the challenge of conducting molecular property prediction when the underlying experimental data, or "labels" are noisy. Most work on noise in QSAR modeling assumes Gaussian label noise and evaluates either representations or model architectures in isolation. Cortés-Ciriano et al. [17] examined 12 algorithms across multiple noise levels but used fixed representations, and studies that compare representations typically use clean data. No work has systematically compared the relative importance of representation versus architecture choice for label noise robustness, asked whether probabilistic methods justify their computational overhead under noise, or tested whether findings generalize across different types of injected noise. This thesis fills that gap. I will also discuss noise mitigation, using the results of my noise robustness study. I explore why many noise mitigation techniques underperform in cheminformatics data and methods to address this problem in the form of novel molecular representations.

The first half of this thesis therefore asks four questions. First, how does the choice of molecular representation and model architecture explain the variance in predictive

performance, and does that division change when you instead consider how much performance a QSAR model is able to retain under increasing label noise? Second, what does the choice of model provide under label noise? Within this question we compare the effects of deterministic and probabilistic modeling. Third, does the type of label noise matter, or just the quantity of it? We explore different techniques of generating and adding label noise to both clean data, as well as data with inherent experimental noise. Fourth, what is the impact of artificial label noise on a model's uncertainty estimation? Does the estimated uncertainty rise with increased label noise and can these uncertainty values point to which labels are corrupted? Together, these questions aim to identify what, if anything, makes a QSAR model robust to noise. To answer them we introduce NoiseInject, an open-source Python package for benchmarking noise robustness and uncertainty quantification (Section 2.6.1).

In an early study with Gaussian noise added to QM9 labels, the configurations of published methods for noisy labels that kept the least performance all compared molecules by a distance. We hypothesized that the distance was at fault. On a given dataset, most features of a general-purpose representation either barely vary or do not relate to the property. Yet each one that varies adds a term to the distance, so together they dominate it (Chapter 4). We propose instead to compute distances over a hybrid representation: a fixed number of task-relevant features selected from several representations. A first version showed that the selected features work as ordinary model inputs too, so the question became how to choose them (Chapter 5). Chapter 6 tests whether distance in the hybrid tracks the label better, and Chapter 7 returns to noisy labels, comparing the single representations with the hybrid. The same problem appears in antibody representations, where the features that matter are few and buried in representations of the whole sequence. For this reason we test the method on antibodies, and on chemical reactions.

The second half asks whether the representation itself can be adapted to the task:

1. How can the best set of features be selected for a given dataset and model, and does the method carry over from small molecules to antibodies and reactions?

2. Does distance in a hybrid representation track the label better than distance in the best single representation, and do methods built on nearest neighbors work better in it?

3. Can noisy labels be found and corrected, and do hybrid representations stay ahead of single representations through that correction?

4. Does weighting features by their importance help in building hybrid representations, in model training and in finding noisy labels?

1.1 Outline of the thesis and contributions

Chapter 2 introduces molecular representations, model families, molecular similarity and activity cliffs, and the sources of noise in molecular data. It defines the seven noise conditions used in Chapter 3 and the three groups of methods for noisy labels (noise detection, noise mitigation and noise-robust training), and closes with artificial noise injection, and evaluation metrics.

Chapter 3 looks into what makes a QSAR model robust to label noise. The primary prediction target is the HOMO–LUMO gap in QM9, whose computed labels carry negligible measurement noise. The same experiments were also run on three assay datasets: LogD, Caco-2 and hERG K_i. We compare seven different types of label noise, referred to as noise conditions. We matched the quantity of noise to the training set label spread for all but one of the noise conditions. We measure noise robustness as the proportion of predictive performance retained as artificial label noise increases. We found that model family was the main factor in noise robustness, and choice of molecular representation matters more for predictive performance than noise robustness. Strong predictive performance on clean labels does not imply robustness. LightGBM and XGBoost predicted well on clean labels but lost the most with increasing label noise, while random forests were the most consistent model across datasets, representations, and noise conditions. We replicated systematic noise by giving every scaffold family its own offset and by clipping labels at a made-up assay limit. Models struggled the most when training on this systematic noise. Label noise raised mean predicted uncertainty for the probabilistic models which estimate it; however, only the BNNs with a variance head, the heteroscedastic GP and the heteroscedastic VBLL networks correctly attributed the rise in uncertainty to the data rather than to the model itself, though not on every dataset and representation.

Chapter 4 takes a turn to look at another topic: molecular similarity. The natural next step to Chapter 3 is noise mitigation. However, a small sweep of twelve published noise-handling methods, four that remove molecules and retrain and eight that change the training, showed disappointing results. None of the methods that incorporated the concept of molecular similarity showed consistently promising results. On a given dataset, standard representations carry many features that do not vary, and some map different molecules to the same vector. Similarity computed over these representations

inherits both of these problems, and methods that rely on similarity suffer as a result. The chapter ends with what a representation would need to mitigate these problems, which Chapter 5 builds.

Chapter 5 presents the adaptive hybrid representation. The adaptive hybrid search customizes the settings of a feature selection process to a given dataset, model, and budget. It takes in several base representations and selects the most important features for that task by a fixed budget. This produces a tailor-made feature set, referred to as the hybrid representation. The actual output of the hybrid search is not the set of features, but rather the settings that define the feature selection method used to generate that hybrid representation.

We compare the hybrid with each base representation, with an ensemble of one model per representation, and with principal components, recursive feature elimination and random features of the same size. We read the features it selects, and test the method on antibodies and reactions, and on data held at separate sites. The hybrid predicted better than the best single representation on every small-molecule dataset, with both downstream models. On antibodies the gain depended on the dataset: on two of them the best single representation was one we built to read only the regions or mutations that vary. On most antibody datasets, the large expression scan included, an ensemble of one model per representation did better. On the reactions the hybrid gained nothing.

Chapter 6 asks whether a hybrid representation repairs the failures of similarity described in Chapter 4. We build hybrids whose features are chosen for distance, ranking each candidate by how well each molecule's five nearest neighbors predict its label. On every small-molecule dataset, distance in the hybrid tracked label differences more closely, and the five nearest neighbors predicted better. On the antibody datasets and the reactions it tracked the label more closely as well, but prediction from neighbors was worse on several of them. The benefit of a hybrid has to be measured on each dataset.

Chapter 7 returns to label noise, with the single representations and the hybrid side by side. We test noise detection, noise mitigation and noise-robust training on injected noise and on the labels as delivered. Corrections worked best when a known flag, such as an assay limit or a scaffold family, said which labels were wrong. On small molecules the hybrid stayed ahead of the single representation in almost every condition, but not always with the network on hERG K_i. The best method changed between the two.

Chapter 8 answers the research questions. It argues that the noise in a dataset should be modeled before a method is chosen. No single method fits every dataset,

and simple methods are often as good as complex ones. The chapter ends with the contributions, limitations and future work.

The software contributions are NoiseInject and the KIRBy repository, which implements the search for hybrid representations and the experiments (Section S.8 of the Additional files).

1.2 Collaborations

This work was supervised by Garrett Morris and co-supervised by Thierry Hanser and Stéphane Werner at Lhasa Limited, which runs the federated learning platform Effiris [18]. Lhasa also supplied the hERG data used in Chapters 4 to 7; in Chapter 5, two of its sets and the public ChEMBL hERG K_i set are treated as data held at separate sites, as in federated learning [18, 19].

The Oxford Protein Informatics Group (OPIG) pointed us to the antibody and protein datasets used in Chapters 5 to 7; all datasets are listed in Section S.7 of the Additional files.

The adaptive hybrid representation, its software and the experiments in Chapters 4 to 7 are the author's own work.

Chapter 2

Molecular representations, models and label noise

2.1 Overview

Chapter 1 set out the questions this thesis asks; this chapter gives the background they rest on. We first describe the molecular representations and models we use. We then cover molecular similarity and activity cliffs. Label noise follows, with its sources in bioactivity data and the ways we inject it, and then the methods for learning with noisy labels. The chapter closes with our NoiseInject package and how results are evaluated.

2.2 Molecular representations in QSAR

The success of machine learning hinges on the quality of input data. Although the quality can refer to noise, which the majority of our work is centered around, in this case we are talking about the type of input. Molecular representations provide different features with which QSAR models and noise detection techniques can be learned. However, a potential bottleneck on larger datasets should be noted; the more complex the data structure, the greater the computational cost. For smaller datasets, the number of parameters is limited by the size of the training set, to prevent overfitting.

One of the most common and intuitive molecular representations is the Simplified Molecular-Input Line-Entry System (SMILES) string, which consists of a sequence of ASCII characters generated by a fixed set of rules to specify a given chemical identity [20]. A common way to use SMILES strings as inputs to QSAR models is to tokenize and one-hot-encode them rather than to use the strings themselves [21]. The strings are tokenized into atomic branches and symbols and mapped to indices of a vocabulary, and

finally converted into binary vectors using one-hot encodings [21]. This representation, referred to as one-hot-encoded SMILES, allows models to process molecular structures as bit-vector-based fingerprints or as sequences of vectors. Sequence-based models such as recurrent neural networks and transformers have proven to be particularly successful in capturing text semantics within a sequence. When applied to raw or one-hot-encoded SMILES strings, these models can effectively capture the structure of the molecule in its textual representation [21].

Another common representation is the molecular fingerprint, a vector containing information about the properties of the molecule in question. In a typical binary fingerprint, each bit accounts for the presence or absence of a given chemical feature [22], and in the case of Morgan fingerprints, a given substructure. One common fingerprint is the Molecular ACCess System (MACCS) key, a substructure key-based fingerprint that encodes predefined atom symbols and properties, as well as bond types [23]. Another is the Extended Connectivity Fingerprint (ECFP), or Morgan fingerprint, which encodes local neighborhoods around each atom and bond connectivity in molecules [23]. These existing molecular representations are well established and have been quite successful when coupled with QSAR models [4, 24]. They are relatively simple, and as the bits can be mapped back to one or more substructures, they are human-interpretable, and furthermore easy-to-compute. However, hashed fingerprints are subject to information loss due to bit collisions, which happens when two different substructures are encoded at the same bit position [22]. They are also based on a fixed procedure and are not trainable or adaptable to a particular dataset. Alternative representations that capture different structural, chemical, or physical features are able to overcome these limitations.

Recent advances have led to the development of new molecular fingerprints. One such method is the Sort & Slice (SNS) technique, an alternative to hash-based folding for pooling ECFP substructures [7]. Sort & Slice begins by sorting all circular substructures in the training set according to their frequency of occurrence. It then "slices" away all but the N most frequent substructures, which are subsequently used to generate an N-dimensional, bit-collision-free binary fingerprint. This approach solves the ambiguity problem of hashed fingerprints, where multiple substructures might be encoded at the same bit position, making it very interpretable [7]. Sort & Slice fingerprints outperform ECFP of diameter four (ECFP4) with standard hash-based folding methods across various molecular property prediction tasks. These experiments consisted of different data splitting techniques, machine learning models, fingerprint diameters, fingerprint dimensions, and atomic invariants [7]. Furthermore, Sort & Slice

outperformed advanced substructure selection schemes such as filtered fingerprints and mutual information maximization [7].

Neural fingerprints are a category of learned representations that overcome the fixed, hard-coded nature of traditional fingerprints. Unlike ECFP or MACCS keys, which rely on predefined hashing schemes or substructure catalogs, neural fingerprints train on molecular data to capture chemical information [25, 26]. These representations are the result of deep learning architectures that have learned dense, continuous embeddings which can be used for downstream tasks.

Mol2vec, which adapts the Word2vec algorithm from NLP to molecular substructures [25], is a common neural fingerprint. Mol2vec treats molecules like "sentences" and circular substructures derived from Morgan fingerprints like "words". The model learns vector embeddings for each substructure through unsupervised training, such that chemically similar substructures are mapped to nearby points in the embedding space [25]. The embeddings of all present substructures are summed to obtain a full representation. This approach produces 300-dimensional dense vectors that avoid the sparsity and bit collisions inherent in hashed fingerprints [25]. Mol2vec has performed strongly on various QSAR benchmarks, matching or exceeding Morgan fingerprints on solubility and bioactivity prediction tasks [25].

Transformer-based architectures have also been applied to molecular representation learning, similarly stemming from NLP ideas. MolBERT applies a bidirectional transformer to learn molecular representations from SMILES strings through self-supervised pre-training [27]. The model has domain-relevant auxiliary adaptations, such as the prediction of calculated physicochemical properties alongside the masked token prediction, to improve the chemical fidelity of the learned embeddings [27]. MolBERT outperformed count-based ECFP4 (ECFC4), RDKit descriptors and the CDDD neural representation on virtual screening and QSAR benchmarks [27]. ChemBERTa is another transformer-based approach which pre-trains masked language models on SMILES data from PubChem [26]. ChemBERTa uses a byte-pair-encoding tokenizer by default and compares it with a SMILES-specific tokenizer [26]. Attention visualization shows neurons selective for chemically relevant functional groups and aromatic rings, and others that track bracket closures in the SMILES string [26]. ChemBERTa-2, pre-trained on up to 77 million PubChem compounds, is competitive with existing architectures on MoleculeNet [28].

Neural fingerprints provide several advantages over traditional fingerprints. They produce dense continuous vectors that avoid sparseness and information loss from bit collisions [25]. They can also be pre-trained once on large unlabeled datasets and

then fine-tuned for specific downstream tasks, opening the door to transfer learning [26, 27]. However, pre-training them is computationally expensive, and they are less directly interpretable than fingerprints, although attention visualization offers some insight [26].

Beyond fingerprints, physicochemical-descriptor vectors (PDVs) are a class of continuous-value molecular descriptors that encode structural and chemical properties of molecules [29]. Unlike binary fingerprints or graph-based representations, PDVs aggregate atom and bond-level quantities into fixed-length feature vectors that capture a wide variety of information about a molecule and its structure [30]. PDVs typically consist of hundreds of properties including constitutional descriptors such as molecular weight, atom and bond counts, and heteroatom ratios; topological indices encoding molecular connectivity patterns; physicochemical properties such as logP (lipophilicity), molar refractivity, and topological polar surface area; and electronic descriptors derived from partial charges and polarizabilities [29, 31]. The `RDKit` cheminformatics toolkit provides a comprehensive suite of more than 200 molecular descriptors that cover these categories [32]. Many PDV descriptors, such as logP, molecular weight and partial charges, have a direct physicochemical meaning, although many of the thousands available have no clear interpretation [31]. They are also less computationally expensive than some 3D methods or neural fingerprints, with many descriptors simply requiring a 2D molecular structure without conformer generation or extensive pre-training. This efficiency makes PDVs particularly appropriate for high-throughput applications. Descriptors based on 3D structure can also be used [30].

However, PDVs have limitations. The dimensionality of PDV feature spaces can lead to problems with multicollinearity [29]. This can be mitigated by feature selection or dimensionality reduction techniques [33, 34]. Additionally, PDVs encode molecular properties as aggregated global features, which may not capture fine-grained structural details such as specific substructure arrangements or local chemical environments, information important for particular endpoints.

One of the more intuitive representations is the molecular graph. These graphs can be categorized into 2D and 3D graphs. 2D graphs rely on adjacency matrices that encode topological atom- and bond-level features but leave out geometric properties such as bond angles or torsion, meaning they may fail to capture stereochemistry. 3D graphs incorporate these spatial features, but are often limited by remaining in a single conformation [35]. A standard molecular graph is constructed such that each molecule is represented as $G = (V, E)$ with node features $\mathbf{x}_i$ and edge features $\mathbf{e}_{ij}$. In one common

featurization, nodes encode atom type, chirality, hybridization and aromaticity, and edges encode bond type, ring membership, direction and stereochemistry [35].

In the standard molecular graph, atoms are mapped to nodes and bonds to edges [6]. These molecular graphs are typically 2D objects; 3D information such as atomic coordinates and bond angles is not part of the graph unless added as node or edge attributes [6]. However, there are alternative molecular graph representations such as graphs with quantum 3D embeddings, weight alignment graphs, association graphs containing pairs of molecules, and geometric interaction graph neural networks that incorporate 3D structures and physical interactions [36–38]. Du et al. [39] developed hybrid 2D and 3D graphs, combining 3D data, guaranteeing precision and completeness by providing unabridged information, and 2D data, providing chemical intuitions for guidance, in a zipper-like arrangement. Voxel-based 3D representations, in which each atom occupies one grid cell, are too sparse for convolutional networks to perform well; Kuzminykh et al. [40] instead spread each atom over nearby voxels with a wave transform.

These graph representations do not highlight specific substructures or specific arrangements of atoms and bonds within each molecule. The substructures contain functional groups that are related to a range of chemical properties and are potentially involved in downstream tasks related to the function of molecules. Bouritsas et al. [41] developed graph substructure networks, a topologically aware message passing scheme based on substructure encoding. The method involves encoding structural roles based on the appearance of specific substructures and using them in message passing updates. Since message-passing graph neural networks (GNNs) are at most as powerful as the Weisfeiler–Lehman test, they cannot isolate sufficient features from the graph structure [41]. Graph substructure networks, on the other hand, capture these substructure-related features. Encoders pretrained on unlabeled molecules, whether graph-based [9] or transformers on SMILES [42], can also produce embeddings that capture structural features.

2.3 Models for molecular property prediction

There are a wide variety of model architectures that can be used for QSAR modeling. Starting off with simpler and more explainable models, we have tree-based methods. Decision trees are non-parametric supervised learning algorithms that perform recursive binary partitioning of the feature space [43]. A decision tree builds a predictive model by learning a series of if-then rules that recursively split the data based on given feature

values, creating a tree where internal nodes represent decision rules, branches represent the results of those decisions, and leaf nodes contain predicted values or class labels [43]. Each recursive split is chosen to maximize homogeneity within the resulting partitions. For classification, Gini impurity is commonly used and for regression, mean squared error reduction [43].

There are several properties that make decision trees a good foundation for QSAR modeling. First, they handle non-linear relationships between molecular descriptors and target properties without requiring explicit feature engineering or transformation [43]. They also do not make assumptions about the underlying distribution of the data, making them robust to violations of parametric assumptions that can limit linear regression or other classical methods [43].

Decision trees handle data in a "contributory" rather than "consensus" manner. In consensus methods such as linear regression or neural networks, every data point influences the prediction at every other point in the feature space. Tree-based methods make predictions based on the subset of training samples that fall into the same leaf node as the test point [43]. This mechanism essentially quarantines outliers in their respective leaf nodes, limiting their impact on predictions for samples in other regions or chemical domains.

Individual decision trees tend to have high variance and often overfit, particularly at large depths [43]. Small modifications made to the training data can result in large differences in the structure of the tree, resulting in unstable predictions [44]. Ensemble methods are used to counteract this instability, aggregating predictions from many trees. Random forests (RFs) aggregate predictions from trees trained on bootstrap samples of data with random subsets of features at each split [45]. Bootstrapping ensures that each tree is trained on a slightly different subset of data, which effectively reduces the influence of any individual sample [45]. This is especially beneficial with noisy samples. The selection of random features at every node decorrelates the trees, which lowers the generalization error of the ensemble [45]. The aggregation of trees further prevents overfitting to noise by smoothing out prediction errors from individual noisy labels.

Neural networks (NNs), a very popular family of models, can also be used for QSAR, although with mixed results. An NN is a system of interconnected nodes which learn by passing information through its layers. There are weights and biases attached to the nodes and various other parameters which the model optimizes to minimize the difference between the predicted output and the actual target value. During training, the network parameters are updated using backpropagation, in which gradients of the

loss function with respect to each weight are propagated backward through the network [46]. The structure of these networks and the hyperparameters used depend on the type of NN. In QSAR, one of the most commonly used architectures is the multilayer perceptron (MLP), an NN containing fully-connected layers with non-linear activation functions which transforms molecular feature vectors (i.e., fingerprints, descriptors) into a property or activity prediction [47].

A typical MLP for molecular property prediction has a few fully connected hidden layers with rectified linear unit (ReLU) activations and dropout regularization to prevent overfitting. Training is performed using backpropagation with gradient descent optimizers such as Adam, to minimize a loss function (mean squared error for regression, cross-entropy for classification) on batches of training data.

However, NN performance on QSAR-related tasks has been very mixed. NNs have a long history in QSAR [48], and tuned deep networks have outperformed shallow methods such as random forests and support vector machines across diverse activity classes [49]. Their complexity and use of gradients make them prone to label memorization [50], which in the context of noise is problematic, as it means that models may memorize the noise themselves. Although models that are memorizing labels may initially appear to have strong predictive performance, they cannot generalize to out-of-distribution samples properly and often provide inaccurate uncertainty estimates. Various approaches have been proposed to improve neural network robustness to label noise, including noise-robust loss functions [51], confident learning to identify mislabeled examples [52], and regularization techniques such as mixup [53] or label smoothing [54].

All the models up to this point have assumed a vector-based input. Molecular graphs, on the other hand, require different types of predictive modeling. Two standard ML methods on molecular graphs include Graph Convolutional Networks (GCNs) and Graph Isomorphism Networks (GINs). GCNs apply the concept of convolution to graph-structured data. Instead of operating on grid-based data, such as images, GCNs aggregate information from neighbors of a node to update its representation [55]. In contrast, GINs address the limitations in GCNs' ability to differentiate between non-isomorphic graphs. Unlike GCNs, which normalize and weight the sum of neighboring node features, GINs use a simple sum followed by a trainable MLP, giving the model flexibility to learn complex transformations of node features. The GIN model's design is informed by the Weisfeiler–Lehman test for graph isomorphism, enabling it to distinguish between molecular graphs that are structurally similar but differ in subtle ways [21, 56]. GIN features have been competitive with or better than fingerprints

and descriptors at classifying pairs of molecules as activity cliffs (ACs; Section 2.4), where small variations in molecular structure can lead to significant changes in chemical or biological activity [57]. Descriptor-based models still predicted the potency of cliff molecules better than deep learning methods [58].

2.3.1 Tree ensembles

Gradient boosting methods such as eXtreme Gradient Boosting (XGBoost) sequentially fit trees to the residuals of previous iterations, progressively reducing the prediction error [59–61]. We used a handful of gradient-boosted tree ensembles in this thesis including Light Gradient-Boosting Machine (LightGBM) [62], and Natural Gradient Boosting (NGBoost) [63].

2.3.2 Support vector machines

Support vector regression fits a function that is as flat as possible while keeping training errors small [64, 65]. It uses the ϵ-insensitive loss. Under this loss, errors smaller than ϵ cost nothing and larger errors cost their size minus ϵ. A constant C trades flatness against tolerance of large errors [65, 66]. The features enter the model only through the kernel. With the radial basis function (RBF) kernel, every feature therefore enters the Euclidean distance between two molecules. In Chapter 3 the continuous representations are standardized and the binary fingerprints are used as built. A wrong label has limited effect. An error inside the ϵ tube changes nothing, and outside it the loss grows only linearly. The weight of each molecule is also capped at C [65]. Under simulated experimental error, the RBF-kernel SVM degraded smoothly and linearly [17]. Its main limitation is computational cost. Training operates on a kernel matrix that grows with the square of the number of training molecules [67].

2.3.3 Gaussian processes

We employed Gaussian process (GP) models using the `Gauche` framework [68], which is optimized for cheminformatics tasks. Standard kernels for GPs are not optimized for the chemical landscape. `Gauche` defines such kernels for strings, fingerprints, and graphs that operate on a range of widely used molecular representations [68]. We used the Tanimoto kernel [69, 70], which is defined for binary vectors $\mathbf{x}, \mathbf{x}' \in \{0,1\}^d$ for $d \geq 1$ as:

$$k_{\text{Tanimoto}}(\mathbf{x}, \mathbf{x}') := \sigma_f^2 \cdot \frac{\langle \mathbf{x}, \mathbf{x}' \rangle}{\|\mathbf{x}\|^2 + \|\mathbf{x}'\|^2 - \langle \mathbf{x}, \mathbf{x}' \rangle}, \tag{2.1}$$

where $\|\cdot\|$ represents the Euclidean norm [68].

The RBF kernel is defined for $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$ as:

$$k_{\text{RBF}}(\mathbf{x}, \mathbf{x}') := \sigma_f^2 \cdot \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right), \tag{2.2}$$

where ℓ is the lengthscale and σ_f^2 the signal variance, which scales both kernels.

2.3.4 Bayesian neural networks

Standard feed-forward networks are deterministic; however, they can be transformed into probabilistic models (Figure 2.1).

Unlike standard deterministic ML methods, Bayesian methods model distributions over predictions using Bayes' rule, allowing for the quantification of uncertainty within parameters and labels. Bayesian methods treat the model parameters as random variables with distributions. Uncertainty can also be estimated from other non-deterministic methods, including deep ensembles [71] and bootstrap modeling [72].

2.4 Molecular similarity and activity cliffs

Much of cheminformatics rests on the similar property principle: structurally similar molecules tend to have similar properties, including biological activity [58, 73]. Nearest-neighbor prediction and GP kernels rely on it, and so do many noise detection methods. Yet similarity is defined only relative to a representation and a coefficient [74]. Two molecules that are near-identical under a substructure fingerprint may be far apart under a PDV. A computed similarity value is easily over-interpreted [74].

The most common measure for binary fingerprints is the Tanimoto coefficient [75]. For two molecules with fingerprints $\mathbf{x}, \mathbf{x}' \in \{0, 1\}^d$,

$$T(\mathbf{x}, \mathbf{x}') = \frac{\sum_{j=1}^{d} x_j x'_j}{\sum_{j=1}^{d} x_j + \sum_{j=1}^{d} x'_j - \sum_{j=1}^{d} x_j x'_j}, \tag{2.3}$$

which is the number of bits set in both molecules divided by the number of bits set in either. Continuous representations are instead compared with a distance, most often the Euclidean distance $\sqrt{\sum_{j=1}^{d}(x_j - x'_j)^2}$. Both measures are sums over the features, and neither weighs a feature by its relation to the property. A bit that is off in both

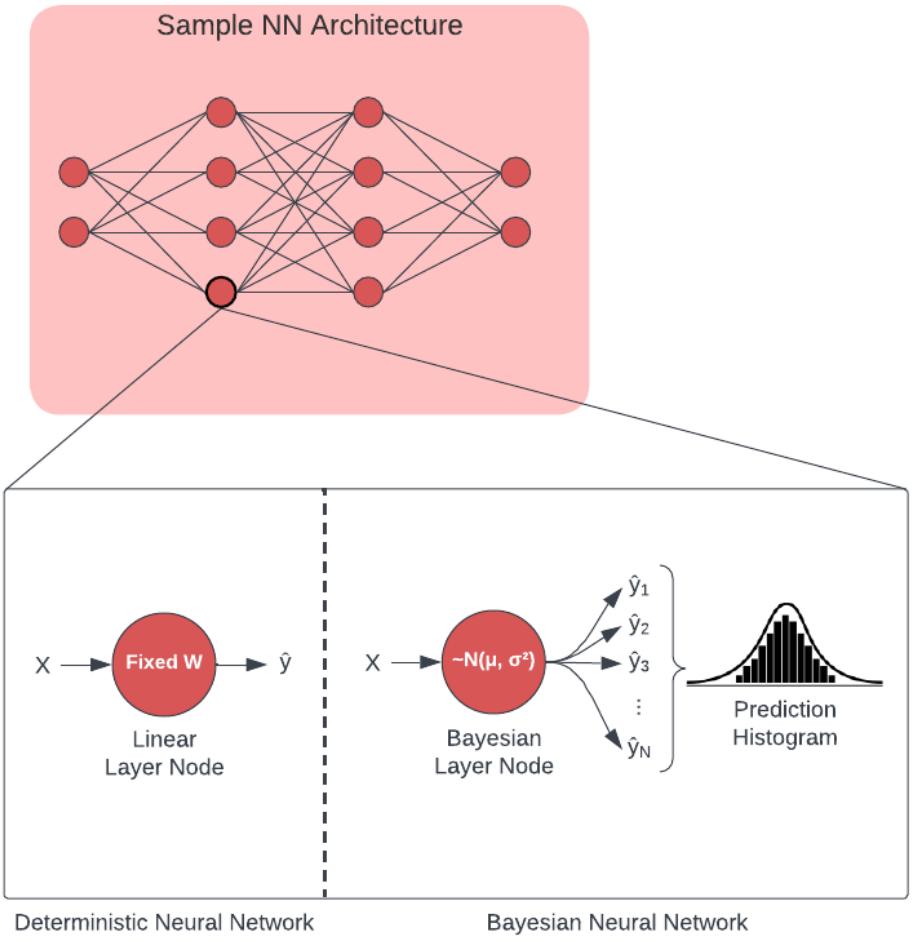


Figure 2.1: One node of a deterministic and of a Bayesian neural network. The deterministic node (bottom left) applies a fixed weight W and returns one prediction $\hat{y}$. The Bayesian node (bottom right) draws its weight from $\mathcal{N}(\mu, \sigma^2)$, so repeated predictions $\hat{y}_1, \ldots, \hat{y}_N$ form a histogram.

molecules contributes nothing to the Tanimoto coefficient, whereas every bit set in either molecule counts; in the Euclidean distance, every feature that differs between the two molecules counts. Chapter 4 returns to this point (Equation 4.3).

Distance-based reasoning relies on the assumption that the data embeddings contain quality information. When the labels are corrupted or the important features are scarce, similarity scores over-rely on compounding false or useless information, and the assumption that similar molecules have similar properties breaks down. Bajusz et al. [75] found that the Tanimoto, Dice and Cosine coefficients and the Soergel distance rank molecules almost identically on binary fingerprints. We take this to mean that changing the formula cannot compensate for a weak representation. This problem is further complicated by activity cliffs (defined below). These molecules, which potentially contain the most important information for model prediction, can be confused

with noise. Another barrier to distance-based reasoning is persistent noise across labels. The task of trying to determine if a molecule's value matches that of its neighbors becomes a lot harder when all of the neighbors have some form of noise as well.

There are many downstream tasks which rely on the concept of similarity. The second half of this thesis relies on many such tasks, and as such focuses on identifying representations that agree with this principle.

An activity cliff is a pair of structurally similar molecules whose potency differs by a large amount because of a small chemical change [76, 77]. Whether a pair is a cliff depends on the similarity measure, and there is no widely accepted definition [58]. The MoleculeACE benchmark [58] uses three measures: the Tanimoto coefficient on ECFPs, the Tanimoto coefficient on scaffold fingerprints and a scaled Levenshtein distance on SMILES strings. A pair forms a cliff if its similarity exceeds 90% under at least one of them and the two molecules differ tenfold or more in K_i or EC_{50} [58]. Across the thirty MoleculeACE targets, molecules in at least one cliff pair make up between 7% and 52% of a dataset [58].

As the number of dimensions grows, the distances to the nearest and to the furthest point become increasingly similar [78]. Molecular representations are routinely high-dimensional and sparse. When many of their features carry no information about the property, distances computed over them are dominated by those features.

2.5 Noise in molecular data

Real-world cheminformatics datasets contain label noise arising from multiple sources across the drug discovery pipeline. Although noise is often modeled as being random, or drawn from a Gaussian distribution, other varieties of noise must be considered when working with experimental data. To properly model experimental noise, we must first understand where the noise is coming from.

One key issue with QSAR models is the standard ML assumption that labels represent true values, as most chemical datasets lack sufficient replications to provide a robust statistical representation of the underlying distributions [10]. A single measurement, or even multiple measurements for a given label, will not guarantee that this label accurately reflects the population mean [10]. Most ML models rely on the assumption that the training data is, in fact, an accurate reflection of the population. Therefore, they treat these potentially noisy labels as point values rather than distributions, which opens the door to overfitting to noise. However, Bayesian methods such as Gaussian processes (GPs) do not make this assumption, and neither does conformal

prediction [10].

Studies involving noise, and specifically the quantification of noise robustness, typically approach label noise by adding independent, zero-mean, Gaussian error to the labels. However, there's evidence to suggest that noise from experimental assays takes on different forms. Kruger and Overington [79] rejected normality of the differences in binding affinity between orthologous targets by Anderson–Darling test, found them approximately Laplace, and found that differences between assays of the same ligand and target closely resemble them. Kalliokoski et al. [13] examined the widest disagreements among 16,844 pairs of repeat pIC_{50} measurements and found that in most cases, the wide gap between repeat values was due to annotation error rather than to an imprecise assay.

Although numerous studies have explored the effects of noise on ML algorithms for molecular property prediction, comparisons between different molecular representations remain limited. Kolmar and Grulke [10] examined the predictive limits of QSAR in the presence of noise, showing that by learning the underlying trends rather than fitting to noise, the models can achieve predictions more accurate than those of their training data. They assumed experimental noise could be modeled as Gaussian-distributed, using the Central Limit Theorem to argue that aggregated experimental measurements from multiple independent error sources exhibit a Gaussian distribution. However, Heid et al. [80] distinguish random noise, which affects all molecules alike, from systematic noise that differs between regions of chemical space. Using synthetic noise applied only to nitrogen-containing molecules, they showed that mean-variance estimation can detect it.

Beyond just the distribution, the assumptions of independence and zero mean may fail as well [81]. Bentz et al. [82] examined P-glycoprotein transport measurements from 23 laboratories and found that, among the laboratories using cell lines, 62% of the variance in log digoxin efflux ratio was due to differences between laboratories. When you pool data from different sources, each source has the potential to carry a shared offset. These sources usually contain families of related compounds, breaking the independence assumption. The zero-mean assumption is not assured either, especially when true values fall above or below an assay's detection limit, called censoring. Svensson et al. [83] count censored labels in 15 industrial assays, of which 13 carry censored labels and eight sit between a quarter and two thirds censored. Chapter 3 tests noise conditions that vary the distribution, the correlation, and the bias of injected error separately, at a matched quantity of noise for every condition except censoring.

Label noise enters bioactivity data at every step from the assay plate to the

database. Table 2.1 lists these sources, with examples of each and the kind of error each tends to produce. The kind of error describes how the error is distributed across molecules, which is the property that the noise conditions in Section 2.5.1 vary.

Source	**Example**	**Kind of Error**
Assay	Edge, row and column effects from evaporation or temperature gradients, and errors in liquid handling [84]; carryover between wells [85]	Error that varies with plate position
Instrument	Higher relative error near a detection limit [86]; autofluorescence of the test compound [87]	Larger error for weak binders; a systematic bias; labels clipped at the limit
Biology	Seeding density, growth rate, cell cycle and serum properties [88]; cell passage number [89]	Variation between repeats of the same experiment
Aggregation across sources	ChEMBL, BindingDB and PubChem pool hundreds of laboratories [90, 91]; independent K_i measurements of one compound–target pair differing by 0.44 log units on average [12]	An offset shared by each laboratory or source, hard to separate from the true value without metadata [92]
Federated learning	Teacher models label a shared public set, and a student model learns from those labels [18]	The teachers' prediction errors on top of the measurement error
Annotation	Mis-registered structures, wrong stereochemistry and salt forms [14]; 0.1–3.4% of structures incorrect, depending on the database [93]; ambiguous protonation states and counterions [94]	A label attached to the wrong molecule; distinct molecules merged

Table 2.1: Sources of label noise in bioactivity data, with reported examples and the kind of error each source produces.

Although the general assumption in much of the noise analysis and mitigation literature is that noise, specifically label noise, will have a negative effect on the model, this is not necessarily the case. There are multiple instances of using noise as a method of regularization. In linear regression, appending observations of pure noise shrinks the coefficient estimates, an implicit regularization, although not an optimal one [95]. Furthermore, adding artificial noise drawn from a Gaussian distribution to the activation layers of a neural network is considered another form of regularization [96]. These studies indicate that the presence of noise may act as a form of regularization in some contexts. Neither corrupts the labels of real observations, so whether label noise regularizes QSAR and quantitative structure–property relationship (QSPR) models remains open.

2.5.1 Artificial noise injection

Our objective is to evaluate the robustness of QSAR models against label noise. To do so, we "inject" artificial noise into the labels during training while test data remain free of noise. Validation labels are given noise of the same type and quantity as the training labels, drawn independently. This artificial noise simulates real-world experimental noise. The clean target label y_i is replaced by a noisy label $\tilde{y}_i = y_i + \epsilon_i$, where ϵ_i is the artificial noise injected at the index i, the value of which is determined by a specified noise strategy. As we increased the quantity of noise, we measured how much predictive performance fell.

We explored seven different ways of adding noise, which we refer to as noise conditions and which are illustrated in Figure 2.2 and summarized in Table 2.2. Three conditions vary the shape of the distribution from which the error is drawn, and three concentrate the error on some molecules or share it within Bemis–Murcko scaffold families. The seventh condition, censoring, clips labels at a limit.

Molecules that share a Bemis–Murcko scaffold, the ring systems and linking chains left once all other side chains are removed [97], form a scaffold family.

For a given label, every condition except censoring delivers the same expected quantity of noise at a set noise level. These levels are defined as a fraction of the standard deviation of clean training labels. The noise added to the label of molecule i is

$$\epsilon_i = \frac{\tau}{G}\, s_i z_i, \qquad G = \sqrt{\frac{1}{n}\sum_{j=1}^{n} s_j^2}\ \ \sigma_z, \tag{2.4}$$

where τ is the target standard deviation of the noise, in the units of the label, and z_i is drawn from the condition's distribution at unit scale. The multiplier s_i is 1 for most molecules and $\lambda = 3$ for those the condition affects, which are a random 10% of molecules under outlier and whole scaffold families covering 20% of the split under grouped-wider. The constant σ_z is the standard deviation of z, equal to 1 for the Gaussian, $\sqrt{\nu/(\nu-2)}$ for the Student-t and $\sqrt{2}$ for the Laplace. The factor G divides both of these out, so that ϵ has standard deviation τ in expectation whatever the distribution and whichever molecules are affected.

Grouped-shifted widens no molecule's error, and instead gives every scaffold family g an offset of its own,

$$\epsilon_i = \frac{\tau}{G}\left(\sqrt{\rho}\, b_{g(i)} + \sqrt{1-\rho}\, w_i\right), \tag{2.5}$$

with one draw b per family and one draw w per molecule. The two variances sum to τ^2, and the family offset carries a share ρ of that total.

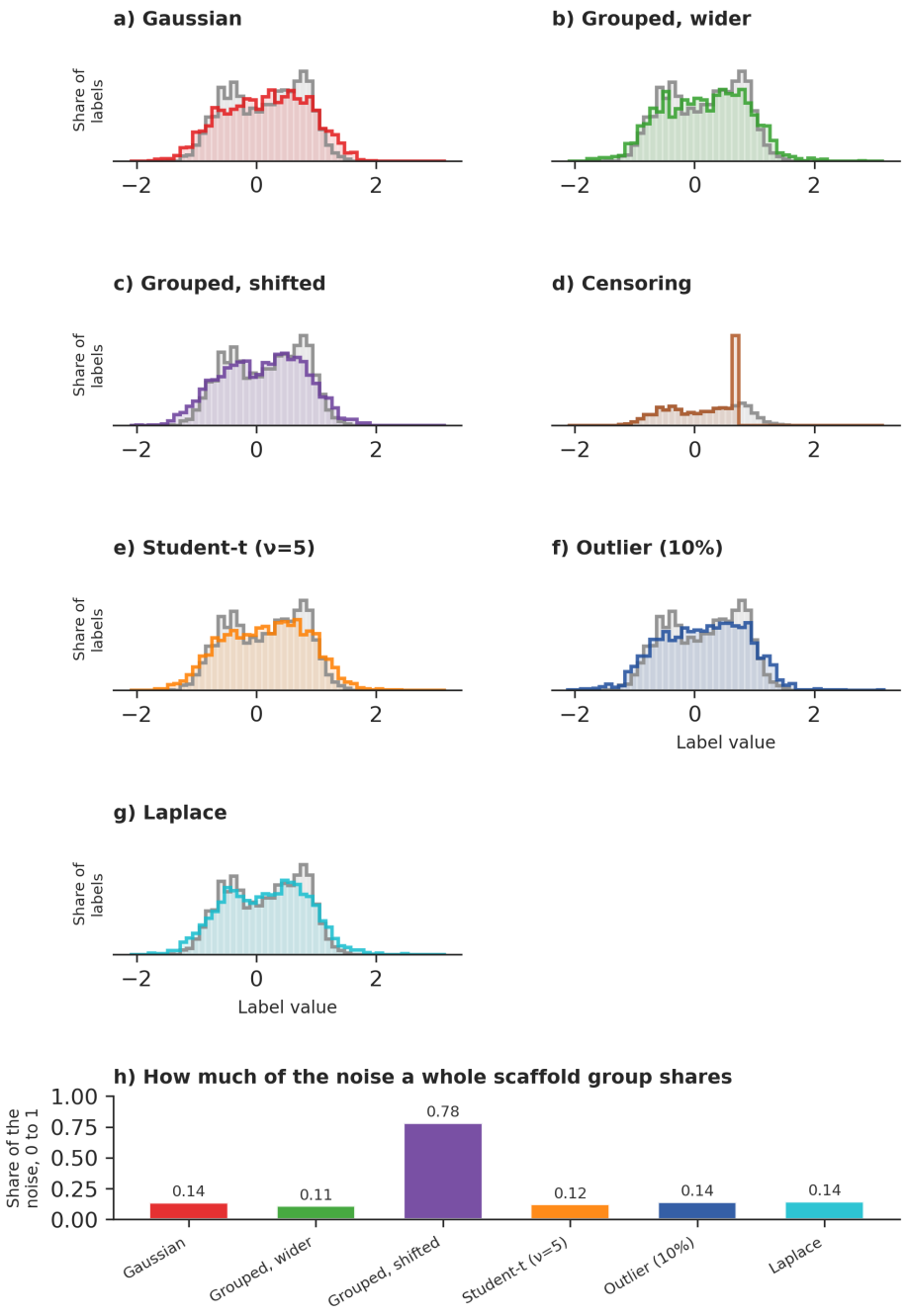


Figure 2.2: Label distributions under the seven noise conditions, for 2,000 synthetic labels in 40 random groups that stand in for scaffold families. a)–g) Clean (gray) and noised (color) labels at a noise level of 0.5 times the standard deviation of the clean labels (25% of labels clipped under censoring). h) Standard deviation of the mean error within a group divided by that of all errors, for every condition except censoring; a value near 1 means a group's labels move together, and noise that ignores the groups gives about 0.14 with about 50 labels per group.

We selected the parameter values from published measurement errors wherever possible, and Table 2.2 notes where no published estimate exists. We found no published estimate of how large a share of a dataset is affected by a difference between laboratories, so under grouped-wider we chose to draw scaffold families until they covered 20% of the training molecules.

Condition	**What It Changes**	**Value**	**Source for the Value**
Gaussian	Shape: every label's error is Gaussian	–	The standard assumption in noise-robustness studies
Student-t	Shape: heavier tails than Gaussian	$\nu = 5$	Differences between repeated measurements are non-normal [79]; no published estimate of ν
Laplace	Shape: sharper peak and heavier tails than Gaussian	–	Approximately Laplace for differences in pK_i between related targets, which resemble differences between repeated measurements [79]
Outlier	Error λ times wider on a random share of molecules (Huber's contamination model [98])	10% of molecules; $\lambda = 3$	1–10% of routine data contain gross errors, upper bound taken [99]; for one solubility method, inter-laboratory error is about three times intra-laboratory error [100]
Grouped-wider	Error λ times wider on whole scaffold families	Families covering 20% of the training molecules; $\lambda = 3$	No published estimate of the share; λ as for outlier [100]
Grouped-shifted	Every scaffold family gets its own offset; no error is widened	$\rho = 0.62$	Laboratory identity explains 62% of the variance in log digoxin efflux ratio across the laboratories using cell lines [82]
Censoring	Labels past a limit are set to the limit; the level is the fraction clipped	Set by the noise level	Censored labels in industrial assays [83]

Table 2.2: The seven noise conditions and their parameter values. ν is the number of degrees of freedom of the Student-t distribution, λ the widening factor (Equation 2.4) and ρ the share of the noise variance carried by the scaffold-family offset (Equation 2.5). For censoring, the noise level is the fraction of labels clipped.

2.5.2 Learning with noisy labels

In this thesis, methods for learning with noisy labels (LNL) fall into three groups: noise detection, noise mitigation and noise-robust training (Table 2.3). The groups follow from what selects the molecules and what is done to them. Song et al. [51] sort deep learning methods for noisy labels differently, into five groups by the part of training they change: robust architectures, robust regularization, robust loss functions, loss adjustment and sample selection. Figure 2.3 shows how a noise mitigation method is evaluated.

Table 2.3: Terms used throughout this thesis for methods that handle noisy labels. A noise mitigation method is named by both halves: what selects the molecules, and what is then done to them. The group follows from the two.

What selects the molecules	
Detector	A score inferred from the features and the labels, saying how wrong each label looks. The score is that molecule's suspicion score.
Known flag	Already recorded in the data, with nothing inferred: a label at the assay limit, how many times a molecule was measured, which source reported it, which scaffold family it belongs to.
Every molecule	No selection. The method acts on all the labels.
What is done to them	
Delete	The molecule leaves the training set.
Down-weight	The molecules and labels are kept and each molecule carries a weight.
Move the label	The label shifts toward a target by some distance. The target is a neighbors' mean, a predicted value, a group mean or an assay limit.
The groups	
Noise detection	A detector alone, with nothing done afterwards.
Noise mitigation	A detector, a known flag or every molecule, and then one of the three actions.
Noise-robust training	Nothing is selected and the data is untouched. The loss, the kernel, a noise term, the training loop or the model changes.

No single method is expected to win. Averaged over all problems, no learning algorithm outperforms another [101]. Neighbor-based cleaning relies on the similar property principle. It dates back to editing a training set by its nearest neighbors [102]. However, activity cliffs break that principle, and QSAR models frequently fail to predict cliffs [57]. Chapter 4 tests the principle, and Chapter 7 tests neighbor-based

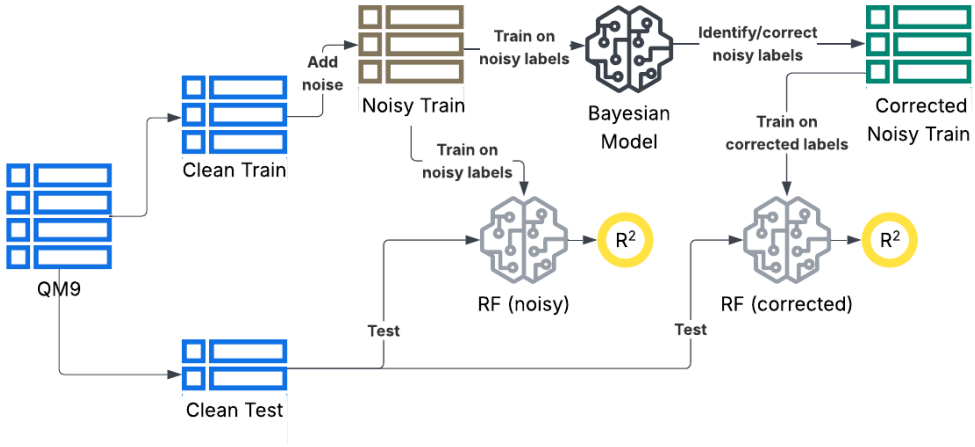


Figure 2.3: Evaluation of a noise mitigation method, with QM9 as the example. A random forest trained on noisy training labels is compared with one trained on labels corrected by a Bayesian model; the test labels stay clean, and both forests are scored on them by R^2 (higher is better).

noise mitigation with the adaptive hybrid representation as the feature space.

2.6 Implementation of the noise conditions

The noise conditions are implemented twice: NoiseInject, an open-source Python package, injects the noise on the three assay datasets of Chapter 3 and computes the robustness and uncertainty metrics. QM9 uses a separate Python and Rust pipeline with its own Rust implementation of the same conditions. An automated check confirms that the two deliver the same noise statistics on the same labels, groups and seeds.

2.6.1 The NoiseInject package

NoiseInject (version 0.3.0) evaluates model robustness and uncertainty calibration under artificial label noise, for both regression and classification. It operates on arrays of labels and on the predictions of any externally trained model, which makes it model-agnostic, and its core depends only on NumPy, pandas, SciPy and scikit-learn. Results are returned as `pandas` data frames with one row per noise level and one aggregate summary row. The software is available under an MIT license and archived on Zenodo [103].

Regression noise

The regression injector sets a noise shape (Gaussian, Student-t or Laplace) and a targeting rule (uniform, grouped-wider, grouped-shifted, outlier or censoring) separately. Each named condition is one combination of the two with its parameters, and the name is recorded with every result. The requested noise τ is the noise level times the standard deviation of the clean training labels. The injector solves for the scale τ/G in closed form (Equation 2.4), and a level of zero returns the labels unchanged. The affected molecules are selected by a separately seeded random number generator, so they are the same at every noise level while the noise draws change. Under grouped-wider, whole scaffold families are added in random order until they cover as close as possible to 20% of the training molecules.

Censoring is parameterized by the fraction f of labels clipped, with its limit at the $(1 - f)$ quantile of the clean training labels. On the training split the censored set is the $k = \text{round}(fn)$ largest labels; on any other split it is whichever molecules lie past the training limit.

Each injection also returns a record of the target and delivered root-mean-square noise, the factor G, the solved scale, the mean of ϵ, the fraction of molecules affected, the group sizes and the random seeds. The delivered noise is checked against a tolerance band, and a draw outside it raises a warning (Additional file 1).

A second function returns the noise scale each molecule receives, without drawing noise; we apply it to held-out molecules. Under uniform and grouped-shifted targeting every molecule gets the same scale, so the package flags these scales as degenerate instead of reporting a rank correlation of zero. A separate injector sizes the noise against a unit supplied by the user; in Chapter 7 this unit is the standard deviation of a model's out-of-fold residuals on the clean training labels. It either adds Gaussian noise or shifts 2%, 5% or 10% of labels. The Gaussian noise has variance the level times the square of the unit. Here the level scales the variance, not the standard deviation as in Equation 2.4. Each shift is the level times the unit, with a random sign or upward only. It can also add Gaussian noise whose standard deviation follows a per-molecule covariate or a hidden log-normal draw, scaled to the same mean variance.

Classification noise

The package provides six label-flipping strategies: uniform flips, flip rates set by class frequency, asymmetric flip rates for binary tasks, a random flip rate for each sample, a flip rate for each class, and flips directed by a confusion matrix. Because the realized flip

rate usually differs from the nominal probability, a binary search finds the probability that changes a target fraction of labels.

Robustness metrics

For regression, the package computes R^2, the root mean squared error (RMSE) and the mean absolute error at every noise level. For classification, it computes accuracy, together with macro- and weighted-averaged precision, recall and F1 score and a per-class breakdown. Robustness is read from the retention curve $\mathrm{ret}(x) = m(x)/m(0)$, where x is the noise level. Here m is a metric bounded above by 1 for which higher values are better: R^2 for regression, and accuracy, F1, precision or recall for classification. The primary summary is the normalized area under this curve, $\mathrm{AUC}_{\mathrm{norm}}$ (Equation 3.2 in Chapter 3), evaluated by the trapezoid rule. It is the mean fraction of clean performance retained across the noise levels and lies approximately within $[0, 1]$. It is also less directly correlated with clean-label R^2 than other robustness metrics we tested (Chapter 3). A secondary summary fits $\mathrm{ret}(x) \approx \exp\bigl(-(x/\tau_W)^\beta\bigr)$ to the curve. The shape parameter β separates curves that hold and then collapse ($\beta > 1$) from curves that fall early and then plateau ($\beta < 1$). The parameter τ_W sets the scale of the decay, and the fit requires at least four noise levels. The package also reports the clean baseline and the retention at the highest noise level, flags any curve that falls below zero retention, and can withhold the curve summaries for models whose clean baseline is too low.

Uncertainty metrics

For a model that returns a predicted standard deviation u_i for each sample, the package computes, at each noise level, the Spearman correlation between u_i and the absolute error $|y_i - \hat{y}_i|$, the empirical coverage $\frac{1}{n}\sum_i \mathbb{1}[|y_i - \hat{y}_i| \le k u_i]$ at $k = 1$ and $k = 2$, against Gaussian targets of 68.3% and 95.4%, the mean interval width $\frac{1}{n}\sum_i 2u_i$, and the expected calibration error (ECE),

$$\mathrm{ECE} = \sum_{b=1}^{10} \frac{|B_b|}{n} \left| \bar{u}_b - \overline{|y - \hat{y}|}_b \right|, \tag{2.6}$$

computed over ten equal-count bins B_b of u; all of these use the clean test labels. Under Gaussian errors a calibrated model has a mean absolute error of $\sqrt{2/\pi}\, u \approx 0.80\, u$, so its ECE is about $0.2\, \bar{u}$, not zero. The Spearman correlation between u_i and the injected noise $|\epsilon_i|$ can only be computed on the corrupted molecules, where the noise is known,

and is reported within each noise level. A correlation pooled across levels is also returned, but it mostly reflects that uncertainty and noise both rise with the level, so it does not measure whether a model can identify which molecules are corrupted.

Three optional wrappers produce u_i from a point predictor: split-conformal prediction around any scikit-learn-compatible model, optionally scaled by the label spread of each molecule's nearest training neighbors; an exact Gaussian process with a Tanimoto or RBF kernel through `Gauche` [68]; and Monte Carlo dropout on any `PyTorch` network [104].

2.7 Evaluation methodology

Chapter 3 reports predictive performance as R^2 on held-out molecules and robustness as $\mathrm{AUC}_{\mathrm{norm}}$ (Equation 3.2). Chapters 4 to 7 state the metric used for each dataset where it is introduced, and report results as per-fold scores and their spread across folds, with no significance tests.

No split or fold in this thesis is random, and no group of molecules is split between training and test. Random splitting inflates performance estimates for molecular data, because it places structurally similar molecules on both sides of the train/test boundary [105–107]. For small molecules the group is the Bemis–Murcko scaffold family (Section 2.5.1). Section 3.2 gives the splits of Chapter 3, and Section 5.2.6 gives the folds of the later chapters and the groups used for datasets that are not small molecules.

2.8 Conclusions

A predictor has two parts, the representation and the model. The representation also shapes what counts as similar, since similarity depends on both the representation and the coefficient. Activity cliffs and uninformative features, in turn, break the similar property principle. Real assay noise is often neither independent nor identically distributed, and it need not be zero-mean. For this reason we defined seven artificial noise conditions. Methods for noisy labels fall into noise detection, noise mitigation and noise-robust training. Whichever group a method belongs to, none is expected to win on every dataset. We implemented the seven conditions in NoiseInject, an open-source package that injects them and reports the robustness and uncertainty metrics for any model. Chapter 3 asks whether the representation or the model architecture governs how strongly a predictor degrades under label noise, through the questions set out in Chapter 1.

Chapter 3

Noise robustness across molecular representations and model architectures

The work in this chapter is adapted from Punt, A., Hanser, T., Werner, S., and Morris, G.M., NoiseInject: A Benchmarking Framework to Study Noise Robustness in QSAR Models, to be submitted to the Journal of Cheminformatics. Most of the figures, tables, and text in this chapter are therefore either identical or similar to the content of that manuscript. The authors' contributions are given in the Statement of Contribution.

3.1 Introduction

In this chapter, we inject the noise conditions of Chapter 2 into training labels and measure how much predictive performance each pairing of representation and model keeps.

While there have been many investigations into the effect of noise on molecular property prediction, direct comparisons between how different molecular representations respond to label noise remain limited. The choice of molecular representation certainly affects predictive performance. Deng et al. [108] evaluated 62,820 models and found that fixed representations, in most datasets RDKit 2D descriptors, outperformed representations learned from SMILES and molecular graphs. Whether these conclusions translate to noise robustness is our question. Between different types of label noise, Heid et al. [80] found no difference in model performance between error distributions with the same mean and standard deviation. We expand on this by considering alternative patterns of noise, namely noise concentrated in structural groups or on a

sparse subset of samples.

This chapter addresses the questions set out in Chapter 1, specifically how the choice of representation and model architecture explain the variance in predictive performance and in noise robustness, what is the impact of the type of label noise, and whether uncertainty estimates track label noise.

3.2 Methods

We looked at six representations paired with thirteen models, and several variant models, across four prediction tasks. Every pairing was run under Gaussian, grouped-wider and grouped-shifted noise at seven noise levels, and the other four noise conditions (Section 2.5.1) were run on selected pairings. The hyperparameters of every model are given in Additional file 2.

3.2.1 Datasets

Although we are investigating the effects of label noise stemming from experimental settings, for the purposes of testing we inject controlled amounts of artificial noise into clean labels. Data were extracted from the QM9 molecular property dataset distributed by PyTorch Geometric [109], which contains 130,831 small organic molecules with pre-computed quantum-mechanical properties obtained from density functional theory (DFT) calculations at the B3LYP/6-31G(2df,p) level of theory. We selected the gap between the highest occupied and the lowest unoccupied molecular orbital (HOMO–LUMO gap) as the prediction target in QM9. We assumed that QM9 labels were free of noise, notwithstanding approximations in the level of theory. Each experiment was replicated 10 times with a distinct random seed and a randomly selected subset of $N = 10{,}000$ molecules, split 80/10/10 into 8,000 training, 1,000 validation and 1,000 test molecules. All models were fitted on those 8,000 training molecules, except the Gaussian processes, which were fitted on a random subset of 5,000 of them; the validation and test molecules were unchanged.

We repeated these experiments on three molecular property datasets with experimentally determined endpoints. From the OpenADMET initiative [110], we used LogD and Caco-2 efflux permeability, with $N = 5{,}039$ and $N = 2{,}161$ respectively. We also selected hERG K_i data from ChEMBL [111], following a protocol inspired by Landrum and Riniker [16]: filter for binding assays, deduplicate by median pChEMBL value, and remove compounds with an inter-assay standard deviation greater than 1.0 log unit,

resulting in $N = 1{,}415$ compounds. Each of the three was partitioned into five folds by grouped k-fold cross-validation (CV), and each fold's training block gave up a further grouped fifth as a validation set for early stopping. A model therefore fits between 3,194 and 3,364 of the 5,039 LogD molecules, depending on the fold, so the Gaussian-process subsample never applies to these datasets. Unlike QM9's 10 replicates, which are 10 independent draws from one pool, each assay dataset is a single fixed set in which every molecule is tested exactly once and nothing is redrawn.

Log D is the logarithm of the distribution coefficient of a molecule between octanol and water, usually measured at pH 7.4. It counts every charge state, whereas log P counts only the neutral form, and a basic amine that is protonated at that pH therefore has a log D below its log P. Caco-2 cells come from a human colorectal carcinoma and grow as a layer that models absorption in the gut. Permeability across the layer is measured in both directions, and the efflux ratio is the permeability from the basolateral to the apical side divided by that in the other direction. A ratio above about 2 suggests the compound is pumped out by a transporter such as P-glycoprotein. The hERG gene encodes a cardiac potassium channel. Drugs that block it can prolong the QT interval and cause arrhythmia, so hERG binding is screened early in drug discovery. The hERG K_i dataset holds binding affinities for this channel, as pK_i values.

All splits and CV folds were grouped by the chirality-blind Bemis–Murcko scaffold, implemented such that each distinct acyclic molecule forms its own group, and groups are filled in random order. Prior to modeling, all molecular structures were sanitized and canonicalized using `RDKit` to remove invalid valence states and standardize atom and bond typing [32]. Prediction targets were standardized to zero mean and unit variance. Experiments that track uncertainty were run once rather than in replicates.

3.2.2 Representations

In this study, we evaluated six molecular representations: extended-connectivity fingerprints of diameter four (ECFP4), Avalon, Sort & Slice (SNS), physicochemical descriptors (PDV), ChemBERTa and Molecular Hypergraph Grammar GNNs (MHG-GNNs). The same six were used on QM9 and on the three assay datasets. We used the standard ECFP with radius $r = 2$ with $d = 2048$ bits [5, 32]. Sort & Slice takes the same substructures as ECFP4, but it ranks them by prevalence and selects the top L ($L = 1024$) [7]. We fitted the Sort & Slice vocabulary on the training molecules alone, refitting it for each QM9 replicate and for each CV fold. Avalon is a 2,048-bit fingerprint, generated with the Avalon toolkit through RDKit [8, 32].

The PDV representation is a vector of 200 physicochemical descriptors computed with RDKit's `MolecularDescriptorCalculator` [31, 32]; the 200 are listed in Additional file 2, and the list is fixed in our code rather than taken from RDKit's default descriptor set, which grows between releases. ChemBERTa embeddings are the mean over the non-padding token embeddings of the `DeepChem/ChemBERTa-77M-MTR` checkpoint, producing 384 features [28]. MHG-GNNs are a GIN-based autoencoder pre-trained on about 1.38 million PubChem molecules using β-VAE loss, producing 1024-dimensional embeddings through iterative message passing and found in the published `mhggnn_pretrained_model_0724_2023` checkpoint [9]. We z-score normalized PDV, ChemBERTa and MHG-GNN per feature, using the training molecules' mean and standard deviation; ECFP4, Avalon and Sort & Slice reach the model as built.

3.2.3 Models

To understand model behavior in the presence of label noise, we explored a range of model families. We fitted tree ensembles, a support vector machine, Gaussian processes and neural networks. Several are variants of one another, differing in the probabilistic machinery they add, and Additional file 2 gives every configuration with its settings.

Random Forests (RFs) are one of the most common choices for QSAR modeling thanks to their robustness and interpretability [112]. Quantile Regression Forests (QRFs) hold the distribution of training labels in each leaf and compute quantiles over all trees [113]. While we used 100 trees for RF, we observed a noisy quantile estimate with 100 trees for QRF, prompting us to use 300 instead. To observe the effect of the quantile mechanism, we then fit a second RF at 300 trees so the two models were identical in all other settings.

We also used eXtreme Gradient Boosting (XGBoost) [59], Light Gradient-Boosting Machine (LightGBM) [62], and Natural Gradient Boosting (NGBoost) [63]. NGBoost extends deterministic gradient boosting by treating the parameters of a chosen parametric distribution as regression targets and learns them through boosting with a natural gradient update rule [63]. NGBoost relies on scoring off validation data, which differs by dataset. On QM9 the validation split is already held out of every fit. However, on the three assay datasets NGBoost holds out its own scaffold-grouped fifth of the rows it is handed, which already exclude the validation molecules.

Support vector machines (SVMs) are a well-established baseline in QSAR modeling [17, 64]. We used a radial basis function (RBF) with $C = 1.0$ and $\gamma = 1/(d\,\mathrm{Var}(X))$, where d is the number of features and $\mathrm{Var}(X)$ the variance of all training feature values,

as the kernel, as it accommodates continuous vectors.

Gaussian processes (GPs) are a non-parametric class of models that use a kernel to produce a Gaussian predictive distribution over every data point [114, 115]. We fitted three variants of GPs, each with a constant mean and a scaled kernel. The first uses an RBF kernel and runs on all six representations. The second uses the Tanimoto kernel [69, 70] and was run on ECFP4 alone; it is not reported in this chapter. The third is a heteroscedastic variant in which the RBF process is given a second network that predicts the observation noise of each molecule. One of the major limitations of using GPs in practice is computational scaling; inference has $O(N^3)$ time complexity and $O(N^2)$ memory, where N is the number of training points [115]. This can be mitigated with sparse GP approximations, but these introduce approximation error and can be more difficult to tune [116]. As such, we used a standard GP and only fit on at most 5,000 training molecules, drawn by a seeded sample. Due to the size of the datasets, this cap only affected QM9. We implemented these GP models in `GPyTorch` [117], with the Tanimoto kernel taken from the `Gauche` framework [68].

We tested two deterministic feed-forward architectures, both fully connected and both implemented in PyTorch [118]: (i) NN-α with two hidden layers of sizes 128 and 64, and dropout after each hidden layer and (ii) NN-β with two hidden layers, each of size 128, with dropout applied before the output layer. Each model was trained using the Adam optimizer with early stopping on validation loss. We used Bayesian transformations to convert NNs into Bayesian neural networks (BNNs), and implemented them using `PyTorch` [118] with ReLU activations and dropout regularization ($p = 0.2$). Additional file 2 gives their full widths, dropout fractions, learning rates and batch size.

To get these networks producing uncertainty estimates, we applied four probabilistic transformations to each architecture, producing eight probabilistic networks. The first replaces every linear layer with a Bayesian layer carrying a Gaussian prior $\mathcal{N}(0, 0.1^2)$ on its weights, producing BNN-α and BNN-β. They are trained on an evidence lower bound whose Kullback–Leibler (KL) divergence term is divided by the number of training molecules. While this is a straightforward approach, it is computationally expensive. We explored alternative techniques like our second variant, where we add an output head that predicts a variance to the BNN and fit the negative log likelihood of a Gaussian with that variance [119]. The third, Variational Bayesian Last Layer (VBLL), replaces the output layer with a Bayesian last layer that produces mean-field variational posteriors $q(\mathbf{W}) = \mathcal{N}(\boldsymbol{\mu}_W, \mathrm{diag}(\boldsymbol{\sigma}_W^2))$ over the last-layer weights and trains by maximizing the evidence lower bound. This can be thought of as minimizing

the reconstruction loss plus KL divergence $D_{\mathrm{KL}}(q(\mathbf{W})\|p(\mathbf{W}))$ from a standard normal prior, scaled by $1/N$ [120]. Because only its output layer keeps a learned observation-noise parameter, VBLL networks report only epistemic uncertainty per molecule, with one aleatoric variance for the whole fit. The fourth makes observation noise a function of the input. All eight probabilistic networks estimate their predictive distributions from 100 stochastic forward passes. Each network and its Bayesian counterpart were tuned to the same set of parameters so that NN-α against BNN-α and NN-β against BNN-β differ by the Bayesian layers alone.

The thirteen base models are RF, QRF, XGBoost, LightGBM, NGBoost, SVM, the RBF-kernel GP, and NN, BNN and VBLL in each of the two architectures. The six variant models are RF (300 trees); GP (het.), the heteroscedastic GP; BNN-α (var. head) and BNN-β (var. head), the two Bayesian networks with a variance head; and VBLL-α (het.) and VBLL-β (het.), the two heteroscedastic VBLL networks.

3.2.4 Noise conditions and levels

The seven noise conditions are defined in Section 2.5.1.

We controlled noise quantity using noise levels, representing a fraction of the standard deviation of clean training labels. We ran experiments across seven levels, 0, 0.2, 0.3, 0.5, 0.75, 1.0 and 1.5. We added the noise to the raw label and standardized the target afterwards, using the clean training mean and spread. To control the quantity of noise for censoring, we recorded every label above the kth largest clean training label as that limit and used this level as the fraction clipped, at 0, 0.1, 0.2, 0.25, 0.3, 0.4 and 0.5. Censoring is common in pharmaceutical assays, where a third or more of labels can be censored [83].

We ran Gaussian, grouped-wider and grouped-shifted noise on every model and representation pairing. The remaining four conditions ran on selected pairs.

On QM9, Laplace, Student-t and outlier noise were run on eight models, namely RF, NGBoost, SVM, the GP, GP (het.), BNN-α (var. head), VBLL-α (het.) and VBLL-β (het.), each on ECFP4, PDV and ChemBERTa. On the three assay datasets they were run on the same models and representations except VBLL-α (het.). Censoring was run on five pairings, namely RF and GP (het.) on ECFP4, and NGBoost, the GP and BNN-α (var. head) on PDV. On the three assay datasets, $\mathrm{AUC}_{\mathrm{norm}}$ under censoring was computed for the same five pairings, and the uncertainty statistics under censoring for QRF and the other models that give an uncertainty estimate, on ECFP4, PDV and ChemBERTa.

3.2.5 QM9 data pipeline

QM9 [109], loaded through PyTorch Geometric, was processed by a separate pipeline where the Python code loads and splits the data, and computes several representations. The data is then passed to the Rust portion of the pipeline which injects the label noise, standardizes the labels, computes the remaining representations, and sends the processed data back to Python which trains and scores the models.

Each replicate, using its own random seed, randomly selects a subset from QM9, which is split by a new scaffold split (Section 3.2.1), and the same grouping defines the scaffold families targeted by the grouped noise conditions.

Python and the Rust program exchange the data through memory-mapped binary files on disk, one record per molecule (Additional file 11).

The Rust program draws the noise for each split separately, in the order its records are stored. The amount of noise, the censoring limit and the affected scaffold families are all computed from the clean training labels. Validation labels get all three, with the amount in label units, from an independent draw seeded from the training seed. The program checks that validation received as much noise as training. Test labels are never corrupted, but every split records the noise scale each molecule would have received, so that held-out molecules can be scored against the pattern applied in training.

For every molecule, the program records its canonical SMILES, clean and noisy labels, standardized value, injected noise, noise scale and noise pattern. Each run also writes a summary of the noise parameters and the amount of noise delivered. The run halts if a record holds a different molecule when it is written than when its noise was drawn, or if any molecule has an all-zero ECFP4 or Avalon fingerprint, which marks a failed featurization. A self-test mode runs the same noise checks on a label file alone.

3.2.6 Uncertainty estimation

Up to this point we have been concerned with how noise affects the predictive performance of QSAR models. However, label noise should also affect the uncertainty these models report. Some models return only a point prediction, while others also return an estimate of uncertainty, either as a predicted distribution or as the spread across an ensemble [121]. Furthermore, some report a single value of uncertainty while others report one that can be decomposed into an aleatoric and an epistemic component [119]. The aleatoric component describes underlying noise the model associates with the data, while the epistemic component represents the uncertainty within the model's fit. We sought to determine if, as expected, the aleatoric component rises with injected

label noise while the epistemic component does not.

For the BNN and VBLL transformations, we took the two components over $T = 100$ stochastic forward passes [119]:

$$\hat{u}^2_{\text{ale}}(x) = \frac{1}{T}\sum_{t=1}^{T}\hat{v}_t(x), \qquad \hat{u}^2_{\text{epi}}(x) = \frac{1}{T}\sum_{t=1}^{T}\big(\hat{\mu}_t(x) - \bar{\mu}(x)\big)^2, \tag{3.1}$$

where $\hat{\mu}_t(x)$ and $\hat{v}_t(x)$ are the mean and the variance returned for molecule x on pass t, and $\bar{\mu}(x)$ is the mean over passes. Here $\hat{v}_t(x)$ is the predicted observation noise variance. It varies per molecule in the variance-head and heteroscedastic variants, and is one number for the whole fit in the VBLL transformation. A full-BNN without a variance head returns no variance, so its uncertainty is epistemic alone.

The other models follow the same split. For GPs, the epistemic component is the posterior variance of the latent function and the aleatoric component is the learned observation noise [115]. All variants of the GP produce one number for the whole fit except heteroscedastic. For QRF models, each tree holds multiple training molecules. So for a given molecule, the aleatoric component is the variance of the labels in the molecule's given leaf. The total aleatoric uncertainty is those variances averaged over the trees. The epistemic component is the variance of the leaf means across the trees. NGBoost predicts one variance for each molecule from a single fit, which is aleatoric alone [63]. In every model, the total uncertainty is the sum of the aleatoric and epistemic components.

No held-out label carries injected noise, so every comparison against the injected noise was made on training molecules. Each training set was divided into five folds grouped on Bemis–Murcko scaffolds, and each molecule was scored by a refit that excluded its own fold. For NGBoost on QM9, three of the five folds were scored to limit its cost. No uncertainty was calibrated after fitting.

3.2.7 Evaluation

We compute statistics on the held-out molecules, for QM9 the test split and for three assay datasets the held-out fold of the scaffold CV. Different combinations of dataset, model, representation, noise condition, and noise level are examined in each scenario. We used the coefficient of determination (R^2) throughout. For molecule i, let y_i be the clean label, $\hat{y}_i$ the prediction, ϵ_i the noise injected into its label and u_i the predicted uncertainty.

To evaluate the effect of label noise, we tracked predictive performance retention

under increasing artificial noise. For each QSAR pairing we recorded $R^2(\tau)$ at each noise level τ, and normalized it to obtain the retention ratio $R^2(\tau)/R^2(0)$. To quantify robustness, we used the normalized area under this retention curve,

$$\mathrm{AUC}_{\mathrm{norm}} = \frac{1}{\tau_{\max}} \int_0^{\tau_{\max}} \frac{R^2(\tau)}{R^2(0)}\, d\tau, \tag{3.2}$$

evaluated using the trapezoidal rule over the seven noise levels, $\tau \in \{0, 0.2, 0.3, 0.5, 0.75, 1.0, 1.5\}$, each a fraction of the standard deviation of the clean training labels. While we tested other metrics of robustness, we found $\mathrm{AUC}_{\mathrm{norm}}$ to be less directly correlated to clean-label R^2 predictions. For censoring, we sweep over a set of fractions to clip the labels prior to normalization.

An $\mathrm{AUC}_{\mathrm{norm}}$ value near 1 indicates that the QSAR model in question has retained predictive performance with artificial noise, meaning it is more robust to noise. Since we normalize the curve by its own clean-label predictive performance, a model with a low clean R^2 has less to lose, so $\mathrm{AUC}_{\mathrm{norm}}$ is reported beside clean R^2. We excluded replicates with a clean $R^2 < 0.3$ from the robustness analysis (Additional file 3), as performance retention ratios become unstable when the clean-label denominator approaches zero. On QM9 one $\mathrm{AUC}_{\mathrm{norm}}$ is computed per replicate, and we report the median over the 10 replicates. On the three assay datasets one value is computed per scaffold fold, and the median over the five folds is reported.

To compare the relative effect of the 13 base models across datasets and noise conditions, we compared their rankings using Spearman's rank correlation coefficient (ρ) between clean R^2 and $\mathrm{AUC}_{\mathrm{norm}}$. We compared probabilistic models with their deterministic counterparts using a two-sided Wilcoxon signed-rank test paired on the replicate, at $\alpha = 0.05$, separately for each representation and noise condition. This test was run only on QM9, as with five scaffold folds the smallest attainable p is 0.0625.

We conducted a separate analysis of variance (ANOVA) for each noise condition to identify the relative contributions of molecular representation and model architecture to both predictive performance and noise robustness. For a given metric y, either R^2 at a noise level of 1.0 or $\mathrm{AUC}_{\mathrm{norm}}$:

$$y_{fmjr} = \mu + s_r + \alpha_f + \gamma_{m(f)} + \beta_j + (\alpha\beta)_{fj} + (\gamma\beta)_{m(f)j} + e_{fmjr}, \tag{3.3}$$

where s_r represents the effect of replicate r, α_f represents the effect of model family f, $\gamma_{m(f)}$ represents the effect of model architecture m within its family, β_j represents the effect of molecular representation j, $(\alpha\beta)_{fj}$ and $(\gamma\beta)_{m(f)j}$ are the interaction terms, and

e_{fmjr} is the residual. The proportion of variance explained by each term was calculated as the effect size

$$\eta^2_{\text{term}} = \frac{SS_{\text{term}}}{SS_{\text{total}}}, \tag{3.4}$$

using Type I (sequential) sums of squares, entered in the order replicate, model family, model architecture within family, molecular representation, interaction. The 10 QM9 replicates provide the residual. The uncertainty on each share is the 95% interval of η^2 over 1,000 bootstrap resamples of whole replicates. These shares describe the models and representations tested here, not the choice of model or representation in general.

ANOVA relies on the assumption that observations both within and across groups are independent. Two features of this study break that assumption. First, each QM9 replicate uses the same data split and the same noise draw for every model and representation, so observations from one replicate are not independent of each other; we therefore entered the replicate as a block. Second, across the whole set of models in this study, some pairs produced similar predictions that are not independent, so a method present in several near-identical versions would count several times towards the share of model architecture. To handle this issue, we grouped the models into families. We considered grouping pairs of models that share a method or an architecture, and compared each pair on clean R^2 and on AUC_{norm} under the three noise conditions run on every representation, separately on each of the six representations, which gives 24 comparisons per pair. A pair was grouped when the mean difference between the two models was smaller than their standard deviation across replicates in at least two thirds of these comparisons (Additional file 4). Five families met this criterion: RF with quantile regression forests (QRF; 16 of 24), XGBoost with LightGBM (23 of 24), NN-α with BNN-α (20 of 24), NN-β with BNN-β (18 of 24), and VBLL-α with VBLL-β (20 of 24). VBLL did not group with the plain or Bayesian network of its own architecture (4 to 10 of 24), as its clean R^2 was lower by 0.02 to 0.09, and NGBoost did not group with XGBoost or LightGBM (2 and 3 of 24). The two architectures of the plain networks, and of the Bayesian networks, also met the criterion (18 and 17 of 24); we grouped by architecture instead, which the evidence supports equally well. The 13 base models therefore form eight families. Within these families, the differences between models accounted for at most 1.0% of the variance in any outcome and noise condition on QM9, and up to 15.1% of predictive performance on clean labels on the three assay datasets. We also ran Tukey's honestly significant difference test between the members of each family, separately for each representation (Additional file 5). With 10 replicates it detects differences smaller than the variation

between replicates, so we report it beside the size of each difference rather than use it to form the families.

Across representations, we considered grouping SNS with the ECFP4 fingerprint, as both encode overlapping circular substructure information. The two differed by more than their variation across replicates on clean R^2 for every model (0 of 13), so all six representations were kept as separate levels (Additional file 6).

To assess the interaction between uncertainty estimation and label noise, we posed three questions. First, whether uncertainty rises with label noise, as measured by the mean u_i over held-out molecules at the highest noise level divided by its value with no added noise, in each replicate or fold. The second pertains to models that report separate aleatoric and epistemic components. We divided the increase of mean epistemic uncertainty between the highest level of noise and clean labels by the mean of the aleatoric component to determine how each responds to label noise. A ratio near 0 means the added noise was attributed to the labels while a ratio of 1 or above means the epistemic component grew as much or more. We also compared the mean aleatoric component with the standard deviation of the injected noise. The third question we had was whether per-sample uncertainty values track injected label noise, as measured by the Spearman correlation between u_i and $|\epsilon_i|$ over out-of-fold training molecules, per fold and per noise level. We also report the Spearman correlation between u_i and the absolute error of the prediction. Finally, we asked whether dividing a molecule's out-of-fold absolute error by u_i ranks the corrupted labels better than the error alone, and report the difference between the two Spearman correlations with $|\epsilon_i|$.

3.3 Results

3.3.1 Variance decomposition

Considering the two main components of a QSAR model, the molecular representation and the model architecture, we sought to understand how each contributes to both predictive performance and noise robustness. ANOVA decomposition divides the variance between model family, model architecture within its family, molecular representation, the pairing of the two, the replicate, and a residual. We ran this decomposition for Gaussian noise, the standard model, as well as methods whose injected noise is related to the feature space, grouped-wider and grouped-shifted (Figure 3.1, Table 3.1). For predictive performance measured in R^2 at a noise level of 1.0, model family and molecular representation account for comparable shares of the variance,

26.6% and 29.0% under Gaussian noise. However, for noise robustness, quantified by the normalized area under the R^2 retention curve (AUC_{norm}; higher values indicate greater robustness), model family accounts for 47.2% of the variance while molecular representation for 8.1%. This pattern is observed across noise conditions and makes sense, as label noise does not impact the features. The interaction between model and molecular representation explains 19.9% of the variance in AUC_{norm}, more than twice the 8.1% explained by representation alone, meaning representation acts on noise robustness mainly through the model it is paired with. As such, we primarily compare representations within each model.

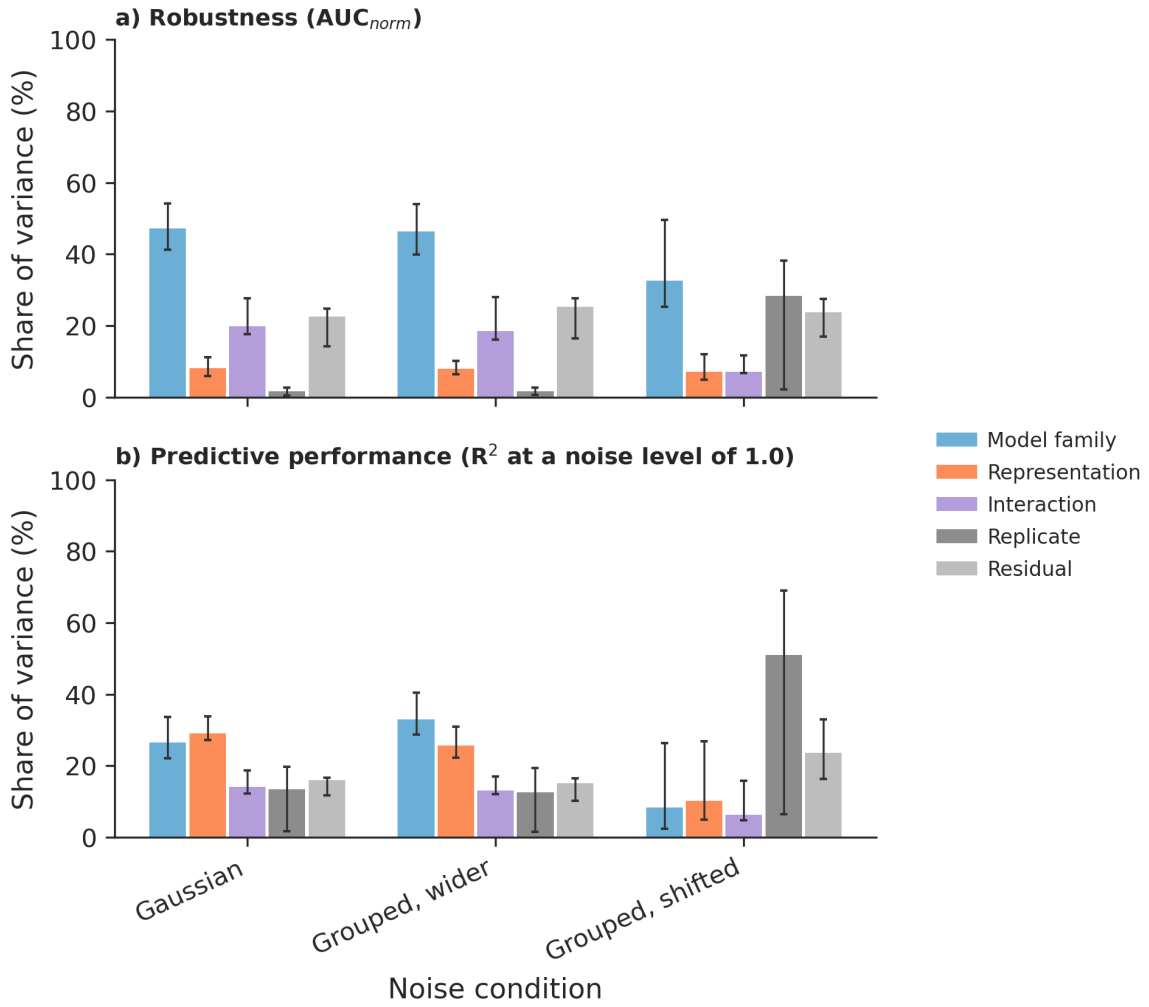


Figure 3.1: Share of the variance explained by each term, on the QM9 HOMO–LUMO gap: a) robustness (AUC_{norm}); b) predictive performance (R^2 at a noise level of 1.0). Whiskers are 95% bootstrap intervals over replicates. Model within family, at most 1.0% in every decomposition on QM9, is not drawn.

We compare the variance decomposition on both clean and noisy labels (Figure 3.2a). For predictive performance, the artificial label noise lowers the impact of the

choice of model family, while the replicate and residual take more. This means that the variance is left more up to chance and to the underlying variation within the data itself with label noise, which is the expected behavior. Robustness divides the variance a bit differently. Representation explains 8.1% of the variance in $\mathrm{AUC}_{\mathrm{norm}}$, against 31.1% of clean R^2, while model family rises to 47.2%. The replicate nearly disappears from $\mathrm{AUC}_{\mathrm{norm}}$, likely because each replicate's curve is divided by that replicate's own clean R^2, so a difficult split lowers both and cancels out.

We include two terms which stem from the data rather than the model, the residual and replicate terms. The replicate measures how much of the variance can be attributed to the train and test split, which in this case is a scaffold split. On QM9 we replicated each experiment 10 times with different random draws of $N = 10{,}000$, each time giving a new scaffold split. On the assay datasets, a replicate refers to one of the five scaffold CV folds. A large replicate term in ANOVA indicates that the folds differ from one another. The residual is variance that no other term accounts for.

While we observe similar patterns between noise conditions, the predictive performance of grouped-shifted noise in particular varies significantly between replicates, resulting in a larger share for the replicate. We find that 28.4% of the variance in $\mathrm{AUC}_{\mathrm{norm}}$ and 51.0% of the variance in R^2 are due to the replicate, with model family falling to 32.7% and 8.4%. This indicates that the offset applied to each scaffold may account for more of the variance in both R^2 and $\mathrm{AUC}_{\mathrm{norm}}$.

Condition	Outcome	Replicate	Model family	Representation	Interaction	Residual
None	Clean R^2	10.6	35.4	31.1	12.1	10.2
Gaussian	$\mathrm{AUC}_{\mathrm{norm}}$	1.7	47.2	8.1	19.9	22.6
Grouped, shifted	$\mathrm{AUC}_{\mathrm{norm}}$	28.4	32.7	7.0	7.2	23.7
Grouped, wider	$\mathrm{AUC}_{\mathrm{norm}}$	1.7	46.3	7.9	18.5	25.3
Gaussian	R^2 at level 1.0	13.4	26.6	29.0	14.2	16.0
Grouped, shifted	R^2 at level 1.0	51.0	8.4	10.3	6.2	23.7
Grouped, wider	R^2 at level 1.0	12.7	33.0	25.7	13.1	15.1

Table 3.1: Share of the variance (%) explained by each term on the QM9 HOMO–LUMO gap, for 13 base models and six representations over 10 replicates. Model within family, at most 1.0% in every row, is left out, so a row does not sum to 100%. Intervals are in Additional file 7.

3.3.2 Predictive performance and noise robustness

The variance decomposition tells us what aspect of QSAR modeling affects predictive performance and robustness, however it does not tell us about the characteristics of

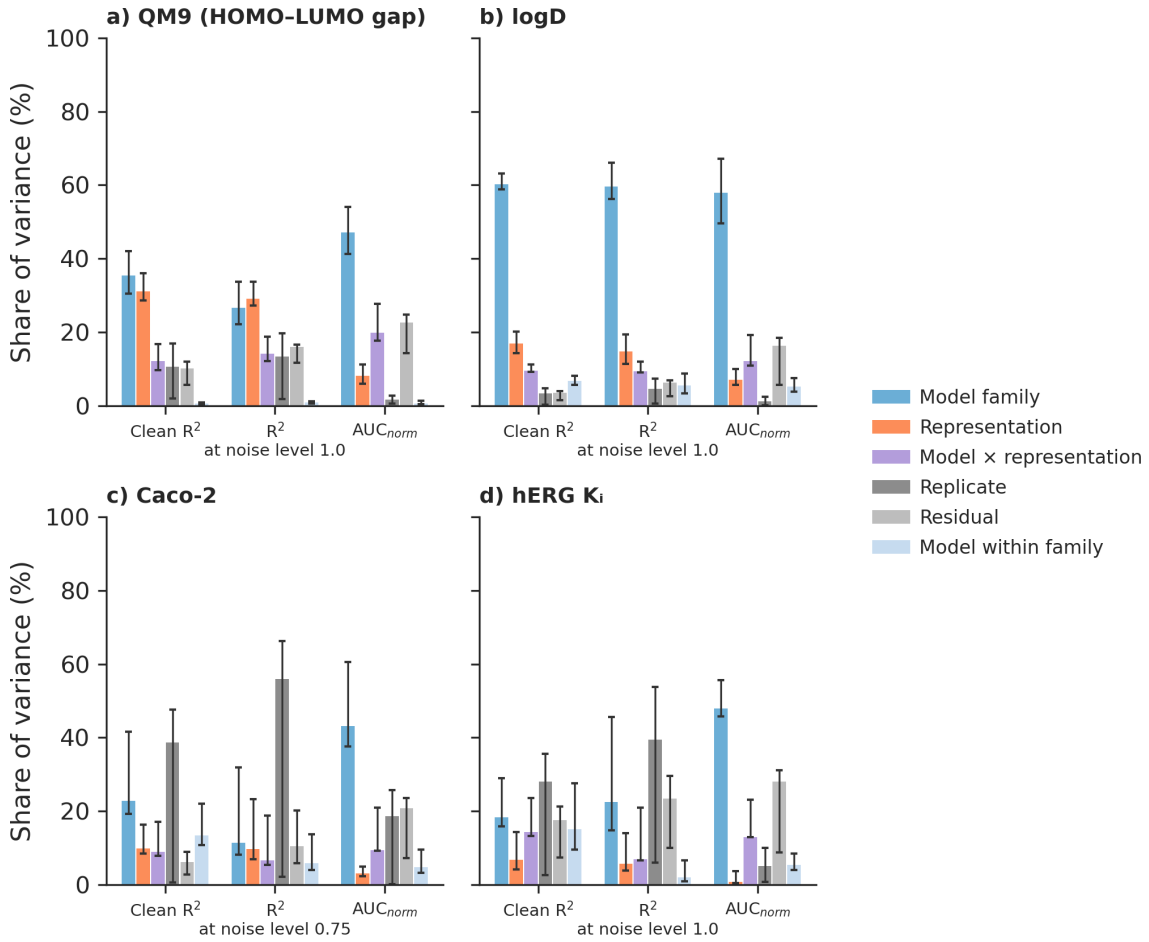


Figure 3.2: Share of the variance explained by each term on a) the QM9 HOMO–LUMO gap, b) LogD, c) Caco-2 and d) hERG K_i, for clean R^2, and for R^2 at a noise level of 1.0 (0.75 on Caco-2) and AUC_{norm} under Gaussian noise. Whiskers are 95% intervals from 1,000 bootstrap resamples of replicates: 10 on QM9 and five scaffold folds on each assay dataset. The values and their intervals are in Additional file 7.

individual models. All models' performance degrades as the labels get increasingly corrupted with noise (Figure 3.3). The loss remains small until the artificial noise reaches about half the spread of the clean training labels, and then it begins to drop. The rate it drops and the quantity of noise retained is not necessarily correlated to the model's predictive performance on clean labels. Across the 13 base models under Gaussian noise at ECFP4, clean R^2 and AUC_{norm} are barely correlated, at a Spearman correlation of -0.10 ($p = 0.73$) (Figure 3.4). If anything, the lower a model's predictive performance on clean labels, the less it has to lose as the labels get noisier.

The link between predictive performance and noise robustness can be examined by model family (Figure 3.4). The forests, boosted trees, SVM, and GP all retain relatively high predictive performance on clean labels while maintaining relatively strong robustness to noise. The plain NNs and BNNs are among the highest in predictive per-

formance on clean labels, and are the four least robust. The two variational networks sit between these groups, with lower predictive performance on clean labels and middling noise robustness. Standard feed-forward networks can make strong predictions under clean labels, however they are not a safe bet under noisy ones.

NGBoost is a standout case in the sense it is highly dependent on its paired molecular representation. Under Gaussian noise it has the highest AUC_{norm} on every one of the six representations, yet also the lowest in predictive performance on four of the six representations. It is one of the worst performing models with respect to R^2 on clean data. Its high robustness is partially due to the fact that when you start with a low predictive performance, you don't have as much to lose. However, it maintains shallow robustness curves across representations. On ECFP4 NGBoost's predictive performance ranks lowest up to a noise level of 0.75 and stays in the lower half at every noise level. However, on PDV it starts second lowest and ends highest, surpassing the other models between noise levels 1.0 and 1.5. So when paired with the right features, NGBoost has the potential to make better predictions under higher quantities of label noise. On grouped-shifted noise, RF has higher AUC_{norm} values than NGBoost on ECFP4, Avalon and ChemBERTa. Even then, NGBoost reports the highest noise robustness for PDV, MHG-GNN and Sort & Slice. While NGBoost performs well under higher label noise, LightGBM performs well under smaller quantities of noise. Its performance drops sharply after noise level 1.0.

As we've seen, the choice of representation can have an impact on noise robustness, but that impact is determined by model family, represented by the interaction term between model and representation in Figure 3.5. Some models are more impacted by representation than others, though. In general, networks span a wider range of AUC_{norm} across the six representations than any of the other models. Between ECFP4 and PDV under Gaussian noise, the four plain and Bayesian networks differ by between 0.037 and 0.065 of AUC_{norm}, while no tree or kernel model shifts by more than 0.011. VBLLs are an exception, as no two variational networks differ by more than 0.019.

3.3.3 Noise conditions on the HOMO–LUMO gap

Many studies involving experimental label noise rely on the assumption that noise can be modeled by a Gaussian distribution. We decided to vary the distribution which noise is drawn from, as well as the methodology by which it's injected into specific labels. Five of the seven conditions draw each label's error on its own, so the error is random. Grouped-shifted gives a whole scaffold family a shared offset, and censoring

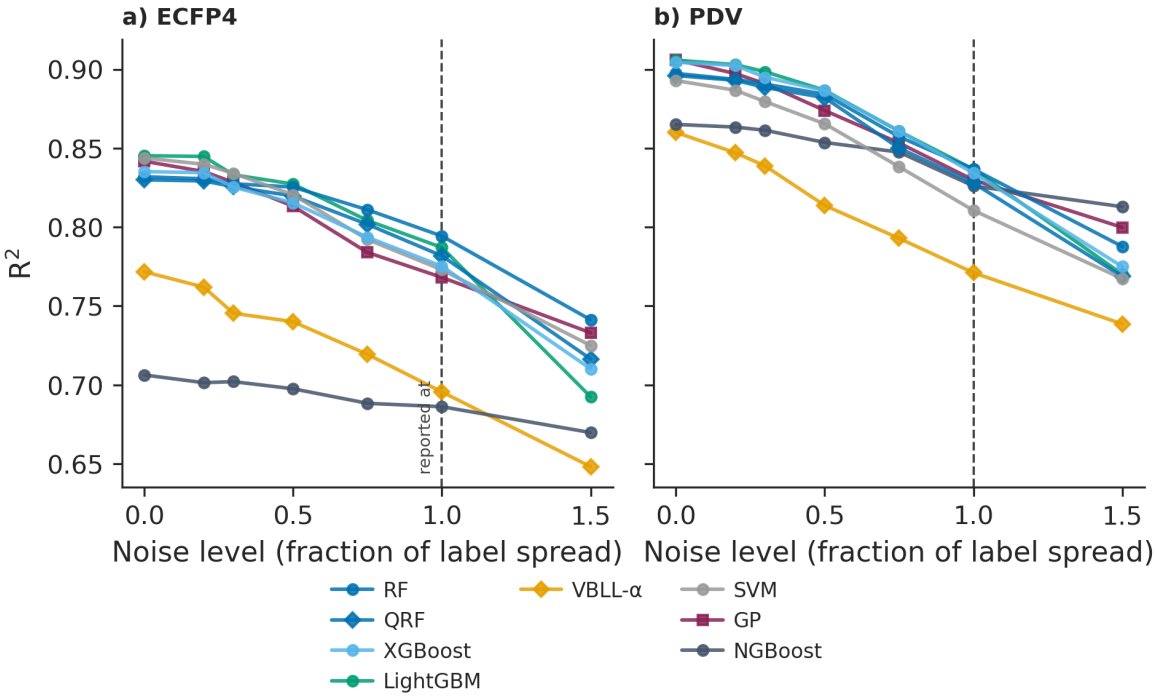


Figure 3.3: R^2 against noise level on the QM9 HOMO–LUMO gap under Gaussian noise, for the eight base models with the highest $\mathrm{AUC}_{\mathrm{norm}}$ on ECFP4: a) ECFP4; b) PDV. Medians over 10 replicates. The dashed line marks the noise level of 1.0 at which R^2 is reported.

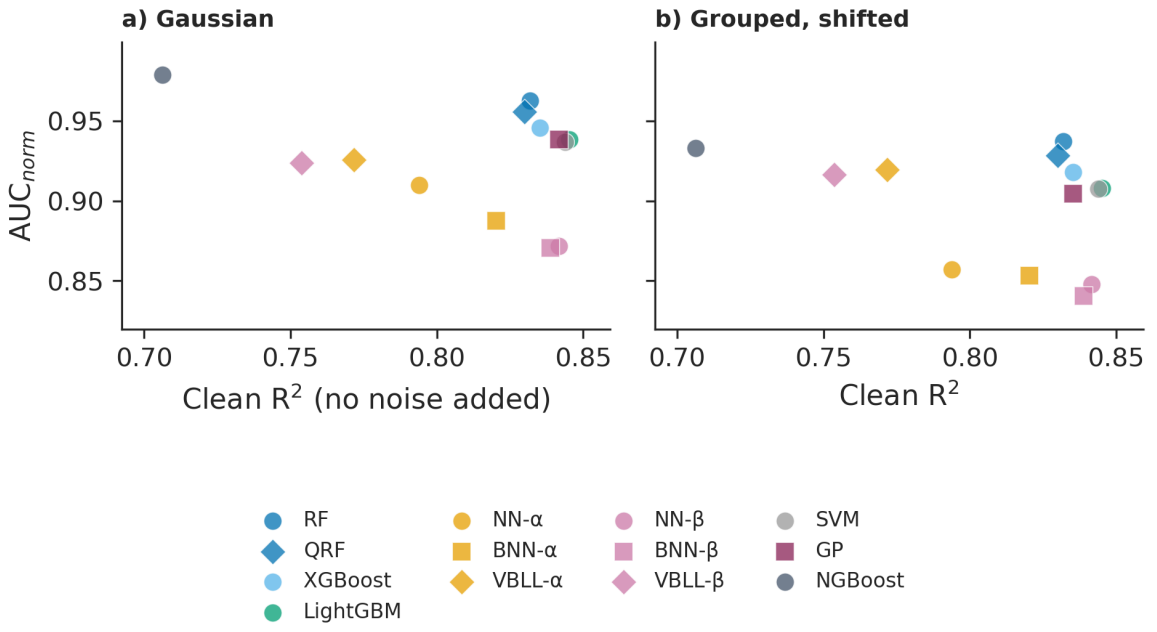


Figure 3.4: Clean R^2 against $\mathrm{AUC}_{\mathrm{norm}}$ for each base model on the QM9 HOMO–LUMO gap with ECFP4, under a) Gaussian and b) grouped-shifted noise.

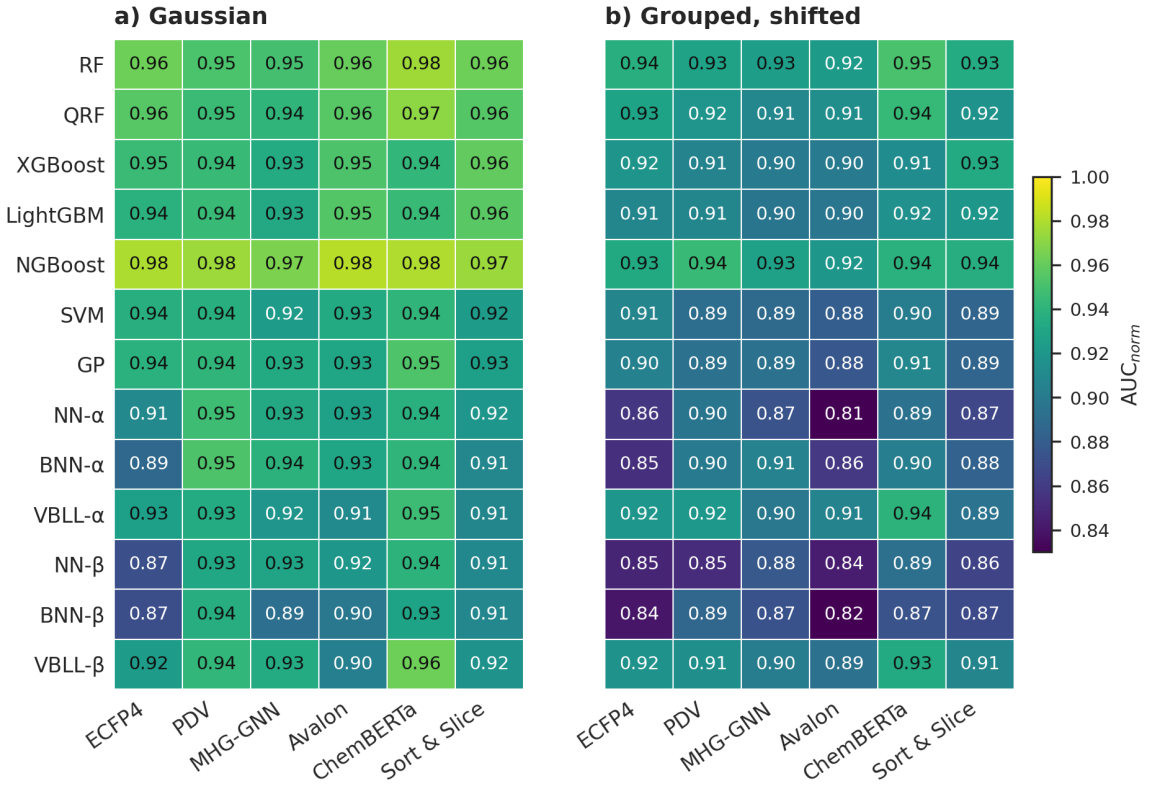


Figure 3.5: AUC_{norm} of the 13 base models on each representation, on the QM9 HOMO–LUMO gap under a) Gaussian and b) grouped-shifted noise. Medians over 10 replicates, on one color scale. Excluded configurations are in Additional file 3; grouped-wider noise is in Additional file 8.

pulls every label above a limit down to that limit, so in both the error is systematic. We controlled the quantity of noise by adding it in proportion to the spread of the clean labels on the training set, so a difference between two conditions is a difference in pattern rather than amount. One condition, censoring, could not be held to a set amount as its levels count the fraction of labels clipped.

We compare Gaussian noise on the HOMO–LUMO gap to Laplace, Student-t and outlier noise in Figure 3.6. The shape of the distribution from which the noise was drawn made little difference. SVM reported higher AUC_{norm} values on non-Gaussian label noise, 0.937 under Gaussian noise against 0.947, 0.952 and 0.955 under Student-t, outlier and Laplace noise on PDV, while NGBoost lost AUC_{norm} under every one. NGBoost's relatively stronger performance on Gaussian makes sense as it fits a per-molecule scale under a Gaussian likelihood, which the other distributions violate. On the three assay datasets with ECFP4, SVM gains AUC_{norm} under all three distributions, and NGBoost loses it in all but one case (Table 3.4). The size of the shift stays small, which is why these results coincide with Heid et al. [80], who found no difference between error distributions drawn at a matched mean and standard deviation. The

shape of the noise does not change how much performance a model keeps, but it does change which model keeps the most.

While alternative shapes of noise may be accurately modeled by the Gaussian distribution, we do not find that is the case for systematic label noise. Relative to Gaussian, grouped-shifted noise lowered the AUC_{norm} across all models and representations (Table 3.3). The models struggle to make predictions when the noise is isolated on molecules of specific scaffold families instead of being applied randomly across every sample. The variational networks retained the most predictive performance, while the plain, deterministic NNs lost the most. The predictive abilities of BNN-α degraded at a slower rate compared to its deterministic counterpart on every representation except PDV, while BNN-β did so only on MHG-GNN and PDV. Grouped-wider noise changed no model beyond its replicate spread except QRF on Avalon and MHG-GNN, which makes predictions particularly well under this noise condition. Censoring created the largest drop in predictive performance, AUC_{norm} was 0.12 to 0.18 lower for the same QSAR models than under Gaussian noise (Table 3.2). Censoring levels count clipped labels, though, so the two are on different scales.

Most of these observations are being made at the upper end of the noise range. With ECFP4, six of the seven most robust models start within 0.015 of one another in R^2 for clean labels, stay together up to a noise level of 0.5 and separate only above 1.0, finishing between 0.692 for LightGBM and 0.741 for RF (Figure 3.3). We ranked the models by predictive performance at each noise level. Of the 13 base models, RF started ranked seventh, but by level 1.0 it was ranked first. SVM on the other hand falls from first to fifth (Figure 3.8). Because AUC_{norm} weights the whole range from 0 to 1.5 equally, and the first half of it separates little, it has the potential to understate just how much the pattern of the noise costs. However, choosing to look at performance of a set level has its drawbacks, as the ranks shift at a different pace between noise conditions.

Model	Representation	Clean R^2	Gaussian	Censoring
RF	ECFP4	0.832	0.963	0.817
GP (het.)	ECFP4	0.840	0.937	0.813
NGBoost	PDV	0.865	0.975	0.809
GP	PDV	0.906	0.941	0.815
BNN-α (var. head)	PDV	0.890	0.951	0.771

Table 3.2: AUC_{norm} under censoring and under Gaussian noise on the QM9 HOMO–LUMO gap, for the five model and representation pairs run under censoring. Medians over 10 replicates. A censoring level is the fraction of labels clipped, so the two AUC_{norm} columns are on different scales.

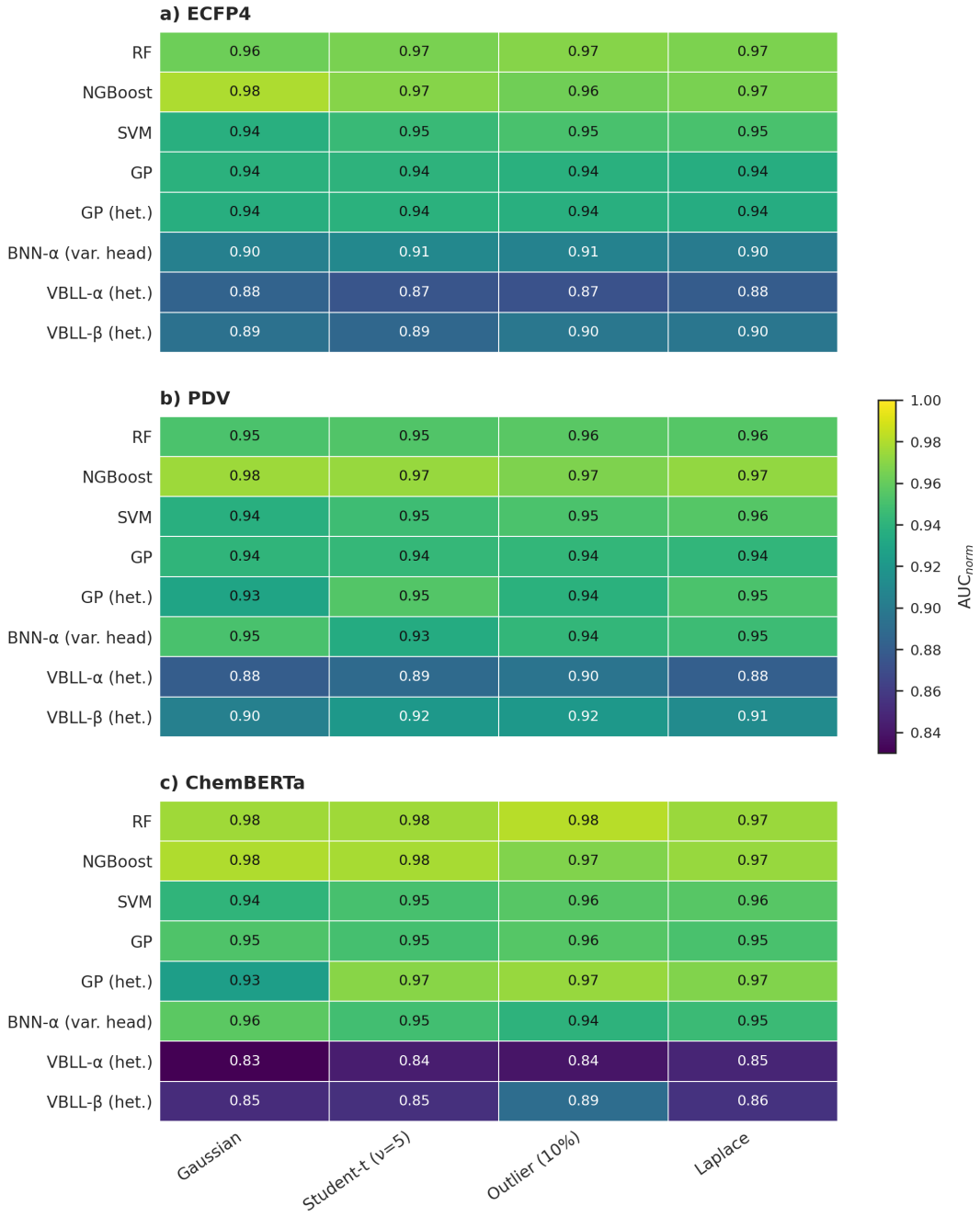


Figure 3.6: $\mathrm{AUC}_{\mathrm{norm}}$ under the noise conditions run on a subset of models, on the QM9 HOMO–LUMO gap: a) ECFP4; b) PDV; c) ChemBERTa. Medians over 10 replicates, on one color scale. Censoring is in Table 3.2.

3.3.4 Robustness on the three assay datasets

So far, we have only discussed results coming on HOMO–LUMO gap from QM9, a computed property whose labels carry no measurement error. We repeated these experiments on three experimentally-obtained datasets, LogD, Caco-2 [110], and hERG K_i [111], replicating our process of adding artificial noise. Unlike QM9, the experimental endpoints carry their own measurement noise in both training and test labels;

Model	ECFP4	PDV	MHG-GNN	Avalon	ChemBERTa	Sort & Slice	Exceeds spread
RF	-0.025	-0.024	-0.025	-0.038	-0.025	-0.028	6 of 6
QRF	-0.027	-0.027	-0.032	-0.042	-0.027	-0.038	5 of 6
XGBoost	-0.028	-0.039	-0.035	-0.052	-0.030	-0.025	5 of 6
LightGBM	-0.030	-0.037	-0.037	-0.058	-0.028	-0.037	5 of 6
NGBoost	-0.046	-0.030	-0.039	-0.062	-0.041	-0.039	6 of 6
SVM	-0.029	-0.045	-0.034	-0.053	-0.040	-0.038	5 of 6
GP	-0.034	-0.048	-0.040	-0.054	-0.045	-0.041	6 of 6
NN-α	-0.053	-0.051	-0.060	-0.115	-0.044	-0.053	4 of 6
BNN-α	-0.034	-0.053	-0.034	-0.073	-0.039	-0.025	2 of 6
VBLL-α	-0.006	-0.014	-0.020	-0.005	-0.007	-0.013	0 of 6
NN-β	-0.024	-0.080	-0.052	-0.076	-0.047	-0.045	4 of 6
BNN-β	-0.030	-0.050	-0.019	-0.075	-0.055	-0.041	2 of 6
VBLL-β	-0.007	-0.028	-0.033	-0.010	-0.028	-0.009	0 of 6

Table 3.3: Change in $\mathrm{AUC}_{\mathrm{norm}}$ from Gaussian to grouped-shifted noise on the QM9 HOMO–LUMO gap, per base model and representation; negative is a loss. The last column counts the representations on which the loss exceeds the replicate spread, the highest minus the lowest $\mathrm{AUC}_{\mathrm{norm}}$ over 10 replicates under Gaussian noise.

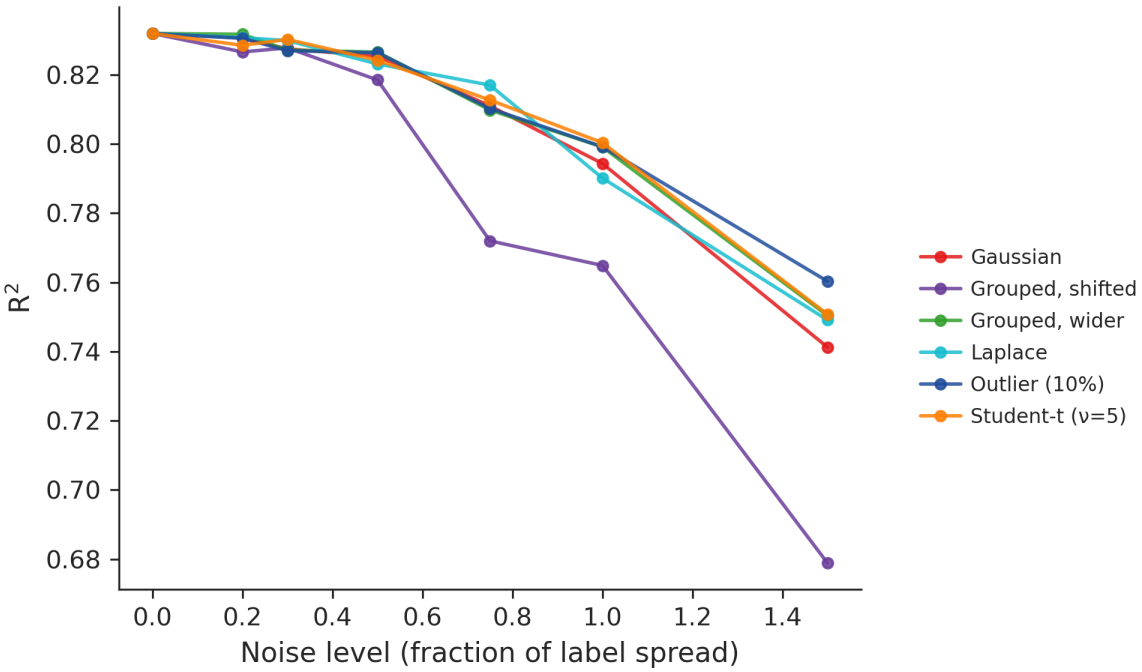


Figure 3.7: R^2 of the random forest against noise level under each noise condition, on the QM9 HOMO–LUMO gap with ECFP4. Medians over 10 replicates. Censoring is not shown, as its level is a fraction of labels clipped.

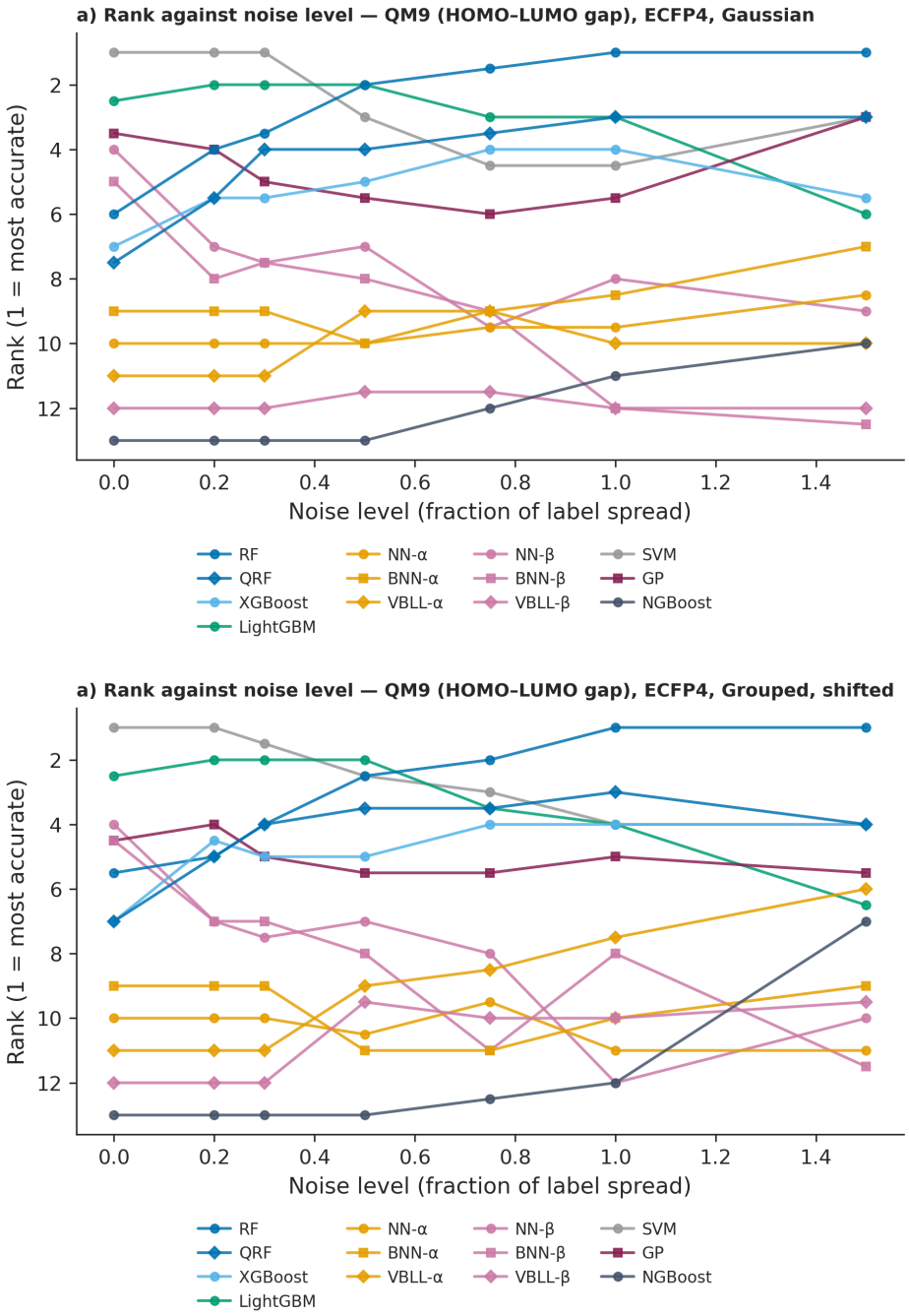


Figure 3.8: Median rank of each base model by R^2 at each noise level, on the QM9 HOMO–LUMO gap with ECFP4, under a) Gaussian and b) grouped-shifted noise. Models are ranked within each replicate; tied medians give half ranks.

the injected artificial noise therefore adds on top of an unknown noise floor. Repeat measurements of the same compound have a standard deviation of about 0.54 log units for pK_i [12, 13]. The hERG K_i labels have a spread of 0.915 log units, so a noise level of 0.6 of that spread is about one unit of that estimated laboratory error. Each model's AUC_{norm} under each noise condition on the three endpoints is given with its clean R^2 beside it (Figure 3.10).

As on QM9, the noise distribution does not change the predictive capabilities of a model (Table 3.4). Laplace, Student-t and outlier noise were run on seven models, RF, the two GPs, SVM, NGBoost, BNN-α (var. head) and VBLL-β (het.), on ECFP4, PDV and ChemBERTa. Each delta below is measured from the same model's AUC_{norm} under Gaussian noise. RF and the two GPs never lose more than 0.03 of AUC_{norm} under these three conditions or under grouped-wider noise, on any dataset or representation. SVM and NGBoost again move in opposite directions, SVM never loses AUC_{norm} under any of the four, while NGBoost loses under all four on Caco-2 and hERG K_i. The two networks move furthest in both directions. It's worth remembering that while we are injecting artificial noise, the assay measurements carry their own source of noise whose properties are unknown, so no "clean" label is truly clean.

Of the three noise conditions run on every model, grouped-shifted noise gives the lowest AUC_{norm} values on every assay dataset (Figure 3.10). Under it, RF ranks first or second for AUC_{norm} with every representation on LogD and with four of six on hERG K_i. R^2 at a noise level of 1.0 depends more on clean performance. By that measure, SVM ranks first to third on LogD, while RF ranks fifth to ninth across datasets.

LightGBM has particularly varied results across the datasets. On LogD, it has relatively high predictive performance and middle-of-the-pack noise robustness. However, on hERG and Caco-2, it loses the most with increased noise, particularly on Gaussian and grouped-shifted noise. The effects are not as pronounced on grouped-wider, although it still ranks among the least robust there. XGBoost degrades in the same way on hERG and Caco-2.

While the results are not identical between the datasets, there are some patterns that emerge (Figure 3.11). The ranks in this paragraph are among the 13 base models, while the figure ranks among all 19. In general, the relative AUC_{norm} model rankings match between QM9 and LogD, as well as between Caco-2 and hERG K_i with every representation except ChemBERTa. RF is the most consistent, among the 13 base models under Gaussian noise it ranks between first and seventh for AUC_{norm} on every dataset and representation, second or third on QM9 and first on LogD at five of six. The GP has the highest predictive performance on clean labels on all three assay datasets

Dataset	Model	Gaussian	Laplace	Student-t (ν=5)	Outlier (10%)	Grouped, wider
LogD	RF	0.935	+0.018	+0.006	+0.007	+0.021
LogD	NGBoost	0.924	0.000	-0.028	-0.032	-0.009
LogD	SVM	0.879	+0.044	+0.023	+0.027	+0.044
LogD	GP	0.891	+0.004	+0.003	-0.001	+0.006
LogD	GP (het.)	0.896	0.000	-0.002	-0.007	-0.003
LogD	BNN-α (var. head)	0.881	-0.013	-0.013	-0.024	-0.029
LogD	VBLL-β (het.)	0.796	+0.010	+0.012	+0.020	+0.068
Caco-2	RF	0.865	+0.035	+0.028	+0.026	+0.053
Caco-2	NGBoost	0.878	-0.069	-0.038	-0.060	-0.147
Caco-2	SVM	0.841	+0.049	+0.051	+0.031	+0.018
Caco-2	GP	0.809	+0.039	+0.034	+0.020	+0.023
Caco-2	GP (het.)	0.815	+0.035	+0.032	+0.011	+0.019
Caco-2	BNN-α (var. head)	0.787	-0.110	+0.092	-0.043	-0.017
Caco-2	VBLL-β (het.)	0.707	+0.036	+0.025	+0.015	+0.032
hERG K_i	RF	0.898	+0.015	-0.006	+0.016	-0.012
hERG K_i	NGBoost	0.842	-0.031	-0.027	-0.042	-0.093
hERG K_i	SVM	0.859	+0.051	+0.035	+0.025	+0.055
hERG K_i	GP	0.831	+0.022	+0.034	+0.025	+0.030
hERG K_i	GP (het.)	0.843	+0.010	+0.036	+0.015	+0.021
hERG K_i	BNN-α (var. head)	0.809	-0.034	-0.032	-0.063	-0.009
hERG K_i	VBLL-β (het.)	0.836	+0.007	-0.047	+0.005	-0.002

Table 3.4: AUC_{norm} under Gaussian noise, and its change under each other noise condition, for a subset of models on the three assay datasets with ECFP4; negative is a loss. Medians over five scaffold folds.

with nearly every representation, and ranks third to ninth for AUC_{norm}. NGBoost, the most robust model on QM9, ranks first to third on LogD, second to tenth on Caco-2 and sixth to ninth on hERG K_i. LightGBM falls furthest, ranking 10th to 13th on Caco-2 and hERG K_i at every representation while being among the highest in predictive performance on clean labels. VBLL-α is the opposite, ranking first or second for AUC_{norm} on hERG K_i and last on LogD at every representation, from one of the lowest clean R^2 values. This fits the pattern of the NNs being exceptionally varied.

The forests, RF and QRF, sit in the middle for predictive performance on clean labels, but they are the most reliably robust to noise across all datasets (Figure 3.10). Their rank barely changes between noise conditions, so no pattern of noise suits them better than another. Caco-2 is the exception, where RF ranks third to seventh rather than first or second.

Other models perform conditionally well, and in some cases that condition is the representation they are paired with. The GP has some of the higher predictive capabil-

ities on clean labels and under Gaussian noise across assay datasets. It is the highest on LogD and Caco-2 under grouped-wider noise as well, but under grouped-shifted noise it falls. The choice of representation has little impact on GP, moving its AUC_{norm} less than repeating the split does on every dataset.

There is a slight consideration with using both AUC_{norm} and predictive performance side-by-side. AUC_{norm} measures the fraction of its own clean R^2 that a model keeps, so predictive performance on clean labels can work against it. A model that already predicts poorly has little to lose and often scores a higher AUC_{norm}, while a model that starts with strong performance has more to lose. The GP and NGBoost are the clearest cases. The GP has among the highest predictive performance on clean labels and a middling AUC_{norm}, while NGBoost has the highest AUC_{norm} on QM9 from one of the lowest clean R^2 values. While NGBoost's predictive performance is highly dependent on its representation, its robustness is not. On QM9 it is both the most robust model and the lowest ranked model for predictive performance up to a noise level of 0.75 with ECFP4. But with PDV it's able to retain some predictive performance under noise, finishing highest across all the models at noise level 1.5.

Upon first glance, SVM does not seem like a standout, however, it is the model whose standing is better with non-Gaussian noise. Among RF, the GP, NGBoost and SVM, the four models run under every condition except censoring, SVM has the lowest AUC_{norm} of the four under Gaussian noise on LogD and the second lowest on Caco-2 with ECFP4 (Table 3.4). It rises to first or second under Laplace, Student-t and outlier noise and under grouped-wider noise in all but one case. Gaussian noise is actually the one condition it is worst at. Its AUC_{norm} stays in the middle, partially because it starts high, and it ranks among the five highest for predictive performance at a noise level on every assay dataset under every noise condition.

The feed-forward networks are the most variable, although in general they have some of the highest predictive performance under clean labels and are generally not robust to noise. They are also dependent on the choice of representation. VBLL networks perform poorly on clean labels, are the least robust to noise on LogD, yet are the most robust on Caco-2 and hERG K_i. The gradient-boosted trees, LightGBM and XGBoost, were also quite variable, with strong predictive performance on clean labels and poor noise robustness on Caco-2 and hERG K_i.

The quantity of noise is an important factor as well. At a noise level of 0.5 the models still rank by predictive performance much as they do on clean labels, while by 1.0 the order has changed, most of all on hERG K_i and then QM9. The order of models by AUC_{norm} is broadly similar across noise conditions, with the forests near the

top under each.

While it is clear that QSAR model performance drops with label noise, different models drop faster than others. How much noise a dataset tolerates before anything happens is itself a property of the dataset. Up to a noise level of 0.3, three tenths of the spread of the clean labels, the models are largely unaffected by the noise on the HOMO–LUMO gap and on LogD, where every base model with ECFP4 still keeps at least 93% of its clean R^2; on Caco-2 one of the 13 and on hERG K_i four of the 13 are already below 90% at that level. By a noise level of 0.5 the lowest retained fraction is 90% on the HOMO–LUMO gap against 76% on hERG K_i (Figure 3.9). Boosted tree ensembles are some of the first to fall. On Caco-2 with ECFP4, LightGBM has among the highest predictive performance on clean labels. It is also the first tree ensemble to fall below 90% of that performance, at a noise level of 0.5. By level 1.5 it retains 26% of its original R^2, compared to 63% for RF on ECFP4. XGBoost drops off at the next recorded noise level. However, NGBoost is the opposite, having some of the lowest predictive performance on clean labels and the flattest curve with injected label noise across all models. On the HOMO–LUMO gap it never falls below 90% of its clean R^2 at any level tested. The networks drop at an inconsistent rate. VBLL-α is the earliest to fall on LogD, dropping below 90% of its clean R^2 at a level of 0.5 and reaching 38% by 1.5. Yet on Caco-2 and hERG K_i it is among the flattest models.

The variance decomposition of predictive performance tells us how much the choice of model and representation matters under clean labels (Figure 3.2; Additional file 7). Extending that concept, we are also interested in how these shares change under increasing label noise and compare to the variance decomposition of $\mathrm{AUC}_{\mathrm{norm}}$. The replicate and residuals account for more variance across all datasets. On Caco-2 and hERG K_i the replicate, which on these datasets is one of five scaffold folds, is the largest share of predictive performance before and after noise is injected. Conversely, it falls well below the choice of model family for robustness. On every dataset the interaction between model and representation explains more of the variance in $\mathrm{AUC}_{\mathrm{norm}}$ than representation alone.

3.3.5 Probabilistic and deterministic counterparts

To further explore the impact of model architecture on noise robustness, we examined deterministic and probabilistic variants of the same core model architecture. The hypothesis was that a model that reports a distribution rather than a point value may be more robust to small amounts of injected label noise. We compared variants across

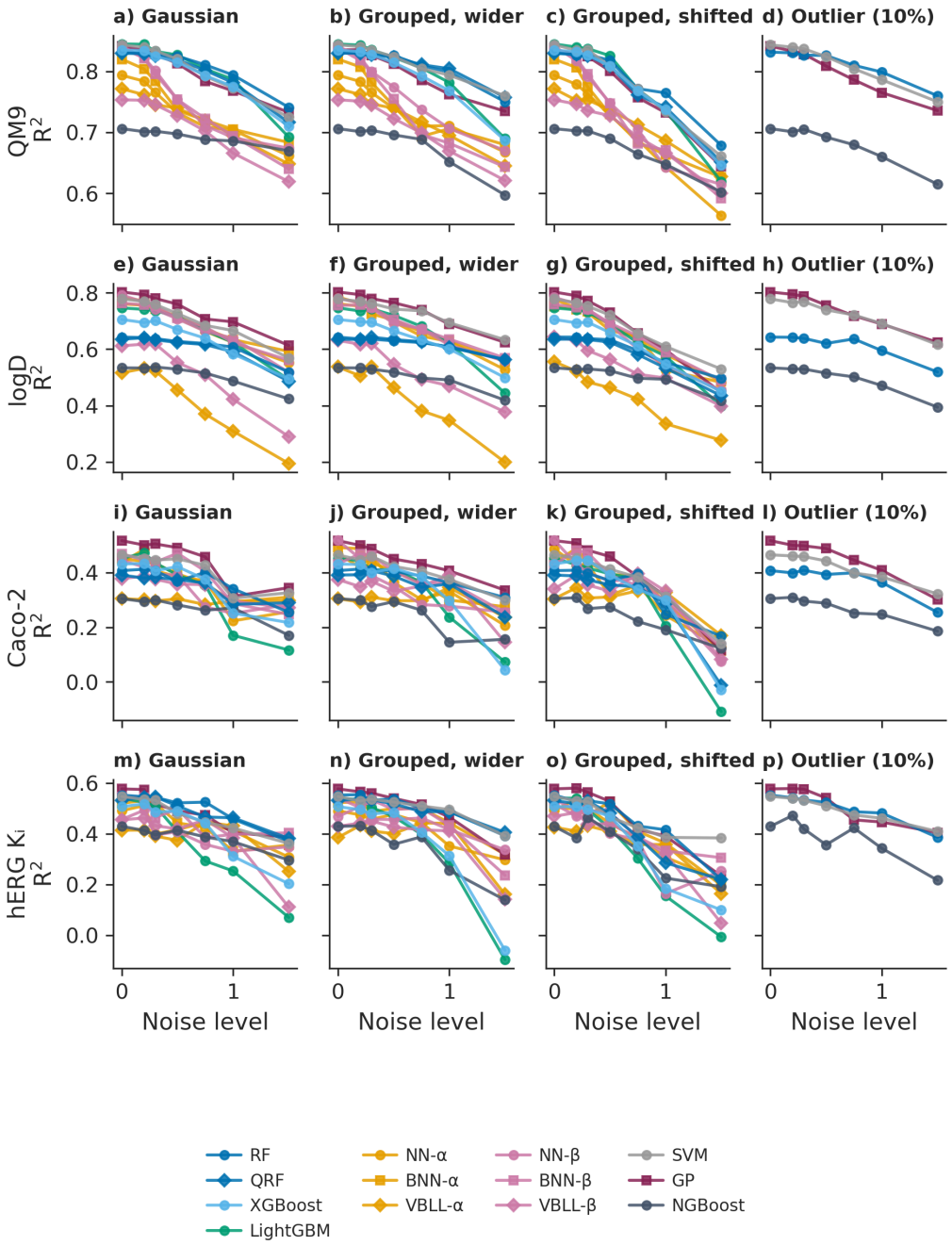


Figure 3.9: R^2 against noise level for the 13 base models on every dataset with ECFP4. Rows are datasets and columns noise conditions: a)–d) QM9 HOMO–LUMO gap; e)–h) LogD; i)–l) Caco-2; m)–p) hERG K_i. Medians over 10 replicates on QM9 and five scaffold folds on the assay datasets.

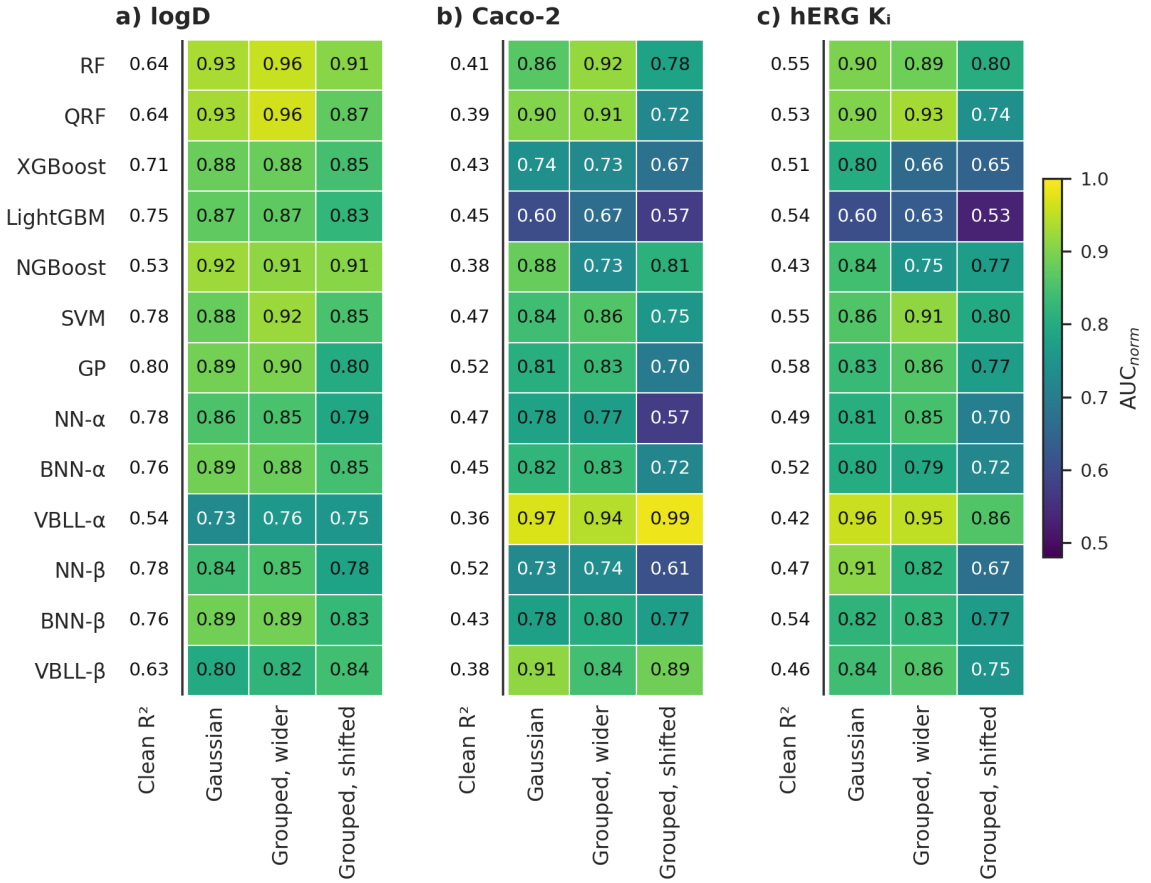


Figure 3.10: AUC_{norm} of the 13 base models with ECFP4 on a) LogD, b) Caco-2 and c) hERG K_i, under the three noise conditions run on every model. The first column is clean R^2. Medians over the scaffold folds that pass the clean-R^2 threshold (Additional file 3), on one color scale. The variant models are in Additional file 9.

three sets of base models: two NNs labeled NN-α and NN-β and RF (Figure 3.12). For each NN, we transform a standard deterministic network into a BNN. The BNNs with a variance head run at different settings from their Bayesian bases, so they are compared only for uncertainty. We compared RF to its probabilistic counterpart, QRF, with the same baseline architecture and hyperparameters. Comparisons of this nature are only possible when the variants carry the same underlying settings (Additional file 2). NN-α and BNN-α therefore share one setting, as do NN-β and BNN-β, and QRF is compared with a random forest of the same 300 trees (Additional file 2).

We compare RF and QRF directly with the same underlying architecture. QRF had lower AUC_{norm} on QM9 under Gaussian and grouped-shifted noise on every representation, and changed it little under grouped-wider noise, with almost no change in clean R^2 (Figure 3.12). On the assay datasets QRF was less robust than RF under grouped-shifted noise on every dataset and representation, and under Gaussian noise on LogD and hERG K_i (Additional file 10). Under grouped-wider noise it was more

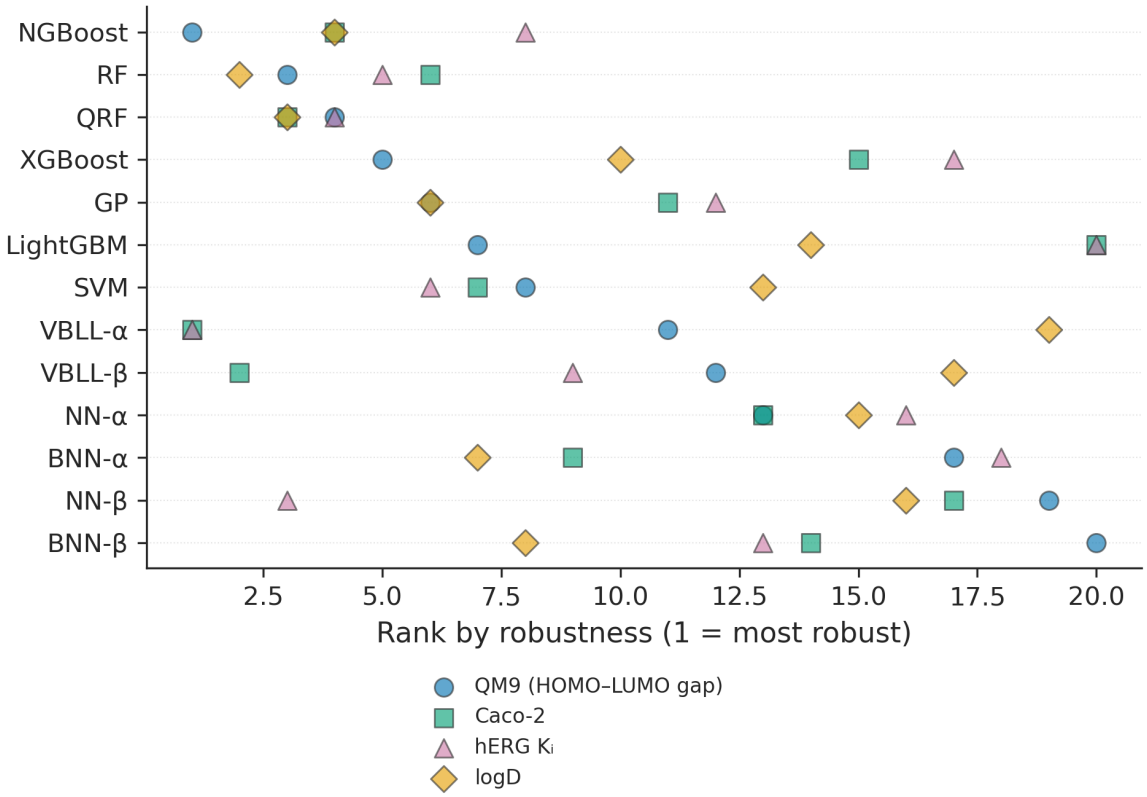


Figure 3.11: Rank by $\mathrm{AUC}_{\mathrm{norm}}$ of each base model on every dataset, with ECFP4 under Gaussian noise. Models are ordered by their rank on the QM9 HOMO–LUMO gap. Ranks are among all 19 models; the six variant models are not drawn.

robust on Caco-2 and mixed elsewhere.

The Bayesian transformations that we used to transform feed-forward deterministic NNs into probabilistic ones produced a slightly larger $\mathrm{AUC}_{\mathrm{norm}}$, but only because they lowered predictive performance on clean labels. BNN-β kept more $\mathrm{AUC}_{\mathrm{norm}}$ than NN-β in most combinations on every assay dataset, and BNN-α on all assay datasets. On QM9 the Bayesian transformations made no significant change on either predictive performance or $\mathrm{AUC}_{\mathrm{norm}}$ for either network.

3.3.6 Uncertainty under increasing label noise

Until now we've discussed how much predictive capabilities a model is able to retain while its training labels are corrupted with noise. Now, we consider how confident the model reports its own predictions are in the presence of noise, and whether or not those estimates match the increased label noise. Kolmar and Grulke [10] found that the mean predicted uncertainty derived from the GPs increases with the amount of label noise in the training data. We go beyond the population level and instead ask if a model's per-sample uncertainty tracks label noise. One key sign that a model is

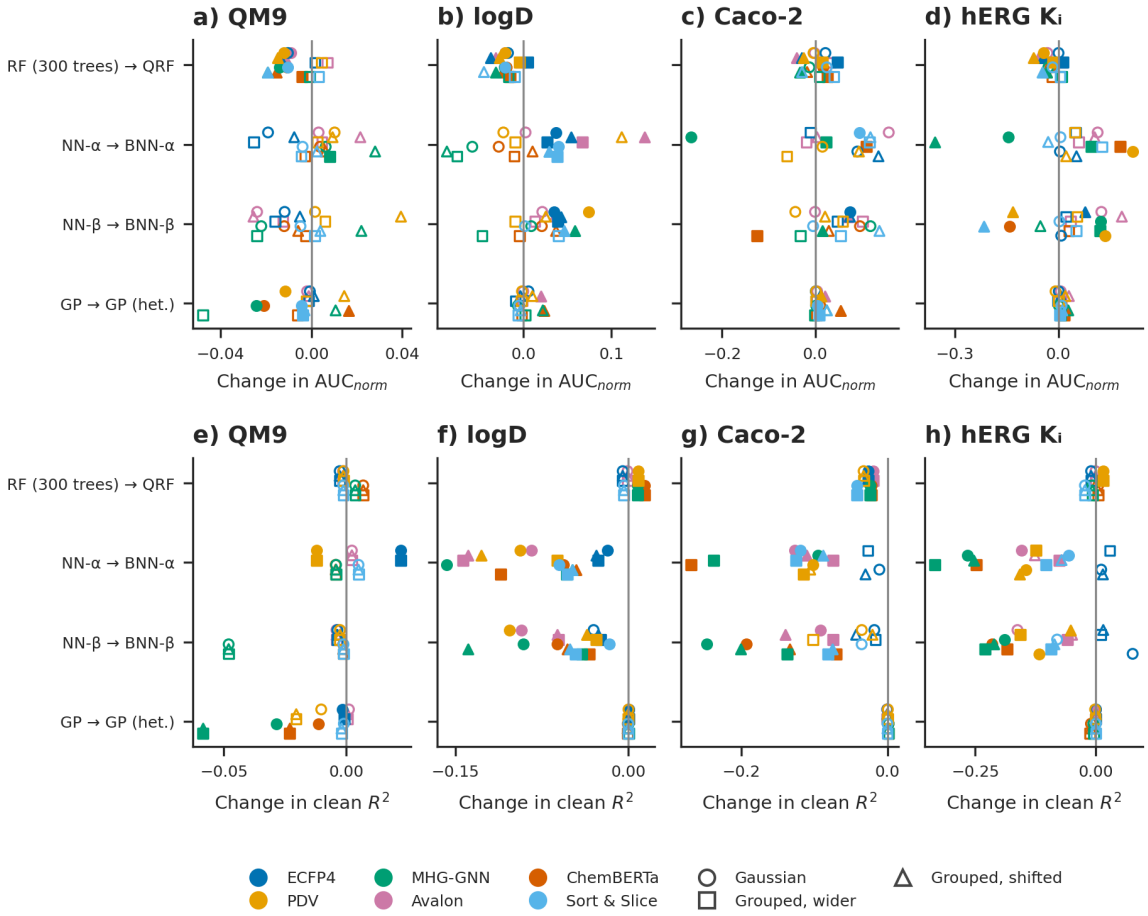


Figure 3.12: Change in AUC_{norm} (top) and clean R^2 (bottom) from each model to its probabilistic counterpart at the same settings, on every dataset: counterpart minus base, the median of paired differences over 10 replicates on QM9 and five scaffold folds on the assay datasets. Color is the representation and shape the noise condition. A filled mark changed in the same direction in every replicate or fold.

able to handle noise is its ability to track it, represented by the per-sample Spearman correlation between predicted uncertainty and noise magnitude.

As expected, mean predicted uncertainty is positively correlated with Gaussian label noise for most models and representations (Table 3.5). There are a few exceptions, namely BNN-β on PDV and VBLL-β (het.) on Avalon for QM9. BNN-α fails to rise in at least one fold on every assay dataset, though with PDV only on LogD and Caco-2, and both full-BNNs with a variance head fail on MHG-GNN for hERG K_i, and BNN-β with a variance head also fails on ChemBERTa and on Caco-2. Kolmar and Grulke [10]'s finding therefore holds for almost every probabilistic model on the computed property, and for most of them on measured labels.

Model	QM9	LogD	Caco-2	hERG K_i	Aleatoric	Epistemic
QRF	4.9	2.7	2.4	2.3	per molecule	per molecule
NGBoost	4.8	3.2	2.6	2.6	per molecule	none
GP	5.3	4.1	2.5	2.6	one per fit	per molecule
GP (het.)	—	4.3	2.7	2.8	per molecule	per molecule
BNN-α	3.4	1.4†	1.0†	1.4	none	per molecule
BNN-α (var. head)	4.1	2.7	1.9	2.1	per molecule	per molecule
VBLL-α	3.5	2.3	1.4	1.2	one per fit	per molecule
VBLL-α (het.)	3.0	1.9	2.8	3.5	per molecule	per molecule
BNN-β	2.7†	1.2†	1.1	1.2	none	per molecule
BNN-β (var. head)	4.3	2.5	2.4	1.7	per molecule	per molecule
VBLL-β	3.3	2.4	1.3	1.2	one per fit	per molecule
VBLL-β (het.)	2.6	2.0	2.7	3.1	per molecule	per molecule

Table 3.5: Mean predicted uncertainty on held-out molecules at a noise level of 1.5, divided by its value with no added noise, with PDV under Gaussian noise. †: the uncertainty fails to rise in at least one fold. A dash: not run. Aleatoric and Epistemic: whether the model reports that component per molecule, as one value per fit, or not at all.

Aleatoric and epistemic attribution

While this tells us that the models are reportedly less certain under noisy data, we don't know what the model is attributing this uncertainty to. To answer this question, we decompose uncertainty estimates into their aleatoric and epistemic components wherever possible. We expect to see aleatoric uncertainty rise with injected label noise, while the epistemic component should remain relatively constant, as it is the aleatoric component that describes uncertainty in the underlying data. The models whose frameworks allowed for this decomposition include full-BNNs with a variance head, the heteroscedastic GP, and QRF (Table 3.5). The two heteroscedastic VBLL networks also report both components, and they separate them more often than any other model, though not on every dataset and representation. They are not drawn in the figures, as they were not scored out of fold.

The full-BNNs with a variance head behave as expected (Figure 3.13), the epistemic component grows by at most 0.43 of the aleatoric growth in every configuration where the ratio can be computed. Despite QRF's noise robustness, both of its aleatoric and epistemic components rise with noise and on some representations, the epistemic component actually rises at a faster rate.

Unlike the predictive performance and robustness, the representation actually dictates whether decomposition works. For QRF, the epistemic growth is 0.61 to 1.14 times the aleatoric growth on the three fingerprints, 1.02 to 1.44 times on PDV, and

1.29 to 2.04 times on ChemBERTa and MHG-GNN. Under Gaussian noise the heteroscedastic GP keeps its epistemic growth below a quarter of its aleatoric growth on PDV, ChemBERTa and MHG-GNN except MHG-GNN on QM9, but not on ECFP4 under grouped-shifted noise, where the two grow by similar amounts.

The noise conditions follow the pattern found in robustness, the three conditions which alternate the noise distribution show similar decompositions to those under Gaussian noise, while grouped-shifted raises the epistemic share of the heteroscedastic GP and of QRF in every pairing of dataset and representation where they were run. This scaffold family-based offset may be appearing as structure in the data rather than label noise to the models.

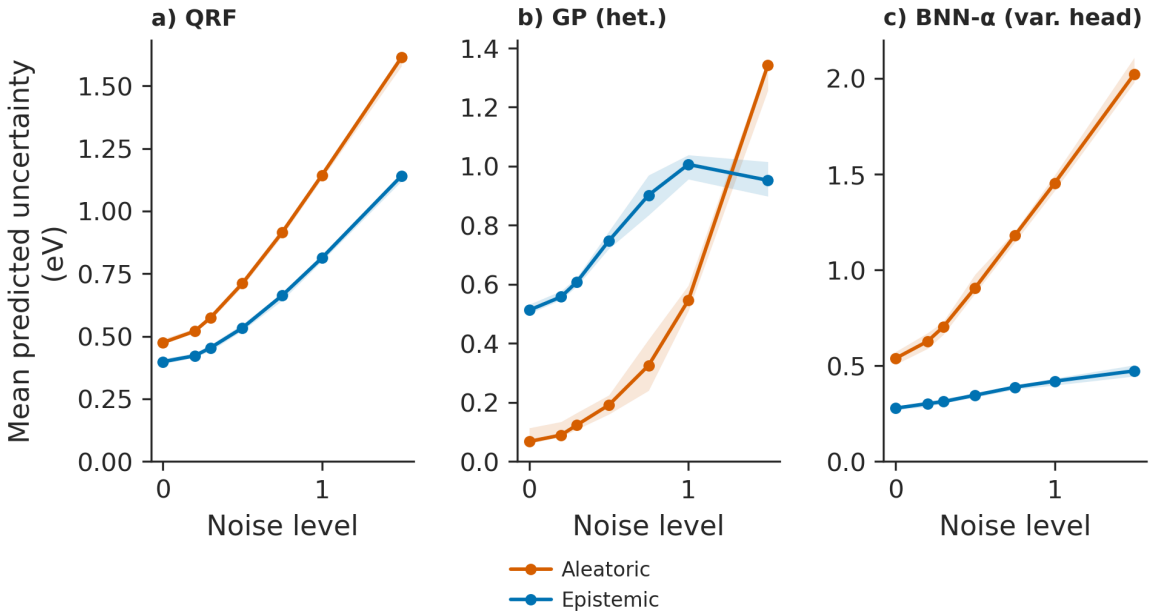


Figure 3.13: Aleatoric and epistemic components of the predicted uncertainty against noise level, one panel per model, on the QM9 HOMO–LUMO gap with ECFP4 under Gaussian noise. Each value is the mean over the molecules of one out-of-fold pass; lines are medians over folds and bands their range. A component that is one value per fit is not drawn (Table 3.5).

The aleatoric component indicates that label noise increased, but it doesn't tell us by how much (Figure 3.14). For each unit of added noise it rises by roughly 0.4 to 1 unit, depending on model, dataset and representation. The aleatoric component can therefore compare noise between datasets or noise levels, but it does not estimate a dataset's internal error.

Uncertainty estimation breaks under censoring as the process of clipping the labels narrows their spread, which in turn shrinks the total uncertainty. None of the models tested treat a clipped label as a noisy one, so they actually grow more confident while

losing information.

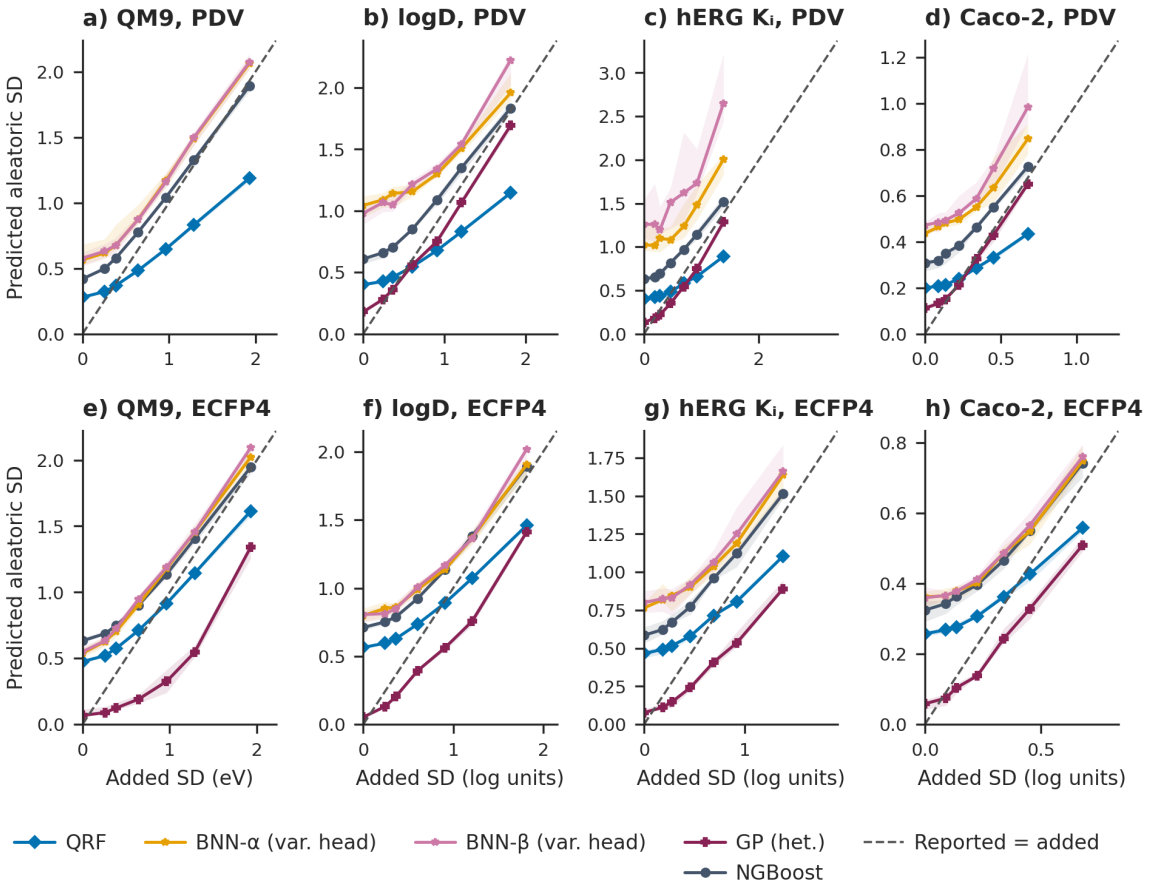


Figure 3.14: Predicted aleatoric standard deviation against the standard deviation of the added noise, under Gaussian noise. Rows are PDV (top) and ECFP4 (bottom); columns are the QM9 HOMO–LUMO gap, LogD, hERG K_i and Caco-2. Each line is one model: the median over folds of the mean over the molecules of one out-of-fold pass, with a band spanning the folds. The dashed line is equality. The heteroscedastic Gaussian process was not run on QM9 with PDV, and the two heteroscedastic variational networks were not scored out of fold.

Per-sample uncertainty and label noise

Finally, we ask whether a model's uncertainty on one molecule tracks the noise on that molecule's label. We assess this using the Spearman correlation between predicted uncertainty and the size of the injected noise over out-of-fold training molecules.

Since Gaussian, Laplace, Student-t and grouped-shifted conditions add noise from the same scale to every label, there is not much for uncertainty to find. It's the same situation with outlier noise, under which a tenth of the data is chosen at random for extra noise, and with grouped-wider in which each scaffold family falls in a single fold, so a model makes predictions based on unseen scaffolds and has no basis for

estimating uncertainty. The only case where per-sample uncertainty tracking is possible is censoring, as the noise is dependent on the value of the label itself. The correlations are then large and mostly negative, meaning the model is least uncertain on the labels that were clipped most. On QM9 with half the labels clipped, on PDV, they run from -0.84 to -0.80 across folds for NGBoost and from -0.47 to -0.41 for the GP, and on LogD with half the labels clipped every fold of every model except the two GPs is at -0.29 or lower. We divided each molecule's error by its uncertainty and found that the division ranked the clipped labels better than the error alone on QM9 and on most of LogD. Models are most confident on the clipped labels, probably because the clipping piles many labels into one value, which the model learns to tightly predict. Ironically, per-molecule uncertainty can be used in reverse to identify clipped labels: the labels a model is most certain about, especially when they fall in a narrow range, are the most likely to have been clipped.

3.4 Conclusions

We investigated how a variety of molecular representations and models perform under increasing label noise. We used ANOVA variance decomposition across three noise conditions on four datasets to show that molecular representation accounts for a larger share of the variance in predictive performance than in robustness. Robustness to label noise, as measured by $\mathrm{AUC}_{\mathrm{norm}}$, the normalized area under the R^2 retention curve, is more impacted by the choice of model. The representation each model is paired with explains more variance than the representation alone. As such, a representation should not be chosen independently of the model it will be used with, and some models benefit from alternative molecular representations across both predictive performance and noise robustness. Model family was the primary factor in every decomposition of $\mathrm{AUC}_{\mathrm{norm}}$ except one, Caco-2 under grouped-shifted noise, where the scaffold fold took the larger share.

Grouped-shifted and censored noise, the two conditions in which the error is systematic, corrupt QSAR model predictions the most. Changing the shape of the distribution from which noise is drawn barely has an impact on model performance, a conclusion which is in agreement with Heid et al. [80]. Low quantities of noise do not make much of an impact, on the HOMO–LUMO gap every base model keeps at least 93% of its clean R^2 up to a noise level of 0.3, and grouped-shifted noise separates from the other conditions only from a level of 0.75 (Figure 3.7). Models' performance begins to degrade more rapidly on the assay datasets than on the HOMO–LUMO gap, which makes

sense as assay data contain inherent experimental noise. Tree and kernel models maintained consistent $\mathrm{AUC}_{\mathrm{norm}}$ across all representations, while the plain NNs and BNNs depended on the representation they were given, as did the predictive performance of NGBoost. Across datasets and noise conditions, RF and QRF were reliably robust, although their predictive performance on clean labels was middling and on Caco-2 they struggled under Gaussian noise. QRF was the slightly less robust of the two, falling below RF under Gaussian and grouped-shifted noise. Other models performed well only under some conditions. The GP had the highest predictive performance on clean labels and under independent noise but fell behind when the noise was shared within a scaffold family, NGBoost predicted well only with the right representation, and SVM led in predictive performance under scaffold-shared noise on some datasets. Transforming a feed-forward network into a Bayesian one increased $\mathrm{AUC}_{\mathrm{norm}}$ on the assay datasets only by lowering clean R^2, and made no significant change on QM9, so switching to a tree-based model is the stronger move under label noise.

With regard to uncertainty estimation, mean predicted uncertainty rose with label noise for almost every probabilistic model on QM9 and for the majority of models on the assay datasets. Of the models whose aleatoric and epistemic components can be split per molecule, the BNNs with a variance head, the heteroscedastic GP and the heteroscedastic VBLL networks correctly attributed error from the injected noise as aleatoric, while QRF read that error as epistemic. The success of a split is also dependent on the representation, which is not necessarily the case for predictive performance or noise robustness. The aleatoric component rises by roughly 0.4 to 1 unit for each unit of noise added, so it can compare noise between datasets or between noise levels, but it does not measure a dataset's internal error. Outside censoring, no model's per-sample uncertainty identified which labels had been corrupted. Under censoring the models' most confident predictions are the clipped labels themselves, and dividing a molecule's error by its uncertainty picked out the clipped labels better than the error alone.

QM9 served as a relatively clean baseline as its properties are computed. The assay datasets containing experimental measurements on the endpoints LogD, Caco-2 [110], and hERG K_i [111] are smaller and cover a more narrow range of the chemical space. As the labels in these data are experimentally-derived and pooled from different laboratories, they contain some unknown amount of noise. We are then adding artificial noise on top of the existing experimental noise. These validation datasets supported the QM9 findings for the forests, with the boosted trees XGBoost and LightGBM the notable exceptions, degrading on the external datasets. Future work should extend this

type of noise robustness benchmarking to classification tasks and larger experimental datasets with better-known uncertainty estimates.

When training on noisy experimental data in the context of QSAR, it is important to keep in mind that a model's robustness to noise is primarily driven by its own mechanism of training, and to an extent the molecular representation the input data comes in, oftentimes with respect to the model. Of the models tested, RF was the most consistent choice across the four datasets, ranking between first and eighth for $\mathrm{AUC}_{\mathrm{norm}}$ on every dataset, representation and noise condition. Where a predictive distribution is wanted, its probabilistic counterpart QRF maintains RF's predictive performance and nearly all of its noise robustness. What QRF cannot do is say where that uncertainty came from. Beyond the forests, the better choice depends on how much noise there is. LightGBM and XGBoost predict well on clean labels and lose the most as noise rises, so they suit data you trust, while NGBoost and the VBLL networks start lower and hold what they have, except the VBLL networks on LogD.

In this research, we define a noise level as a fraction of the spread of the clean training labels. It is therefore relative to the dataset rather than an absolute measurement error, and cannot be mapped onto a given dataset exactly. There are other mechanisms used to estimate the noise in a dataset directly. When an assay's controls are available, the Z′-factor can be used to estimate noise [86]. You can also use screening data to test for row, column and plate effects, which produce systematic rather than independent error [122]. When you lack controls, replicates, and metadata, a label can be checked against the data itself. Confident learning, developed for classification, flags labels by comparing each label to the class probabilities a model predicts for that molecule when it was left out of training [52]. You can use the point at which the model error matches the data's error to generate a predictive limit [10, 123]. Beyond formal methods, you should consider how a dataset was assembled. How many independent replicates were done, how many laboratories contributed, whether assay protocols were consistently applied, and how varied are the compounds in chemical space?

The first limitation of this study is data availability. While HOMO–LUMO gap from QM9 is a good baseline as its computed labels carry no measurement error, it is only a single task. As the noise is meant to replicate experimental data, this analysis would be improved by more quality-controlled, relatively clean and well-replicated experimental data across different targets. However, data of that nature are hard to find publicly.

Second, $\mathrm{AUC}_{\mathrm{norm}}$ is a ratio to clean R^2, so a model that starts from a low clean baseline has less to lose, and a few replicates scored above 1.05. We tried other measures of noise robustness, but they were either too strongly correlated with clean R^2 or did

not capture the right quantity. The change in R^2 at a single noise level, for example, misses the shape of the curve. $\mathrm{AUC}_{\mathrm{norm}}$ is not a perfect metric, but it describes the whole curve, and across the 13 base models it is barely correlated with clean R^2.

Third, it is hard to draw conclusions that hold across so many conditions. No model or representation wins everywhere, although RF comes closest. When working with cheminformatics data, you don't know in advance how a model and representation will perform on the given prediction task. The underlying data is probably noisy, but you don't know how noisy it is or what shape that noise takes.

Given that, our recommendation is to test a handful of models against a handful of representations rather than tuning a single pairing, since on every dataset the pairing explains more of the variance in robustness than the representation does alone. The pattern of the noise should also guide that choice. Under Laplace, Student-t and outlier noise, RF was usually the most robust model and SVM usually ranked second on the assay datasets. NGBoost lost $\mathrm{AUC}_{\mathrm{norm}}$ under all three, except Laplace noise on LogD. Under grouped-wider noise, QRF was the most robust model and SVM was close behind. NGBoost lost $\mathrm{AUC}_{\mathrm{norm}}$ on every assay dataset. Grouped-shifted noise was the hardest condition. Every model lost $\mathrm{AUC}_{\mathrm{norm}}$ on every dataset and representation. RF lost the least of the models that also predicted well, and on the HOMO–LUMO gap the plain networks lost the most. Gaussian noise was the only condition with a clear specialist. NGBoost was the most robust model on the HOMO–LUMO gap under Gaussian noise with every representation, but not on Caco-2 or hERG K_i.

These results follow from the loss function each model fits. NGBoost fits a per-molecule scale under a Gaussian likelihood, so it loses ground when the noise is not Gaussian. SVM's loss grows only linearly outside its margin, so large errors cost it less, and it gains under the same conditions. Each boosting stage fits the residuals of the stage before it, and those residuals are largest in the noisy families. The GP is the most affected by error that is correlated with structure, because its kernel measures structure. None of this matters if the data are clean, as on the HOMO–LUMO gap and LogD little separated the models below a noise level of 0.3.

In this chapter each model was given a standard molecular representation, and the choice of representation explained relatively little of the variance in noise robustness. We observe that this choice does impact the performance of different models to varying degrees, and the representations seem to capture different signals on each dataset that we tested. Chapter 4 asks how well these representations compare molecules on a single dataset.

Chapter 4

Limitations of the similarity assumption

4.1 Introduction

The previous chapters have been focused on the impact of label noise. Before we delve into techniques that are meant to train on, identify, and mitigate such label noise, we first focus on a tangential point, molecular similarity. All of the aforementioned noise-adjacent methods, as well as the QSAR modeling itself, rest on the similar property principle (SPP; Section 2.4), which posits that structurally similar compounds display similar chemical properties [73]. This serves as the conceptual foundation for QSAR modeling. The concept of molecular similarity is also used in various contexts to compare neighboring molecules. This comes into play when identifying noisy molecules that in theory stick out from their neighbors. There are some important exceptions to the SPP, namely activity cliffs (ACs) [58, 77]. Furthermore, there is no single way to define similarity between compounds. This is where the principle begins to break down in practice. When defining similarity between two molecules, we typically rely on some pairwise mathematical function, or coefficient, that takes in two molecular representations as input. Two factors vary here, the representation and the coefficient. Bajusz et al. [75] compared eight coefficients and found Tanimoto, Dice, Cosine and Soergel the best and largely equivalent, while similarities derived from Euclidean and Manhattan distances were not recommended on their own. In this chapter we ask if the standard methods of comparing molecules satisfy the SPP, that is, whether they quantify how similar two molecules actually are.

General-purpose molecular representations are designed to work across chemical domains. Consequently, when considering data from a specific domain, many of their

features show little to no variance. Some representations assign separate molecules identical feature vectors. Since common fingerprints do not encode stereochemistry [124], they are often unable to differentiate stereoisomers, which can have substantially different measured activity [125]. As these coefficients compare every feature, irrelevant features dilute the result. Different representations also disagree about which molecules are close based on the information they carry.

The problem of whether neighboring molecules can predict chemical activity has been measured before. The modelability index of Golbraikh et al. [126] scores a binary dataset by a class-weighted share of nearest-neighbor pairs that share an activity class, producing a single value for how strongly the similarity assumption holds on that particular combination of representation and data. It is not clear whether a low value is due to the data being intrinsically hard to model or to the representation not capturing adequate features for the task. Other work in this area relates to activity cliffs. Sheridan et al. [127] compared experimental error, activity distribution and activity-cliff metrics as predictors of how well a dataset can be modeled, and found the cliff metrics the strongest of the three. However, they found that these metrics cannot separate real cliffs from those produced by uncertain activity measurements. Medina-Franco [128] points out that some of these activity cliffs are artifacts of the chosen representation, similarity threshold, or noise in the data. Activity cliffs are distinct from noise, as they carry some of the most valuable information about the interaction between structure and activity. None of these measures ask specifically about whether the specific features within these representations matter for the property at hand.

We first consider how much the representation impacts similarity measures. Riniker and Landrum [129] benchmarked 12 fingerprints and two simple baseline fingerprints for similarity-based virtual screening on 88 targets from three public collections, and found that the screening performance of the 12 fingerprints was highly correlated. Only the two baselines differed significantly from the rest. Nearly all of 25 pretrained neural embeddings were found to have a negligible improvement over ECFP across 25 datasets [130]. However, when looking into the differences between the structure of fingerprints, Huber and Pollmann [131] found that fingerprints which store raw counts are more effective at defining similarity than folded fingerprints when compared against a graph-based structural reference. Folding introduces bit collisions, which disproportionately distort similarity for molecules with many set bits [131]. ACs follow the same pattern: across 15 configurations of representation and similarity measures on 3 datasets, the representations disagreed about which pairs form cliffs [132], supporting the point Medina-Franco [128] makes about ACs being artifacts of their representation.

Any method that incorporates similarity measures, such as AC detection, or cleaning a training set by comparing each sample's label with its nearest neighbors' labels, whether by removing the samples that disagree [102] or by smoothing their labels, inherits this choice, as do the noise estimation and mitigation methods of this thesis. Coupry and Pogány [133] learn a molecular similarity from graph embeddings, but independently of any endpoint. Zheng and Tropsha [134] did choose the similarity for the task. They predict each molecule's activity as the average over its k most similar molecules by Euclidean distance over descriptors, and choose the descriptors by simulated annealing to maximize the leave-one-out q^2. Itskowitz and Tropsha [135] also optimize the number of neighbors and their weighting. Both select features for a nearest-neighbor model of one endpoint, whereas this thesis selects features across several representations.

The purpose of this chapter is to establish the problem that Chapter 5 sets out to solve. Section 4.3.1 asks, for a given dataset, how much the features of standard representations vary and how many molecules map to identical feature vectors. Section 4.3.2 asks what happens when these features are used to quantify similarity, and Section 4.3.3 follows the consequences for methods built on these similarity measures, including a small sweep of published noise-handling methods.

4.2 Methods

4.2.1 Distance metrics for continuous molecular descriptors

While binary fingerprints are compared with the Tanimoto coefficient (Chapter 2), we need distance metrics for continuous feature vectors. The small sweep of published noise-handling methods (Section 4.3.3) used the following four:

- Euclidean (L_2): standard geometric distance, $d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$
- Manhattan (L_1): sum of absolute differences, $d(x, y) = \sum_i |x_i - y_i|$, more robust to outlier features
- Cosine: angle-based, $d(x, y) = 1 - \frac{x \cdot y}{\|x\| \|y\|}$, invariant to magnitude
- Mahalanobis: covariance-adjusted distance, $d(x, y) = \sqrt{(x - y)^\top S^{-1} (x - y)}$, where S is the feature covariance estimated on the training molecules [136]

All features are standardized to zero mean and unit variance prior to distance computation.

4.3 Results

4.3.1 Flaws in standard representations

General-purpose representations are built to span a broad range of chemical domains. When a dataset covers only a small region of the chemical space, a lot of the features within these generalized representations become irrelevant, describing chemistry which is absent from the data at hand. Pairwise comparisons of these representations rely on these irrelevant features.

Constant and near-constant features

To investigate similarity estimation, we first look at a simple measure: how many features vary within a dataset. A single dataset often contains a set of relatively similar molecules. Many features within a generalized representation contain information that describes all, or almost all, of the molecules in the set, failing to capture the variation between them. We define a constant feature to be one that takes a single value in every molecule of a dataset and a near-constant feature to be one that carries the same value in at least 95% of the molecules. We counted constant and near-constant features from a set of representations (described in Table 5.12) across hERG K_i, Caco-2, LogD, QM9, MDR1–MDCK efflux, human liver microsome (HLM) clearance, rat liver microsome (RLM) clearance and the Lhasa hERG blocker training set. MDR1–MDCK efflux and the two clearance datasets come from Biogen [137]. HLM clearance and the Lhasa hERG set are not in Table 5.12; their base representations are those with counts in Additional file 12.

The substructure-based fingerprints have the most near-constant features. The learned embeddings, which are taken from the last layers of a NN, have none except MHG-GNN (Additional file 12, Figure 4.1). ECFP4 is the worst offender. On hERG K_i (1,412 molecules) 1,815 of the 2,048 features are near-constant. That jumps to 1,997 out of 2,048 near-constant features for the HOMO–LUMO gap on QM9. 292 of MHG-GNN's 1,024 features are zero in every molecule across all seven datasets that used it. They come directly from the saved encoder output rather than the data, and the cause was not investigated.

Some distances ignore constant features. Tanimoto similarity (Equation 2.3; the distance is $1 - T$) ignores bits off in both molecules, but a bit on in every molecule raises every molecule's measured similarity. It does not use the 105 constant ECFP4 bits on hERG K_i. Constant features add zero to a Euclidean or Manhattan distance;

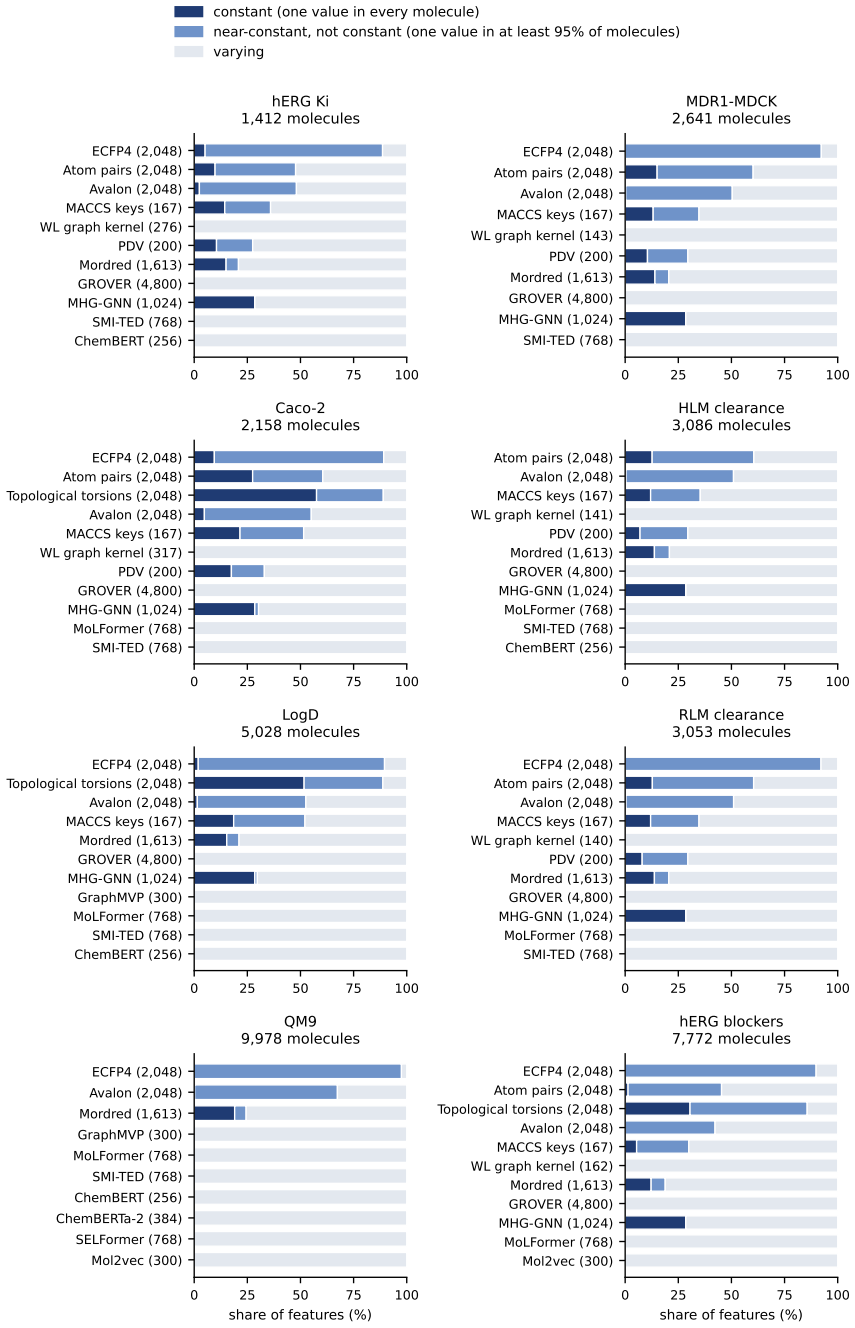


Figure 4.1: Shares of constant (dark blue), near-constant (mid blue) and varying (gray) features of each representation on eight datasets. The number of features of each representation is in parentheses.

however, near-constant features are amplified. That is useful if those features define an AC. If not, they are contributing disproportionately to the measure of similarity.

Molecules with identical feature vectors

A standard fingerprint- or descriptor-based representation can produce the same feature vector for two separate molecules. In turn, models using this representation will

produce a single prediction for the molecules with identical features. Identical feature vectors do not always mean that the representation has merged two different molecules, since a dataset pooled from several sources can hold two measurements of the same molecule.

Many fingerprints are computed without stereochemistry [124]. In this research, ECFP4 is built with stereochemistry off, and the 2D descriptor sets (PDV, Mordred) also ignore it, so almost every molecule they produce identical representations for is a stereoisomer of another (Table 4.1). Any representation that reads from these subsequently merges stereoisomers. Additionally, methods using the Weisfeiler–Lehman relabeling scheme are bounded by the 1-Weisfeiler–Lehman test (Chapter 2), so some non-isomorphic graphs also receive the same vector. This is seen in the graph-kernel features which tie 105 of 1,412 hERG K_i molecules, 56 of which differ only in stereochemistry and 49 in more than that.

Whether or not a learned embedding is able to differentiate stereoisomers is dependent on its input as well as its individual implementation. SMI-TED, a pretrained SMILES encoder, ties 56 hERG K_i, 633 LogD and 177 Caco-2 molecules, all stereoisomers. A tokenizer can drop stereochemistry prior to training. The tokenizer of the ChemBERTa-77M-MTR checkpoint (ChemBERTa-2 in Additional file 12, called ChemBERTa in Chapter 3) gives 72 of the 1,415 hERG K_i molecules of Chapter 3 the same token sequence as another molecule. Since stereoisomers can differ in measured activity [125], these representations lack potentially useful chemical information.

Two molecules are tied when a representation gives them exactly the same feature vector. Table 4.1 counts ties on hERG K_i, LogD and Caco-2 for each dataset's base representations and for its augmentation features, the small feature sets added to the hybrid after it is built (Section 5.2.3). No tie came from one compound listed twice (checked by InChIKey, which differs between stereoisomers) or from a row of zeros.

Apart from the MACCS keys, most tied molecules are stereoisomers. Atom pairs, PDV, Mordred and SMI-TED tie only stereoisomers, and ECFP4, Avalon and topological torsions add a few molecules that differ in more than their stereochemistry. On hERG K_i, ECFP4, atom pairs, PDV, Mordred and SMI-TED tie the same 56 molecules, so joining any of these five representations leaves all 56 tied. The WL graph kernel also ties 49 hERG K_i and 26 Caco-2 molecules that are not stereoisomers, and the MACCS keys tie the most molecules on every dataset, mostly not stereoisomers. The 3D augmentation features and the learned embeddings ChemBERT, GROVER, MHG-GNN, MoLFormer and GraphMVP tie none, and joining all of a dataset's representations leaves no molecule tied.

A model has to give every molecule in a tied group the same prediction. The best single prediction for a group is its mean label, so ties set an error that no model can go below,

$$\mathrm{RMSE}_{\min} = \sqrt{\frac{1}{n}\sum_{G}\sum_{i \in G}(y_i - \bar{y}_G)^2}, \tag{4.1}$$

where n is the number of molecules, G runs over the tied groups and $\bar{y}_G$ is the mean label of group G. The matching best R^2 is $1 - \mathrm{RMSE}^2_{\min}/\mathrm{Var}(y)$. On hERG K_i this lowest error is at most 0.134 pK_i (MACCS keys), small next to the label standard deviation of 0.915 pK_i.

Single groups can still disagree by a lot. Independent pK_i measurements of the same compound against the same target differ by 0.44 log units on average, with a standard deviation of 0.54 [12], so a gap larger than 0.54 inside a tied group is larger than typical measurement noise. On hERG K_i, 6 of the 26 ECFP4 groups and 38 of the 153 MACCS groups differ by more than 0.54. The widest ECFP4 group is a pair of stereoisomers 1.14 pK_i apart (Figure 4.2). An ECFP4 model can at best predict their mean, 5.59 pK_i, which leaves each molecule 0.57 pK_i from its measured value.

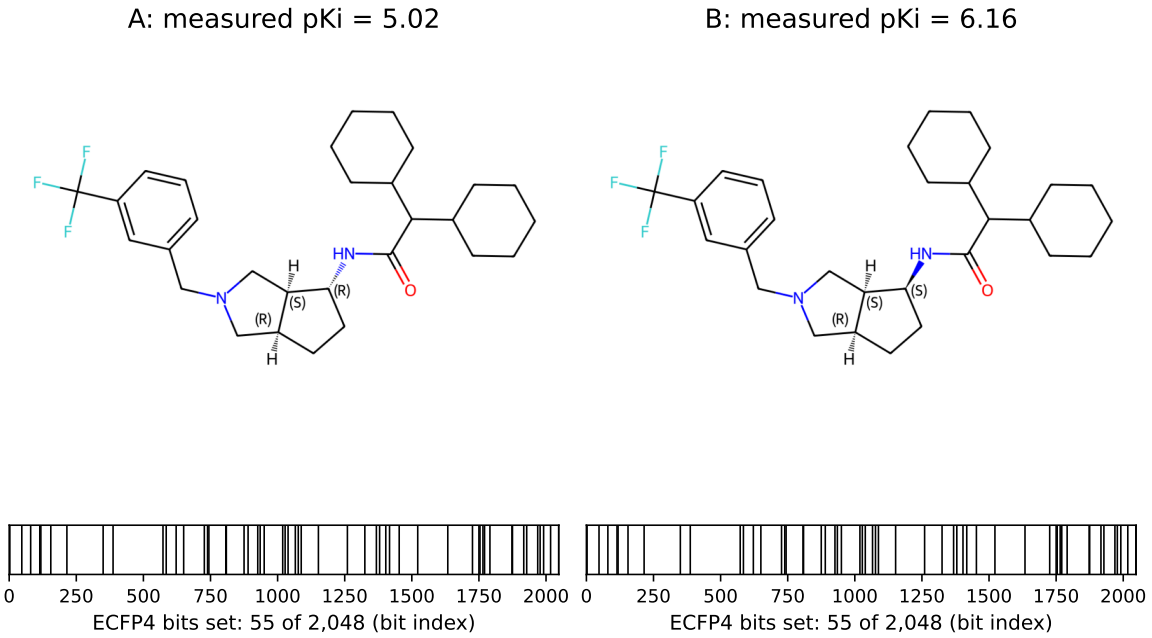


Figure 4.2: Two hERG K_i stereoisomers with identical ECFP4 vectors, which differ at the carbon bearing the amide nitrogen (R in A, S in B). The top row shows the structures with their measured pK_i values, and the bottom row shows the 55 of 2,048 ECFP4 bits set in both.

A tied pair with different labels is also the most extreme activity cliff possible. The

structure–activity landscape index of molecules i and j,

$$\mathrm{SALI}_{ij} = \frac{|y_i - y_j|}{1 - \mathrm{sim}(i, j)} \tag{4.2}$$

[138], is infinite when their similarity is 1 and their labels differ, and here the cliff is made by the representation rather than the chemistry. A method that judges a label by its nearest neighbors sees a tied neighbor at distance 0 with a different label, and may flag a correct label as noise (Section 4.3.3).

Dataset	Representation	Molecules	Tied	Stereo-only	Best RMSE	Best R^2	Largest spread
hERG K_i (pK_i)	ECFP4, Atom pairs, PDV, Mordred, SMI-TED	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Avalon	1,412	66 (4.7%)	56	0.049	0.997	1.20
	WL graph kernel	1,412	105 (7.4%)	56	0.071	0.994	1.49
	MACCS keys (aug.)	1,412	377 (26.7%)	30	0.134	0.978	1.80
LogD (log units)	ECFP4	5,028	641 (12.7%)	633	0.044	0.999	1.00
	Mordred, SMI-TED	5,028	633 (12.6%)	633	0.043	0.999	1.00
	Avalon	5,028	647 (12.9%)	629	0.047	0.998	1.40
	Topological torsions	5,028	675 (13.4%)	631	0.047	0.998	1.10
	MACCS keys (aug.)	5,028	2,343 (46.6%)	437	0.220	0.965	2.80
Caco-2 (log units)	ECFP4	2,158	179 (8.3%)	177	0.027	0.996	0.74
	Atom pairs, PDV, SMI-TED	2,158	177 (8.2%)	177	0.027	0.996	0.74
	Avalon	2,158	181 (8.4%)	177	0.027	0.996	0.74
	Topological torsions	2,158	186 (8.6%)	177	0.029	0.996	0.74
	WL graph kernel	2,158	203 (9.4%)	177	0.028	0.996	0.74
	MACCS keys (aug.)	2,158	804 (37.3%)	139	0.111	0.935	1.72

Table 4.1: Molecules that share their feature vector with at least one other molecule, one row per representation that ties any molecules; representations with identical entries share a row. Tied gives the number and share of such molecules, and Stereo-only those tied only with their own stereoisomers. Best RMSE and best R^2 are the best any model can reach given the ties. Best RMSE and Largest spread, the largest label range within one tied group, are in the units beside the dataset name, and (aug.) marks an augmentation representation.

4.3.2 Flaws in similarity and other pairwise computations

Section 4.3.1 described some of the limitations of standard representations. Here we ask what happens when two molecules are compared through one of them, as these pairwise comparisons (Chapter 2) can violate the SPP.

The Tanimoto coefficient is the most popular similarity metric in cheminformatics [139], and is commonly computed on ECFPs, although the coefficient applies to any bit vector. It has been described as the "coefficient of choice" for molecular similarity [140], while the Morgan fingerprint, ECFP4 in this research, ranks among the strongest

fingerprints for virtual screening and target prediction [141]. ECFP4 and ECFP6 specifically, alongside topological torsion, rank among the best of 28 fingerprints tested for ordering diverse structures by similarity [142]. However, it is not a one-size-fits-all solution. Across 96,456 human metabolites, 72.9% had an ECFP4 nearest neighbor at Tanimoto distance 0 [141]. Stereochemistry had been removed before that comparison, so these are constitutionally different molecules that ECFP4 cannot tell apart.

A similarity coefficient cannot tell you which features matter to the prediction task at hand. And the SPP holds only relative to such a measure. To understand the impact of irrelevant features on similarity measures, we make the following computation. First, we categorize a representation's features into those relevant to the task, R, and those irrelevant to the task, I, and take the squared Euclidean distance as a sum over both:

$$d^2(\mathbf{x}, \mathbf{y}) = \sum_{j \in R} (x_j - y_j)^2 + \sum_{j \in I} (x_j - y_j)^2. \tag{4.3}$$

Every non-constant irrelevant feature adds one further term, so the irrelevant portion grows linearly with $|I|$. After standardization each feature contributes the same amount on average over all pairs, so the share of the distance carrying signal is about $|R|/(|R|+|I|)$. When we use a general-purpose representation like ECFP4 on a dataset spanning a small portion of the chemical space, $|I|$ can be a lot larger than $|R|$. Constant features contribute nothing, while near-constant features have a strong impact. The worst-case scenario involves a single rare bit set in only a handful of molecules. After standardization, uncommon bits take on larger values, outweighing common features that may be more relevant. If this bit is irrelevant to the task at hand, the similarity measure suffers.

Larger representations compound the problem. As the number of features grows, the nearest and farthest molecules end up at nearly the same distance, meaning that these pairwise distances are less able to discriminate, as explained in Chapter 2. Two measures of this effect are the relative contrast, defined as the difference between the largest and smallest measured distance divided by the smallest [143], and hubness, a measure of how unevenly the molecules appear in other molecules' nearest neighbors [144]. These errors propagate along larger data pipelines, starting from the representation into similarity, from similarity into the set of neighbors a molecule is compared against, and from there into a prediction of some kind.

The four measurements below use the six small-molecule regression datasets of Table 5.12 (Chapter 5) and HLM clearance. Binary representations were compared by Tanimoto distance, and all others by Manhattan distance on standardized features

(Section 4.2), with constant features removed.

Activity cliffs

Chapter 2 gives the MoleculeACE definition of an activity cliff, which combines three similarity measures. Here we use one. Two molecules are similar if their ECFP4 Tanimoto similarity is at least 0.55 [145], and a molecule is a cliff if a similar molecule has a label that differs from its own by more than a set gap. We chose the gap on each dataset so that 15% of the molecules are cliffs, so the gap follows the spread of the labels (Table 4.2). On hERG K_i the gap is 1.76 pK_i, and 211 of the 1,412 molecules are cliffs. On QM9 only 1,093 pairs of the 9,978 molecules are similar, and the gap is 0.05 eV, 0.06 times the median absolute deviation of the labels. Because the gap is set by a share of molecules, on QM9 a cliff marks a small label difference, not a cliff in the usual sense. On the three Biogen datasets (RLM clearance, MDR1–MDCK efflux and HLM clearance) it is 0.20 to 0.34 log units, 0.50 to 0.60 times the median absolute deviation.

The count of cliffs depends on the representation. We kept each dataset's gap and replaced ECFP4 with each other base representation, calling similar the same number of closest pairs as ECFP4 does. On hERG K_i the number of cliff molecules then ranges from 138 (atom pairs) to 267 (WL graph kernel). Of the pairs that ECFP4 calls similar, only 57% to 74% are also among another representation's closest pairs, and on QM9 only 1% to 24% (Section S.3 of the Additional files).

Distance and label difference

We asked whether distance tracks label difference in two ways. Over all pairs of molecules, we took the Spearman correlation between the distance of a pair and the absolute difference of its labels, a one-number summary of the neighborhood-behavior plot of Patterson et al. [146]; datasets above 2,000 molecules were sampled down to 2,000. Over each molecule's closest pairs only, we predicted each label as the mean label of the molecule's five nearest neighbors, taken from other scaffold groups, and took the Spearman correlation between predicted and measured labels.

Across the 70 combinations of dataset and representation, the all-pairs correlation ranged from -0.04 to 0.23 and was below 0.10 in 44 of them. The five-neighbor prediction, a correlation between predicted and measured labels, ranged from 0.29 to 0.89 (Table 4.2; every representation is in Section S.3 of the Additional files). On hERG K_i with ECFP4, the all-pairs correlation is 0.06, while the five-neighbor prediction

reaches 0.72, with an RMSE of 0.63 pK_i against a label standard deviation of 0.92 pK_i.

Figure 4.3 shows why the two differ. Of the 996,166 hERG K_i pairs, 93% have an ECFP4 Tanimoto similarity below 0.20, and only 1.1% reach 0.55. Over all pairs the median label difference is 0.81 pK_i, but above a similarity of about 0.4 it is 0.20 to 0.30 pK_i, although 8.3% of the 774 pairs at 0.8 or above still differ by more than 1 pK_i. Mordred shows the same pattern: the closest 5% of pairs have a median difference of 0.44 pK_i, and every other 5% has 0.78 to 1.11 pK_i. The useful information sits in the few closest pairs. The all-pairs correlation is set by the other 95%, whereas the five-neighbor prediction uses only the closest pairs.

Agreement on neighbors

We compared the five nearest neighbors of each molecule, from other scaffold groups, under ECFP4, SMI-TED and Mordred (PDV on Caco-2, where Mordred is not a base representation; on HLM clearance ECFP4 is not a base representation and its stored features were used). On hERG K_i any two of the three share on average 0.47 to 0.52 of each molecule's five neighbors, and on the three Biogen datasets and QM9 only 0.13 to 0.21 (Table 4.3).

Agreement with shared structure

For 500 random pairs per dataset we found the maximum common substructure (MCS) with RDKit [32], allowing 2 seconds per pair and leaving out pairs that ran out of time. The MCS distance is $1 - n/(a + b - n)$, where n is the number of atoms in the common substructure and a and b are the numbers of heavy atoms in the two molecules. Its Spearman correlation with ECFP4 Tanimoto distance ranged from 0.20 (hERG K_i and HLM clearance) to 0.51 (Caco-2 and LogD).

Every method in Section 4.3.3 that takes a similarity as input inherits the dilution described here.

4.3.3 Consequences for similarity-based methods

We have identified some issues within the representations and similarities computed over them. While this calls into question the predictive limits of QSAR and other structure-based modeling, it also calls into question downstream applications of molecular similarity. Nearest-neighbor methods, where a molecule's neighbors, as measured by similarity, are the inputs to several other processes, are by definition sensitive to the

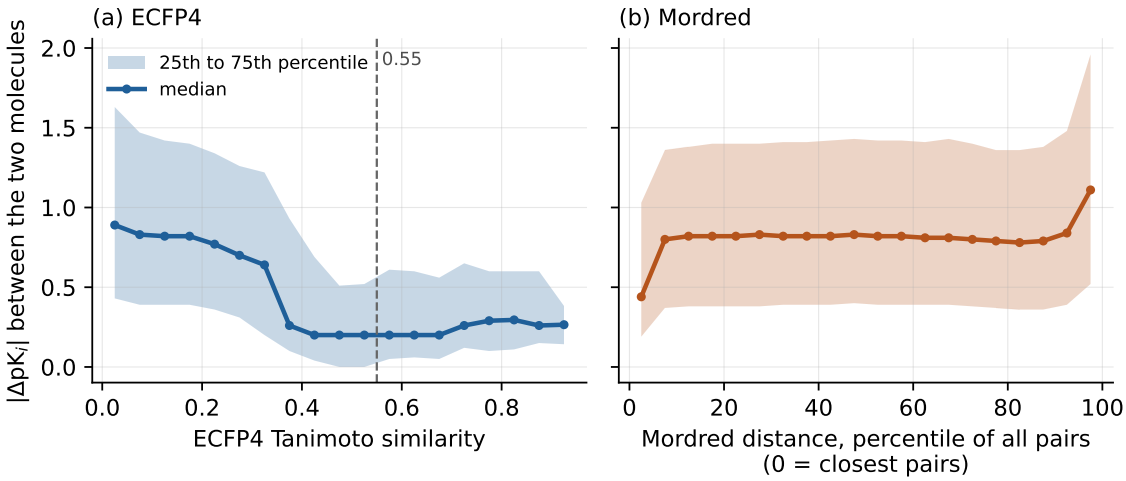


Figure 4.3: Absolute pK_i difference against similarity or distance for all pairs of hERG K_i molecules, with the median in each bin (markers) and the interquartile range (shaded). a) ECFP4 Tanimoto similarity in bins of 0.05 (bins with fewer than 20 pairs omitted), with the activity-cliff threshold of 0.55 dashed. b) Manhattan distance on standardized Mordred features, as a percentile of all pairs.

		All pairs			Five neighbors			Cliffs	
Dataset	Molecules	ECFP4	Desc.	SMI-TED	ECFP4	Desc.	SMI-TED	Molecules	Gap
hERG K_i	1,412	0.06	0.07	0.09	0.72	0.72	0.69	211	1.76 pK_i
LogD	5,028	0.12	0.06	0.10	0.76	0.80	0.76	731	2.90 log
Caco-2	2,158	0.08	0.18	0.14	0.59	0.62	0.57	324	1.21 log
RLM clearance	3,053	0.02	0.07	0.04	0.36	0.49	0.46	458	0.34 log
MDR1–MDCK efflux	2,641	0.02	0.16	0.10	0.51	0.58	0.59	397	0.20 log
HLM clearance	3,086	0.00†	0.07	0.03	0.37†	0.49	0.45	462	0.27 log
QM9	9,978	0.19	0.18	0.13	0.88	0.84	0.85	1,495	0.05 eV

Table 4.2: All-pairs correlation, five-nearest-neighbor prediction and activity cliffs for each dataset. All pairs gives the Spearman correlation between pair distance and absolute label difference, and Five neighbors the Spearman correlation between each label and the mean label of its five nearest neighbors. Cliffs gives the number of cliff molecules and the label gap that defines a cliff. Desc. is Mordred, or PDV on Caco-2, and † marks ECFP4 where it is not a base representation of the dataset.

quality of similarity measurements. They are used within other processes like predicting a molecule's activity using its neighbors' labels, detecting outliers, either ACs or noise, defining an applicability domain by distance to the training set [147], grouping molecules into clusters for data splits [148], identifying activity cliffs, and, in federated learning, weighting each site's vote on a molecule by that molecule's distance to the site's own data [149]. If this measure of similarity, or equivalently distance, is flawed, then all of these downstream applications will feel the effects of it.

Dataset	Desc.	Five-neighbor overlap			ECFP4 against MCS		
		ECFP4–Desc.	ECFP4–SMI-TED	Desc.–SMI-TED	Spearman	Pairs	Timed out
hERG K_i	Mordred	0.52	0.50	0.47	0.20	495	5
LogD	Mordred	0.38	0.41	0.41	0.51	494	6
Caco-2	PDV	0.38	0.41	0.38	0.51	492	8
RLM clearance	Mordred	0.15	0.21	0.18	0.28	500	0
MDR1–MDCK efflux	Mordred	0.15	0.20	0.17	0.26	500	0
HLM clearance	Mordred	0.15	0.21	0.18	0.20	500	0
QM9	Mordred	0.13	0.18	0.21	0.46	500	0

Table 4.3: Agreement between representations for each dataset. Five-neighbor overlap is the mean fraction of a molecule's five nearest neighbors that two representations share. ECFP4 against MCS gives the Spearman correlation between ECFP4 Tanimoto distance and MCS distance, the number of pairs used, and the number left out because the MCS search timed out.

"A small sweep of published noise-handling methods" reports a sweep in which the 20 configurations with the lowest retention under noise all relied on a pairwise distance measurement somewhere in their process. "Noise detection by neighbors" focuses directly on neighbor-based noise detection (Table 2.3), the most clear-cut use of similarity.

A small sweep of published noise-handling methods

In Chapter 3, model family explained the largest share of variance in noise robustness, and representation at most 8.1% on QM9 (Table 3.1). Earlier in this project we therefore ran a small sweep of published noise-handling methods, to ask whether training methods built for noisy labels improve robustness and whether representation still matters under them.

The three groups of methods for noisy labels and the five categories of Song et al. [51] are described in Section 2.5.2, and such methods have been studied across machine learning, including QSAR. Most methods in the LNL literature were built for image classification with flipped labels [51], so they do not transfer automatically to continuous labels.

The study tested 12 published noise-handling methods on the QM9 HOMO–LUMO gap (10,000 molecules, scaffold splits), adding Gaussian noise with σ from 0 to 1.0 in steps of 0.1 to the training labels only. The representations were ECFP4, Sort & Slice [7], PDV, canonical SMILES and randomized SMILES. Depending on the method, a configuration could also add the model's own uncertainty or a distance between

molecules (Tanimoto, Euclidean, Manhattan, cosine or Mahalanobis). Others added an alternative GP kernel or loss function, for 221 configurations in all. Four methods detect suspected molecules, remove them and retrain (noise mitigation): small-loss selection, confident learning, distance-based selection and multistage cleaning. The other eight change the training (noise-robust training).

We measured noise robustness by retention, R^2 under noise divided by R^2 with clean labels, as a percentage: the retention ratio of Chapter 3 (Equation 3.2). The study predates $\mathrm{AUC}_{\mathrm{norm}}$, the area under the retention curve. Retention correlates more closely with clean R^2, so it separates robustness from accuracy less well, but it is a good enough proxy for comparing methods with and without a distance. In an analysis of variance over the adapted components, using a distance explained the most variance in R^2. All 20 configurations with the lowest retention used a distance, and none with a distance reached both a clean R^2 above 0.6 and a retention above 80%.

The methods that held up estimate noise from the model's own loss or uncertainty. DivideMix [150] retained the most, 86.7% (standard deviation 0.4%), followed by small-loss selection [151] at 84.2% (2.1%) and the uncertainty curriculum, which orders molecules by the model's own loss [152, 153], at 82.7% (3.5%). The heteroscedastic GP, a GP whose noise is predicted per molecule by a second model [154], retained 79.0% at $\sigma = 0.6$, averaged over 15 configurations, with R^2 of 0.71 on clean labels and 0.56 at that level. The exploratory study does not record the noise level for the other methods, so this value cannot be ranked against theirs. Regularization alone retained less, 73.0% for Mixup and 74.0% for sharpness-aware minimization. Under these methods, the choice of representation explained even less variance than in Chapter 3.

The study added Gaussian noise to every training label of a single dataset and property, the QM9 HOMO–LUMO gap. It also added a distance to methods designed without one, so poor adaptation is an alternative explanation for their low retention. Chapter 7 runs a new implementation of the same 12 methods on more datasets, on hybrids and under the other noise conditions of Section 2.5.1 (Section 7.3.8).

The distances may have failed for two reasons, which the evidence here does not separate. First, a single representation may lack the information a distance needs, with and possibly even without noise. PDVs encode global molecular properties and ECFP4 contains substructure information, and any one of these sources alone may not be sufficient for robust similarity estimation. Second, some representations are sparse, with many features irrelevant to the molecules of a given dataset (Section 4.3.2). We sought to address the second with the adaptive hybrid representation (Chapter 5), which concatenates the top-ranked features by importance from several base representations.

Noise detection by neighbors

Neighbor-based noise detection treats a label far from its neighbors' labels as probably wrong, an idea that goes back to editing nearest-neighbor training sets [102]. The wider literature on filtering noisy labels concerns classification [155, 156], but Martín et al. [157] adapt fourteen classification noise filters to regression, seven of them nearest-neighbor filters that use the neighbors' mean label. They find that the filters improve regression methods on 20 general regression benchmarks with injected label noise, none of them QSAR datasets. The neighbor-based noise scores, sample removal, sample weighting, label smoothing and the recovery rate are defined in Chapter 7 (Sections 7.2.2 and 7.2.3, Equation 7.6).

The method needs a distance that picks the right neighbors, and neighbors with clean labels. Earlier sections of this chapter argue that the first condition fails, and the second fails whenever noise is spread over the whole training set. A large gap from the neighbors' labels can mean a noisy label, a noisy neighbor or the wrong neighbors, and the score cannot tell which. Activity cliffs also look like noise to such a detector (Chapter 2, "Molecular similarity and activity cliffs").

The exploratory study also used neighbors to clean labels, on 5,000 randomly chosen QM9 molecules (HOMO–LUMO gap, scaffold split) with Gaussian noise on the training labels at $\sigma = 0.3$ and $\sigma = 0.6$. Each cleaning method was judged by retraining a random forest on ECFP4 with clean, noisy and cleaned labels, giving the recovery rate, the share of the R^2 lost to noise that cleaning recovers (Equation 7.6). Neighbor-based scores compared each label with those of the molecule's k nearest training molecules under Euclidean, Manhattan or cosine distance. Table 4.4 holds only the configurations built on a single representation. This study is separate from the sample-removal study of Chapter 7.

With the neighbor-based scores used as sample weights at strength $\alpha = 0.1$, mean recovery was negative for every representation at both noise levels (Table 4.4). Of all 860 cleaning configurations, 8 had positive recovery at both noise levels; 6 of these used a single representation, and 4 of those 6 were neighbor-based. Label smoothing toward neighbor or model estimates gave positive recovery in 5.0% of configurations at $\sigma = 0.3$ and 0.8% at $\sigma = 0.6$. Mean recovery was negative at every weighting and smoothing strength tried, from 0.1 to 0.7.

Judging a label by its neighbors usually made the model worse and rarely helped at both noise levels, but it did not always fail. The best configuration at $\sigma = 0.6$ was neighbor-based (PDV, Euclidean distance, local variance ratio score) and recovered

	Mean recovery (%)		Positive (% of configurations)	
Representation	$\sigma = 0.3$	$\sigma = 0.6$	$\sigma = 0.3$	$\sigma = 0.6$
ECFP4	−3.0	−7.3	33.3	16.7
PDV	−2.4	−3.4	50.0	27.8
MHG-GNN	−0.1	−7.2	38.9	5.6

Table 4.4: Recovery rate in R^2 on the QM9 HOMO–LUMO gap when neighbor-based noise scores are used as sample weights, for each representation used to find the neighbors, at two levels σ of Gaussian label noise. Recovery is the share of the R^2 lost to noise that cleaning regains, so negative values mean cleaning made the model worse. Positive is the share of configurations with recovery above zero.

+17.2%. Because noise was added to every training label, the neighbors' labels are noisy too; the exploratory study gave this as the reason label smoothing failed.

Uncertainty-based methods, which use no neighbors, showed potential at lower noise but degraded at higher noise. Sample weighting did better than label smoothing but usually still hurt. Over all sample-weighting configurations, not only those in Table 4.4, recovery was positive in 14.2% of configurations at $\sigma = 0.3$ and 4.7% at $\sigma = 0.6$. The mean recoveries were −22.9% and −20.9%.

4.4 Conclusions

On a single dataset, often very little of a standard representation varies. On hERG K_i, 1,815 of the 2,048 ECFP4 features take one value in at least 95% of the 1,412 molecules (Section 4.3.1). In addition, several representations give different molecules, mostly stereoisomers, the same feature vector, and no model can separate those molecules, whatever their labels. ECFP4 gives 56 of the 1,412 hERG K_i molecules a vector shared with another molecule, all of them stereoisomers ("Molecules with identical feature vectors").

A distance sums over every feature, so when only a few features matter for the property, the rest dominate the sum (Section 4.3.2). Across 70 combinations of dataset and representation, the Spearman correlation between pair distance and label difference was below 0.10 in 44. Predicting each label from its five nearest neighbors gave Spearman correlations of 0.29 to 0.89 with the measured labels, so the useful information sits in a few close pairs rather than in the distance as a whole.

In the small sweep of published noise-handling methods on QM9, the configurations with the lowest retention all used a distance, and judging a label by its neighbors' labels usually made the model worse (Section 4.3.3). The sweep added a distance to some

methods designed without one, so poor adaptation may explain part of this.

A representation that avoids these flaws would need five properties (Table 4.5). Its features would be chosen for the task, using only each fold's training labels. It would be compact, so that irrelevant features do not dominate a distance, with the number kept searched rather than fixed. It would draw on several representations, since each misses information the others capture; plain concatenation does this but is not compact (Chapter 5, "Combining representations"). It would work both as a model input and as a space for similarity. Finally, it would be traceable: each chosen feature leads back to its representation and can be read chemically wherever the base feature can.

Feature selection cannot fix every flaw. Molecules identical in every base representation stay identical in the hybrid, and a selected hashed bit still merges every substructure folded into it. Selection also needs enough labeled molecules to estimate which features matter.

Chapter 5 builds the adaptive hybrid representation and asks how to select its features for a given prediction task (Section 5.2.2).

Flaw (where shown)	Requirement	Where handled
Features that do not vary ("Constant and near-constant features")	Features chosen for the task, within a budget	"Feature importance scoring"
Molecules merged into one vector ("Molecules with identical feature vectors")	Features drawn from several representations	"Combining representations"
Distance dominated by irrelevant features ("Flaws in similarity and other pairwise computations")	Compact, with the number of features searched	"The search space"
Poor performance of neighbor-based methods ("Consequences for similarity-based methods")	Usable as a space for similarity	"Similarity in a hybrid" (Chapter 6)
Features that are hard to read chemically	Traceable, so that each chosen feature leads back to its representation and can be read chemically wherever the base feature can	"Blueprints and their chemical interpretation"

Table 4.5: Flaws of standard representations and similarity, the requirement each sets for a representation, and where the requirement is addressed, in Chapter 5 unless stated.

Chapter 5

Building adaptive hybrid representations

5.1 Introduction

So far, Chapter 4 explained how the standard methods of estimating molecular similarity, or conversely distance are flawed and potentially break the SPP. Specifically, the representations used in these similarity calculations contain irrelevant features that either do not vary or more generally do not pertain to the prediction task at hand. This chapter explores a solution. With the understanding that each representation contains a set of features which contain a certain type of information about that data, we aim to select the most informative features across a larger set of representations, while filtering out the irrelevant ones. We create an adaptive hybrid representation, a set of features bounded by a budget that is selected from several base representations by a search using one dataset and downstream model. The goal is to select a set of features that is customized for a given model and prediction task. As discussed in Chapter 3, performance can vary significantly across datasets and models. Rather than hope the features you give a model will be useful, this process lets you prescreen them.

In this chapter we build the adaptive hybrid representation, referred to as the hybrid representation, first reporting on when and why it helps. The questions this chapter and Chapter 6 address are set out in Chapter 1. This chapter measures predictive performance on the labels as published, apart from one separate-site experiment with flipped labels (Additional file 33). Distance measures between hybrids are covered in Chapter 6, and noise detection and learning with noisy labels in Chapter 7.

5.1.1 Combining representations

Not every molecular representation is equal. In fact, they hold very different types of information. RDKit2D descriptors matched or outperformed features derived from embedded GNNs and pretrained language models on many datasets across 62,820 models on a variety of molecular property prediction benchmarks [108]. Descriptor-based models held 73% of top-three positions on 11 benchmarks [158]. In 1,310 ChEMBL assays, feedforward networks on ECFP fingerprints outperformed graph convolutions and recurrent networks trained on SMILES [159]. We know that the choice of representation matters, as is discussed in Chapter 3, yet the combinations of representation and model are hard to explore by hand [160]. Beyond just molecules, general protein language models predict antibody expression better than antibody-specific ones, while the antibody-specific models are better on binding [161], and no pathology foundation model is best on every clinical task [162]. We turn our attention to combining features from these different representations.

Most research in this field relies on a single representation, and few combine features from different representations beyond concatenation. One of the reasons just pooling features from different representations together doesn't work is dimensionality. If you add a standard ECFP4 ($d = 2,048$) with Mordred descriptors ($d = 1,613$) and an MHG-GNN embedding ($d = 1,024$), you end up with 4,685 features, which is far too many considering many datasets from experimental assays contain only a few hundred labeled samples. With such ratios, more features mean weaker predictions [163, 164]. Therefore, we consider ways of combining representations.

The most straightforward method of combining representations is concatenation. This however degrades with non-homogeneous feature sets and obscures which representation is responsible for the score [165]. Deng et al. [108] found that appending RDKit2D descriptors to GROVER lowered RMSE on only 9 of 24 datasets tested. Late fusion trains one model per source and aggregates the predictions [166]. MOGONET is of this kind, combining per-source predictions in label space [167]. While this is effective, it loses interactions between the source representations, an issue which is addressed by intermediate fusion, for example cross-attention between sources [168]. Multiple kernel learning does this with kernels rather than whole models [169, 170], and produces no interpretable feature sets. Laufkötter et al. [171] found that combining bioactivity-based fingerprints with structure-based ones like ECFP4 improved both predictive performance and scaffold-hopping. Similarly, Turkina et al. [172] combined features from six non-hashed fingerprints with feature importance scores and showed

that the additional features consistently improved predictive accuracy over the original fingerprints across a range of models. Most of this work involves limited combinations, adding together standard fingerprints and linear representations [23, 171]. Although the features derived from embeddings are not as conceptually straightforward, they contain another axis of information.

When you want to filter a feature set for the features most important to the prediction task, you can use a dimensionality reduction technique or feature selection methodology. Principal component analysis (PCA) uses linear combinations of the original features, ordered by the variance they explain [34]. Their combined features are dependent on the original feature units unless each feature is z-scored first which matters when data from different modalities like fingerprint bits, descriptors and embeddings are joined [34]. PCA never sees the labels, and components with small variance can still be important in prediction [173]. Partial least squares (PLS), the supervised counterpart in chemometrics, builds components that covary with the property [174]. Neither gives components that can be traced to one substructure, descriptor or representation, making them difficult to interpret.

Feature selection keeps a subset of the original features, unlike PCA or PLS which builds new ones. Of its three families [33], filter methods score features against the labels before any model is fitted, as minimum-redundancy maximum-relevance selection (mRMR) ranks features by their mutual information with the label minus their redundancy with already-chosen features [175]. Wrapper methods score subsets of features by fitting the model [176]. Embedded methods select features while the model is fitted, as the lasso does [177]. Recursive feature elimination repeatedly fits a model and drops the lowest-ranked features [178].

When selecting features, the question is not only what features are most important, but also, what features does the model prefer? One of the ways to answer this is to consider how many features relative to the size of a dataset a given model works best with. Distance-based models such as k-nearest neighbors (kNN) and SVM use every feature in every distance computation (Chapter 4), whereas tree ensembles choose one feature per split and can essentially ignore irrelevant ones. Dablander et al. [7], who created a substructure-based fingerprint called Sort & Slice, suggest that using labels to select features may result in overfitting. Table 5.1 compares these alternatives with the adaptive hybrid methodology developed in this thesis.

Method	Uses the labels	Keeps original features	Several representations	Retrain to add one
Concatenation [165]	no	yes	yes	yes
Late fusion and ensembling [166, 179]	only if pooling weights are learned	yes	no	no, one more model
Intermediate fusion [168]	yes	no	yes	yes
Multiple kernel learning [169, 170]	yes	no, kernels only	yes	yes
PCA and PLS [34, 174]	PCA no, PLS yes	no	yes, on a concatenation	yes
Flat feature selection [33]	yes	yes	yes, on a concatenation	yes
Group lasso [180]	yes	yes, whole groups	yes	yes
Adaptive hybrid representation	yes	yes	yes	search rerun, method unchanged

Table 5.1: Methods for combining or reducing representations. The columns give whether each method uses the labels, keeps the original features, lets one model see several representations, and must refit to add a representation.

5.1.2 The adaptive hybrid representation

The task of selecting the best features from a set of representations and combining them is not simple. How do you measure what features are "best"? Are these features "best" relative to the representation they came from or to the whole pool of features? Should you select more features from representations that seem to have more signal to begin with, or take a set number of features from every representation? How many base representations should you select features from? How many features should you select? When scoring potential combinations of features, what type of evaluator should you use?

In a dream world, there would be a single answer to each one of these questions that applies to every dataset. Unfortunately we live in reality, and as I discovered, the answer to every single one of these questions is "depends on the data." The task of selecting the best features then became a search space optimization challenge. The question actually became for a given prediction task, defined by the dataset, prediction target, and model, what is the best methodology for selecting features? The hybrid search does not output just a list of features, rather it outputs an ideal search configuration to find features for your data. It tells you how to score features (feature

importance score), what rules you should use to select them (allocation method), how to score potential hybrid feature combinations (internal evaluator). In this research, I run this hybrid search multiple times across different maximum number of features (budgets), number of input representations, and models (downstream evaluators) on each dataset. While it is not formal "training" in the ML sense, the hybrid search is essentially a feature selection method that is fit on the training data. The representations used in this thesis are listed in Tables 5.9, 5.10 and 5.11.

Formally, an adaptive hybrid representation is a set of features, capped at a fixed budget, selected from several base representations of the same molecules. The output of a hybrid search is itself a feature selection method that when run on the given dataset, produces the hybrid representation. This search is run on a single dataset and downstream model, and the resulting hybrid differs by model, dataset, and even split. Its intended use case is when there are multiple base representations that describe different aspects of a molecule, and you want to combine these different features for a single training task. The hybrid ranks features by a fitted model's importance scores, as embedded methods do, and keeps a fixed number of the top-ranked ones, as a filter does. Classical methods treat all features as one flat list. The nearest classical relative, the group lasso, keeps or drops predefined groups of variables whole [180], whereas the hybrid splits the budget among representations (Section 5.2.2). Because only original features are kept, each traces back to one representation, which makes blueprints possible.

5.1.3 Blueprints and their chemical interpretation

As explained above, the adaptive hybrid representation relies on a search. A completed search is saved as a blueprint, which records the settings needed to select the features that make the hybrid representation. Because each selected feature traces back to its original base representation and relative importance rank, the search can tell you about your own prediction task. The best configuration is saved for each downstream model and budget, giving one blueprint per pair, allowing us to compare blueprints and see if different models prefer different features.

Depending on the origin of the feature, some are chemically interpretable. By definition, the PDV contains values that relate back to chemical properties like molecular weight or logP. An ECFP4 bit can be shown as the atom environment that sets it, and a MACCS key stands for a known substructure. Beyond small molecules, the readable unit of an antibody is a sequence position and its residue. This feature-level explain-

ability fails when features are derived from learned embeddings. That will only tell you if the learned embedding itself is useful as a representation, but nothing about the selected feature.

While a set of thousands of features may seem overwhelming and irrelevant, identifying substructures or descriptors that recur among the selected features can point to parts of the molecule that may be involved in binding or activity. If the selected features make sense chemically for that endpoint, it's a good sign that the selection mechanism is finding real signal instead of fitting noise. Conversely, if the hybrid search is done over a range of budgets or models, and the blueprints are showing a vastly different set of features, that may be an indication that the features at hand are struggling to provide information about the prediction task. A predictor that scores proposed molecules in generative design [181] could also take a hybrid as input, which we leave to future work.

5.1.4 Applications beyond small molecules

The hybrid search is a generalized search that takes in multiple feature matrices from the same samples and a supervised prediction task, and outputs a feature selection method that produces a filtered set of features from those original vectors. The default search objective scores prediction only, meaning the methodology applies to tabular tasks or cases where you have different measurements of the same inputs. Although this was developed with small molecules and QSAR modeling in mind, it is by no means limited to that use case. While there are some functionalities in this research that are tailored to chemical tasks, such as blueprints mapping features back to their chemical interpretations and the use of nearest-neighbor objectives (Chapter 6) to aid in the computation of molecular similarity, the core functionality of the hybrid process is domain-independent.

One caveat concerns sparse representations such as one-hot encoded sequences, where each feature is set in only a few samples and so scores low importance on its own. Here, the one-hot encoding is a base representation on three antibody datasets: nanobody thermostability, change in binding free energy ($\Delta\Delta G$) and CHO polyreactivity. The blocks substitution matrix (BLOSUM) encoding is a base representation on humanness.

While this setup was initially intended for small molecules, we extend this research to antibodies. Twelve antibody datasets are used in this chapter, from deep mutational scans of one antibody to panels of distinct antibodies (Table 5.7). One Buchwald–

Hartwig yield dataset tests the method on reactions, a separate domain.

Another caveat of this setup is that the hybrid can help only on datasets with enough samples (see Section 5.4) and with base representations whose features carry importance. When all the representations score poorly, there are not enough useful features to pull from. Additionally, if there is only a single base representation that carries signal, there is nothing for the hybrid to improve upon, which is shown in Section 5.3.1.

5.2 Methods

Figure 5.1 outlines the hybrid search.

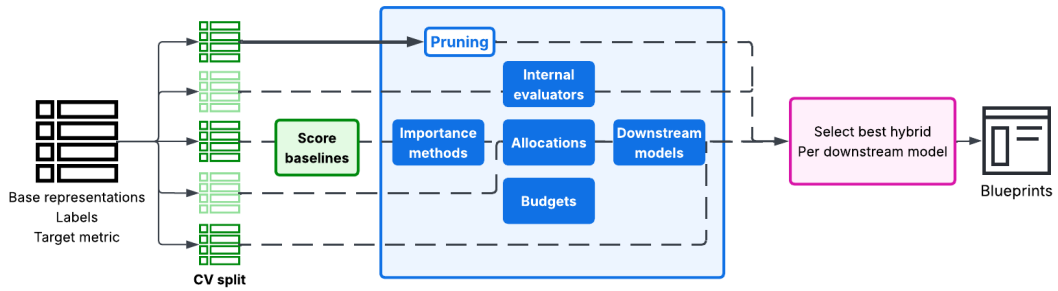


Figure 5.1: Schematic overview of the hybrid search. The inputs (left) are split into five cross-validation folds, and the single representations are scored as baselines. Each fold is searched over importance methods, allocation methods, budgets, internal evaluators and downstream models (center). The best hybrid for each downstream model is stored as a blueprint (right).

5.2.1 From base representations to a hybrid

The input to the search is K frozen representation matrices $\mathbf{X}^{(k)} \in \mathbb{R}^{n \times d_k}$, $k = 1, \ldots, K$, for the same n samples, whose widths d_k range from tens to thousands of features, as well as a label vector $\mathbf{y}$, one group label per sample for cross-validation (for example, its scaffold group), the evaluation metric, the task type, one or more downstream models and a list of budgets. It returns a hybrid of at most B features before any augmentation features added outside the budget (Section 5.2.3), where the budget B is a maximum rather than a target, and a blueprint that records the settings necessary to rebuild the hybrid representation on another set of samples, such as another subset of the same dataset.

When combining representations, we are careful to avoid data leakage. Because the feature selection process trains on the features and their labels, pooling all the

features prior to selection would leak validation-level information into the training, which is a well-documented bias in feature selection [182, 183]. As such, we run each hybrid search within a cross-validation fold. When a single representation is scored, we impute missing values with training-fold medians. A representation wider than 5,000 features is scored on its own using only its top 500 features (200 on datasets under 1,000 samples). These are chosen in each fold by a LightGBM model fitted on the training part. This affects only that representation's score, not the features the hybrid can select from it.

The data are split at three levels within CV runs. Every split keeps each group whole, a scaffold for small molecules, a sequence cluster for antibodies, and some predefined grouping for datasets from other domains (Section 5.2.6). With fewer than five groups, the inner and internal splits use as many folds as there are groups (Additional file 19). Figure 5.2 shows the three levels as nested boxes.

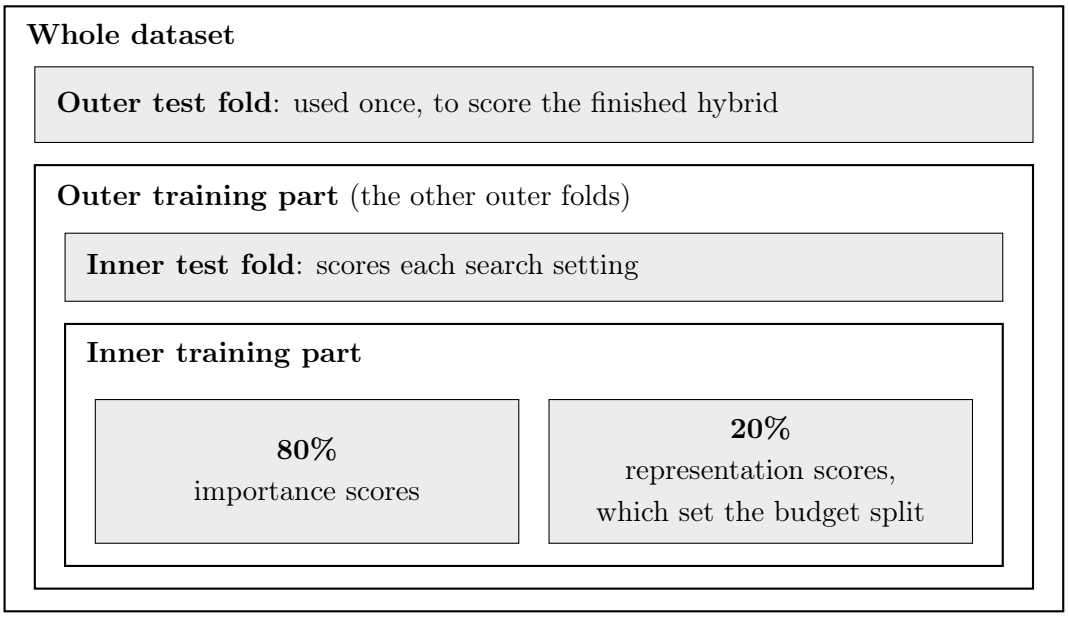


Figure 5.2: The three levels of splitting, each inside the one before. Every split keeps each group whole.

For each representation k, the importance method gives every feature a score $s_j^{(k)} \geq 0$, computed on the 80% part of the internal split (Figure 5.2). These scores rank features only within their own base representation. The standalone score r_k is the score of the internal evaluator, trained on representation k in the 80% part and scored on the 20% part, in the dataset's metric. These scores cannot be compared across metrics as they stand: a lower RMSE is better and it has no fixed upper limit, whereas a Spearman correlation is higher when better and lies between -1 and 1. Before the budget is split, each r_k is therefore converted to a score $\tilde{r}_k$ on which 0 means the representation

predicts no better than a trivial predictor and 1 means it predicts perfectly. The trivial predictor depends on the metric. For Spearman, Pearson, the Matthews correlation coefficient (MCC) and R^2, $\tilde{r}_k = \max(0, r_k)$. For AUROC and the concordance index, $\tilde{r}_k = \max(0, 2(r_k - 0.5))$. For RMSE, $\tilde{r}_k = \max(0, 1 - \mathrm{RMSE}_k/\mathrm{RMSE}_0)$, where RMSE_0 is the error of predicting the training mean, and mean absolute error is rescaled likewise against the training median. For accuracy and weighted F1, $\tilde{r}_k = \max(0, (r_k - r_0)/(1 - r_0))$, where r_0 is the score of always predicting the majority training class.

An optional representation count N keeps the N representations with the highest standalone scores on the fold and discards the rest for that iteration of the search. Each remaining representation receives a weight $w_k = g(\tilde{r}_k)$, with g set by the allocation method, and a share $b_k^* = B\, w_k / \sum_l w_l$ features, rounded by largest remainder and capped at its width. A representation left out by the representation count, or whose standalone fit fails, receives no features. Thus $\sum_k b_k = \min(B, \sum_k d_k)$ and $b_k \leq d_k$, and a representation with $w_k > 0$ keeps at least one feature. The global pool does not use these shares (see below).

Let S_k be the top b_k features of representation k by $s_j^{(k)}$, among those with $s_j^{(k)} > 0$; if none scores above 0, its full b_k is taken. The cut exists because tree importance gives exactly 0 to every feature no tree split on, and filling the budget with those would pick them by their position in the matrix. The hybrid is $\mathbf{H} = [\mathbf{X}^{(1)}_{:,S_1}, \ldots, \mathbf{X}^{(K)}_{:,S_K}]$, with $\sum_k |S_k| \leq B$, so it can be narrower than its budget.

The hybrid search returns a list of settings for feature selection, referred to here as a configuration. It is scored by fitting the downstream model on the hybrid in each fold of the search and averaging the held-out scores. Each pair of downstream model and budget has its own search, and thus its own winner. Because this score is chosen over a range of selected configurations, it is optimistic [184, 185]. However, so is the score of the highest-performing base representation it is compared with, which is also the best of several. The whole search runs inside each outer training fold, and the winning settings are rebuilt there and scored once on the untouched outer test fold, both for the winner at each budget and for each downstream model's best across budgets. The inner search score minus the outer test score is reported against the budget in Section 5.3.1.

Two variants of this search are used later: under label noise, training quantities use noisy labels while held-out scoring uses clean labels (Chapter 7). Under the nearest-neighbor objective, each sample's label is predicted from its five nearest neighbors in the feature space (Chapter 6).

As explained above, a hybrid search returns a list of settings that function as a feature selection method to generate the hybrid representation itself. These settings

are recorded in a blueprint that can be used on new folds of the same dataset or even new data. The blueprint contains additional information, including some fields for analysis, meta-data, and scores. These scores are used only for comparative purposes as in this research, the hybrids are rebuilt per-fold and are scored as such.

Additional file 20 lists every field of a blueprint and its type.

The blueprint does not record the specific features that are returned by the hybrid, as it is meant to be re-run using the same settings on new, labeled data. Alternatively, you can reuse the features of a labeled fit and apply them to data with no labels under the assumption that the same features matter between sets.

The blueprint is extended by an additional explainability layer. Since it does not store feature names, this additional step runs on a completed hybrid, records where each selected feature came from and, wherever possible, what that feature means. An ECFP4 bit is named by the atom environment that sets it, written as a SMILES fragment from a molecule with the bit set. MACCS keys are named by their SMILES arbitrary target specification (SMARTS) pattern and a short name, and descriptor features by descriptor name. Protein composition, descriptor and k-mer features, antibody composition, one-hot, BLOSUM, structural and inverse-folding features and PCA components are named too. MAP4 and DRFP features get only a bit index, and Avalon, Mordred, atom-pair, graph-kernel and learned-embedding features only the representation name and dimension index. The Mordred descriptor names in Figure 5.6 were added in a separate step.

5.2.2 The search space

To run a hybrid search, we first measure feature importance, select how many features to take from each contributing base representation, and choose how to score a candidate feature set. And for each of those steps, there's no one right way to do it. Different prediction tasks prefer different methods of scoring feature importance or candidate features, different sets of input representations, and so forth. The search tries to find the ideal settings for the hybrid feature selection process, tailored to a given dataset and downstream model. All combinations of settings values are explored, as the settings interact. The best budget depends on the downstream model, as shown below for nanobody thermostability. This search grid also tests the chapter's claim that no single setting suits every dataset.

Six settings in total are searched, namely the downstream model, the budget, the representation count, the importance method, the allocation method and the internal

evaluator. Table 5.5 gives the values each setting can take, and Table 5.6 the smaller set searched on each dataset in this chapter (Section 5.2.4). The importance methods tested are LightGBM gain, TreeSHAP or DeepLIFT (described below). Feature-level importance depends on the model as well as on the data [186], and as such we observe that within the search, a different ranking of features produces a different hybrid output.

The downstream model is trained on the hybrid representation. This model can be a supervised model of any form, classification or regression. We used random forest, XGBoost, LightGBM, SVM, a deep neural network (DNN) and a Gaussian process. Each dataset in this chapter uses two of them (Section 5.2.8).

The budget is the maximum number of features in the hybrid, taken from eight values (10, 25, 50, 100, 250, 500, 1,000 and 2,000 features). When the budget is too small, informative features can be left out, yet when it is too large it will pick up on too many uninformative ones, especially ones that give small datasets room to overfit.

We select a set of up to 10 base representations per dataset, chosen prior to the search (Table 5.12). The selections were made based on each representation's relative performance on the prediction task. We took into account what type of information each representation contained, and tried to span a wide set across the selected 10. For example, while there are many high-performing graph-based embeddings in QSAR modeling, for the purposes of testing a feature selection process which takes features from different sources of information, it is better to have a mix of graph-based embeddings, as well as other types of molecular representations like substructure fingerprints, property vectors, sequential embeddings, etc. The representation count (2, 3, 5 or all) sets how many may contribute features, keeping those with the highest standalone scores on each fold. The number of representation counts to test is searched in the sweep as datasets differ in how many representations have features worth selecting, and pooled feature selection over all representations has no such setting. While we tested various methods of pooling the features, we found computing feature importance scores and concatenation techniques on a global pooled feature set was less often best in preliminary runs than when the feature scoring is kept on a per-representation basis (Additional file 16). Since the features contain different types of information at different scales, pooling the features too early makes comparisons within the pool challenging. We found that it's better to determine which features are important to each representation first, and go from there.

Every allocation method is paired with every count. In preliminary runs, some datasets performed best with 2 or 3 representations and others with all 10. The full

runs show the same split. On LogD, where counts of 2, 3, 5 and all were searched, the hybrid used all 10 representations in nine of its ten searches (five folds for each of the two downstream models). On QM9 with LightGBM, which searched 3 and all, it used 3 on every fold.

Feature importance scoring

We run a feature importance scoring method on each base representation, which gives each feature a single positive score. These scores remain relative within a given representation, so they are not compared or normalized between representations. Split-based tree importance is biased toward features with many possible split points [187], which matters here because we use binary fingerprint bits alongside continuous embeddings. No single importance method works best across datasets and downstream models. After testing a wider variety of methods found across literature, we settled on three methods whose rankings did not correlate strongly with each other and whose selected features performed well as model input (Additional file 16).

We score features using LightGBM gain importance, SHapley Additive exPlanations (SHAP) values for tree ensembles (TreeSHAP) and DeepLIFT (Deep Learning Important FeaTures) [188]. LightGBM gain measures how much a feature reduced the training loss where trees split on it [62]. TreeSHAP gives each feature's exact Shapley value for each molecule's prediction from a tree model [189]. DeepLIFT measures how much a feature moves the output of a small network, trained with Adam [190], away from its output on an all-zero reference input, which after standardization places every feature at its training mean [188, 191, 192]. On Caco-2, the median share of features scoring above 0 was 30% for LightGBM gain, 70% for TreeSHAP and 90% for DeepLIFT (preliminary, over 10 representations and 3 folds). Table 5.2 gives the settings of each method, including the pre-screen of representations wider than 2,000 features. Each dataset searches one, two or all three importance methods depending on its individually-decided search settings (Table 5.6).

Feature allocation methods

The allocation method decides how many of the B features each representation contributes while the importance method decides which. Of the five methods searched, four give each representation a fixed share computed from its rescaled standalone score (linear, squared, softmax and rank-decay allocation). The fifth, the global pool, lets the candidate features offered by every representation compete in one importance ranking.

Method	Model fitted per representation	Score of one feature	Classification labels	Representations over 2,000 features
LightGBM gain	LightGBM, 50 trees, depth 6	Summed loss reduction of its splits; 0 if never split on	One model on the class labels	2,000 highest-variance features fitted; the rest score 0
TreeSHAP	RF, 50 trees, depth 10	Mean absolute SHAP value over training molecules	Binary: positive class; multiclass: mean over classes	As for LightGBM gain
DeepLIFT	NN, ReLU layers of 64 and 32 units, 100 epochs of Adam, learning rate 0.001	Mean absolute attribution against an all-zero reference (Rescale rule)	Binary: positive class; multiclass: mean over classes	Every feature scored

Table 5.2: Settings of the importance methods, fitted per representation and fold. Unlisted settings are library defaults.

The four fixed-share methods compute budget shares from the standalone score on internal validation. Only linear allocation makes the share proportional to that score. All four assume that representations with stronger predictive performance contribute more valuable features. This assumption fails when a strong representation relies on a few select features. In that case, a large share of features will be given to that representation, picking up on weaker features and potentially missing out on stronger ones from other representations. Fixed shares ignore complementarity, so two representations carrying the same information both receive a share. Only the global pool ranks features from different representations against each other, however ranking feature importance on a global pool is dubious as the features are not necessarily appropriate to compare.

The first method of feature allocation we tried was a greedy forward selection, in which the hybrid is built one block of features at a time [33, 176]. In each round, the next block is added from representations in order of which improves the internal validation score. This in theory lets it choose features for how well they work together. In practice, this is very expensive. With blocks of 10 features and a budget of 500 and 10 representations, it needs up to 500 model fits per configuration and fold against 10 for a score-based method. It was also less accurate: in preliminary runs on Jain antibody neonatal Fc receptor (FcRn) retention it placed 16th of 17 allocation methods, with a Spearman correlation of 0.360 against 0.453 for linear allocation. We therefore

removed it from the search (Additional file 16).

The fixed-share methods differ only in how w_k is computed from $\tilde{r}_k$ (Table 5.3; worked example in Table 5.4), with settings fixed for every dataset. Because every rescaled score lies between 0 and 1, softmax cannot give one representation most of the budget: with 10 representations the best takes at most 23.2%. Rank decay uses only the order: with 10 representations the best receives 27.3% and the tenth 1.8%, whatever the gaps between their scores.

Method	Weight w_k	Fixed setting	How features are chosen	When a representation receives 0 features
Linear	$\tilde{r}_k$	none	Top b_k of its own ranking	$\tilde{r}_k = 0$
Squared	$\tilde{r}_k^2$	power 2	Top b_k of its own ranking	$\tilde{r}_k = 0$
Softmax	$\exp\left((\tilde{r}_k - \max_l \tilde{r}_l)/T\right)$	$T = 1$	Top b_k of its own ranking	never from its score
Rank decay	$e^{-0.3\rho_k}$, ρ_k = rank by score, 0 for the best	decay 0.3	Top b_k of its own ranking	never from its score
Global pool	$\tilde{r}_k^2$, sizing its offer to a pool of $2B$ candidates	power 2; pool $2B$	Top B of one joint ranking over all offered candidates	$\tilde{r}_k = 0$, or none of its candidates in the joint top B

Table 5.3: The five allocation methods. w_k is the weight computed from representation k's rescaled standalone score $\tilde{r}_k$, B is the budget, b_k is the share of representation k, ρ_k is its rank by score and T is the softmax temperature.

Method	$\tilde{r}_1 = 0.6$	$\tilde{r}_2 = 0.5$	$\tilde{r}_3 = 0.2$
Linear	46	39	15
Squared	55	39	6
Softmax	39	35	26
Rank decay	44	32	24

Table 5.4: Worked example of the four fixed-share allocation methods for a budget of 100 features and three representations. Each entry is the number of features a representation receives, with shares rounded by largest remainder.

The global pool sizes each representation's offer with squared weights $\tilde{r}_k^2$, as shares of a pool of $2B$ candidate features. Each offer is capped at the representation's width and holds only features scoring above 0. The same importance method is fitted once on the joined offers, and the hybrid keeps the top B features that score above 0. Each representation's count is therefore an outcome of the joint ranking, with no minimum of one feature, and a feature's rank depends on what the other representations offered.

Search configuration and internal evaluation

The most useful features for an RF may not be the most useful for an SVM, and vice versa. The relative importance of a feature is dependent on the model [186]. Within the search grid, we use an internal evaluator to fit on one candidate feature set at a time. This process is separate from the final downstream model evaluation. In a standard search, four internal evaluators are used: RF, DNN, LightGBM, and the matched evaluator. When the internal evaluator is set to `match`, the search scores each candidate hybrid on the same model type as the downstream model, for each of the twelve model types the code supports. So internal evaluators outside the standard sweep may be tested. For a downstream model outside of the pre-defined set, if no matched evaluator exists in the code, the search uses the other internal evaluators that were requested. Each internal evaluator is fitted on the internal training fold and scores on the internal validation fold within that CV iteration using the dataset's specified evaluation metric.

In the full grid, a random forest, LightGBM or DNN downstream model is searched with three internal evaluators, because its matched evaluator is already one of the other three. An XGBoost, SVM or Gaussian process downstream model is searched with four, because the matched evaluator adds one of its own family. Four datasets searched fewer (Table 5.6). EGFR binding has no random forest evaluator, expression titer and Buchwald–Hartwig yield have two evaluators, and AC-SINS self-association has no Gaussian-process evaluator. Only the DNN evaluator shares its downstream model's architecture and training settings. Even it draws its 20% early-stopping samples at random, whereas the downstream DNN splits them by the outer split's groups. Additional file 13 gives every evaluator setting.

The internal evaluator's scores determine how the budget is split and which representations are kept. The internal evaluator ranks candidate feature sets, it explicitly does not rank features within a representation. This process of selecting a feature set by fitting a model on it, and keeping the set that scores the best is known as the wrapper method [176]. This is distinct from a filter method which scores features without fitting a model.

The search tests every combination of the six settings on a dataset under 5-fold cross-validation. Over every value of Table 5.5, this is 10,080 configurations with all six downstream models, or 8,160 without the Gaussian process (Additional file 15). With each dataset's own values, the searches reported in this chapter scored 108 to 3,360 configurations per outer fold (Table 5.6). Each dataset's grid is a small cut of the

full search, made to save compute (Section 5.2.4); at worst, it finds a less informative hybrid than the full search would have.

Before the full runs, we ran preliminary design experiments on 1 to 9 datasets to choose the values each setting may take (Additional file 16). A value was kept if it gave the best score for some dataset, downstream model and budget. A setting was fixed at one value when the spread between its values was smaller than the spread of one value across folds. On hERG K_i, for example, five ways of rescaling each representation's score differed by less than one of them varied across folds. Hence the rescaling is one fixed rule per metric.

5.2.3 Augmentation

Some feature sets are small, such as log-likelihood ratios (3 features) and the 3D shape and pharmacophore features (12 to 60 features). Others, such as mutation features that record differences from a parent antibody, may carry more meaning beside a full representation. On some datasets we therefore add them as augmentation, calling the finished hybrid **H** the base and the feature sets that supply extra features the donors. Donors are set per dataset (Augmentation column of Table 5.12). Uncapped augmentation adds the extra features outside the budget, and capped augmentation puts them in place of the weakest base features. An earlier search of each whole dataset covered 16 datasets of an earlier list. On none did either augmented condition beat no augmentation by more than 1% (3% on datasets under about 1,000 samples). We therefore ran the searches in this chapter without augmentation on all 20 datasets (Section 5.2.4). Only the nearest-neighbor search on SMAC colloidal stability and HIC retention keeps uncapped augmentation (Chapter 6). Additional file 21 gives how donor features are ranked and added, and the three conditions.

5.2.4 Search grid per dataset

Searching every value of Table 5.5 in every fold of every dataset was too slow, so each dataset searches a smaller grid (Table 5.6). From an earlier search of each whole dataset, we kept the two downstream models with the best scores. Likewise, we kept the budgets whose ten best configurations scored within 1% of the best budget (3% on datasets under about 1,000 samples), widened to at least three consecutive budgets. Importance methods, allocation methods, internal evaluators and representation counts that scored well below the best were dropped.

Two datasets, QM9 and hERG blocking, were too slow for the earlier search over ev-

ery value. They were searched over a shorter fixed list, called the fast tier (LightGBM gain importance; linear, squared and rank-decay allocation; three or all representations), and their grid stays within it. The search with the nearest-neighbor objective has its own grid, given in Chapter 6.

Setting	Values
Downstream models	Random forest, XGBoost, LightGBM, SVM[a], DNN, Gaussian process[b]
Internal evaluators	Matched evaluator, random forest, LightGBM, DNN
Feature budgets (features)	10, 25, 50, 100, 250, 500, 1,000, 2,000
Allocation methods	Linear, squared, softmax, rank decay, global pool
Importance methods	LightGBM gain, DeepLIFT, TreeSHAP
Representation counts	2, 3, 5, all representations

[a]Not fitted above 10,000 molecules (5,000 when augmentation features are added outside the budget). [b]Not fitted above 1,000 molecules.

Table 5.5: Values the hybrid search can take for each setting. The matched evaluator belongs to the same model family as the downstream model.

Additional file 22 follows hERG K_i through these cuts, from the 8,220 configurations of its earlier search, which included kNN among six downstream models, to 224, and its final grid still contains the best configuration.

5.2.5 Evaluation protocol

Every experiment splits each dataset into outer folds as set out in Section 5.2.6. The search sees only the training samples of each outer fold, and the selected features are rebuilt on them and applied unchanged to the held-out samples. Only three experiments limit each held-out feature to the range it takes on the training samples: the comparison with the best single representation, the frequent and rare scaffolds, and the comparison of many and few representations.

Because of time and computational constraints, not every experiment in Chapters 5 to 7 ran on every fold or every dataset. Some experiments here ran on the first three outer folds, most noise experiments in Chapter 7 ran on the first outer fold only, and some experiments have no result on some datasets. We report the results that exist, give the number of folds beside each one, and name the datasets where a result is missing. We do not repeat this caveat at every result.

The hybrid is compared with the best single base representation at its full number of features, which is what a user would otherwise run. This comparator is chosen per

Dataset	Downstream models	Budgets (features)	Importance	Allocation	Internal evaluators	Representation counts	Configurations
Small molecules							
QM9†	DNN, LightGBM	500–2,000	G	L, Q, R	RF, LGBM, DNN	3, all	108
hERG K_i	SVM, DNN	250–2,000	G, T	L, Q, S, R	RF, LGBM, DNN, SVM	all	224
Caco-2 efflux	DNN, SVM	500–2,000	G, T, D	L, Q, S, R	RF, LGBM, DNN, SVM	all	252
LogD	DNN, SVM	500–2,000	G	L, Q, R	RF, LGBM, DNN, SVM	2, 3, 5, all*	252
Rat microsomal clearance	SVM, DNN	500–2,000	G, T, D	all five	RF, LGBM, DNN, SVM	all	315
MDR1–MDCK efflux	SVM, DNN	500–2,000	G, T, D	all five	RF, LGBM, DNN, SVM	all	315
hERG blocking†	LightGBM, XGBoost	100–2,000	G	L, Q, R	RF, LGBM, DNN, XGB	3, all	210
Antibodies							
Anti-VEGF scan expression	SVM, DNN	250–2,000	G, T, D	all five	RF, LGBM, DNN, SVM	all	420
Anti-VEGF scan binding‡	RF, SVM	250–1,000	G, T	all five	RF, LGBM, DNN, SVM	all	210
Nanobody thermostability	DNN, SVM	50–2,000	G, T	all five	RF, LGBM, DNN, SVM	2, 3, 5, all*	1,680
EGFR binding	DNN, SVM	250–2,000	G, T	all five	LGBM, DNN, SVM	2, 3, 5, all*	800
Angiopoietin-2 binding	GP, SVM	25–100	G, T	all five	RF, LGBM, DNN, GP, SVM	3, 5, all	720
Antibody humanness	XGBoost, RF	10–2,000	G, T, D	all five	RF, LGBM, DNN, XGB	2, 3, 5, all	3,360
Binding free energy	DNN, SVM	100–500	G, T	L, S, R	RF, LGBM, DNN, SVM	3, 5, all	378
SMAC colloidal stability	DNN, SVM	500–2,000	G, T, D	all five	RF, LGBM, DNN, SVM	3, 5, all	810
HIC retention	DNN, GP	250–2,000	G, T, D	all five	RF, LGBM, DNN, GP	5, all	840
AC-SINS self-association	LightGBM, GP	50–2,000	G, T, D	L, Q, S, R	RF, LGBM, DNN	2, 5, all	1,296*
Expression titer	DNN, RF	250–2,000	T, D	all five	RF, DNN	2, 3, 5, all	640
CHO polyreactivity	DNN, GP	250–2,000	T, D	all five	RF, LGBM, DNN, GP	2, 5, all	840
Chemical reactions							
Buchwald–Hartwig yield	RF, LightGBM	500–2,000	T, D	all five	RF, LGBM	2	120

†Fast tier. ‡Only the first outer fold was searched. *The representation counts were reduced after the search ran, and AC-SINS scored 1,451 configurations on one of its five folds.

Table 5.6: Search grid of each dataset, as run. One row is one dataset. Budgets are consecutive values of Table 5.5. Importance: G, LightGBM gain; T, TreeSHAP; D, DeepLIFT. Allocation: L, linear; Q, squared; S, softmax; R, rank decay; all five adds the global pool. Internal evaluators include the matched evaluator of each downstream model, except the Gaussian process on AC-SINS (RF, random forest; LGBM, LightGBM; XGB, XGBoost; GP, Gaussian process). Configurations is the number scored in one outer fold's search, over both downstream models.

downstream model and per fold from the search's scores on the outer training samples, and it can differ between folds. It is then refit on those samples and scored on the same held-out samples as the hybrid. The hybrid is held to its budget, and the comparator is consequently often many times wider. Both widths are recorded for every fold, since a difference between the two scores is partly a difference in width. The main quantity per fold is the hybrid's score minus that of the best single representation, signed so that a positive value means the hybrid scored better (for RMSE the sign is flipped).

5.2.6 Datasets and splitting

The hybrid experiments use 20 datasets (Table 5.7): seven of small molecules, 12 of antibodies and one of reactions. Their curation is given in Section S.7 of the Additional files and their representations in Table 5.12. Antibody datasets outnumber small-molecule ones, but small molecules are the focus of this chapter, and results are read per dataset, not pooled across a domain.

Pre-processing

Section S.7 of the Additional files gives the shared and per-dataset pre-processing steps, with the counts after each step. The LogD and Caco-2 labels are clipped at their 1st and 99th percentiles, so on Caco-2, for example, 22 molecules share the highest label and 23 the lowest.

Splitting rules

As in Chapter 3 (Section 3.2), no group is split between folds, and classification folds are also stratified with a fixed seed. Every dataset is split into five outer folds, though some experiments ran on fewer (Section 5.2.5), and no split is random (Section 2.7). On a given dataset, every representation, model and hybrid is scored on the same folds.

Groups by domain

The small-molecule datasets, QM9 included, are grouped by Bemis–Murcko scaffold (Section 3.2), here keeping stereochemistry. On QM9 the largest group is the empty scaffold of acyclic molecules, with 1,041 of the 9,978 molecules. The two datasets from Biogen [137] (rat microsomal clearance and MDR1–MDCK efflux) have 1.2 and 1.1 molecules per scaffold, respectively, so holding out a scaffold there is close to holding out single molecules. Chapter 3 used a different scaffold key and different curation for

Dataset	Label (unit)	Task, metric	n	Folds grouped by (groups)	Best single representation
Small molecules					
QM9 [109, 193]	HOMO–LUMO gap (eV)	regression, RMSE ↓	9,978	scaffold (3,452)	Mordred (both)
hERG K_i [111]	pK_i	regression, RMSE ↓	1,412	scaffold (769)	SVM: MHG-GNN; DNN: GROVER
Caco-2 efflux [110]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,158	scaffold (1,086)	DNN: MHG-GNN; SVM: PDV
LogD [110]	logD at pH 7.4 (no unit)	regression, RMSE ↓	5,028	scaffold (2,413)	DNN: Mordred; SVM: MHG-GNN
Rat microsomal clearance [137, 194]	$\log_{10}$ CL_{int} (mL/min/kg)	regression, RMSE ↓	3,053	scaffold (2,625)	SVM: PDV; DNN: MHG-GNN
MDR1–MDCK efflux [137, 194]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,641	scaffold (2,318)	PDV (both)
hERG blocking	blocker or not (3,630 : 4,142)	binary, AUROC ↑	7,772	scaffold (4,002)	MHG-GNN (both)
Antibodies					
Anti-VEGF scan expression [195, 196]	expression enrichment ratio (no unit)	regression, Spearman ↑	4,275	mutated position (225)	ESM3 CDR (both)
Anti-VEGF scan binding [195, 196]	binding enrichment ratio (no unit)	regression, Spearman ↑	4,275	mutated position (225)	RF: ESM3; SVM: varies
Nanobody thermostability [197]	melting temperature (°C)	regression, Spearman ↑	672	95% identity cluster (309)	SaProt (both)
EGFR binding	mutational scan score	regression, Spearman ↑	659	mutated position (36)	SVM: Ab-RoBERTa CDR; DNN: varies
Angiopoietin-2 binding	mutational scan score	regression, Spearman ↑	397	mutated position (20)	SaProt (both)
Antibody humanness [198]	human or not (198 : 366)	binary, AUROC ↑	564	95% identity cluster (461)	RF: AntiBERTy CDR; XGBoost: varies
Binding free energy [199, 200]	$\Delta\Delta G$ (kcal/mol)	regression, Spearman ↑	429	complex (30)	SVM: Mutation features; DNN: ESM-2 shift from parent
SMAC colloidal stability [201, 202]	SMAC retention time	regression, Spearman ↑	242	competition fold (5)	DNN: ProtT5; SVM: varies
HIC retention [202]	HIC retention time	regression, Spearman ↑	242	competition fold (5)	ESM-2 (joined chains) (both)
AC-SINS self-association [202]	AC-SINS shift at pH 7.4	regression, Spearman ↑	242	competition fold (5)	LightGBM: IgT5; GP: AbLang-2
Expression titer [202]	expression titer	regression, Spearman ↑	239	competition fold (5)	RF: IgT5; DNN: varies
CHO polyreactivity [202]	polyreactivity to CHO cell proteins	regression, Spearman ↑	197	competition fold (5)	ProtT5 (both)
Chemical reactions					
Buchwald–Hartwig yield [203]	yield (%)	regression, RMSE ↓	3,955	additive (22)	RF: CGR fingerprint; LightGBM: MHG-GNN (difference)

Table 5.7: The 20 datasets of the hybrid experiments after curation, grouped by domain. n is the number of samples, and the fold groups are given with the number of groups in parentheses. For classification tasks, the class counts are in parentheses in the label column. CL_{int} is the intrinsic clearance, and arrows show whether lower or higher is better. The hERG blocking data were supplied by Lhasa Limited. The last column gives, for each of the dataset's two downstream models, the single base representation the search ranked first on the training samples of at least three of the five outer folds.

Small molecules and reactions		Antibodies	
Dataset	Short name	Dataset	Short name
QM9	QM9	Anti-VEGF scan expression	VEGF-E
hERG K_i	HK_i	Anti-VEGF scan binding	VEGF-B
hERG IC_{50}	HIC_{50}	Nanobody thermostability	Nb-Tm
hERG blocking	HB	EGFR binding	EGFR
Caco-2 efflux	Caco-2	Angiopoietin-2 binding	Ang2
LogD	LogD	Binding free energy	$\Delta\Delta G$
Rat microsomal clearance	RLM	SMAC colloidal stability	SMAC
MDR1–MDCK efflux	MDR1	HIC retention	HIC-R
Buchwald–Hartwig yield	BH	AC-SINS self-association	AC-SINS
		Expression titer	Titer
		CHO polyreactivity	PR-CHO
		Antibody humanness	Hum

Table 5.8: Short names of the datasets, used in the figures.

its versions of LogD, Caco-2, hERG K_i and QM9, and Section S.7.2 of the Additional files lists the differences.

The reaction dataset is Buchwald–Hartwig C–N couplings from high-throughput plates. Following Ahneman et al. [203], it is grouped by additive: each held-out fold holds four or five additives unseen in training (Section S.7 of the Additional files).

In the Koenig deep mutational scan of the anti-vascular endothelial growth factor (VEGF) antibody G6, each variant carries one mutation. Variants are grouped by their mutated position, giving 225 groups. With all mutations at one position on the same side of a split, a model cannot look up a per-position effect at test time [204]. The nanobody and humanness sets are grouped by single-linkage clusters at 95% sequence identity, the identity threshold Kenlay et al. [161] used to cluster their pretraining sequences.

The two AbAgym deep mutational scans (EGFR and angiopoietin-2 binding) are grouped by mutated position, as for the Koenig scans, giving 36 and 20 groups. The $\Delta\Delta G$ set joins antibody–antigen entries from SKEMPI 2.0 [199] and AB-Bind [200]. It is grouped by protein complex, giving 30 groups, and hence no complex is on both sides of a split. The five Ginkgo developability assays are measured on the same panel of antibodies. Two are retention times, in hydrophobic interaction chromatography (HIC) and standup monolayer adsorption chromatography (SMAC). The others are affinity-capture self-interaction nanoparticle spectroscopy (AC-SINS), expression titer, and polyreactivity to Chinese hamster ovary (CHO) cell proteins. We use the five cross-validation folds released with the competition [202] as the groups, which makes the outer folds the competition's own.

Consequences of the grouping

Some groupings give very unequal folds: the Ginkgo competition folds hold 35 to 53 antibodies, and per-fold scores on these datasets spread more widely.

5.2.7 Representations by domain

Any representation that gives one fixed-length vector per sample can serve as a base representation (Section 5.2.1). The first version of the method used three molecular representations: PDV, ECFP4 and MHG-GNN. These three are now one small-molecule example among many. Every base representation is frozen: it is computed once from the raw input (a SMILES string, a sequence or a reaction) and is never trained on the labels.

To choose them, every candidate representation was first scored on its own with a gradient-boosted tree model [62] under the dataset's grouped folds. Candidates were then compared by the Spearman correlation between their out-of-fold predictions, since two representations whose predictions correlate highly carry much the same signal. At most ten were then chosen by hand, looking at each candidate's own score next to these correlations and favoring representations that encode the input in different ways. Binding free energy has eight.

Which representation is best depends on the downstream model as well as the dataset. The best one is the same under both of a dataset's downstream models on three of the seven small-molecule datasets and five of the twelve antibody datasets (Table 5.7). On hERG K_i, for example, MHG-GNN is best with the SVM and GROVER with the DNN. Augmentation features do not count toward the ten, and no search in this chapter uses them (Section 5.2.3).

If an input cannot be parsed or embedded, it gets an all-zero row, so that every representation of a dataset keeps the same rows. In computed descriptors, a missing or infinite value is set to 0, and values are stored unscaled. A pretrained model gives one vector per input, the mean of its last-layer token vectors with padding excluded, unless the tables say otherwise. Scaling and filling of missing values happen inside each fold, on the training rows only, as described in the construction of the hybrid.

Small molecules

The small-molecule representations (Table 5.9) are the fingerprint, descriptor, graph and language-model families described in Chapter 2. The only additions are four sets of 3D shape and pharmacophore features, used as augmentation only. All four

are computed from a single conformer per molecule, embedded with a fixed seed and optimized with the Merck molecular force field. A single conformer may misdescribe flexible molecules.

Chemical reactions

Reaction datasets (Table 5.10) use three kinds of representation. The condensed graph of reaction (CGR) merges reactants and products into one graph in which the bonds that change are marked [205, 206]. Here it is hashed into a circular fingerprint after atom mapping [207]. Reaction language models read the reaction SMILES directly [208, 209]. Any molecular representation can also be applied to the reactants, to the product, or to their difference. A one-hot encoding of which aryl halide, ligand, base and additive was used carries no chemistry. It is kept because on plate data such identity features can predict yield about as well as chemical descriptors [210].

Antibodies

Antibody representations (Table 5.11) start from the heavy- and light-chain amino-acid sequences. Besides one-hot and BLOSUM62 encodings [211], most are frozen language models, either trained on all proteins or on antibody repertoires only. Most models embed each chain separately and join the two vectors, so their width doubles on datasets with both chains. Several representations read one model in different ways, for instance averaging over earlier layers or over the complementarity-determining region (CDR) residues only, with CDRs found by IMGT numbering [212] using ANARCI [213]. Datasets of point variants of one parent antibody also use features that describe each mutation.

Table 5.9: Representations used for small molecules. In the Role column, B marks a base representation, A an augmentation and B/A a representation that is a base representation on some datasets and an augmentation on others.

Representation	What it encodes	Features	Source	Role
ECFP4	hashed circular substructures, radius 2	2048	[5]	B
Atom pairs	hashed atom-type pairs with bond distance	2048	[214]	B
Topological torsions	hashed paths of four bonded atoms	2048	[215]	B/A
Avalon	hashed paths, rings and atom environments	2048	[8]	B
MACCS keys	166 predefined substructure keys	167	[216]	B/A

Representation	What it encodes	Features	Source	Role
WL graph kernel	Weisfeiler–Lehman label counts, 5 rounds, labels present in at least 5% of molecules	140–317	[217]	B
PDV	RDKit 2D physicochemical descriptors	200	[32]	B
Mordred	2D molecular descriptors	1613	[218]	B
GROVER	frozen pretrained graph transformer	4800	[219]	B
MHG-GNN	frozen pretrained graph encoder	1024	[9]	B
GraphMVP	frozen 2D graph encoder pretrained with 3D views	300	[220]	B
MoLFormer	frozen SMILES language model; public checkpoint trained on 10% of its corpus	768	[42]	B
SMI-TED	frozen SMILES encoder–decoder	768	[221]	B
ChemBERT	frozen SMILES language model; community checkpoint trained on ChEMBL following the cited method	256	[222]	B
ChemBERTa-2	frozen SMILES language model	384	[28]	B
SELFormer	frozen SELFIES language model	768	[223]	B
Mol2vec	summed substructure word vectors	300	[25]	B
3D pharmacophore	counts of 7 pharmacophore types, and mean and spread of distances for the 21 type pairs, one conformer	49	[32]	A
USR	shape moments from 4 reference points, one conformer	12	[224]	A
USRCAT	USR per pharmacophore atom type, one conformer	60	[225]	A
Coulomb matrix	sorted eigenvalues of the Coulomb matrix, one conformer	50	[226]	A
MolCLR	frozen graph encoder pretrained by contrastive learning	512	[227]	A

Table 5.10: Representations used for chemical reactions. Role is read as in Table 5.9, a dash in the Role column marks a representation used on no dataset, and the suffix in parentheses states whether a molecular representation is computed on the reactants, on the product or as their difference.

Representation	What it encodes	Features	Source	Role
CGR fingerprint	circular fingerprint, radius 2, hashed from the condensed graph of reaction	2048	[205, 206]	B
rxnfp	frozen reaction language model	256	[208]	B
ReactionT5	frozen reaction language model	768	[209]	B
Component one-hot	which aryl halide, ligand, base and additive	45	[203]	B
GROVER (reactants)	GROVER averaged over reactants	4800	[219]	B
Mordred (reactants)	Mordred averaged over reactants	1613	[218]	B

Representation	What it encodes	Features	Source	Role
Mol2vec (reactants)	Mol2vec averaged over reactants	300	[25]	B
ECFP4 (reactants)	ECFP4, bitwise OR over reactants	2048	[5]	–
Avalon (difference)	product minus reactant Avalon bits	2048	[8]	B
MHG-GNN (difference)	product minus reactant embedding	1024	[9]	B
MACCS (difference)	product minus reactant MACCS keys	167	[216]	B
Mol2vec (difference)	product minus reactant Mol2vec	300	[25]	–
MACCS (reactants)	MACCS keys, bitwise OR over reactants	167	[216]	A
MACCS (product)	MACCS keys of the product	167	[216]	A
3D pharmacophore (reactants)	3D pharmacophore averaged over reactants	49	[32]	A
3D pharmacophore (product)	3D pharmacophore of the product	49	[32]	A

Table 5.11: Representations used for antibodies. LM is a language model. Feature counts marked /chain double on the datasets with heavy and light chains. A dash in the Source column marks a standard encoding or one built for this thesis, and Role is read as in Table 5.9.

Representation	What it encodes	Features	Source	Role
One-hot	one-hot residues, both chains padded to fixed length	5600	–	B
BLOSUM62	BLOSUM62 row per residue, both chains padded	5600	[211]	B
Amino-acid composition	fraction of each amino acid, per chain	40	–	B/A
ESM-2	protein LM, last layer, mean over residues	1280/chain	[228]	B
ESM-2 (joined chains)	ESM-2 on heavy and light chain joined into one string	1280	[228]	B
ESM-2 early layers	ESM-2, mean over layers 0–16	1280/chain	[228]	B
ESM-2 middle layers	ESM-2, mean over layers 20–33	1280/chain	[228]	B
ESM-2 CDR	ESM-2, mean over CDR residues only	1280/chain	[228]	B
ESM-2 per CDR	ESM-2, CDR1/2/3 pooled separately then joined	3840/chain	[228]	B
ESM-2 variable positions	ESM-2, mean over the 20 highest-entropy positions	1280/chain	[228]	B
ESM-2 shift from parent	ESM-2 variant minus parent embedding	1280/chain	[228]	B
ESM-2 log-likelihood ratio	masked mutant vs parent log-probability: sum, mean, min	3	[229]	A
ESM3	protein LM	1536/chain	[230]	B

Representation	What it encodes	Features	Source	Role
ESM3 CDR	ESM3, mean over CDR residues	1536/chain	[230]	B
ESM C	protein LM	1152/chain	[231]	B
ProtT5	protein LM	1024/chain	[232]	B
ProtT5 CDR	ProtT5, mean over CDR residues	1024/chain	[232]	B
SaProt	structure-aware protein LM	1280/chain	[233]	B
AbLang-2	paired antibody LM, both chains read together	480	[234]	B
IgT5	paired antibody LM, both chains read together	1024	[161]	B
AntiBERTy	antibody LM	512/chain	[235]	B
AntiBERTy CDR	AntiBERTy, mean over CDR residues	512/chain	[235]	B
Ab-RoBERTa	antibody LM	768/chain	[236]	B
Ab-RoBERTa CDR	Ab-RoBERTa, mean over CDR residues	768/chain	[236]	B
Mutation features	wild-type and mutant residue, substitution score, changes in hydrophobicity, weight and charge, position; averaged over mutations	48	–	B/A
ESM-2 layer 24	ESM-2, layer 24 of 33 only	1280/chain	[228]	B
ESM-2 per region	ESM-2, each of the seven IMGT regions (FR1–FR4, CDR1–CDR3) pooled separately, then joined	8960/chain	[228]	B
SaProt CDR	SaProt, mean over CDR residues	1280/chain	[233]	B
ESM C CDR	ESM C, mean over CDR residues	1152/chain	[231]	B
Structural	solvent-accessible surface area and model confidence from a predicted structure, plus the paratope features	28	[237]	A
Paratope	sequence-based summaries of the CDRs, from their boundaries	8	–	A
AntiBERTy log-likelihood ratio	masked mutant vs parent log-probability under AntiBERTy	3	[235]	A

Table 5.12: Base and augmentation representations of each dataset, grouped by domain. The hybrid search selects features from the base representations. Augmentation features would be added after the hybrid is built, but no search in this chapter uses them (Section 5.2.3). A dagger marks datasets searched at the fast tier.

Dataset	Task	Base representations	Augmentation
Small molecules			

Dataset	Task	Base representations	Augmentation
QM9† [109]	HOMO–LUMO gap, regression	Mordred, Avalon, SMI-TED, ECFP4, SELFormer, ChemBERTa-2, Mol2vec, ChemBERT, GraphMVP, MoLFormer	Topological torsions, 3D pharmacophore, USRCAT, Coulomb matrix
hERG K_i [111]	hERG binding, pK_i, regression	MHG-GNN, Mordred, WL graph kernel, Avalon, GROVER, PDV, ECFP4, Atom pairs, ChemBERT, SMI-TED	MACCS keys, 3D pharmacophore, Coulomb matrix, USR, USRCAT
Caco-2 efflux [110]	efflux ratio (log), regression	WL graph kernel, MHG-GNN, ECFP4, PDV, GROVER, Avalon, Topological torsions, SMI-TED, Atom pairs, MoLFormer	MACCS keys, 3D pharmacophore, USRCAT, Coulomb matrix, USR
LogD [110]	LogD, regression	MHG-GNN, Mordred, GROVER, ECFP4, SMI-TED, MoLFormer, Avalon, Topological torsions, ChemBERT, GraphMVP	MACCS keys, 3D pharmacophore, USRCAT
Rat microsomal clearance [137]	intrinsic clearance (log), regression	PDV, MHG-GNN, SMI-TED, Mordred, MoLFormer, GROVER, WL graph kernel, Atom pairs, Avalon, ECFP4	3D pharmacophore, MACCS keys, USRCAT, Coulomb matrix, USR
MDR1–MDCK efflux [137]	efflux ratio (log), regression	Mordred, PDV, MHG-GNN, Avalon, GROVER, SMI-TED, Atom pairs, WL graph kernel, MACCS keys, ECFP4	3D pharmacophore, Coulomb matrix
hERG blocking†	blocker vs non-blocker, binary	MHG-GNN, Mordred, GROVER, Atom pairs, Avalon, WL graph kernel, ECFP4, Mol2vec, Topological torsions, MoLFormer	USRCAT, Coulomb matrix, MolCLR
Antibodies			
Anti-VEGF scan expression [195]	expression enrichment of variants, regression	ProtT5, ESM3 CDR, SaProt, IgT5, ESM-2 early layers, ESM C, AbLang-2, ESM-2 (joined chains), ESM-2, ESM-2 middle layers	Mutation features, ESM-2 log-likelihood ratio, Amino-acid composition
Anti-VEGF scan binding [195]	binding enrichment of variants, regression	ESM3, AbLang-2, ProtT5, IgT5, ESM C CDR, SaProt, ESM-2 early layers, ESM-2 variable positions, ESM-2, ESM-2 (joined chains)	ESM-2 log-likelihood ratio, Mutation features, AntiBERTy log-likelihood ratio, Amino-acid composition
Nanobody thermostability [197]	melting temperature, regression	SaProt, ESM3, ESM C, ESM-2 variable positions, ESM-2 CDR, ESM-2 early layers, ProtT5 CDR, IgT5, One-hot, ESM-2 shift from parent	Mutation features

Dataset	Task	Base representations	Augmentation
EGFR binding	mutational scan, regression	ESM-2 (joined chains), ESM3, Ab-RoBERTa, Ab-RoBERTa CDR, AbLang-2, Amino-acid composition, SaProt CDR, ESM C, ESM-2 early layers, Mutation features	none
Angiopoietin-2 binding	mutational scan, regression	SaProt, ESM-2 middle layers, ESM-2 layer 24, AntiBERTy, ESM3, ProtT5, ESM C, Ab-RoBERTa CDR, ESM-2 CDR, Mutation features	none
Antibody humanness [198]	human vs non-human, binary	IgT5, AntiBERTy CDR, Ab-RoBERTa, AbLang-2, BLOSUM62, ESM C, ESM-2 per CDR, ESM-2 early layers, ProtT5, ESM-2 middle layers	none
Binding free energy	$\Delta\Delta G$, regression	One-hot, AntiBERTy CDR, ESM-2 layer 24, ProtT5 CDR, ESM-2 (joined chains), AntiBERTy, Mutation features, ESM-2 shift from parent	Amino-acid composition
SMAC colloidal stability [201]	retention time, regression	ProtT5, ESM-2 (joined chains), ESM-2 variable positions, ESM3, IgT5, ESM-2 early layers, AbLang-2, ESM C, SaProt, AntiBERTy	Paratope, Structural
HIC retention [202]	retention time, regression	ESM-2 (joined chains), Amino-acid composition, ESM3, ProtT5, ESM-2 early layers, IgT5, ESM-2 variable positions, ESM C, AbLang-2, ESM-2 middle layers	Paratope, Structural
AC-SINS self-association [202]	AC-SINS shift, regression	IgT5, AbLang-2, AntiBERTy CDR, ESM-2 (joined chains), Ab-RoBERTa, SaProt, Amino-acid composition, ESM C, ESM-2 early layers, ProtT5	Paratope, Structural
Expression titer [202]	titer, regression	SaProt, IgT5, ESM-2, ESM C, ESM-2 variable positions, ESM-2 (joined chains), ESM-2 middle layers, AbLang-2, ProtT5, ESM3	Paratope, Structural
CHO polyreactivity [202]	polyreactivity, regression	ESM-2 (joined chains), ESM C, ProtT5, SaProt, One-hot, Amino-acid composition, ESM-2 per region, IgT5, ESM-2 variable positions, AbLang-2	Paratope, Structural
Chemical reactions			

Dataset	Task	Base representations	Augmentation
Buchwald–Hartwig yield [203]	yield (%), regression	CGR fingerprint, GROVER (reactants), Avalon (difference), Mordred (reactants), MHG-GNN (difference), ReactionT5, Component one-hot, Mol2vec (reactants), rxnfp, MACCS (difference)	MACCS (product), MACCS (reactants), 3D pharmacophore (product), 3D pharmacophore (reactants)

5.2.8 Models and metrics

The downstream models (Chapter 2) are random forest, XGBoost, LightGBM, an SVM, a DNN and a Gaussian process [115]. Each dataset uses the two that scored best in the earlier search of the whole dataset (Table 5.6). All six are used: the Gaussian process is one of the two models on four small antibody datasets (angiopoietin-2 binding, HIC retention, AC-SINS self-association and CHO polyreactivity). Tree ensembles are the baseline to beat, because a hybrid is a tabular feature matrix and tree-based models still lead on medium-sized tabular data of about 10,000 samples [238].

By default, every model uses the same settings on every dataset, so that score differences come from the input. An optional per-fold random search tries 20 draws of a model's settings on 3 group-aware inner folds of the outer training fold. In the search, the DNN is not tuned this way and relies on early stopping instead. On one outer fold of five datasets, tuning helped the SVM most and made the random forest worse on Buchwald–Hartwig yield (Additional file 23). The Gaussian process and the SVM are skipped above the dataset sizes in Table 5.5, because their fits no longer finish in useful time. Other models left out on named datasets are listed in Additional file 14, and all model settings are in Additional file 13.

The kNN model is not a default downstream model, but a third downstream model in the search with the nearest-neighbor objective (Chapter 6), on every dataset. It takes $k = 5$ neighbors by default, lowered to the square root of the number of training samples when a fold has fewer than 25.

Each dataset is scored in the metric its field normally uses, so that it can be compared with published work (Table 5.7). Regression on small molecules, QM9 included, and on reaction yields uses RMSE in the label's units. Antibody regression uses Spearman ρ, because these assays rank candidates and their scales differ between laboratories. Binary classification uses AUROC, which does not depend on a decision threshold. We do not average metrics across datasets.

Tables give the mean over the 5 folds, and the text gives the number of folds on which one side is ahead. We use no significance tests and no confidence intervals, because 5 fold scores are correlated through shared training molecules. No unbiased estimator of the variance of k-fold cross-validation exists that is valid under all distributions, and estimates that ignore the overlap underestimate it [239].

Two settings differ between runs. In later runs, the DNN clips each prediction to the range of its training labels. Without the clip, one molecule with an extreme descriptor value could make a fold's error several orders of magnitude larger on rat microsomal clearance. Most DNN results in this chapter come from runs without the clip, and the experiments where such an error occurred have no rat microsomal clearance result. In later runs, the Gaussian process also starts its kernel length scale at the square root of the number of features rather than at 1. On inputs of over 1,000 features, a start of 1 left some fits predicting one value for every sample. The hybrids of the four Gaussian-process datasets were chosen by searches that started at 1. We report all results as computed and name the setting beside each result it affects.

Summed over its 5 outer folds, the search of one dataset took 2.3 hours (hERG K_i) to 28.2 hours (MDR1–MDCK efflux) on small molecules, and 11.1 hours on Buchwald–Hartwig yield. On antibodies it took 0.4 hours (CHO polyreactivity) to 38.6 hours (the anti-VEGF expression scan). Each fold ran as its own job on a CPU node of the Oxford ARC or HTC cluster, with 8 cores and 64 GB of memory by default. The times count only each fold's last attempt, so a fold resumed from a stopped run is undercounted. One hERG K_i fold recorded only 90 seconds.

5.2.9 Comparators, controls and ablations

The hybrid is the best of many configurations; hence its gain may only reward a large search. Each experiment below asks one question about this or a related issue. Table 5.13 lists them, with the settings each varies and holds fixed.

Unless stated otherwise, every experiment below uses the dataset's grid (Table 5.6), its two downstream models, five outer folds and the dataset's own metric. From each fold's search it takes, for each downstream model, the two single representations and the two hybrids with the best scores on the training samples.

Best single representation and PCA

As a second reference beside the best single representation, every base representation is joined, standardized on the training samples and reduced by PCA [34] to each budget

Question	Datasets	What varies	Held fixed	Reported in
Does the hybrid beat the best single representation, and PCA at the same width?	19	hybrid, single representation, PCA of all representations joined	folds, models, the search's grid	Section 5.3.1
Does one model on the hybrid beat an ensemble of one model per representation?	16	hybrid, two ensembles, their membership and pooling	folds, models, representations	Section 5.3.1
At the same budget, is it better to draw on many representations or few?	18	top 1, 2 or 3 representations, an even split over all, the search's split	budget, importance method, models	Section 5.3.1
Does the hybrid help on rare scaffolds as well as common ones?	14	the half of the data, and which samples the search saw	the 2,000-sample subset, models	Section 5.3.1
Does the search overfit as the budget grows, or with one fold fewer?	13 and 14	budget, up to 8,000 features; training samples	every other setting of the grid	Section 5.3.1
Does the gain come from several representations or from fewer features?	14 regression	hybrid, mRMR, recursive elimination, random features	budget, models	Section 5.3.2
Must the internal evaluator match the downstream model?	13	internal evaluator	every other setting of the grid	Section 5.3.2
Does importance help during training?	20	feature and sample weights, above-median features	the hybrid, models	Section 5.3.2
Does a smaller hybrid replace tuning the model?	12	budget, the SVM's C or the forest's depth	the hybrid's settings	Section 5.3.2
Does the hybrid take in a new embedding?	11	which embedding is in the pool; random features in its place	models, grid	Section 5.3.2
Are the hybrid's prediction intervals calibrated?	19	hybrid or single representation	models, 90% conformal intervals	Section 5.3.2
Are the selected features the same on every fold?	14	fold	models, grid	Section 5.3.3

Table 5.13: The experiments of this chapter. One row is one experiment. Datasets is the number of the 20 datasets with a result, and every experiment uses five outer folds unless its section says otherwise.

of the dataset's grid. The downstream model is fitted on the principal components as they are. On each fold, the hybrid is compared with PCA at the hybrid's own budget; in this way, neither side's width is chosen on the held-out samples. PCA cannot have more components than training samples, and where the budget is larger the two widths differ.

Ensembles

The ensemble fits one model per representation and pools their predictions (late fusion), whereas the hybrid fits one model on one fused feature vector (early fusion) [165]. Every choice for the ensemble is made on an inner 3-fold group split of the outer training samples. The first ensemble uses the same downstream model on every representation, and the second uses each representation's own better model. Each picks its members, the top 3 representations or all of them, and one of seven ways to pool predictions, from the mean and median to trimmed, outlier-removed and score-weighted means.

Many or few representations

On each outer fold, the base representations are ranked by their summed feature importance on the training samples. The top 1, 2 or 3 of them supply the B features with the highest importance among their own features. These are compared with the search's own hybrid at B and with an even split of B over every base representation, each using the importance method that the fold's best hybrid used. The search's hybrid differs from the hand-built hybrids in two ways at once. It ranks representations by their standalone scores rather than by summed importance, and it stops at the last feature with importance above zero.

Frequent and rare scaffolds

Datasets above 2,000 samples are first reduced to about 2,000 by drawing whole scaffold families with a fixed seed. Scaffolds at or above the 80th percentile of group size form the frequent half, and the rest form the rare half. Each half gets its own search on each outer fold. The hybrid of a search on all the subset's training samples is also rebuilt and scored on each half. Three other splits were run as well (by heavy-atom count, label spread within a scaffold and feature spread within a scaffold), which are not reported here.

Overfitting with the budget

The budgets are the standard list (Table 5.5) plus 4,000 and 8,000 features. They also include 0.5, 0.75, 1, 1.25, 1.5 and 2 times the smallest outer training set, which places one budget where the number of features equals the number of samples. At every budget, the winning configuration is rebuilt on the outer training samples and scored on the held-out fold. The selection bias at a budget is the search's own score minus the held-out score; a positive value means the search promised more than the held-out fold gave. We report how the bias changes with the budget, and the budget the search picks on each fold from its training samples alone. We do not compare hybrids wider than the training set against narrower ones by their best held-out budgets, because that choice reads the held-out labels on both sides.

One fold fewer

A second check repeats each outer fold's search with one more fold removed from its training samples, for each of the other four folds. The held-out fold is the same and the two searches differ by one fold of training samples. Each search, however, picks its own budget, so the two hybrids can differ in width as well.

Feature reduction at the same width

On regression datasets, every method receives B features from all base representations joined, fitted on the fold's training samples. The methods are the two best hybrids at budget B from the fold's search, mRMR [175], recursive feature elimination [178] and B random features. mRMR ranks features by mutual information and stops at 200 features for run time. It consequently has no result on any small-molecule dataset, where every budget is 250 features or more. Recursive elimination fits a 50-tree random forest of depth 5 and removes 10% of the features at each step. Each of the dataset's two downstream models scores every method, and only the features differ.

Internal evaluator

One search per outer fold is run for each evaluator setting. The matched evaluator belongs to the downstream model's family, while a near-family evaluator pairs tree models with each other, the SVM with a Gaussian process, and the DNN with a narrower DNN. A far-family evaluator pairs tree models with an SVM, the SVM with LightGBM, and the DNN with k-nearest neighbors. A fourth setting scores the DNN with a wider DNN, and a fifth searches the dataset's own internal evaluators (Table 5.6).

Each setting's search picks its own budget, and the number of features can thus differ between settings. On every dataset, the SVM's near-family evaluator is a Gaussian process whose length scale started at 1 (Section 5.2.8).

Importance during training

Importance is computed on the training samples by the method the search picked. Sample weighting is not run for the Gaussian process, which takes no per-sample weights, nor on multiclass tasks. The five conditions are:

1. No weighting: the features as selected.
2. Feature weighting: standardized features multiplied by the square root of their importance, with the weights normalized to average 1.
3. Sample weighting: samples weighted by how well their label agrees with their 10 nearest neighbors in the rescaled space.
4. Combined weighting: feature and sample weighting together.
5. Above-median selection: only the features strictly above the median importance.

The weight of sample i is $1 - |y_i - \bar{y}_{N(i)}|/(\max y - \min y)$, clipped to lie between 0.1 and 1, where $\bar{y}_{N(i)}$ is the mean label of its 10 nearest neighbors by Manhattan distance. Tree splits depend only on the order of values [240], so for tree models feature weighting differs from no weighting only in that zero-importance features drop out. The rescaling does change which samples are nearest neighbors, and so changes sample weighting. Sample weighting down-weights rather than removes a suspect label, unlike neighbor-based filtering (Section 4.3.3). We found no QSAR study that does this, and learned per-sample weights for noisy labels come from general machine learning [241].

Fewer features or a tuned model

For the SVM and the random forest, the best hybrid at a small budget is fitted at the model's default setting. It is compared with a hybrid rebuilt at the largest width we allow: the smallest of the pool width, twice the number of samples in the whole dataset and 2,048 features. This wider hybrid is fitted at the setting that scores best on the other folds (the SVM's C or the forest's maximum depth). Datasets whose two downstream models include neither family are not run.

Adding a new embedding

Each embedding of the pool is taken in turn as the new one, and the search is run with it and without it. As a control, the embedding's selected features in the hybrid are replaced by the same number of Gaussian random features, with no new search. On LogD and hERG blocking the experiment ran on 2,000 molecules, drawn as whole scaffold families.

Calibration

Split-conformal 90% prediction intervals are built on the training samples and scored on the held-out samples by the fraction of held-out labels they cover and by their mean width.

Stability and blueprints

Selection on fold m, for $m = 1, \dots, F$, gives a binary vector $\mathbf{z}_m \in \{0,1\}^d$. Here d is the number of features in all base representations joined, not the size of the union of the selected features. We measure the stability of the selection with the index of Nogueira et al. [242],

$$\hat{\Phi} = 1 - \frac{\frac{1}{d}\sum_{f=1}^{d} s_f^2}{\frac{\bar{k}}{d}\left(1 - \frac{\bar{k}}{d}\right)}, \qquad s_f^2 = \frac{F}{F-1}\hat{p}_f(1-\hat{p}_f).$$

Here $\hat{p}_f$ is the fraction of folds that selected feature f, and $\bar{k}$ is the mean number of features selected. $\hat{\Phi} = 1$ means the same features on every fold, and $\hat{\Phi} \approx 0$ is what random selection gives. Each fold picks its own budget, and the index allows for the resulting difference in set size through $\bar{k}$. To read a blueprint, each selected feature is traced to its base representation, and its importance in the fitted model is measured with SHAP: TreeSHAP for tree models [189], the DeepLIFT-based SHAP explainer for the DNN, and Kernel SHAP on 50 held-out samples for the SVM and the Gaussian process [243].

5.2.10 Transfer, collaboration studies and separate sites

The first study asks whether search settings found on a large dataset can be reused on a small one instead of a new search. The settings are the importance method, allocation method, budget B and internal evaluator. The source is LogD, with 5,028 molecules, and the target is hERG pK_i, with 1,412 molecules. Both use the seven representations the two datasets share: Avalon, ChemBERT, ECFP4, GROVER, MHG-GNN, Mordred

and SMI-TED. The source settings come from one search on every LogD molecule, none of which is in the target. The closest prior work, Meta-QSAR [244], chose the learning method and representation for a new QSAR dataset from results on over 2,700 others and, like this study, reused decisions rather than selected features. We have not found a paper that applies feature indices selected on one QSAR dataset unchanged to another, although our literature search was not complete.

Four comparisons run inside every held-out fold of the target: the best single representation, a hybrid searched on the target's training molecules as the upper reference, the source settings with features re-selected on the target, and the source's selected features applied unchanged. The last two ask separately whether the search decisions and the chosen features transfer. We use five folds grouped by scaffold family, RMSE in pK_i units and the two downstream models decided for both datasets, the DNN and SVM.

Two further studies come from a collaboration with the Oxford Protein Informatics Group. The first repeats the activity-cliff QSAR setting of Dablander et al. [57] on three targets, with five scaffold folds. Dopamine D2 (6,333 compounds) and factor Xa (3,605) come from ChEMBL, and the SARS-CoV-2 main protease (1,924) comes from the COVID Moonshot project. Each target's search draws on eight representations, which do not include GROVER or SMI-TED. The downstream models are LightGBM and the DNN, and the metric is the mean absolute error (MAE) in log units of activity.

The second study counts the molecules that a representation cannot tell apart, meaning that two or more molecules get an identical feature row, on every representation of each dataset (Chapter 4), and asks whether a hybrid removes the problem. The representation with the most such molecules on hERG K_i, the WL graph kernel, is the target. Two hybrids add features of two partner representations to it. One adds ECFP4 and PDV, which are also blind to stereochemistry. The other adds ECFP4 and MHG-GNN, which together give no two hERG K_i molecules the same row. We report the score of each hybrid against the graph kernel alone, and the number of molecules that still share a row in the hybrid.

Federated learning trains one model across data held by several parties without moving the data [245]. It has been run across pharmaceutical companies [246], whose structures and assay results are commercial assets that cannot be pooled [18, 19]. Our collaborator, Lhasa Limited, uses the teacher-student process of Chapter 2 (Table 2.1) [149]. We use their code for the fingerprints, site models and site clustering, and re-implement from it the step that combines the site models' predictions.

Our addition is that feature selection, as well as model fitting, takes place at the

sites. One search on the training molecules of all sites together chooses the settings (allocation method, importance method and internal evaluator). Each site then selects features with these settings on its own molecules and reports only feature indices, importance values and one score. A coordinator splits the total budget B across sites in proportion to their scores, with at least one feature for each site whose selection succeeded. The main experiment below instead gives each site an even share, and no experiment compares the two splits, so we cannot say what the site scores buy. A feature already taken by another site is replaced by the site's next-ranked feature, which keeps all features in the hybrid distinct. Indices and importances are smaller, and harder to invert, than model weights or gradients. In related work, each client runs mutual-information feature selection on its own data and a server aggregates the results [247].

All four separate-site experiments run on Caco-2 and LogD, with five folds grouped by scaffold family and three seeds, the DNN and SVM, and budgets of 500 to 2,000 features. In the main one, each site holds the same molecules but one family of representations: substructure fingerprints, physicochemical descriptors or pretrained embeddings. It compares the best single family, picked on the training molecules, with one feature selection over all families, and both with one selection per family at an even share of the budget. When a family is narrower than its share, the rest goes to the other families. The even split can, as a result, build fewer features than one selection over all families. The built widths are recorded on every fold.

Three further experiments, with flipped labels, with one representation per site and with features selected per cluster, are described in Additional file 33.

The separate-site work has two further limitations beside the simulated sites (Section 5.4). It depends on the collaborator's implementation of the published process. That process trains a student on a large separate public set, a role the training molecules take here as a compromise, with held-out molecules excluded from it.

5.3 Results

We first compare the hybrid with single representations, with ensembles and with simpler ways of choosing features. We then ask where its gain comes from and what its blueprints show, and end with the collaboration and separate-site studies.

5.3.1 Performance of the hybrid

Every number in this section is a held-out score, in the dataset's own metric, over five outer folds unless stated otherwise, and most DNN results come from runs without the prediction clip (Section 5.2.8).

Against the best single representation

On small molecules the hybrid scores better than the best single representation on every dataset, with both downstream models (Table 5.14), and on the six regression datasets it does so on every fold. On hERG blocking the hybrid is ahead on most folds with each model. On every small-molecule dataset the hybrid also beats, with both models, the single representation that was best on the held-out molecules, a choice no user could make in advance (Table 5.14); on the Buchwald–Hartwig reactions it does not. On Caco-2 with the DNN, for example, the best single representation in hindsight is PDV. To see which representations the hybrid relies on, we shuffled each representation's features in turn on the held-out molecules (Additional file 24). On hERG K_i, Caco-2 and MDR1–MDCK efflux, the drop in score spreads about equally over five to seven representations, none carrying a third of it. On QM9 with LightGBM and on hERG blocking the hybrid leans on one representation, which carries most of the score on QM9 and about half on hERG blocking. The hybrid still beats every single representation there.

Table 5.14 gives the antibody results, which differ from one dataset to the next. On the anti-VEGF expression scan, one of the two largest antibody sets, the hybrid is ahead with the SVM on all five folds. On $\Delta\Delta G$ the mean gain is the largest in the chapter with both models, though the gain moves by more than its own size from fold to fold. On EGFR binding, expression titer and AC-SINS self-association the hybrid's mean is higher with both models, while it wins on only two to four of the five folds.

In an antibody representation, as in a molecular fingerprint, the features that matter are few and buried among many that carry nothing: a language model's embedding of the whole sequence mixes the CDRs that vary with a framework that mostly does not. Many of the antibody representations in the pools were built for this thesis to separate the two (Table 5.11). On four antibody datasets the hybrid beats even the single representation that was best in hindsight, and on two of them that representation is one of ours (Table 5.14). On the anti-VEGF expression scan it is the CDR readout of ESM3 with the SVM, and on $\Delta\Delta G$ it is the mutation features with the SVM and the change from the parent embedding with the DNN.

Dataset	Model	Best single base representation	Hybrid	Best in hindsight	Its score
RMSE ↓					
Caco-2 efflux	SVM	0.3036	0.2862	PDV	0.300
	DNN	0.3042	0.2775	PDV	0.298
LogD	SVM	0.4930	0.4302	MHG-GNN	0.493
	DNN	0.4497	0.4089	MHG-GNN	0.447
MDR1–MDCK efflux	SVM	0.4678	0.4370	PDV	0.468
	DNN	0.4732	0.4438	PDV	0.473
QM9 HOMO–LUMO gap	LightGBM	0.3720	0.3463	Mordred	0.372
	DNN	0.3619	0.3155	Mordred	0.362
Rat microsomal clearance	SVM	0.5209	0.4818	PDV	0.521
	DNN	0.5318	0.4974	PDV	0.533
hERG K_i	SVM	0.5881	0.5590	MHG-GNN	0.588
	DNN	0.6031	0.5691	MHG-GNN	0.593
Buchwald–Hartwig yield	RF	14.4056	15.2383	Avalon (difference)	14.2
	LightGBM	16.1679	15.6211	CGR fingerprint	15.2
Spearman ↑					
AC-SINS self-association	LightGBM	0.4631	0.5242	IgT5	0.472
	GP	0.5357	0.5547	AbLang-2	0.542
Angiopoietin-2 binding	SVM	0.4543	0.5216	AntiBERTy	0.577
	GP	0.4550	0.5049	SaProt	0.601
Anti-VEGF scan binding	RF	0.4325	0.5112	SaProt	0.445
	SVM	0.4538	0.5214	ProtT5	0.490
Anti-VEGF scan expression	SVM	0.7029	0.7729	ESM3 CDR	0.725
	DNN	0.7163	0.7352	ESM3 CDR	0.724
Binding free energy	SVM	0.0819	**0.2178**	Mutation features	0.203
	DNN	0.1317	**0.3542**	ESM-2 shift from parent	0.222
CHO polyreactivity	GP	0.5009	0.4419	ProtT5	0.510
	DNN	0.4003	0.4133	ESM C	0.431
EGFR binding	SVM	0.2821	0.3138	Ab-RoBERTa CDR	0.357
	DNN	0.1810	**0.2160**	Ab-RoBERTa	0.290
Expression titer	RF	0.2504	0.2706	IgT5	0.362
	DNN	0.3083	**0.3668**	ProtT5	0.372
HIC retention	GP	0.6000	0.5515	ESM-2 (joined chains)	0.600
	DNN	0.5311	0.5293	ESM-2 (joined chains)	0.531
Nanobody thermostability	SVM	0.6901	0.6899	SaProt	0.6901
	DNN	0.6939	0.6764	SaProt	0.694
SMAC colloidal stability	SVM	0.3485	0.3480	ProtT5	0.394
	DNN	0.2766	0.2677	ESM-2 early layers	0.313
AUROC ↑					
hERG blocking	XGBoost	0.8532	0.8644	Mordred	0.855
	LightGBM	0.8555	0.8666	MHG-GNN	0.860
Antibody humanness	RF	0.9659	0.9614	AntiBERTy CDR	0.967
	XGBoost	0.9511	0.9555	IgT5	0.963

Table 5.14: Held-out score of the best single base representation and the hybrid, per dataset and model, mean over outer folds, in each dataset's own metric. Bold: the four rows where the hybrid gains most over the best single base representation, as a fraction of the single score. Best in hindsight is the single representation best on the held-out samples, with its score.

In the $\Delta\Delta G$ hybrid, the representations built for this thesis carry most of the drop in score under shuffling with both models. On the other antibody datasets they carry much less, so they matter most where the antibodies differ by defined mutations. On AC-SINS self-association, another of the four, the hybrid draws mainly on two antibody language models, IgT5 and AbLang-2. Across the antibody datasets the hybrids draw on about one and a half to five representations by the same shuffling (Additional file 24). On angiopoietin-2 binding the best representation in hindsight with the SVM, AntiBERTy, was never ranked first on any fold's training antibodies.

On nanobody thermostability, SMAC colloidal stability, HIC retention and CHO polyreactivity the hybrid adds nothing (Table 5.14). On HIC retention it is behind with the Gaussian process, and here the search picked the same ESM-2 representation on every fold, so the single representation was not the problem. The thermostability label also pools five assays into one. On angiopoietin-2 binding, whose labels were read off a published heatmap, the hybrid is higher with both models on three of five folds (Table 5.14). Antibody humanness is a task one representation already solves, so the hybrid has nothing to add.

On the Buchwald–Hartwig reactions the hybrid lowers the RMSE with LightGBM and raises it with the random forest (Table 5.14), both by less than the spread across folds. The search drew on only two of the ten representations on every fold (Additional file 28): the reaction fingerprint and the change in the MHG-GNN embedding from reactants to product. The folds hold out whole additives, so on this split the reaction representations carry little that the hybrid could combine.

Against ensembles, and many representations against few

With the same downstream model on every representation, an ensemble of one model per representation beats the hybrid with both models on most antibody datasets with a result (Table 5.15). On the anti-VEGF expression scan the DNN ensemble is ahead of the hybrid on every fold. On small molecules the hybrid holds up better: the same-model ensemble beats it with both models only on hERG blocking, and on the Buchwald–Hartwig reactions. The hybrid is ahead of both ensembles on QM9 with the DNN and on hERG K_i with the SVM, on every fold. On antibodies the representations are language models of one sequence that make similar predictions, and averaging them smooths out the noise of each model on a small training set. On small molecules the representations carry different information, and one model on their selected features can use interactions that an average of separate models never sees.

At the same budget, drawing on several representations beats drawing on one (Ta-

Dataset	Model	Hybrid	Same-model ensemble	Best-model ensemble
RMSE ↓				
QM9	DNN	**0.317**	0.340	0.340
	LightGBM	0.346	0.360	
HK_i	SVM	**0.560**	0.582	0.583
	DNN	0.570	0.554	
Caco-2	DNN	0.278	0.271	0.281
	SVM	0.286	0.299	
LogD	DNN	0.409	0.4167	0.4175
	SVM	0.430	0.475	
MDR1	SVM	0.437	0.4550	0.4552
	DNN	0.444	0.442	
BH	RF	15.2	14.7	14.6
	LightGBM	15.6	15.1	
AUROC ↑				
HB	LightGBM	0.867	0.8719	0.8724
	XGBoost	0.864	0.874	
Hum	XGBoost	0.955	0.9684	0.9684
	RF	0.962	0.969	
Spearman ↑				
VEGF-E	SVM	0.772	0.781	0.783
	DNN	**0.735**	**0.785**	
Nb-Tm	DNN	0.6788	0.6785	0.6790
	SVM	0.687104	0.687096	
EGFR	SVM	0.316	0.369	0.408
	DNN	0.216	0.219	
$\Delta\Delta G$	SVM	0.213	0.233	0.260
	DNN	0.346	0.189	
SMAC	DNN	0.265	0.300	0.394
	SVM	0.345	0.402	
AC-SINS	LightGBM	0.524	0.510	0.517
	GP	0.557	0.525	
Titer	DNN	0.368	0.390	0.387
	RF	0.271	0.331	
PR-CHO	GP	0.443	0.485	0.508
	DNN	0.413	0.449	

Table 5.15: Hybrid against ensembles that fit one model per base representation and pool the predictions by one of seven rules (Section 5.2.9). One row is one dataset and downstream model. The same-model ensemble uses the row's model on every representation. The best-model ensemble gives each representation its own better model and is the same for both rows of a dataset. Held-out means over five outer folds, in the dataset's own metric. Dataset short names as in Table 5.8.

ble 5.16; every budget in Additional file 25). Filled from the top-ranked representation alone, the budget scores worse than the search's hybrid at every budget searched, with both models, on most datasets with a result, and at all but one budget on the rest. On small molecules, how the budget is split matters much less. An even split over the same representations, at the same width, comes close to the search's split there, except QM9 with the DNN. On antibodies the split can matter. At the smallest budget, the search's split is well ahead on nanobody thermostability with the DNN, angiopoietin-2 binding with the SVM and SMAC colloidal stability with the DNN. The even split is well ahead on $\Delta\Delta G$ with the SVM and expression titer with the random forest, and on EGFR binding at every budget. On small molecules, the gain comes more from which representations are drawn on, and from ranking features within each one, than from the allocation rule that sizes their shares.

Table 5.16: Search's hybrid against the budget filled from the top-ranked representation alone and against an even split over the same representations. One row is one dataset and downstream model, at the smallest budget (number of features) its search covers; every budget is in Additional file 25. Held-out means over five outer folds, in the dataset's own metric. Dataset short names as in Table 5.8.

Dataset	Model	Budget	Top representation	Search's hybrid	Even split
RMSE ↓					
QM9	DNN	500	0.357	**0.314**	**0.328**
	LightGBM	500	0.372	0.347	0.346
HK_i	SVM	250	0.599	0.564	0.562
	DNN	250	0.574	0.569	0.571
Caco-2	DNN	500	0.307	0.2827	0.2829
	SVM	500	0.302	0.285	0.283
LogD	DNN	500	0.455	0.4165	0.4175
	SVM	500	0.499	0.438	0.440
RLM	SVM	500	0.547	0.4876	0.4881
	DNN	500	0.560	0.506	0.511
MDR1	SVM	500	0.474	0.438	0.441
	DNN	500	0.495	0.452	0.460
BH	RF	500	15.5	15.4	14.9
	LightGBM	500	16.2	15.5	16.1
AUROC ↑					
HB	LightGBM	100	0.849	0.858	0.862
	XGBoost	100	0.848	0.863	0.856
Hum	XGBoost	10	0.894	0.941	0.936
	RF	10	0.932	0.952	0.944
Spearman ↑					

Dataset	Model	Budget	Top representation	Search's hybrid	Even split
Nb-Tm	DNN	50	0.553	**0.624**	**0.572**
	SVM	50	0.534	0.643	0.622
EGFR	SVM	250	0.254	0.321	0.357
	DNN	250	0.222	0.247	0.251
Ang2	GP	25	0.348	0.506	0.490
	SVM	25	0.397	0.549	0.477
$\Delta\Delta G$	SVM	100	0.125	0.240	0.358
	DNN	100	0.026	0.342	0.267
SMAC	DNN	500	0.259	0.308	0.255
	SVM	500	0.282	0.346	0.296
HIC-R	GP	250	0.431	0.552	0.499
	DNN	250	0.421	0.447	0.446
AC-SINS	LightGBM	50	0.323	0.521	0.516
	GP	50	0.336	0.521	0.491
Titer	DNN	250	0.230	0.296	0.292
	RF	250	0.247	0.292	0.381
PR-CHO	GP	250	0.294	0.471	0.483
	DNN	250	0.304	0.406	0.448

On the three small-molecule regression datasets with a result, the hybrid also gains on rare scaffolds, with each half searched separately (Table 5.17). On the rare-scaffold half of LogD the hybrid's gain is larger than on the frequent half with both models, and so it is on Caco-2 with the DNN. On hERG K_i the hybrid is ahead in both halves, by less on the rare half, and on hERG blocking it is ahead on the rare half with LightGBM only. On rare scaffolds few training molecules resemble the held-out ones, so a single representation has less to go on.

Against PCA of all representations joined, at the budget each fold's search picked, the results split by model. With the SVM the hybrid is ahead on every fold of hERG K_i, Caco-2, LogD, rat microsomal clearance and MDR1–MDCK efflux. With both models it is ahead on every fold of QM9 and hERG blocking. With the DNN, PCA is ahead on LogD on all 5 folds, 0.392 against 0.409 RMSE, and the hybrid is ahead on only 2 of 5 folds of Caco-2. On hERG K_i, MDR1–MDCK efflux and rat microsomal clearance the DNN hybrid and PCA are within 0.0020 RMSE. Additional file 26 compares this with two other experiments, one of which disagrees on LogD and Caco-2.

On antibodies neither method wins throughout. With the SVM, PCA is ahead on 4 of 5 folds of the anti-VEGF expression scan and angiopoietin-2 binding, and on 3 of 5 of $\Delta\Delta G$. With the DNN, PCA is ahead on 3 of 5 folds of the expression scan, and the hybrid on 4 of 5 of $\Delta\Delta G$, 0.354 against 0.239 Spearman.

Dataset	Model	Scaffolds	Molecules	Best single representation	Hybrid
RMSE ↓					
HK_i	SVM	Frequent	840	0.632	0.588
		Rare	572	0.686	0.655
	DNN	Frequent	840	0.633	0.596
		Rare	572	0.696	0.667
Caco-2	DNN	Frequent	1,256	0.291	0.282
		Rare	745	0.367	0.327
	SVM	Frequent	1,256	0.295	0.280
		Rare	745	0.338	0.330
LogD	DNN	Frequent	1,351	**0.607**	**0.587**
		Rare	649	**0.670**	**0.573**
	SVM	Frequent	1,351	0.642	0.575
		Rare	649	0.696	0.620
AUROC ↑					
HB	LightGBM	Frequent	1,288	0.765	0.763
		Rare	712	0.791	0.810
	XGBoost	Frequent	1,288	0.753	0.757
		Rare	712	0.793	0.789

Table 5.17: Best single representation and hybrid on molecules from frequent and from rare scaffolds, each half searched separately, from about 2,000 molecules of each dataset. One row is one dataset, downstream model and half. Held-out means over five outer folds, in the dataset's own metric. Dataset short names as in Table 5.8.

Overfitting of the search

With budgets searched up to 8,000 features, and at half to twice the number of training samples, the selection bias is small on the larger small-molecule datasets. On the small antibody datasets the search promises much more than it delivers (Table 5.18). On these datasets the folds also often pick a hybrid wider than their own training set, for example on every fold of EGFR binding with the SVM. The search's internal scores can choose between configurations with a few thousand molecules, but not with 150 to 540 training antibodies, where its choice is partly noise.

On permuted class labels, selecting features on the full dataset before cross-validation gave Ambroise and McLachlan [182] an error close to zero, against close to chance with selection redone inside each fold. Apart from the search's own scores in Table 5.18, every hybrid score in this chapter is on samples the search never saw, so the gains above are not this bias. The bias still exists inside the search, and the numbers above show where it is large: on small datasets, where many configurations are compared on few samples [248].

Dataset	Model	Score		Training samples	Budget picked on fold				
		Search	Held out		1	2	3	4	5
RMSE ↓									
HK_i	SVM	0.584	0.559	1,129	564	250	564	564	500
	DNN	0.591	0.592	1,129	2,000	100	4,000	250	2,000
Caco-2	DNN	0.281	0.283	1,726	1,726	8,000	8,000	1,726	1,000
	SVM	0.288	0.286	1,726	500	500	500	500	1,000
LogD	DNN	0.426	0.413	4,022	8,044	4,022	2,000	5,028	3,016
	SVM	0.448	0.431	4,022	1,000	2,011	1,000	1,000	2,011
BH	RF	14.8	15.6	3,057	250	100	100	50	100
	LightGBM	15.3	15.6	3,057	1,000	1,000	100	250	50
AUROC ↑									
HB	LightGBM	0.862	0.863	6,112	7,640	6,112	3,056	1,000	2,000
	XGBoost	0.859	0.861	6,112	1,000	6,112	2,000	3,056	3,056
Hum	XGBoost	0.967	0.958	439	878	500	658		
	RF	0.968	0.964	439	250	100	549		
Spearman ↑									
Nb-Tm	DNN	0.678	0.623	537	1,000	1,000	1,000	1,000	403
	SVM	0.668	0.679	537	403	250	250	403	268
EGFR	SVM	0.397	0.327	517	4,000	8,000	4,000	2,000	8,000
	DNN	**0.371**	**0.152**	517	10	100	25	4,000	100
$\Delta\Delta G$	SVM	0.365	0.194	343	25	1,000	10	100	50
	DNN	0.340	0.177	343	172	172	257	4,000	514
SMAC	DNN	0.417	0.310	189	500	100	2,000	2,000	142
	SVM	0.377	0.366	189	500	1,000	500	2,000	4,000
AC-SINS	LightGBM	0.538	0.505	189	236	4,000	94	500	500
	GP	0.560	0.557	189	236	284	94	500	2,000
Titer	DNN	0.426	0.294	186	500	186	186	93	4,000
	RF	0.419	0.268	186	250	2,000	186	372	2,000
PR-CHO	GP	0.517	0.478	152	8,000	8,000	8,000	8,000	1,000
	DNN	0.500	0.405	152	4,000	25	8,000	2,000	500

Table 5.18: The search's own score of the hybrid it picked, on each fold's training samples, against the held-out score of the same hybrid, with budgets searched up to 8,000 features and at 0.5 to 2 times the number of training samples. One row is one dataset and downstream model. Scores are means over five outer folds (three for antibody humanness), in the dataset's own metric. Training samples is the smallest training set of any fold; the last five columns give the budget (number of features) each fold picked. Dataset short names as in Table 5.8.

Leaving one more fold of training samples out of each fold's search makes the held-out score worse on every small-molecule dataset with a result, with both models (Table 5.19). On hERG K_i, Caco-2 and LogD it is worse by 0.009 to 0.037 RMSE. The two searches differ in two ways at once, the training samples and the budget each one picks, so we cannot say which of the two causes the loss.

5.3.2 Sources of the gain

This section asks whether the gain is more than a cut in the number of features, and how it depends on the downstream model and on reusing the importance scores in training. It ends with three smaller checks: tuning the model, adding a new embedding and calibration.

Selection at a fixed number of features

Figure 5.3 and Additional file 27 compare the hybrid with recursive feature elimination and random features over all base representations joined. Recursive elimination ranks all joined features in one list, the pooled ranking the hybrid avoids (Section 5.2.2). On QM9, Caco-2, LogD and MDR1–MDCK efflux the hybrid has the lower error at every budget with both models. The exception is hERG K_i with the DNN, where recursive elimination and even random features beat the hybrid at 1,000 and 2,000 features. On the Buchwald–Hartwig reactions recursive elimination is ahead at most budgets. On antibodies (Figure 5.4) random features are beaten at most budgets, but recursive elimination is ahead on $\Delta\Delta G$ with the SVM and on CHO polyreactivity at every budget. mRMR has a result only at 50 and 100 features, on nanobody thermostability and $\Delta\Delta G$. It is close to the hybrid on the first and far behind it on the second.

Dependence on the downstream model

Figures 5.3 and 5.4 show the budget each fold's search picked, and Additional file 28 lists it. We did not test whether the best settings follow measurable properties of a dataset, so we only describe these picks. Much of what looks like a dataset pattern was fixed in advance by each dataset's grid (Table 5.6). The QM9, LogD and hERG blocking searches could use only LightGBM gain. SMAC colloidal stability, HIC retention, expression titer and CHO polyreactivity searched only budgets wider than their 152 to 198 training antibodies. Where the grid left a choice, the downstream model moved the pick as much as the dataset did. Dataset size did not order the budgets either:

Dataset	Model	Held-out score		Stability	Features selected	
		Own search	One fold fewer		Fewest	Most
RMSE ↓						
HK_i	SVM	0.560	0.593	0.323	250	1,000
	DNN	0.568	0.605	0.273	250	1,831
Caco-2	DNN	0.278	0.288	0.399	993	1,964
	SVM	0.286	0.295	0.424	500	1,000
LogD	DNN	0.417	0.440	0.420	2,000	2,000
	SVM	0.430	0.460	**0.497**	1,000	2,000
BH	RF	15.2	15.3	0.156	479	1,000
	LightGBM	15.6	16.0	0.144	500	1,994
AUROC ↑						
HB	LightGBM	0.867	0.854	0.447	1,000	1,940
	XGBoost	0.864	0.853	0.327	475	1,940
Hum	XGBoost	0.955	0.954	0.122	50	500
	RF	0.962	0.959	0.098	100	1,000
Spearman ↑						
VEGF-E	SVM	0.772	0.755	0.270	2,000	2,000
	DNN	0.745	0.735	0.199	2,000	2,000
Nb-Tm	DNN	0.615	0.602	0.081	250	1,000
	SVM	0.687	0.636	0.130	250	500
EGFR	SVM	0.316	0.223	0.075	224	1,824
	DNN	0.203	0.143	0.085	187	1,817
$\Delta\Delta G$	SVM	0.213	0.240	0.113	100	498
	DNN	0.206	0.189	0.100	100	250
SMAC	DNN	0.317	0.253	0.099	128	2,000
	SVM	0.345	0.321	0.140	142	2,000
AC-SINS	LightGBM	0.524	0.483	0.100	100	2,000
	GP	0.557	0.512	0.122	250	2,000
Titer	DNN	0.318	0.281	0.169	500	1,683
	RF	0.271	0.292	0.115	250	2,000
PR-CHO	GP	0.443	0.447	0.090	500	2,000
	DNN	0.455	0.369	0.107	250	2,000

Table 5.19: Held-out score of each fold's own search and of the same search run with one more fold of training samples left out, and stability of the selected features across the five folds. One row is one dataset and downstream model. Scores are means over five held-out folds in the dataset's own metric. Stability is the index of Nogueira et al. [242]: 1 for the same features on every fold, about 0 for random selection. The last two columns give the fewest and most features selected on any fold. Dataset short names as in Table 5.8.

QM9, the largest dataset with 7,982 training molecules per fold, picked 1,000 features on 4 of 5 folds with the DNN, below LogD's 2,000.

On Caco-2 the search picked more features for the DNN than for the SVM, and on nanobody thermostability it picked 1,000 features for the DNN on four of five folds against 250 or 500 for the SVM (Figure 5.4). On LogD it picked 2,000 for the DNN on every fold, the top of the grid. With budgets searched up to 8,000 features, the DNN picked 2,000 to 8,044 features on LogD (Table 5.18). The SVM's smaller picks fit what Chapter 4 showed for distances: a kernel computed over many features is diluted by the ones that carry nothing, while a network can learn to ignore them. On angiopoietin-2 and $\Delta\Delta G$ the search picks at most a few hundred features. On expression titer the random forest's pick varies widely between folds, the selection noise of the previous section showing up in the budget. No one budget or representation count would serve every dataset and model, which is why both are searched.

Across the configurations of a search, the choice of downstream model accounts for more of the variation in score than the budget on most datasets with a result (Table 5.20). On LogD, for example, it accounts for about half of the variance and the budget for almost none. The budget still interacts with the model's own settings (Additional file 29). With the SVM strongly regularized (C of 0.01 or 0.1), the search picks the same budget on every fold of LogD, rat microsomal clearance, MDR1–MDCK efflux and Caco-2. With $C = 10$ it picks larger budgets on LogD and rat microsomal clearance, since a model that is allowed to fit more can use more features. The width of the DNN does not move its pick in the same way: on LogD it takes the top of the grid at every width, on all folds but one.

Scoring candidate hybrids with the downstream model itself, rather than a model of another family, makes almost no difference on small molecules (Additional file 30). On LogD, Caco-2 and hERG K_i with the SVM the three evaluator settings end within a few thousandths of an RMSE unit of each other. A cheap evaluator ranks candidate feature sets about the same way as the model that will use them, so on these datasets the search does not need the expensive one. On some antibody datasets it matters: the matched evaluator is best with both models on EGFR binding, expression titer and colloidal stability, and on EGFR binding with the SVM it is well ahead of a LightGBM evaluator.

Using importance during training

Using the importance scores again during training does little on small molecules (Table 5.21). The best of the four conditions gains little over the plain hybrid on any

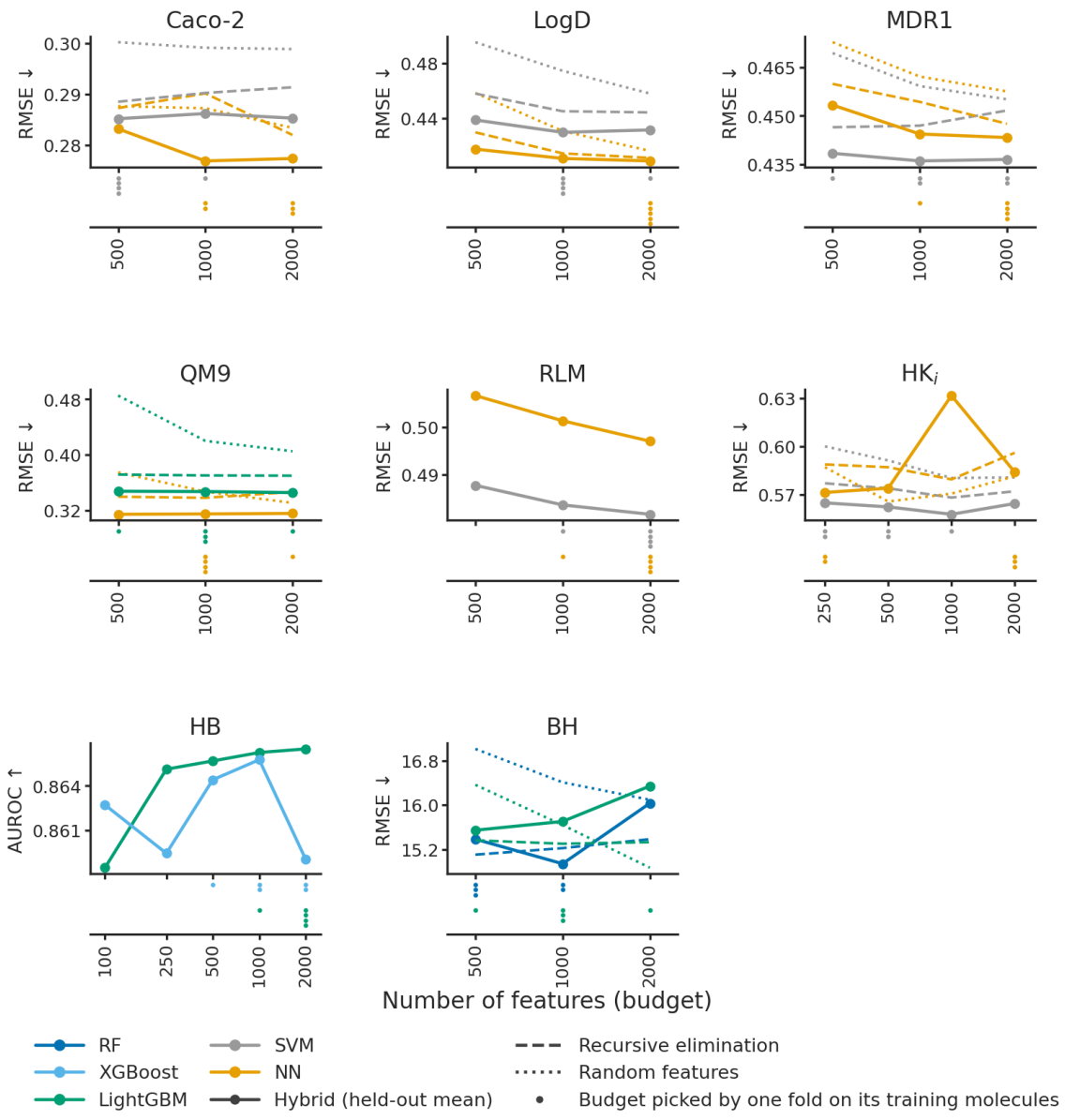


Figure 5.3: Small molecules and the Buchwald–Hartwig reactions. Held-out score against the number of features, one panel per dataset, one color per downstream model, in each dataset's own metric (arrow gives the better direction). Solid: the best hybrid of each fold's search. Dashed: recursive feature elimination. Dotted: randomly chosen features. Lines are means over five outer folds. Dots under each panel: the budget each outer fold picked on its training molecules, one dot per fold. Dataset short names as in Table 5.8.

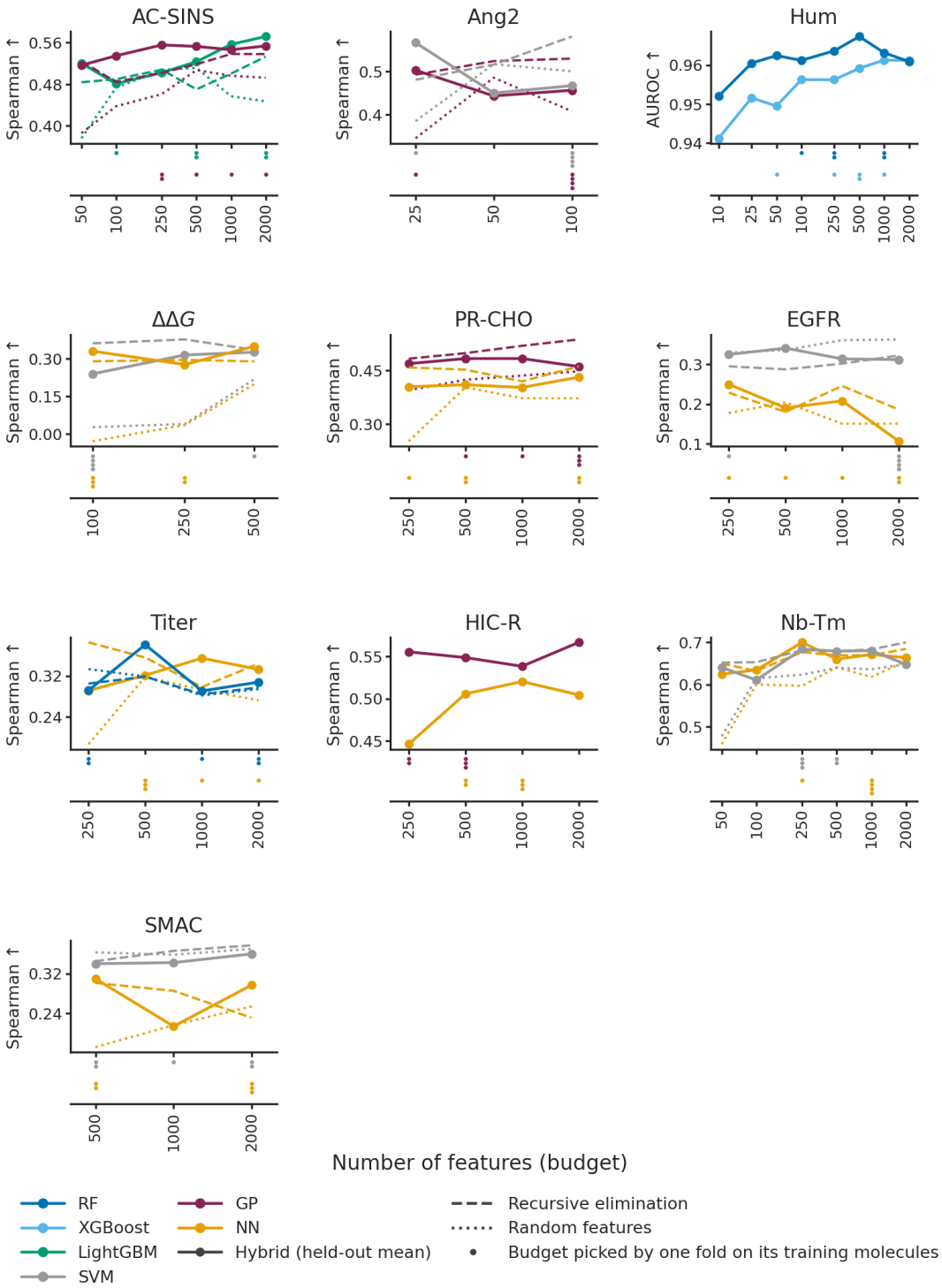


Figure 5.4: Antibodies. Held-out score against the number of features, one panel per dataset, one color per downstream model, in each dataset's own metric (arrow gives the better direction). Solid: the best hybrid of each fold's search. Dashed: recursive feature elimination. Dotted: randomly chosen features. Lines are means over five outer folds. Dots under each panel: the budget each outer fold picked on its training molecules, one dot per fold. Dataset short names as in Table 5.8.

Dataset	Downstream model	Budget	Interaction	Residual
Small molecules and reactions				
QM9	84.1	1.0	0.2	14.8
HK_i	59.7	1.0	2.4	37.0
Caco-2	0.0	6.6	43.6	49.9
LogD	**48.5**	**1.4**	1.6	48.5
MDR1	2.0	3.0	4.0	90.9
BH	82.6	1.6	0.2	15.6
HB	14.3	51.2	0.0	34.5
Antibodies				
Nb-Tm	7.2	25.4	1.9	65.5
EGFR	60.5	3.1	2.6	33.8
$\Delta\Delta G$	14.2	6.1	0.4	79.3
SMAC	34.2	0.2	4.8	60.7
HIC-R	47.4	6.8	1.0	44.8
AC-SINS	37.6	17.0	0.2	45.1
Titer	14.5	1.0	1.7	82.7
PR-CHO	33.6	20.0	0.7	45.7
Hum	0.9	41.6	0.1	57.5

Table 5.20: Share of the variance in the search's own score on the training samples, across a search's configurations, explained by the downstream model, the budget, their interaction and the residual (eta squared, in percent). One row is one dataset. Dataset short names as in Table 5.8.

small-molecule dataset, and feature scaling makes LogD with the SVM worse. The selection has already used the importance scores, so on small molecules there is little left for them to add. On antibodies some condition helps on most datasets, and by more. Feature scaling helps colloidal stability with the DNN most, and keeping the above-median features also helps $\Delta\Delta G$ with the SVM. Both act as further regularization on training sets of a few hundred antibodies. Which condition helps changes from one dataset and model to the next, so none of them is a default.

Table 5.21: Held-out score of the hybrid trained without weights, with features scaled by their importance, with molecules weighted, with both, and with only the features above the median importance. One row is one dataset and downstream model. Held-out means over five outer folds, in the dataset's own metric. The Gaussian process takes no sample weights. Dataset short names as in Table 5.8.

Dataset	Model	Un-weighted	Features scaled	Samples weighted	Both	Above median
RMSE ↓						

Dataset	Model	Un-weighted	Features scaled	Samples weighted	Both	Above median
QM9	DNN	0.316	0.318	0.313	0.317	0.321
	LightGBM	0.346	0.344	0.3470	0.3472	0.345
HK_i	SVM	0.560	0.574	0.571	0.584	0.567
	DNN	0.5921	0.582	0.587	0.578	0.5915
Caco-2	DNN	0.280	0.2762	0.2765	0.275	0.282
	SVM	0.286	0.2894	0.295	0.297	0.2887
LogD	DNN	0.409	0.416	0.408	0.41677	0.41685
	SVM	**0.430**	**0.462**	0.442	0.470	0.433
RLM	SVM	0.482	0.4883	0.4881	0.492	0.487
	DNN	0.499	0.500	0.497	0.498	0.503
MDR1	SVM	0.437	0.442	0.441	0.445	0.440
	DNN	0.444	0.451	0.4463	0.449	0.4458
BH	RF	15.21	15.24	15.4	15.3	15.5
	LightGBM	15.57	15.60	15.72	15.69	15.5
AUROC ↑						
HB	LightGBM	0.858	0.8603	0.857	0.856	0.8602
	XGBoost	0.863	0.860	0.856	0.858	0.851
Hum	XGBoost	0.9562	0.95649	0.9580	0.9579	0.95650
	RF	0.96115	0.96116	0.962	0.9606	0.959
Spearman ↑						
VEGF-E	SVM	0.772	0.7675	0.773	0.7678	0.770
	DNN	0.7348	0.738	0.740	0.743	0.7349
VEGF-B	RF	0.511	0.515	0.516	0.514	0.512
	SVM	0.507	0.505	0.509	0.506	0.508
Nb-Tm	DNN	0.628	0.607	0.632	0.592	0.594
	SVM	0.610	0.615	0.619	0.622	0.626
EGFR	SVM	0.314	0.243	0.327	0.249	0.290
	DNN	0.231	0.2219	0.2216	0.203	0.217
Ang2	GP	0.456	0.515			0.492
	SVM	0.467	0.456	0.469	0.455	0.434
$\Delta\Delta G$	SVM	0.239	0.261	0.246	0.262	0.312
	DNN	0.329	0.381	0.348	0.377	0.319
SMAC	DNN	**0.261**	**0.357**	0.309	0.355	0.311
	SVM	0.354	0.314	0.355	0.321	0.313
HIC-R	GP	0.551	0.537			0.499
	DNN	0.519	0.508	0.556	0.545	0.446
AC-SINS	LightGBM	0.481	0.4884	0.474	0.496	0.4882
	GP	0.535	0.501			0.527
Titer	DNN	0.344	0.355	0.349	0.363	0.340
	RF	0.149	0.150	0.161	0.165	0.225
PR-CHO	GP	0.425	0.421			0.461
	DNN	0.470	0.439	0.471	0.441	0.387

Choosing fewer features does not replace tuning the model on small molecules (Ta-

ble 5.22). A hybrid of up to 2,048 features with the model's regularization setting tuned beats the search's smaller hybrid at the default setting. It does so on all four small-molecule datasets with a result, for example 0.411 against 0.430 RMSE on LogD with the SVM. On nanobody thermostability, $\Delta\Delta G$, expression titer and the Buchwald–Hartwig reactions the smaller hybrid is ahead. On EGFR binding, the anti-VEGF expression scan and antibody humanness the tuned wider hybrid is ahead. The budget and the model's own regularization both control how much the model can fit. With thousands of molecules the model's setting does that better, but with a few hundred antibodies neither does so on every dataset.

Dataset	Model	Search's hybrid, default setting	Wider hybrid, setting tuned	Features in the wider hybrid
RMSE ↓				
HK_i	SVM	0.558	0.544	2,048
Caco-2	SVM	0.287	0.275	2,048
LogD	SVM	**0.430**	**0.411**	2,048
MDR1	SVM	0.438	0.435	2,048
BH	RF	14.97	15.04	2,048
Spearman ↑				
VEGF-E	SVM	0.771	0.773	2,048
Nb-Tm	SVM	0.676	0.667	1,344
EGFR	SVM	0.289	0.367	1,318
$\Delta\Delta G$	SVM	0.291	0.265	858
Titer	RF	0.292	0.267	478
AUROC ↑				
Hum	RF	0.963	0.965	1,128

Table 5.22: The search's hybrid at the model's default setting against a wider hybrid with the regularization setting tuned on each fold's training samples (C for the SVM, tree depth for the random forest). One row is one dataset and downstream model. Held-out means over five outer folds, in the dataset's own metric. The last column is the wider hybrid's number of features. Dataset short names as in Table 5.8.

On small molecules the hybrid uses a new embedding added as one more base representation (Additional file 31): its selected features beat Gaussian random features swapped into the same places in the same hybrid. No new search is run for the random features, so this is a weaker test than giving a search random features to choose from. The embedding's features win for every embedding on hERG K_i and Caco-2. On LogD, run on 2,000 molecules, they win for every embedding except Uni-Mol2 with the DNN. On SMAC colloidal stability, expression titer and the Buchwald–Hartwig reactions they win for fewer than half of the embeddings. On these two antibody datasets another

language model of the same sequence adds little.

With 90% conformal intervals, the hybrid's intervals on small molecules cover close to 90% of the held-out molecules, as the single representation's do (Additional file 32). They are also narrower, for example on LogD with the SVM. On several small antibody datasets the hybrid's intervals cover too few held-out antibodies, fewer than the single representation's, for example on angiopoietin-2 binding and $\Delta\Delta G$ with the SVM. The calibration set is part of a training fold of a few hundred antibodies, and the hybrid chosen on those antibodies is optimistic about them, the same selection bias as above.

5.3.3 Blueprints on specific datasets

Figure 5.5 shows one fold of each dataset: several hybrids draw on all ten representations in similar amounts, for example on LogD with the DNN and on the anti-VEGF expression scan with the SVM. Elsewhere one or two representations carry the hybrid: on EGFR binding almost all its features come from ESM-2 run on the joined chains.

LogD can be read chemically (Figure 5.6). The SVM's hybrid picked 372 features on all five folds, 38 of them Mordred descriptors, 36 ECFP4 bits and the rest from eight representations whose features carry no names. Among the descriptors are Crippen's calculated logP (SLogP), the number of basic groups (nBase), several surface areas binned by partial charge (PEOE_VSA), and counts of secondary amine and aromatic nitrogen environments. Among the bits are tertiary amine environments, in which a nitrogen is bonded to three carbons (`CN(C)C`), and aryl chloride environments. Unlike logP, which counts only the neutral form of a molecule, logD at pH 7.4 counts every charge state. At that pH a basic amine is largely protonated, which lowers logD below logP. A model of LogD should therefore need both lipophilicity and basicity, and the hybrid picked both on every fold. Lipophilicity comes from Mordred's SLogP, and basicity from Mordred's nBase and from ECFP4 amine bits.

The features picked are partly the same from fold to fold. On the stability index of Nogueira et al. [242], the small-molecule hybrids score higher than the antibody and Buchwald–Hartwig hybrids, and all score well below 1 (Table 5.19). Many features are correlated within and across representations, so a feature on one fold can stand in for a different one on another; an index well below 1 therefore need not mean the hybrid fits noise. A blueprint should still be read for what recurs across folds, as for LogD above. On antibodies, features drawn from several language models of one sequence are largely interchangeable, and on a few hundred antibodies which of them is picked changes from fold to fold.

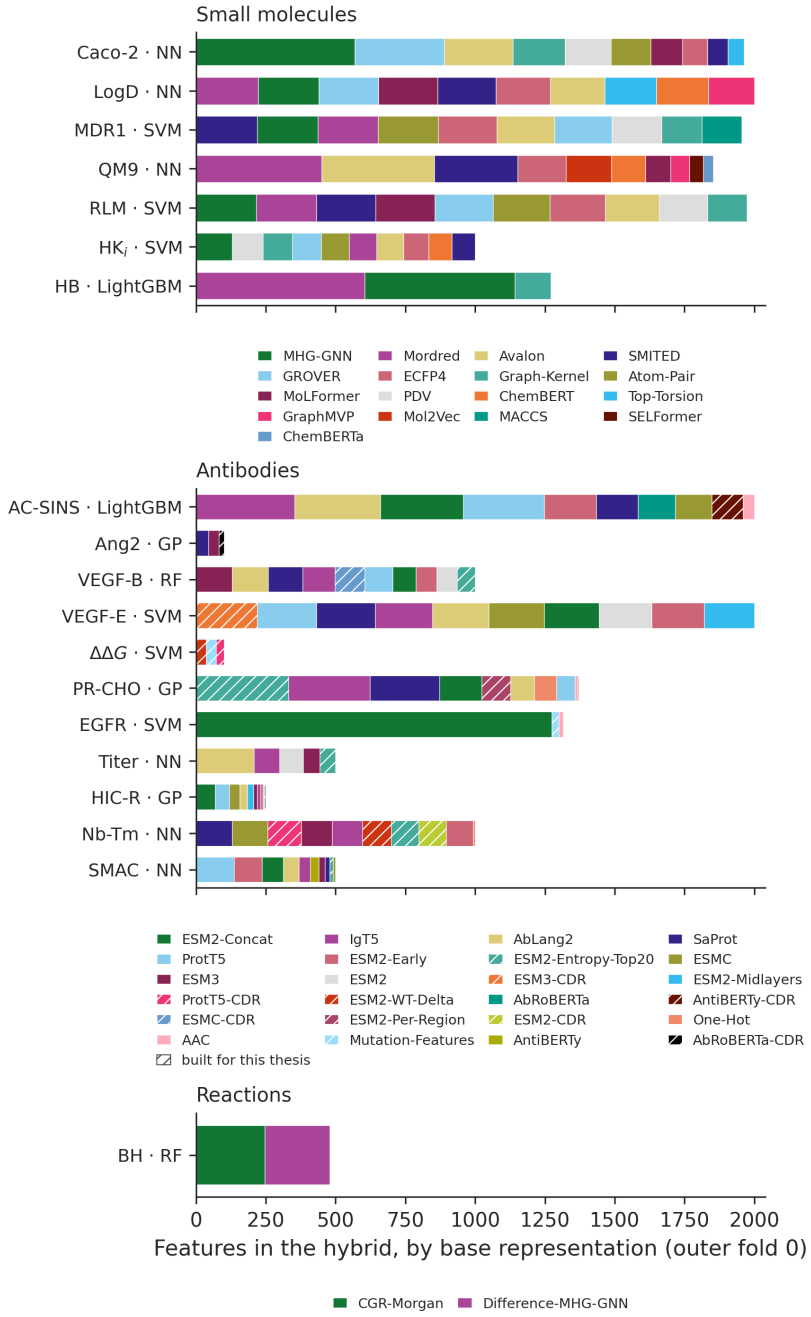


Figure 5.5: Number of features the hybrid took from each base representation, one bar per dataset, on the first outer fold with the dataset's first downstream model. Hatched segments are representations built for this thesis. Dataset short names as in Table 5.8.

5.3.4 Collaboration studies

The small-molecule result carries over to the three activity-cliff targets of Dablander et al. [57], data the hybrid was not developed on. For every target and model pair except factor Xa with LightGBM, the hybrid has a lower error than the single representation the same search ranked first (Table 5.23). With the DNN on factor Xa it is lower on all five folds. These targets have many activity cliffs, which is where a single similarity-

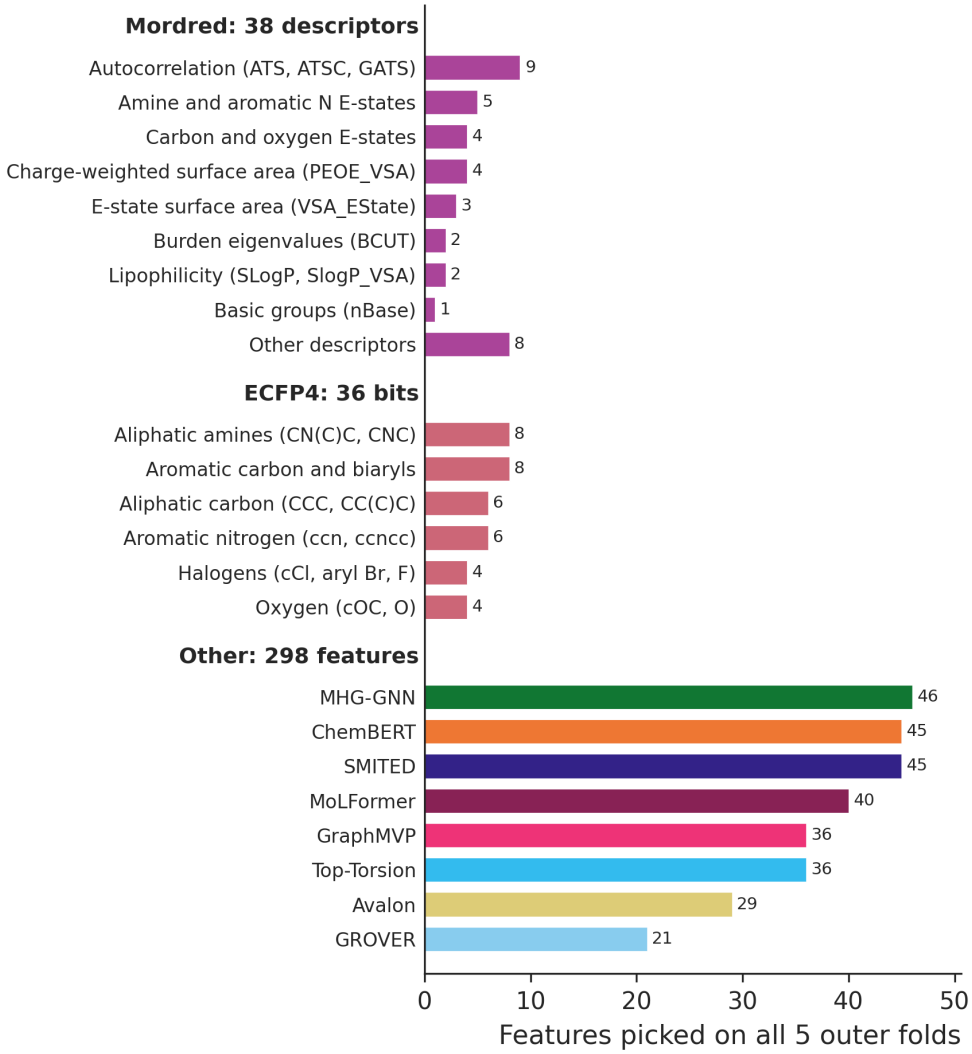


Figure 5.6: Number of features the SVM's LogD hybrid picked on all five outer folds. Mordred descriptors are grouped by descriptor family and ECFP4 bits by substructure. Every other representation is one bar.

based representation struggles most (Chapter 4).

On Caco-2, MACCS keys and the WL graph kernel both give some molecules a row that another molecule also has (Table 4.1). Amino-acid composition does the same to 432 of the 659 variants in the EGFR scan, which differ by single mutations. None of the hybrids built on these datasets leaves two samples with the same row.

On hERG K_i the graph kernel leaves some molecules sharing a row (Table 4.1). Adding features from ECFP4 and MHG-GNN lowers its RMSE with the SVM on all five folds, and no two molecules share a row (Table 5.24). Adding ECFP4 and PDV instead improves it almost as much while some molecules still share a row. Most of the gain therefore comes from the information the added representations give every molecule, and little from breaking the ties.

Target	Model	Top-ranked single representation	Hybrid
Dopamine D2	LightGBM	0.519	0.514
	DNN	0.523	0.499
Factor Xa	LightGBM	0.685	0.714
	DNN	**0.737**	**0.661**
SARS-CoV-2 main protease	LightGBM	0.465	0.452
	DNN	0.500	0.490

Table 5.23: Single representation the search ranked first and hybrid on the three activity-cliff targets of Dablander et al. [57]. One row is one target and downstream model. Held-out MAE in log units of activity, lower is better, mean over five outer folds.

	RMSE		
Representations	SVM	DNN	Molecules sharing a row
Graph kernel alone	**0.611**	0.624	105
Graph kernel, ECFP4 and PDV	0.575	0.580	56
Graph kernel, ECFP4 and MHG-GNN	**0.571**	0.581	0

Table 5.24: The WL graph kernel on hERG K_i alone and as a hybrid with features of two partner representations. One row is one set of representations. Held-out RMSE in pK_i units, lower is better, mean over five outer folds. The last column counts the molecules whose feature row another molecule also has.

Search settings found on LogD were reused on hERG K_i, with the features selected again on hERG K_i. They beat hERG K_i's best single representation with both models (with the SVM on all five folds), but stay behind a search run on hERG K_i itself (Table 5.25). Reusing settings saves the search on the smaller dataset at some cost in error.

Model	Best single representation	Search on hERG K_i	LogD settings, features reselected	LogD settings and features
SVM	**0.589**	0.563	**0.567**	0.569
DNN	0.604	0.567	0.589	0.593

Table 5.25: Search settings found on LogD reused on hERG K_i. One row is one downstream model. Columns: the best single representation on hERG K_i; a search run on hERG K_i; the LogD settings with features selected again on hERG K_i; the LogD settings and features reused unchanged. Held-out RMSE in pK_i units, lower is better, mean over five outer folds.

5.3.5 Features chosen across separate sites

Feature selection can run where data cannot be pooled. When each site holds one family of representations, one selection over all families beats the best family and the best single representation, on Caco-2 and LogD with both models (Table 5.26). On LogD with the SVM, for example, it is well ahead of the best family, and the best single representation is further behind. Selecting at each site with an even share of the budget does as well as one selection over all families on every dataset and model, although it can build a different number of features. Each site can therefore select on its own and send only feature indices.

Dataset	Model	Best single representation	Best family	One selection, all families	Selection per family
Caco-2	DNN	0.305	0.288	0.284	0.283
	SVM	0.304	0.301	0.2847	0.2845
LogD	DNN	0.453	0.446	0.419	0.420
	SVM	0.493	**0.460**	**0.43823**	0.43822

Table 5.26: Feature selection when each site holds one family of representations of the same molecules. One row is one dataset and downstream model. Columns: the best single representation; the best single family; one selection over all families; one selection per family at an even share of the budget. Held-out RMSE, lower is better, mean over three seeds of the five-fold mean.

Additional file 33 reports three further experiments: one hybrid over sites that each hold one representation, selection at sites with flipped labels, and selection per cluster of Caco-2.

5.4 Conclusions

This chapter asked how the best set of features can be selected for a given prediction task (Chapter 1). We found that the answer depends on the dataset and the downstream model, and that a search can find it. On every small-molecule dataset the hybrid beats the best single representation with both downstream models, and it beats even the representation that was best in hindsight. A likely reason is that the representations carry different information: on hERG K_i, Caco-2 and MDR1–MDCK efflux the hybrids draw on five to seven of them about equally. On small molecules this supports the hypothesis behind the hybrid, that the feature set is part of the problem.

At the same number of features the hybrid beats recursive elimination and random features on all but one small-molecule dataset with a result. Its gain is therefore more

than a reduction in width. Drawing on several representations beats drawing on the top one alone on most datasets. On small molecules, which representations are drawn on matters more than how the budget is split. Since we found no setting that suits every dataset and model, the settings should be searched for each new task rather than fixed in advance. The DNN picked more features than the SVM on Caco-2 and LogD, and cheaper internal evaluators were good enough on small molecules but not on some antibody datasets. Finally, the selected features can be read: on LogD the hybrid picks lipophilicity and basicity features on every fold.

On antibodies the hybrid may help for a similar reason: the features that matter may be few and buried in representations of the whole sequence. On $\Delta\Delta G$, AC-SINS self-association and both anti-VEGF scans it beats every single representation, even the one best in hindsight. On $\Delta\Delta G$ and the expression scan, that representation is one we built to read the regions and mutations that vary. On most antibody datasets, however, the large expression scan included, an ensemble of one model per representation does better. On reactions the method gains nothing on the additive split. On data held at separate sites, selecting at each site does as well as one selection over all representations. On small molecules the hybrid can also take in new embeddings added to the pool, but on two antibody datasets it gains little from them.

The fourth question asked whether weighting features by their importance improves the hybrid and model training. In construction, the features ranked by importance beat random features of the same number on most datasets. In training, by contrast, using the importance scores again adds little on small molecules and helps on some small antibody datasets, with no condition that works everywhere.

The hybrid needs data. Its gains are consistent on datasets of a few thousand samples and inconsistent on a few hundred, where the search's own scores overstate what the hybrid delivers (Section 5.3.1). Most of our antibody datasets are that small, so their results say as much about dataset size as about antibodies. The antibody pools also offer few kinds of representation, mostly language models of the same sequence. The search is costly and must be rerun when the data, representations or model change. Where a simpler selection, such as recursive elimination or a tuned model on a wider hybrid, does as well, the search costs more than it returns. The separate sites are simulated from two public datasets, Caco-2 and LogD.

This chapter used the hybrid only as input to a model. Whether hybrids keep more of their performance under label noise, and whether their features stabilize noise mitigation, are the subjects of Chapters 6 and 7. Chapter 6 first asks whether a feature set chosen for the task also gives better neighbors, which every method that relies on

similarity could use, including the detection of noisy labels.

Chapter 6

Similarity in an adaptive hybrid representation

6.1 Introduction

Chapter 5 built the adaptive hybrid representation, a set of features selected from several base representations for one task, as input to a model. Chapter 4 showed that in most standard representations the distance between two molecules barely rises with the difference in their labels. Any method that trusts a molecule's neighbors inherits this, such as predicting a label from the nearest neighbors or finding a noisy label by comparing it with its neighbors' labels. The first idea behind the hybrid was to compute distances on the features relevant to the task alone, and this chapter tests that idea.

A hybrid can be built for similarity in two ways. In the first, the search of Chapter 5 uses a nearest-neighbor model as its downstream model, and we measure distances in the hybrid it selects. In the second, the search ranks each candidate hybrid by how well each molecule's five nearest neighbors in it predict its label, and fits no downstream model. We ask two research questions:

1. Does distance in a hybrid track label differences better than distance in the best single representation?

2. Do the methods built on that distance work better in a hybrid? We test prediction from nearest neighbors, activity cliffs, Bayesian optimization, distance to the training molecules as a guide to prediction error, retrieval of neighbors and clustering.

This chapter uses 14 datasets: six small-molecule, seven antibody and one reaction

dataset. Antibody datasets outnumber small-molecule datasets, so we give every result by domain rather than pooled.

6.2 Methods

Datasets, folds (grouped by scaffold family for small molecules) and metrics are as in Chapter 5, with five outer folds. Of its 20 datasets, we leave out six: rat microsomal clearance, both anti-VEGF scans, angiopoietin-2 binding, antibody humanness and HIC retention. Some experiments have no result on some datasets because their runs did not finish in time (Section 5.2.5). MDR1–MDCK efflux has no row in Tables 6.5, 6.6, 6.7, 6.12, 6.10 and 6.13. QM9 has no row in Tables 6.12 and 6.10. Table 6.6 has no weighted values on Caco-2, hERG blocking, hERG K_i, expression titer and SMAC colloidal stability. In each outer fold, everything an experiment compares is chosen from searches on that fold's training molecules alone, and then scored on its held-out molecules. Features are scaled on the training molecules: features that only take the values 0 and 1 are left as they are, and the rest are standardized. In the experiments, but not inside the searches, held-out values are cut to the training range as well, which stops a single extreme molecule from dominating a distance. Where a statement covers all 14 datasets, "molecule" also stands for an antibody or a reaction.

The fold split, the sample of molecule pairs, the label permutations, the random feature subsets and the k-means starts all use fixed seeds. The spread we report is therefore the spread across the five outer folds of one split. We report the mean over the five folds, and the per-fold values where the spread matters, without significance tests. The simulation of choosing molecules to measure is repeated over three starting sets instead.

6.2.1 Feature spaces compared

Each outer fold runs three searches on its training molecules. The first, the general search, is the search of Chapter 5, run for each of the dataset's two downstream models (Table 5.6). On regression tasks the SVM is a support vector regressor. The second, the distance search, uses a k-nearest-neighbor model as its downstream model, with Euclidean distance and five equally weighted neighbors, or fewer when a fold has under 25 training molecules. It has its own lists of importance methods, allocation methods, numbers of representations and budgets. Its budgets are capped at 500 features, and on some datasets its other lists differ from the general search's too. On hERG K_i,

for example, the general search tries 250, 500, 1,000 and 2,000 features, while the distance search tries 100, 250 and 500. The third is the similarity search, described in Section 6.2.2. From these three searches, we take the following feature spaces:

- the nearest-neighbor model's best and second-best single representations, ranked by the distance search;
- each downstream model's best and second-best single representations, ranked by its general search;
- each downstream model's two best hybrids, four in all: the predictive hybrids;
- the top hybrid of the distance search: the distance hybrid;
- the top hybrid of the similarity search: the similarity hybrid, on the datasets where that search is defined.

Unless stated otherwise, "the single representation" in the results means the nearest-neighbor model's best single representation, and "the hybrid" means the distance hybrid. Retrieval and clustering also score one control: the principal components of all representations joined, with as many components as the hybrid has features.

6.2.2 Similarity in a hybrid

The search of Chapter 5 can be written as follows. Each configuration θ fixes an importance method, an allocation method, a number of representations and a budget B. Given base representations $R_1, \ldots, R_m$, a configuration builds a feature set $S_\theta \subset R_1 \cup \cdots \cup R_m$ with $|S_\theta| \leq B$. The search returns S_{θ^*}, where $\theta^* = \arg\max_\theta s(S_\theta)$ and s is the search's score. In the general search, s is a downstream model's score on the search's own held-out folds. In the similarity search, $s(S) = q_5(S)$, how accurately each molecule's label is predicted from its five nearest neighbors in S (Equation 6.1), and no downstream model is fitted. All other settings and the folds are shared with the general search. The neighbor score, computed on a validation set taken from the fold's training molecules, also replaces the internal evaluator when the budget is split.

Zheng and Tropsha [134] and Itskowitz and Tropsha [135] chose descriptors to maximize nearest-neighbor prediction within one descriptor set, and we modify this by choosing features across several representations. The similarity hybrid tests whether choosing features for how well the neighbors predict a label does better than choosing them for a model's prediction.

The score uses the Manhattan distance of Chapter 4, on features scaled as above. Each standardized feature adds a term of similar size to this distance. A representation's weight in the distance, which the allocation method sets, is thus roughly its number of features divided by the budget. A fingerprint weighs less than this per feature, because a feature that only takes the values 0 and 1 adds less. The neighbor score uses no importance weighting. In this score, Manhattan distance and inverse-distance weights are design choices that we do not compare with alternatives.

To rank whole candidates, we compute the score on the held-out molecules of each fold of the search, inside the outer training fold. These molecules act as each other's neighbors. Their groups (scaffold families, for small molecules) are split into up to 5 parts, and each molecule's 5 nearest neighbors are drawn from the other parts. A neighbor from the same scaffold family is an easy match and would inflate the score. This is the only place in the chapter where a molecule's own group is kept out of its neighbors. Each neighbor is weighted by the inverse of its distance [249]:

$$\hat{y}_i = \frac{\sum_{j \in N_5(i)} y_j / d_{ij}}{\sum_{j \in N_5(i)} 1 / d_{ij}}. \tag{6.1}$$

A neighbor at distance 0 receives all of the weight, shared equally if there are several. On regression data, the score q_5 is the Spearman correlation $\rho(\hat{\mathbf{y}}, \mathbf{y})$. On binary data, $\hat{y}_i$ is the distance-weighted fraction of neighbors in class 1, and $q_5 = 2(\mathrm{AUROC} - 0.5)$, where AUROC is the area under the receiver operating characteristic curve. Both scores put chance at 0, which the budget split needs, because each representation's share of the budget is its score divided by the sum of the scores. If any score is negative, all scores are first shifted so that the lowest is 0. The modelability index of Golbraikh et al. [126] (Chapter 4) is related, but uses a single nearest neighbor and a hard class match.

A fold of the search is undefined when it has fewer than 10 held-out molecules or fewer than 2 groups, or when its labels or predictions are constant. A candidate hybrid with an undefined fold is refused. We do not run the similarity search where its score is undefined:

- The four Ginkgo developability datasets in this chapter (AC-SINS self-association, SMAC colloidal stability, CHO polyreactivity and expression titer) use the competition's five folds as their five groups. Inside an outer fold, each held-out block of the search is then a single group, so no antibody has a neighbor outside its own group.

- On the change in binding free energy ($\Delta\Delta G$) caused by a mutation, one protein complex holds 69 of the 429 rows. On outer folds 1 to 4 it falls alone into one held-out block of the search, and every candidate is refused. We skip this dataset on all five folds.

6.2.3 Nearest-neighbor settings

Every experiment that predicts from neighbors or measures retrieval scores each feature space at every combination of the settings in Table 6.1, instead of at one chosen setting. A held-out molecule's prediction is the weighted mean of its neighbors' labels,

$$\hat{y}_i = \frac{\sum_{j \in N_k(i)} w_{ij}\, y_j}{\sum_{j \in N_k(i)} w_{ij}}, \tag{6.2}$$

where $N_k(i)$ is the k training molecules nearest to i under distance d, and w_{ij} is the neighbor weight. On classification data, each class receives the summed weight of its neighbors. Unlike in the similarity score, a training molecule from the held-out molecule's own group (scaffold family, for small molecules) can be one of its neighbors. The one experiment that does not use these settings is the one on all representations joined (Section 6.2.4).

Setting	Levels	Source	Status
Distance d	Manhattan; cosine; Minkowski with $p = 0.5$, $d(\mathbf{x}, \mathbf{y}) = \left(\sum_j \lvert x_j - y_j \rvert^{0.5}\right)^2$	Chapter 4 (Manhattan, cosine); this chapter (Minkowski)	Reused
Neighbors k	5, 10, 20	Standard	Reused
Neighbor weight w_{ij}	$1/r_{ij}$, where r_{ij} is the rank of j among the k neighbors of i, nearest first; $1/d_{ij}^2$	None found; Shepard [250]	New; reused
Importance weighting	None; $\mathbf{w} = (\mathbf{s}/\bar{s})^a$; $\mathbf{w} = \mathbf{s}^{1/2}$	Wettschereck et al. [251]	Modified

Table 6.1: Nearest-neighbor settings. One row is one setting, with its levels; every combination of levels is scored. In the last row, $\mathbf{s}$ is the vector of feature importances, $\bar{s}$ its mean and a an exponent set by the distance. Minkowski distance is left out on datasets above 5,000 molecules (hERG blocking, LogD and QM9).

We found no published source for the rank scheme and count it as new. We sweep it and $1/d^2$ rather than choose between them.

Importance weighting is feature-weighted nearest neighbors [251], modified so that the weights come from the importance scores of the hybrid search. Each feature is

multiplied by its weight before the distance is computed, $\mathbf{x}' = \mathbf{x} \odot \mathbf{w}$, with negative importances set to 0. For a single representation, the importance $\mathbf{s}$ is computed on the fold's training molecules, with LightGBM gain in most experiments. For a hybrid, it is the importance its search used to select the features, with two exceptions. Distance against label difference refits LightGBM gain on the hybrid's features together, and retrieval recomputes the search's importance on the hybrid as a whole. We use two formulas for $\mathbf{w}$:

- for distance against label difference and the best number of features, $\mathbf{w} = (\mathbf{s}/\bar{s})^a$, with $a = 1$ under Manhattan distance, $a = 0.5$ under cosine and $a = 2$ under Minkowski with $p = 0.5$;
- for every other experiment that weights, $\mathbf{w} = \mathbf{s}^{1/2}$ under every distance.

The exponent a of the first formula is set so that each feature's term in the distance is multiplied by its relative importance, $s_f/\bar{s}$, under all three distances. The second formula has no such matching. Under Manhattan distance, a feature with four times the average importance counts four times as much as an average feature under the first formula, and only twice as much under the second. The weighted distances of Sections 6.3.1 and 6.3.2 are therefore the same distance only under cosine. Some experiments divide the second formula by a constant, which changes neither the ranking of distances nor any prediction from neighbors. The square root was first written for Euclidean distance and kept for the others, and no experiment in this chapter varies the exponent.

6.2.4 Experiments

Table 6.2 lists every similarity experiment, and the paragraphs below define what each one estimates.

Distance against label difference

For one feature space, this experiment estimates the Spearman correlation between the distance of two held-out molecules and the absolute difference in their labels. A positive value means that molecules further apart differ more in their labels. We estimate it in each outer fold from a sample of up to 50,000 pairs. We measure it first in each of the dataset's representations alone, under every distance of Table 6.1, with no selection and no importance weighting. We then measure it in the feature spaces of Section 6.2.1, unweighted and weighted.

Question	Datasets	Varied	Held fixed	Score	Reported in
Does distance track label differences?	All 14	Feature space; distance; importance weighting	Up to 50,000 held-out pairs per fold	Spearman between pair distance and absolute label difference	Section 6.3.1
Best number of features	10 with a DNN, no MDR1–MDCK efflux	Budget	Each downstream model's top hybrid at each budget of its search	Dataset metric and Spearman as above, same hybrid	One line, Section 6.3.1
Prediction from nearest neighbors	All 14	Feature space; settings of Table 6.1	Comparison fixed before the run: Manhattan, 5 neighbors, rank	Dataset metric	Section 6.3.2
Importance weighting in four models	12, no QM9 or MDR1–MDCK efflux; joined: 13 regression	Model; importance weighting; feature space	Gaussian process up to 5,000 molecules, SVM up to 10,000; joined: 10 neighbors, $1/d$, Manhattan	Dataset metric, weighted against unweighted	Section 6.3.2
Choosing molecules to measure	hERG K_i, Buchwald–Hartwig	Surrogate model; feature space	Three starting sets of at least 5% of the molecules; 100 picks by expected improvement	Area under the best-found-so-far curve	One line, Section 6.3.2
Activity cliffs	Five small-molecule, no MDR1–MDCK efflux	Feature space; downstream model	ECFP4 Tanimoto at least 0.55; gap set for 15% of training molecules; at least 10 cliff and 10 other molecules	Dataset metric on cliff and other held-out molecules	Section 6.3.3
Distance against prediction error	All 14	Feature space; distance; number of neighbors; importance weighting; model fit	Mean distance to the nearest training molecules	Spearman between distance and prediction error	Section 6.3.4
Retrieval and clustering	All 14	Feature space; settings of Table 6.1; 3, 5 or 10 clusters	k-means under squared Euclidean distance	Label gap to the nearest neighbors; label coherence of clusters	Section 6.3.4
Label information in the kept features	All 14	Kept features, random subset, principal components	At most 50 components; three label permutations	Summed mutual information, held-out molecules	One line, Section 6.3.4

Table 6.2: Similarity experiments. One row is one experiment. Datasets are counted from the 14 of this chapter. All use the five outer folds, except choosing molecules to measure, which uses three starting sets.

Prediction from nearest neighbors

We predict each held-out molecule's label from its nearest training molecules (Equation 6.2), and make two comparisons. The first was fixed before the run, and nothing in it is picked on the folds it is scored on. It compares the distance hybrid against the single representation, both unweighted, under Manhattan distance with 5 neighbors weighted by the inverse of their rank. The second picks a setting again for each held-out fold, using the other four folds' held-out scores. On the hybrid side, it picks among up to six hybrids (the four predictive hybrids, the distance hybrid and the similarity hybrid), weighted and unweighted, at every setting of Table 6.1, up to 216 combinations. On the single side, it picks among the nearest-neighbor model's two single representations, unweighted only, which gives 36. On the three datasets with two distances, the numbers are 144 and 24, and where the similarity hybrid is undefined

they are 180 and 36. So the two sides differ in two ways at once. The hybrid side picks from five or six times as many combinations, and only the hybrid side can use importance weighting.

Importance weighting in four models

We score every feature space with and without importance weighting, using four models that read a distance. The first three are prediction from nearest neighbors at every setting, a Gaussian process with an RBF kernel and an SVM with an RBF kernel. The fourth is each downstream model given four extra features from its nearest training molecules, following q-RASAR [252]. The four extra features are the neighbors' mean label and its spread, the distance to the nearest neighbor and the mean distance to the neighbors. They are computed at the three neighbor settings that rank best in a screen on the training molecules. The Gaussian process and the SVM run on regression data only. A separate experiment applies the weighting to all representations joined, on regression datasets, once for each importance method.

Activity cliffs

An activity cliff is a pair of structurally similar molecules with very different activity [58, 77]. Only the two hERG datasets measure an activity, but we keep the term for QM9, Caco-2 and LogD. We decide which molecules are cliff molecules in each outer fold, from its training molecules. We keep the structural side of the rule of Hu et al. [145]: two molecules are similar when their ECFP4 Tanimoto similarity is at least 0.55. We modify its activity side, because a gap of two log units flags no molecule at all on Caco-2. Instead, we choose the gap so that 15% of the training molecules are cliff molecules. A training molecule is a cliff molecule when it is in at least one similar pair whose labels differ by more than the gap. On classification data, a similar pair counts when its two molecules are in different classes. A held-out molecule is a cliff molecule when it has at least two similar training molecules whose labels span more than that gap; its own label is never read. Each downstream model is fitted on its two best single representations and its two best hybrids. Each fit is scored separately on the cliff molecules and on the other held-out molecules. The main comparison is the first hybrid against the best single representation.

Distance against prediction error

This follows Sheridan et al. [253], who found that a molecule's similarity to the training molecules tracks how well it is predicted. Each downstream model is fitted twice, on all representations joined and on its own first hybrid, and we record its error on every held-out molecule. On regression data this is the absolute error, and on classification data it is one minus the predicted probability of the true class. We then compute the Spearman correlation between that error and the molecule's mean distance to its nearest training molecules, at 5, 10 and 20 neighbors and every distance. We do this in each feature space of Section 6.2.1, unweighted and weighted.

Retrieval and clustering

Retrieval measures how close a molecule's neighbors are to it in label. For each held-out molecule, the label gap (lower is better) is the weighted mean absolute difference between its label and those of its nearest training molecules, at every setting of Table 6.1. On classification data, it is one minus the weighted share of neighbors in the molecule's class. For clustering, we fit scikit-learn's k-means [254] on the training molecules with 3, 5 and 10 clusters, and assign each held-out molecule to a cluster. Label coherence (higher is better) is one minus the mean standard deviation of the held-out labels within a cluster, divided by the standard deviation of all held-out labels. On classification data, it is the mean share of the largest class in each cluster. Clusters with fewer than two held-out molecules are left out.

Three smaller checks

To find the best number of features, we take each downstream model's top hybrid at every budget of its search. At each budget we record, for that same hybrid, its prediction score and the correlation between distance and label difference, and find the budget where each peaks. On hERG K_i and Buchwald–Hartwig only, we simulate choosing which molecules to measure, a form of Bayesian optimization. Starting from a set of whole groups (scaffold families on hERG K_i, additives on Buchwald–Hartwig) holding at least 5% of the molecules, a surrogate model picks 100 molecules one at a time by expected improvement [255]. We repeat this from three starting sets, and the score is the area under the best-found-so-far curve. Last, for each representation a predictive hybrid draws from, we compare the label information in the features it kept against principal components and a random subset of the same size. Each block of features is reduced to at most 50 principal components. A mutual-information

estimate is summed over the components, and the mean over three label permutations is subtracted. We use scikit-learn's estimators, which follow Kraskov et al. [256] for continuous labels and Ross [257] for classes. This sum counts information shared by two components twice.

6.3 Results

We first ask whether distance in a hybrid tracks label differences more closely than in a single representation, and then whether the methods built on that distance gain from it.

6.3.1 Distance against label difference

As in Chapter 4, distance in a standard representation rises only weakly with label difference (Table 6.3). On the small-molecule datasets, the best of 30 pairs of representation and distance, picked on the held-out molecules, reaches at most 0.281 Spearman, on MDR1–MDCK efflux. It varies widely between folds, for example on hERG K_i.

On Buchwald–Hartwig and the small-molecule datasets other than MDR1–MDCK efflux, no representation has a correlation at or below zero. On MDR1–MDCK efflux, EGFR binding, CHO polyreactivity, AC-SINS self-association and SMAC colloidal stability, at least one representation does (Table 6.3). SMAC colloidal stability is weakest: its best single representation reaches 0.016 Spearman, and about half of its combinations of representation and distance are at or below zero.

In the distance hybrid, distance tracks label differences more closely on all six small-molecule datasets, all seven antibody datasets and Buchwald–Hartwig (Table 6.4). It is higher in every fold on LogD and lower in two folds on Caco-2.

The predictive hybrids, built for each downstream model, do not always do this. On Caco-2, hERG blocking, EGFR binding, expression titer, SMAC colloidal stability and Buchwald–Hartwig, some track label differences less closely than their model's best single representation (Table 6.5). On CHO polyreactivity all four are lower, for example 0.106 to 0.064 Spearman for the DNN's first hybrid. A hybrid used for distance therefore has to come from the distance search.

Eight datasets have weighted values (Table 6.6). On five of them, importance weighting alone closes most of the gap between the single representation and the distance hybrid. On LogD, AC-SINS self-association and Buchwald–Hartwig, the weighted single representation passes the unweighted distance hybrid, on LogD with

	Best pair of representation and distance				All pairs	
Dataset	Pair	Mean	Lowest fold	Highest fold	Median	Lowest
Small molecules						
QM9	ChemBERTa-2, Manhattan	0.279	0.206	0.333	0.186	0.038
Caco-2	Topological torsions, Manhattan or Minkowski	0.229	0.170	0.338	0.133	0.004
MDR1-MDCK efflux	Atom pairs, Manhattan or Minkowski	**0.281**	0.258	0.305	0.127	−0.052
hERG K_i	SMI-TED, Manhattan	0.117	0.028	0.237	0.081	0.040
LogD	MoLFormer, Manhattan	0.147	0.104	0.193	0.085	0.050
hERG blocking	MoLFormer, cosine	0.041	0.021	0.054	0.021	0.003
Antibodies						
EGFR binding	AbLang-2, cosine	0.091	0.002	0.184	0.044	−0.022
Nanobody thermostability	One-hot, Manhattan or Minkowski	0.187	0.121	0.256	0.121	0.075
AC-SINS self-association	AbLang-2, cosine	0.050	0.015	0.077	0.022	−0.031
SMAC colloidal stability	AbLang-2, Minkowski	**0.016**	−0.074	0.104	**−0.0004**	−0.027
CHO polyreactivity	SaProt, cosine	0.133	0.034	0.256	0.088	−0.041
Expression titer	ESM-2 middle layers, Minkowski	0.142	0.044	0.266	0.099	0.005
Binding free energy ($\Delta\Delta G$)	ProtT5 CDR, Manhattan	0.168	−0.017	0.487	0.140	0.073
Reactions						
Buchwald–Hartwig yield	rxnfp, Manhattan	0.135	0.122	0.156	0.080	0.046

Table 6.3: Distance against label difference in single representations. One row is one dataset. Values are the Spearman correlation between the distance of two held-out molecules and their absolute label difference (higher is better), over pairs of one representation and one distance, unweighted. Best pair: picked on the held-out molecules; mean over five outer folds, and its lowest and highest fold. All pairs: median and lowest of the fold means.

Dataset	Best single base representation	Hybrid built for distance
Spearman ↑		
Caco-2 efflux	0.1517	0.1969
LogD	0.0765	0.2687
MDR1-MDCK efflux	0.2286	0.2883
QM9 HOMO-LUMO gap	0.1298	0.3779
hERG Ki	0.0729	0.2625
hERG blocking	0.0257	0.0552
AC-SINS self-association	0.0210	0.0718
Binding free energy	0.1181	0.1919
CHO polyreactivity	0.1033	0.1276
EGFR binding	0.0378	0.0737
Expression titer	0.0097	0.1063
Nanobody thermostability	0.1201	0.2110
SMAC colloidal stability	0.0002	0.0501
Buchwald-Hartwig yield	0.1074	0.1431

Table 6.4: Spearman correlation between the distance between two samples and the difference in their labels, per dataset, mean over outer folds.

0.276 against 0.269 Spearman. On EGFR binding, CHO polyreactivity and $\Delta\Delta G$, weighting closes little of the gap.

Instead of the representation the nearest-neighbor model ranked first for prediction, we can pick, with hindsight on the held-out molecules, the representation and distance whose distance best follows the label (Table 6.3). That pick beats the distance hybrid on Caco-2, EGFR binding, expression titer and CHO polyreactivity (Table 6.6). On Caco-2 the similarity hybrid beats that pick in turn.

On QM9, Caco-2, LogD, nanobody thermostability and CHO polyreactivity, the DNN's hybrid predicts best at more features than the number at which its distances best track label differences (Table 6.7). On hERG K_i and expression titer both peak at the same number of features, and on EGFR binding, SMAC colloidal stability and $\Delta\Delta G$ the prediction score peaks at fewer. On LogD the DNN's hybrid predicts better at 2,000 features than at 500, while distance in the same hybrid falls from 0.231 to 0.154 Spearman.

6.3.2 Prediction from nearest neighbors

At the setting fixed before the run, prediction from the five nearest neighbors is more accurate in the distance hybrid on all six small-molecule datasets (Table 6.8). It is better in every fold on LogD, and worse in one of five folds on hERG K_i and Caco-2 (Additional file 34).

Picking the setting again for each fold does not change this: the hybrid still wins on

Dataset	Model	Best single representation	First hybrid	Second hybrid
Small molecules				
QM9	DNN	0.186	0.274	0.305
	LightGBM	0.186	0.215	0.212
Caco-2	DNN	0.147	0.196	0.206
	SVM	0.184	0.179	0.208
hERG K_i	SVM	0.093	0.215	0.212
	DNN	0.074	0.180	0.151
LogD	DNN	0.087	0.154	0.149
	SVM	0.099	0.1951	0.1948
hERG blocking	LightGBM	0.02444	0.02440	0.031
	XGBoost	0.024	0.026	0.028
Antibodies				
EGFR binding	SVM	0.039	0.035	0.054
	DNN	0.041	0.042	0.060
Nanobody thermostability	DNN	0.120	0.158	0.171
	SVM	0.120	0.163	0.162
AC-SINS self-association	LightGBM	−0.007	0.037	0.039
	GP	0.017	0.042	0.024
SMAC colloidal stability	DNN	0.007	−0.024	−0.0003
	SVM	−0.011	0.002	0.007
CHO polyreactivity	GP	0.124	**0.094**	**0.093**
	DNN	0.106	**0.064**	**0.096**
Expression titer	DNN	0.068	0.057	0.078
	RF	0.031	0.099	0.107
Binding free energy ($\Delta\Delta G$)	SVM	0.144	0.173	0.185
	DNN	0.132	0.218	0.197
Reactions				
Buchwald–Hartwig yield	RF	0.104	0.081	0.074
	LightGBM	0.067	0.0833	0.0831

Table 6.5: Distance against label difference in each downstream model's best single representation and its two predictive hybrids. One row is one dataset and model. Values are the Spearman correlation between the distance of two held-out molecules and their absolute label difference (higher is better), mean over five outer folds, Manhattan distance, unweighted.

all six small-molecule datasets, on LogD from 0.638 to 0.565 RMSE (Table 6.9). This comparison favors the hybrid (Section 6.2.4).

On the antibody datasets and Buchwald–Hartwig, the result depends on the comparison. At the fixed setting the hybrid wins on most antibody datasets and loses on $\Delta\Delta G$, AC-SINS self-association and Buchwald–Hartwig (Table 6.8). With the setting picked again for each fold, it wins only on EGFR binding, $\Delta\Delta G$ and SMAC colloidal stability, and loses on the other four and on Buchwald–Hartwig (Table 6.9). The largest

Dataset	Single representation		Distance hybrid		Similarity hybrid	
	Unweighted	Weighted	Unweighted	Weighted	Unweighted	Weighted
Small molecules						
QM9	0.130	0.329	0.378	0.460	0.368	0.462
Caco-2	0.152		0.197		**0.277**	
hERG K_i	0.073		0.263		0.234	
LogD	0.077	**0.276**	0.269	0.309	0.230	0.312
hERG blocking	0.026		0.055		0.064	
Antibodies						
EGFR binding	0.038	0.047	0.074	0.064	0.057	0.046
Nanobody thermostability	0.120	0.207	0.211	0.227	0.182	0.212
AC-SINS self-association	0.021	0.097	0.072	0.155		
SMAC colloidal stability	0.0002		0.050			
CHO polyreactivity	0.103	0.107	0.128	0.130		
Expression titer	0.010		0.106			
Binding free energy ($\Delta\Delta G$)	0.118	0.131	0.192	0.214		
Reactions						
Buchwald–Hartwig yield	0.1074	0.182	0.143	0.208	0.1069	0.189

Table 6.6: Distance against label difference in each feature space, with and without importance weighting. One row is one dataset. Values are the Spearman correlation between the distance of two held-out molecules and their absolute label difference (higher is better), mean over five outer folds, Manhattan distance. The single representation is the one the nearest-neighbor model ranked first in each fold.

gain that holds in both is on SMAC colloidal stability, from 0.092 to 0.291 Spearman with the setting picked again.

Multiplying features by their importance mostly helps prediction from nearest neighbors (Table 6.12, exceptions in bold), but in the other models, weighting the single representation gains little or loses. The SVM also loses on hERG K_i, while on LogD it barely moves (Table 6.10). With neighbor-label features the direction depends on the setting: on LogD the SVM improves with ten neighbors under cosine distance and gets worse with twenty.

Weighting also improves prediction from all representations joined (Table 6.11). With 10 neighbors, weighting by LightGBM gain improves all five regression small-molecule datasets, for example QM9 from 0.575 to 0.484 RMSE. At least one impor-

	Prediction score, by number of features						Spearman ↑, by number of features					
Dataset	50	100	250	500	1,000	2,000	50	100	250	500	1,000	2,000
Small molecules, prediction score in RMSE ↓												
QM9				0.316	0.315	0.312				0.323	0.282	0.254
Caco-2				0.284	0.279	0.278				0.216	0.205	0.211
hERG K_i			0.569	0.571	0.580	0.575			0.235	0.200	0.155	0.135
LogD				0.415	0.413	**0.408**				**0.231**	0.187	0.154
Antibodies, prediction score in Spearman ↑												
EGFR binding			0.247	0.198	0.209	0.114			0.048	0.064	0.035	0.028
Nanobody thermostability	0.624	0.638	0.698	0.659	0.668	0.662	0.193	0.188	0.172	0.164	0.156	0.147
SMAC colloidal stability				0.309	0.216	0.299				−0.014	−0.006	−0.014
CHO polyreactivity			0.406	0.410	0.404	0.439			0.101	0.094	0.094	0.059
Expression titer			0.296	0.315	0.357	0.337			0.077	0.061	0.124	0.120
Binding free energy ($\Delta\Delta G$)		0.342	0.267	0.339				0.202	0.181	0.205		

Table 6.7: Prediction score and distance against label difference of the DNN's top hybrid at each number of features of its search. One row is one dataset; datasets without a DNN have no row. Prediction score: held-out score in the dataset's metric (direction in the group heading). Spearman: correlation between the distance of two held-out molecules and their absolute label difference in the same hybrid, Manhattan distance, unweighted. Means over five outer folds.

Dataset	Best single base representation	Hybrid
RMSE ↓		
Caco-2 efflux	0.3386	0.3246
LogD	0.6766	0.5831
MDR1-MDCK efflux	0.5543	0.5202
QM9 HOMO-LUMO gap	0.6692	0.4205
hERG Ki	0.6280	0.5822
Buchwald-Hartwig yield	14.4572	**16.1811**
Spearman ↑		
AC-SINS self-association	0.3912	**0.3884**
Binding free energy	0.1424	**0.0899**
CHO polyreactivity	0.2059	0.3591
EGFR binding	0.1760	0.2126
Expression titer	0.2521	0.2622
Nanobody thermostability	0.5005	0.5458
SMAC colloidal stability	0.1268	0.2418
AUROC ↑		
hERG blocking	0.8143	0.8335

Table 6.8: Held-out score of predicting each molecule from its five nearest training molecules, per dataset, mean over outer folds, in each dataset's own metric. Bold: a dataset where the hybrid is worse.

tance method helps on every antibody dataset and Buchwald–Hartwig. Weighting loses with DeepLIFT on Caco-2 and with LightGBM gain and TreeSHAP on SMAC colloidal stability. Without a comparison against shuffled weights, we cannot say how much of the gain comes from the importance itself.

Whether the hybrid wins depends far more on the dataset than on how the neighbors are found (Additional file 34). Averaged over the other settings, the gain on LogD is similar for each distance and number of neighbors, and the hybrid loses on $\Delta\Delta G$ under all three distances. Weighting both sides shrinks the gain on every dataset where the hybrid wins, on LogD from 0.089 to 0.025 RMSE. Part of what the hybrid does for a distance is therefore what weighting does. Where the hybrid loses, the spread across folds is larger than the loss, for example on $\Delta\Delta G$ and AC-SINS self-association. These datasets have a few hundred training antibodies, against one to eight thousand training molecules on the small-molecule datasets (Table 5.7), so we cannot separate size from domain. The settings the search chose do not line up with winning either. On LogD and QM9 it picks the same allocation and importance method in every fold, while CHO polyreactivity wins every fold with four different choices.

Retrieval (Section 6.3.4) shows the same pattern (Additional file 35). On LogD the hybrid's neighbors have closer labels under each distance and number of neighbors. On

Dataset	Metric	Single representation	Hybrid
Small molecules			
QM9	RMSE ↓	0.617	0.422
Caco-2	RMSE ↓	0.326	0.319
MDR1-MDCK efflux	RMSE ↓	0.522	0.480
hERG K_i	RMSE ↓	0.598	0.564
LogD	RMSE ↓	0.638	**0.565**
hERG blocking	AUROC ↑	0.830	0.843
Antibodies			
EGFR binding	Spearman ↑	0.257	0.342
Nanobody thermostability	Spearman ↑	0.572	0.561
AC-SINS self-association	Spearman ↑	0.456	0.441
SMAC colloidal stability	Spearman ↑	0.092	**0.291**
CHO polyreactivity	Spearman ↑	0.350	0.260
Expression titer	Spearman ↑	0.322	0.248
Binding free energy ($\Delta\Delta G$)	Spearman ↑	0.113	0.145
Reactions			
Buchwald–Hartwig yield	RMSE ↓	13.8	14.0

Table 6.9: Prediction of each held-out molecule from its nearest training molecules, with the feature space and setting picked again for each outer fold. One row is one dataset; mean over five outer folds, in the dataset's own metric (arrow gives the direction). Single representation: the nearest-neighbor model's two best, unweighted. Hybrid: up to six hybrids, unweighted or weighted. Buchwald–Hartwig RMSE is in percentage points of yield.

Dataset	Unweighted	Weighted
Small molecules, RMSE ↓		
Caco-2	0.307	0.310
hERG K_i	0.607	**0.630**
LogD	0.508	**0.507**
Antibodies, Spearman ↑		
EGFR binding	0.287	0.247
Nanobody thermostability	0.690	0.672
AC-SINS self-association	0.550	0.499
SMAC colloidal stability	0.282	0.158
CHO polyreactivity	0.3751	0.3749
Expression titer	0.376	0.296
Binding free energy ($\Delta\Delta G$)	0.209	0.206
Reactions, RMSE ↓		
Buchwald–Hartwig yield	14.9	14.8

Table 6.10: Importance weighting in an SVM with an RBF kernel, fitted on the nearest-neighbor model's best single representation. One row is one regression dataset; mean over five outer folds, in the dataset's own metric (heading row gives metric and direction). Buchwald–Hartwig RMSE is in percentage points of yield.

Dataset	Unweighted	Weighted by		
		LightGBM gain	DeepLIFT	TreeSHAP
Small molecules, RMSE ↓				
QM9	0.575	**0.484**		
Caco-2	0.3425	0.337	**0.3428**	0.338
MDR1-MDCK efflux	0.549	0.516	0.540	0.519
hERG K_i	0.642	0.620		0.624
LogD	0.696	0.669		
Antibodies, Spearman ↑				
EGFR binding	0.251	0.307		0.285
Nanobody thermostability	0.443	0.487		0.493
AC-SINS self-association	0.311	0.434	0.341	0.375
SMAC colloidal stability	0.134	**0.121**	0.143	**0.122**
CHO polyreactivity	0.240		0.302	0.413
Expression titer	0.233		0.244	0.257
Binding free energy ($\Delta\Delta G$)	0.050	0.146		0.066
Reactions, RMSE ↓				
Buchwald–Hartwig yield	19.1		17.9	16.5

Table 6.11: Prediction from the ten nearest neighbors in all representations joined, unweighted and with features weighted by each importance method. One row is one regression dataset; mean over five outer folds, in the dataset's own metric (heading row gives metric and direction). Empty cell: an importance method the dataset's search does not use (Table 5.6). Buchwald–Hartwig RMSE is in percentage points of yield.

$\Delta\Delta G$ the hybrid loses under all three distances, by less than the spread across folds at the setting fixed before the run. The distance does matter on the antibody datasets: on all of them but EGFR binding and $\Delta\Delta G$, cosine distance gives the hybrid its smallest lead. On SMAC colloidal stability, cosine reverses the result: labels 0.028 closer under Manhattan distance, 0.057 further apart under cosine.

On hERG K_i and Buchwald–Hartwig, we also simulated choosing which molecules to measure next. On both, the best search runs in a hybrid.

6.3.3 Activity cliffs

We report activity cliffs on five small-molecule datasets only, all but MDR1–MDCK efflux. The antibody datasets have too few held-out cliff antibodies for a score; on $\Delta\Delta G$, 11 of the 429 held out over five folds. The rule for held-out molecules is looser than the one for training molecules, so the share of cliff molecules differs between the two. It is highest on LogD and hERG blocking, nearly half of the held-out molecules in some folds, and lowest on QM9.

		Single representation		Distance hybrid	
Dataset	Metric	Unweighted	Weighted	Unweighted	Weighted
Small molecules					
Caco-2	RMSE ↓	0.339	0.334	0.325	0.3225
hERG blocking	AUROC ↑	0.814	0.823	0.834	0.837
hERG K_i	RMSE ↓	0.628	0.597	0.582	0.580
LogD	RMSE ↓	0.677	0.6065	0.583	0.5765
Antibodies					
EGFR binding	Spearman ↑	0.176	0.207	0.213	0.223
Nanobody thermostability	Spearman ↑	0.500	0.533	0.546	0.552
AC-SINS self-association	Spearman ↑	0.391	0.416	0.388	0.422
SMAC colloidal stability	Spearman ↑	0.127	0.171	0.242	**0.234**
CHO polyreactivity	Spearman ↑	0.206	0.271	0.359	**0.357**
Expression titer	Spearman ↑	0.252	**0.218**	0.262	**0.229**
Binding free energy	Spearman ↑	0.142	**0.124**	0.090	0.135
Reactions					
Buchwald–Hartwig yield	RMSE ↓	14.5	14.3	16.2	**16.5**

Table 6.12: Prediction of each held-out molecule from its five nearest training molecules, with and without features multiplied by their importance, in the nearest-neighbor model's best single representation and in the distance hybrid. One row is one dataset; values are means over five outer folds, Manhattan distance, neighbors weighted by the inverse of their rank. The arrow gives the direction of the metric; Buchwald–Hartwig RMSE is in percentage points of yield. Bold marks a weighted value worse than the unweighted value beside it.

On LogD, Caco-2, hERG K_i and hERG blocking, each model's first hybrid beats its best single representation on cliff molecules and on the other molecules (Table 6.13). On Caco-2, hERG K_i and hERG blocking the gain is larger on cliff molecules; on Caco-2 the DNN goes from 0.340 to 0.288 RMSE on them. The gain on cliff molecules is slightly smaller on LogD, but both models gain on them in every fold. A separate run of the same comparison flags the same cliff molecules and agrees in direction on all four datasets.

On QM9 the hybrid is better on the other molecules for both models (Table 6.13). Only two folds hold the 10 cliff molecules a score needs. Averaged over those two folds, the hybrid is worse on cliff molecules for both models, for the DNN from 0.242 to 0.280 RMSE. Fold by fold, each model's hybrid is better in one of the two and worse in the other. With 48 cliff molecules in all, we draw no conclusion from QM9.

6.3.4 Prediction error, retrieval and clustering

A molecule's distance to its nearest training molecules rises weakly with the error made on it (Table 6.14). The distance hybrid tracks error better for both models on QM9,

		Cliff molecules		Other molecules	
Dataset	Model	Single	Hybrid	Single	Hybrid
RMSE ↓					
QM9	DNN	0.242	**0.280**	0.358	0.313
	LightGBM	0.261	0.267	0.371	0.343
Caco-2	DNN	0.340	**0.288**	0.289	0.272
	SVM	0.331	0.308	0.290	0.275
hERG K_i	SVM	0.561	0.530	0.593	0.565
	DNN	0.570	0.527	0.610	0.579
LogD	DNN	0.430	0.388	0.465	0.417
	SVM	0.469	0.407	0.504	0.440
AUROC ↑					
hERG blocking	LightGBM	0.793	0.808	0.888	0.895
	XGBoost	0.791	0.811	0.887	0.888

Table 6.13: Prediction on held-out cliff molecules and on the other held-out molecules, by each downstream model's best single representation and its first hybrid. One row is one dataset and model; mean over the outer folds that hold at least 10 molecules of the kind: two folds for the QM9 cliff molecules, five for every other cell.

Caco-2, MDR1–MDCK efflux, hERG K_i and LogD, and worse for both on hERG blocking. The largest gain is on hERG K_i, from 0.200 to 0.276 Spearman with the SVM's errors, while on LogD it is small.

On nanobody thermostability, $\Delta\Delta G$, expression titer, CHO polyreactivity and AC-SINS self-association, the hybrid tracks error better for both models (Table 6.14). It tracks error worse for both on SMAC colloidal stability, where the DNN goes from 0.136 to 0.030 Spearman, and on Buchwald–Hartwig. Distance in the single representation does not rise with error for either model on Buchwald–Hartwig, $\Delta\Delta G$ and expression titer.

With the setting picked again for each fold, the hybrid retrieves neighbors with closer labels on all six small-molecule datasets, and QM9's label gap falls from 0.601 to 0.393 (Table 6.15). On hERG blocking, LogD, MDR1–MDCK efflux and QM9, the hybrid's k-means clusters are also more coherent at 3, 5 and 10 clusters (Table 6.16). On Caco-2 and hERG K_i the result changes with the number of clusters; on Caco-2 the hybrid loses at 3 clusters and wins at 5 and 10.

With the setting picked again, the hybrid retrieves neighbors with closer labels on every antibody dataset except $\Delta\Delta G$, where the label gap rises from 1.439 to 1.507 (Table 6.15). It is also worse on Buchwald–Hartwig. Principal components of all representations retrieve better than the hybrid only on EGFR binding, and tie on CHO polyreactivity. Clustering is mixed: at every cluster count the hybrid's clusters are more

Dataset	Model	Single representation	Distance hybrid
Small molecules			
QM9	DNN	0.072	0.114
	LightGBM	0.137	0.209
Caco-2	DNN	0.116	0.123
	SVM	0.124	0.131
MDR1-MDCK efflux	SVM	0.230	0.288
	DNN	0.216	0.268
hERG K_i	SVM	0.200	**0.276**
	DNN	0.195	0.238
LogD	DNN	0.147	0.150
	SVM	0.216	0.220
hERG blocking	LightGBM	0.139	0.019
	XGBoost	0.146	0.041
Antibodies			
EGFR binding	SVM	−0.020	0.040
	DNN	0.105	0.067
Nanobody thermostability	DNN	0.0148	0.0149
	SVM	0.030	0.041
AC-SINS self-association	LightGBM	0.089	0.157
	GP	0.029	0.054
SMAC colloidal stability	DNN	0.136	**0.030**
	SVM	0.090	0.003
CHO polyreactivity	GP	0.082	0.092
	DNN	0.016	0.047
Expression titer	DNN	−0.095	0.110
	RF	−0.037	0.125
Binding free energy ($\Delta\Delta G$)	SVM	−0.052	0.100
	DNN	−0.067	0.047
Reactions			
Buchwald–Hartwig yield	RF	−0.002	−0.032
	LightGBM	−0.014	−0.044

Table 6.14: Distance to the nearest training molecules against prediction error. One row is one dataset and downstream model. Values are the Spearman correlation between a held-out molecule's mean distance to its 5 nearest training molecules and the model's error on it (higher is better), mean over five outer folds, Manhattan distance, unweighted. The model is fitted on all representations joined; distance is measured in the nearest-neighbor model's best single representation and in the distance hybrid.

Dataset	Single representation	Hybrid	Principal components
Small molecules			
QM9	0.601	**0.393**	0.570
Caco-2	0.302	0.286	0.300
MDR1-MDCK efflux	0.527	0.476	0.549
hERG K_i	0.528	0.512	0.543
LogD	0.593	0.543	0.579
hERG blocking	0.285	0.272	0.284
Antibodies			
EGFR binding	0.271	0.270	**0.261**
Nanobody thermostability	8.804	8.162	9.208
AC-SINS self-association	7.444	6.818	7.302
SMAC colloidal stability	0.459	0.424	0.474
CHO polyreactivity	0.164	0.1546	0.1546
Expression titer	117.424	115.030	115.474
Binding free energy ($\Delta\Delta G$)	1.439	**1.507**	1.567
Reactions			
Buchwald–Hartwig yield	12.0	12.1	16.3

Table 6.15: Label gap between each held-out molecule and its nearest training molecules (lower is better), with the feature space and setting picked again for each outer fold, unweighted. One row is one dataset; mean over five outer folds, in the dataset's label units (on hERG blocking, one minus the weighted share of neighbors in the molecule's class; on Buchwald–Hartwig, percentage points of yield). Single representation: the nearest-neighbor model's two best. Hybrid: every hybrid of the dataset. Principal components: of all representations, as many as the hybrid has features.

coherent on nanobody thermostability and CHO polyreactivity, and less coherent on EGFR binding and SMAC colloidal stability (Table 6.16).

Except on QM9 and Buchwald–Hartwig, the features a hybrid keeps carry no more label information than principal components of the same number (Additional file 36).

6.4 Conclusions

Chapter 4 found that distance in standard representations barely rises with label difference. On the six small-molecule datasets, we found that distance in a hybrid built for distance tracks label differences more closely than in the single representation. Prediction from nearest neighbors improves on all six, both at the setting fixed before the run and with the setting picked again. In addition, the neighbors retrieved in the hybrid have closer labels. On LogD, Caco-2, hERG K_i and hERG blocking, each model's first hybrid also predicts cliff molecules better than its best single representation. The

	3 clusters		5 clusters		10 clusters	
Dataset	Single	Hybrid	Single	Hybrid	Single	Hybrid
Small molecules						
QM9	0.114	0.330	0.100	0.348	0.130	0.357
Caco-2	0.082	0.056	0.081	0.093	0.107	0.114
MDR1-MDCK efflux	0.104	0.172	0.091	0.161	0.124	0.180
hERG K_i	0.334	0.280	0.292	0.302	0.224	0.223
LogD	0.053	0.139	0.063	0.149	0.100	**0.225**
hERG blocking	0.615	0.757	0.614	0.746	0.657	0.740
Antibodies						
EGFR binding	0.099	0.091	0.191	0.163	0.216	0.148
Nanobody thermostability	0.154	0.218	0.153	0.205	0.259	0.317
AC-SINS self-association	0.044	0.033	0.027	0.070	0.243	0.286
SMAC colloidal stability	0.241	0.180	0.467	0.350	0.537	**0.461**
CHO polyreactivity	0.044	0.094	0.084	0.152	0.112	0.327
Expression titer	0.080	0.071	0.241	0.212	0.231	0.244
Binding free energy ($\Delta\Delta G$)	0.028	0.148	0.134	0.200	0.224	0.218
Reactions						
Buchwald–Hartwig yield	0.036	0.072	0.092	0.075	0.083	0.104

Table 6.16: Label coherence of k-means clusters of the held-out molecules (higher is better), in the single representation and the hybrid picked again for each outer fold. One row is one dataset; mean over five outer folds. Coherence is one minus the mean within-cluster standard deviation of the labels over their overall standard deviation; on hERG blocking, the mean share of the largest class in each cluster.

similarity hybrid tracks label differences more closely than the distance hybrid only on Caco-2 and hERG blocking, of the eight datasets where it has a value (Table 6.6).

Weighting features by importance does part of the same work without any selection. On LogD it lets the single representation track label differences better than the unweighted hybrid. The hybrid's distance does, however, track prediction error worse on hERG blocking, and except on QM9 and Buchwald–Hartwig its kept features carry no more label information than principal components of the same size.

On the antibody datasets and Buchwald–Hartwig, distance in the hybrid also tracks label differences more closely, but the methods built on it do not always gain. With the setting picked again, prediction from nearest neighbors is worse in the hybrid on nanobody thermostability, AC-SINS self-association, CHO polyreactivity, expression titer and Buchwald–Hartwig. Retrieval is worse on $\Delta\Delta G$ and Buchwald–Hartwig.

Methods that find noisy labels by comparing each label with its neighbors' labels rest on the distance studied here. Chapter 7 tests whether the hybrid's distance makes noisy labels easier to find and to act on.

Chapter 7

Detecting and handling label noise

7.1 Introduction

In Chapter 6, on the six small-molecule datasets, distance in a hybrid built for distance tracked the label more closely than distance in a single representation. Prediction from nearest neighbors also improved on all six. This chapter asks whether that helps with label noise. Can we find the noisy labels in a QSAR dataset, and does acting on them improve a model's held-out score? We ask this on two kinds of labels. The first has damage injected on purpose, so we know which labels were damaged and by how much. The second is the labels as delivered, which carry assay error that nobody has marked.

Every method in this chapter reads a feature space. A detector compares a molecule's label with its neighbors' labels, or with a model's prediction, and both depend on the features. The adaptive hybrid representation (Chapter 5) leaves out features that do not relate to the label yet add a term to the distance (Chapter 4). For every method we therefore ask whether it improves the held-out score and, where it does, whether it improves the hybrid more than the best single representation.

The methods fall into the three groups of Table 2.3. This chapter expands the sweep of twelve published methods in Chapter 4 (Section 4.3.3). It also adds methods that select molecules by a known flag, such as a label at an assay limit, which none of the published methods uses. QM9, whose labels are computed rather than measured, is the clean-label control (Section 7.2.5).

7.2 Methods

This chapter uses the datasets of Chapters 5 and 6 (Table 5.7), plus one more, hERG IC_{50} (HIC_{50} in the figures, Table 5.8), which is used only here. Its labels are pIC_{50}

values for the hERG channel from ChEMBL. The IC_{50} is the concentration of a molecule that gives half of the maximal inhibition in an assay, and pIC_{50} is its negative base-10 logarithm in molar units. hERG IC_{50} is the one dataset with real repeat measurements. In this chapter, "hERG" on its own means the hERG blocking dataset, whose labels are blocker or non-blocker. Folds and metrics are those of Chapter 5. Eight datasets of Table 5.7 are not used here: rat microsomal clearance, MDR1–MDCK efflux, anti-VEGF scan expression and binding, EGFR binding, angiopoietin-2 binding, antibody humanness and HIC retention.

Each dataset has two downstream models. In each outer fold, a search on that fold's training molecules ranks each model's two best single representations and two best hybrids. Every method is tested on all four. In most experiments that search sees the clean training labels. Only the hybrid's features are then re-ranked, and its budget re-split between base representations, on the damaged labels; in this way the hybrid's features move with the damage. The base representations and the way the hybrid is built, however, are chosen with clean labels, which a real user would not have. There are two exceptions. The expanded published sweep repeats the search for every damage kind, level and draw, trained and scored on the damaged labels alone, and the experiment on cleaning before the feature search (Section 7.2.3) compares cleaning before the search with cleaning after it.

We keep apart three tasks that are often treated as one: finding the damaged labels, sizing the damage, and improving the model's held-out score. A method can succeed at the first and fail at the third. Figure 7.1 shows how one method is run and scored in one outer fold.

7.2.1 Noise conditions

We damage a regression dataset in six ways (Table 7.1). Four are noise conditions from Section 2.5.1: Gaussian, outlier, grouped-shifted and censoring. Censoring clips labels at a fixed value that imitates an assay's reporting limit, the edge of its measurable range. The clip is at the top of the label range. For potency labels such as hERG K_i, real censoring falls mostly at the low end; the clip imitates the rarer case. Under outlier noise, a random 5% of molecules get an error three times wider, where Chapter 3 used 10%. Because every other label still gets the ordinary error, the 5% carry about a third of the squared error, not all of it. The last two kinds raise a random 10% of the training labels by one fixed amount, in the two units of Table 7.1. The grouped-wider condition and the Student-t and Laplace shapes of Chapter 3 are not used here.

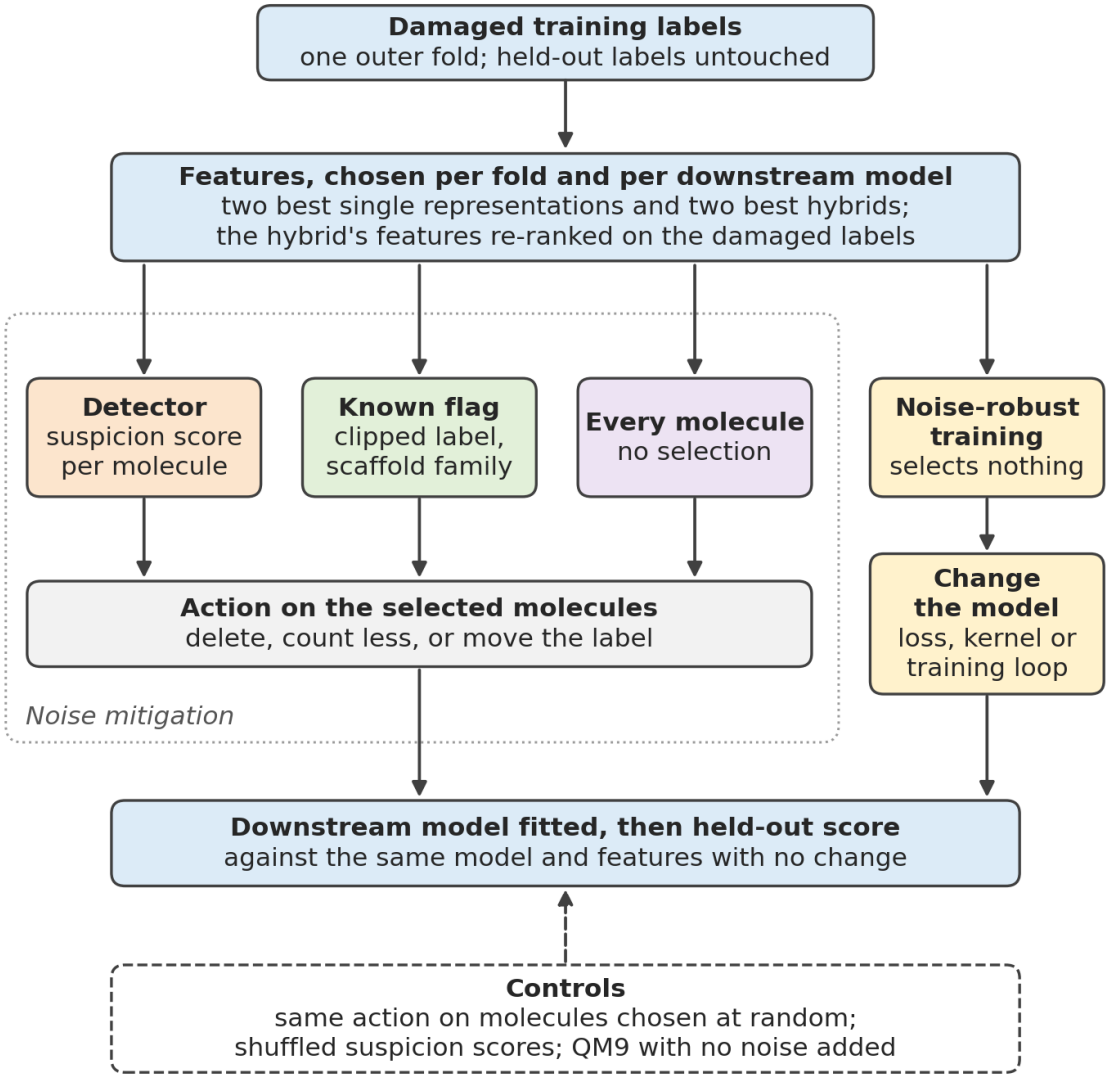


Figure 7.1: How a noise method is run and scored in one outer fold. Noise mitigation selects molecules by a detector, a known flag or every molecule, then acts on them. Noise-robust training selects nothing and changes the model. Dashed: the controls.

A classification dataset has its labels flipped instead, and the level is the share of training labels flipped. Six rules decide which labels flip. A label can flip uniformly, more often in the rarer classes or more often in one direction. It can also flip at a rate that varies by molecule, in proportion to class frequency, or along a confusion matrix. With no matrix given, the last rule is the same as the first, which leaves five distinct rules on a two-class dataset.

Most experiments first run a screen of two levels (Table 7.2). The screen leaves out the shift sized by model error. The held-out predictions it produces correlate 0.93 to 0.97 with those of the plain shift in 10%. Two draws of the plain shift correlate about as closely with each other. An experiment that is taken further runs all six kinds at the

Damage kind	What is done to the training labels	Unit of the level	Labels moved
Gaussian	Every label gets Gaussian error (Equation 2.4)	SD of the clean training labels	All
Outlier	As Gaussian, but a random 5% get error three times wider	SD of the clean training labels	All
Grouped-shifted	Every scaffold family gets its own offset (Equation 2.5); small molecules only	SD of the clean training labels	All
Censoring	Labels above the $(1 - \text{level})$ quantile of the clean training labels are set to that quantile	Share of labels clipped	About the level
Shift in 10%	A random 10% are raised by one fixed amount	SD of the clean training labels	10%
Shift in 10%, sized by model error	As above, in a different unit	SD of the out-of-fold residual	10%

Table 7.1: The six damage kinds on a regression dataset. SD is standard deviation. The out-of-fold residual is that of a LightGBM model fitted to the clean training labels.

full-run levels, on datasets of 5,000 molecules or fewer. Each non-zero level is drawn three times, except censoring, which involves no random draw and runs once. The random seed depends on the damage kind and the draw, not on the level or the fold. The levels therefore nest: every molecule's error at level 1.0 is twice its error at 0.5, and a molecule carries the same damage in every fold it trains in.

Damage	Screen	Full run
Regression, other than censoring	None and 1.0	0, 0.5, 1.0 and 1.5
Censoring, share clipped	None and 25%	0, 10%, 25% and 40%
Classification, share flipped	10%	Up to 40%

Table 7.2: Damage levels in the screen and in the full run. Regression levels are in the units of Table 7.1.

Only the training labels are damaged. As in Chapter 3, the held-out labels of the outer fold are left as delivered: every score in this chapter is against the delivered labels. Where a search runs on damaged labels, as in the expanded published sweep, the inner folds of that search are scored on damaged labels too.

Three settings involve no injection: the labels as delivered, whose errors nobody has marked; hERG IC_{50}, where the disagreement between repeat measurements is real noise; and QM9 with no noise added, whose labels are clean. Three experiments add

damage of their own. The experiment on break-even precision gives a random 5% of molecules wide errors and leaves every other label alone. The experiment that changes the model adds Gaussian noise sized by a LightGBM model's error. The experiment on measuring sources simulates several measurements per molecule.

7.2.2 Noise detection

We group the detectors (Table 2.3) by what they read: an out-of-fold prediction, the neighbors' labels, or a model's predicted spread (Table 7.3). Every detector of the first two kinds reads the molecule's own label. Three of the third kind, the heteroscedastic GP, the variance-head network and NGBoost, predict a spread from the features alone and never read it. Such a detector can only find a wrong label if the damage is tied to structure. Most detectors only rank molecules, but a few of the third kind return a spread in the label's own units, which can feed a weight or a move.

The gap over neighbor spread is the buddy check used to screen weather-station readings, which divides a reading's gap from its neighbors' mean by their standard deviation [258]. RegENN applies a similar threshold to filter noisy regression labels [259], but we found no earlier use of either on molecular labels. The cliff-aware gap divides by the spread of all labels plus that of the neighbors. A molecule among neighbors that disagree with each other, as near an activity cliff, scores lower as a result. SLOM lowers an outlier score in the same way where a neighborhood is unstable, but for spatial data, where neighbors are found by location [260].

The variance-head network predicts a mean $\mu(x)$ and a variance $\sigma^2(x)$ and is trained with

$$\mathcal{L} = \frac{1}{2}\left(\frac{(y-\mu)^2}{\sigma^2} + \log\sigma^2\right)\cdot\sigma^{2\beta}, \tag{7.1}$$

where $\beta = 0.5$ keeps the variance from hiding the errors in the mean [261].

A neighbor detector needs a space to find neighbors in, a neighbor count k and a way to weight the neighbors. We try k of 5, 10 and 20, and weight each neighbor by 1/rank or by $1/d^2$. The setting is chosen inside each fold, without the held-out molecules. We hold back 20% of the training scaffold families and predict each held-back label as the weighted mean of its neighbors' labels among the other training molecules. The three settings that predict best are kept. The space is a single representation, a hybrid, or one of the similarity spaces of Chapter 6.

Two experiments change the space itself. The first multiplies each z-scored feature by the square root of its importance, rescaled so that the multipliers average 1, before the neighbors are found. The importance is computed against the damaged training

Detector	Reads	Suspicion score of molecule i	Source
Out-of-fold forest residual	Prediction	$\lvert y_i - \hat{y}_i \rvert / \sigma_y$, with $\hat{y}_i$ from a random forest of 100 trees on 5 scaffold-group folds; on classification, $1 - \hat{p}_i(y_i)$	Brodley and Friedl [155]; Northcutt et al. [52]
Out-of-fold LightGBM residual	Prediction	$\lvert y_i - \hat{y}_i \rvert$, with $\hat{y}_i$ from LightGBM on 5 scaffold-group folds	Brodley and Friedl [155]
Neighbor-label gap	Neighbors	$\lvert y_i - \bar{y}_{\mathcal{N}} \rvert / \sigma_y$; on classification, the share of neighbors in another class	Wilson [102], Martín et al. [157]
Gap over neighbor spread	Neighbors	$\lvert y_i - \bar{y}_{\mathcal{N}} \rvert / \max(s_{\mathcal{N}}, c)$, with $c = \max(0.01\,\sigma_y, 10^{-3})$	Båserud et al. [258]; Kordos and Blachnik [259]
Cliff-aware gap	Neighbors	$\dfrac{\lvert y_i - \bar{y}_{\mathcal{N}} \rvert}{\sigma_y} \cdot \dfrac{1}{1 + s_{\mathcal{N}}/\sigma_y}$	New; framing after Maggiora [76], Stumpfe and Bajorath [77]; related to Chawla and Sun [260]
Gaussian process	Spread and prediction	Posterior standard deviation plus $\lvert y_i - \hat{y}_i \rvert$, out of fold, each inner fold capped at 1,000 molecules	Rasmussen and Williams [115]; the sum is new
Heteroscedastic GP	Spread	Predicted noise standard deviation, from a GP fitted jointly with a small network for the noise	After Kersting et al. [154]
Variance-head network	Spread	Predicted standard deviation, trained with the β-NLL loss of Equation 7.1	Seitzer et al. [261]
NGBoost	Spread	Predicted standard deviation, 50 boosting stages	Duan et al. [63]

Table 7.3: Detectors used in this chapter. $\bar{y}_{\mathcal{N}}$ and $s_{\mathcal{N}}$ are the mean and standard deviation of the labels of molecule i's k nearest training neighbors, and σ_y is the standard deviation of the training labels. "New" means no published source was found.

labels, as feature-weighted nearest neighbors do [251]. The second compares four families of distance for the neighbor detectors. They are the plain features under Euclidean and Manhattan distance, the features weighted by importance raised to 0.5 or 1.0, forest proximity, and the hybrid's own features. The forest proximity of two molecules is the share of trees in which they fall in the same leaf [262], and their distance is one minus it. The forest has 100 trees with at least 5 molecules per leaf, fitted to the fold's damaged training labels.

One experiment compares several further detectors with the out-of-fold residual.

One is the value of each molecule to a nearest-neighbor predictor, as a Shapley value [263, 264]. Another is the cleanlab regression score [265], which draws its internal folds at random rather than by scaffold family. The rest divide the residual by a spread for that molecule, from NGBoost, a quantile forest or a conformal interval.

Two experiments test the out-of-fold LightGBM residual under the shift in 10% sized by model error. The first ranks molecules by the residual itself. The second divides each residual by a second LightGBM model's out-of-fold estimate of how large that molecule's residual usually is. It then turns the result into conformal p-values against a calibration set of training molecules [266]. Molecules are flagged with the Benjamini–Hochberg rule [267], which bounds the expected share of false flags at 5%, 10% or 20%. The bound holds only if the calibration labels are clean. The calibration set is a random 20% or 50% of the training molecules or, where a dataset has repeat measurements, the molecules with three or more measurements that agree. Both experiments grade the ranking by average precision.

A detector can be graded only where the damage moved some labels and not others, so that it is known which molecules were damaged. That is outlier noise, censoring, the two shifts in 10% and every label flip. Under Gaussian and grouped-shifted noise every label moves, and no detection score exists. We grade a detector by the average precision of its suspicion scores against the set of damaged molecules. A detector that ranks at random scores the damaged share of the training molecules; that share is the chance level. Some experiments report the area under the ROC curve instead, where chance is 0.5.

7.2.3 Noise mitigation

Table 7.4 describes each noise mitigation method by its two halves (Table 2.3). The methods reduce to a few selectors crossed with three actions.

Most methods read the suspicion score rescaled to $[0, 1]$, $\tilde{s}_i = (s_i - \min s)/(\max s - \min s)$. Deleting drops the molecules above a quantile of s (Table 7.4). This is the filtering of Brodley and Friedl [155], which Zhao et al. [268] tested on QSAR modeling sets. Down-weighting gives molecule i the weight $1/(1 + \alpha\tilde{s}_i)$, and all weights are then divided by their mean. A Huber weight, $\min(1, 1.345\,\hat{\sigma}/s_i)$, lowers only the molecules whose score is above $1.345\,\hat{\sigma}$ [98]. A proportional move takes the label part of the way to a target t_i,

$$y'_i = y_i + \alpha\,\tilde{s}_i\,(t_i - y_i), \tag{7.2}$$

so the most suspect label moves a share α of the way and the least suspect does not

move. The target is the weighted mean of the molecule's neighbors' labels [269, 270], or an out-of-fold LightGBM prediction, which is close to the target blending of Reed et al. [271]. One further rule, the score-scaled shrink, moves a share $0.5\tilde{s}_i/(1+0.5\tilde{s}_i)$ of the way to the out-of-fold prediction. It is modeled on James–Stein shrinkage [272], which pulls an estimate toward a common value by an amount set by its variance. Here the variance cancels out of the share, so the share depends only on the score.

Selected by a detector

The main cleanup experiment takes the out-of-fold forest residual as its detector and crosses it with each action above. The cliff-aware gap is graded as a detector beside it. A second experiment asks whether to delete a flagged molecule or keep it, and keeps it either with a lower weight or with a moved label. Its control deletes the same number of molecules at random, at five seeds. A third experiment replaces the forest residual with each of five detectors in turn: the cliff-aware gap, and the GP, the heteroscedastic GP, the variance-head network and NGBoost. Each deletes its top 5%, 10% or 20%. Where the detector has its own prediction, it also moves every label toward that prediction. The share moved is given by Equation 7.2, or by $c_i = \alpha s_i/(1+\alpha s_i)$, which uses the score in its own units.

Four experiments weight molecules by a neighbor or model detector, with no deletion. Three use the weight $1/(1+3\tilde{s}_i)$, under which the most suspect molecule counts a quarter as much as the least suspect. They differ in where the neighbors are found: in features scaled by importance on clean labels, in the importance-scaled space above, and in each of the four families of distance. The fourth uses $\exp(-0.1\,\tilde{s}_i)$ with the gap over neighbor spread, the GP or NGBoost as the detector. With that weight, the most suspect molecule counts about 0.9 times as much as the least suspect.

Close-pair weights look only at pairs of near-identical molecules. A pair is close if its distance is in the lowest 1% of all training pairs. The label noise is estimated from the closest fifth of those pairs as $\hat{\sigma}^2 = \overline{\Delta y^2}/2$, since two noisy copies of one value differ by twice the noise variance. Both molecules of any close pair whose labels differ by more than $3\sqrt{2}\,\hat{\sigma}$ get weight 0.25. Residual flags give weight 0.25 to the top 5%, 10% or 20% of molecules by out-of-fold LightGBM residual. The same experiment also asks how precise a detector has to be before acting on its flags helps. It flags sets of molecules of the same size with a known share of truly damaged molecules, from none to all, and finds the share at which the gain from acting crosses zero. We call that share the break-even precision.

Cliff put-back runs after a cleanup and undoes it on activity-cliff molecules: deleted

Method	Selected by	How many	Action	Target, how far
Selected by a detector				
Delete	Forest residual or neighbor gap	Top 5, 10, 20%	Delete	–
Detectors compared	Five detectors (Section 7.2.2)	Top 5, 10, 20%	Delete or move	Detector's own prediction; α 0.25, 0.5, 1
Down-weight	Forest residual or neighbor gap	All	Count less	$1/(1+\alpha\tilde{s}_i)$
Huber weight	As above	All	Count less	$\min(1, 1.345\,\hat{\sigma}/s_i)$
Label smoothing	As above	All	Move	Neighbors' mean; Equation 7.2, $\alpha = 0.25$
Move toward prediction	As above	All	Move	Out-of-fold LightGBM; Equation 7.2
Score-scaled shrink	As above	All	Move	Out-of-fold LightGBM; $0.5\tilde{s}_i/(1+0.5\tilde{s}_i)$
Neighbor weights	Neighbor gap, in a chosen distance	All	Count less	$1/(1+3\tilde{s}_i)$
Detector weights	Gap over neighbor spread, GP or NGBoost	All	Count less	$\exp(-0.1\,\tilde{s}_i)$
Close-pair weights	Near-identical pairs whose labels disagree	Flagged pairs	Count less	Weight 0.25
Residual flags	Out-of-fold LightGBM residual	Top 5, 10, 20%	Count less	Weight 0.25
Cliff put-back	Any of the above, then a cliff test	Cliff molecules	Undo	Cliff molecules restored
Selected by a known flag				
Constant lift	Label at the assay limit	Clipped labels	Move	Limit plus one shared lift
Truncated-normal mean	As above	Clipped labels	Move	Equation 7.4
Censored EM	As above	Clipped labels	Move	As above, 3 rounds
Scaffold-group offset	Scaffold family	All	Move	Minus the family's mean residual
Source offset	Measuring source	All	Move	Minus the source's offset
Source weights	Measuring source	All	Count less	Inverse error variance
Count and spread weights	Repeat measurements	All	Count less	Count, or inverse error variance
Shrink by spread	Repeat measurements	All	Move	Out-of-fold LightGBM; further for a wider spread
Every molecule				
Huber teacher blend	–	All	Move	Out-of-fold Huber LightGBM; halfway

Table 7.4: Noise mitigation methods, by what selects the molecules. $\tilde{s}_i$ is the suspicion score rescaled to $[0, 1]$, α is the strength and $\hat{\sigma}$ is 1.4826 times the median suspicion score. Weights are divided by their mean. The four deleting methods of the published sweep are in Table 7.5.

molecules return, lowered weights go back to 1 and moved labels go back to their training value. Here the cleanup acts on the top 1%, 5% or 20% of flagged molecules. A training molecule is a cliff molecule if it has a partner with ECFP4 Tanimoto similarity of at least 0.55 [145] whose label is far from its own [76, 77]. Far means more than twice the median absolute deviation of the labels. On datasets whose label is not a biological activity, such as LogD, the same test finds pairs that differ sharply in that property, and we keep the name. The cliff set is rebuilt in every fold from the damaged training labels. Its control puts back the same number of molecules drawn at random from the non-cliff molecules. We found no earlier method that puts examples back after a cleanup. Some cleaning methods exempt a whole class decided in advance [273, 274], and Brodley and Friedl [155] name the risk of discarding correct exceptions without giving a rule.

The hybrid's features are chosen against the labels, which means the labels can be cleaned before the features are chosen or after. One experiment compares the two orders, with nine cleanups that use the out-of-fold forest residual as the detector, under injected damage and on labels as delivered. The search and the fit each use the labels as given or the cleaned labels, four combinations in all. Down-weighting cannot reach the search, because no weight enters the feature ranking or the budget split. Its two cleaned-search arms consequently do not exist. No source we found crosses features chosen on cleaned or recorded labels with a fit on cleaned or recorded labels. The nearest design ranks features on one version of the data and fits on another, but its second step is class-imbalance sampling, not label cleaning [275].

On labels as delivered, nothing says which molecules are wrong. A gain can come from removing hard molecules rather than wrong ones. One experiment runs seven cleanups on delivered labels with the neighbor gap and the forest residual as detectors. Its control shuffles each detector's scores across the training molecules, at three seeds, and reruns the cleanup with the same share and strength. The same number of molecules is touched, but not the ones the detector picked.

Selected by a known flag

When a label is clipped at an assay limit, we know two things that a detector has to guess: which labels are wrong, and in which direction. The limit L is read off the clipped training labels as their largest value, if at least two molecules and 1% of the training molecules sit on it. An out-of-fold LightGBM fit to the clipped labels gives each molecule a prediction p_i. Let σ be the standard deviation of $y_i - p_i$ over the

molecules below the limit, and m_i the inverse Mills ratio,

$$m_i = \frac{\phi(a_i)}{1 - \Phi(a_i)}, \qquad a_i = \frac{L - p_i}{\sigma}, \tag{7.3}$$

where ϕ and Φ are the standard normal density and distribution function. The truncated-normal mean replaces each clipped label by the mean of a normal distribution centered on its prediction and cut at the limit, following the censored-normal model of Tobin [276],

$$y'_i = \max(L,\ p_i + \sigma m_i)\,. \tag{7.4}$$

Censored EM repeats this for three rounds, refitting the downstream model out of fold to the current labels each round, after the iterated estimator of Buckley and James [277]. The constant lift raises every clipped label to $L + \overline{\sigma m}$, the mean of σm_i over the clipped molecules. That shared amount is larger than the truncated-normal mean's lift by $L - p_i$ on average. Two arms test what the flag is worth. Deleting the clipped molecules uses the flag but throws the molecules away. A per-molecule lift raises each clipped label by its own σm_i, and its control deals the same lifts to the wrong clipped molecules, at three seeds. Svensson et al. [83] fitted censored regression to drug-discovery labels, which is the closest precedent.

A scaffold family is also a known flag. Under grouped-shifted noise every family carries its own offset. A family's mean out-of-fold residual estimates that offset, and subtracting it from every label in the family undoes it. Three versions differ in how much they trust a small family. The first subtracts the whole mean residual from every family, and the second only from families of three or more training molecules. The third is a random-intercept model, fitted by up to five rounds of expectation maximization in the style of mixed-effects random forests [278, 279]. It shrinks each family's offset toward zero, more for a family with few molecules.

A source is a known flag when each molecule was measured by several of them. One experiment simulates eight sources, and each molecule is measured by one to four of them, drawn from hERG IC_{50}'s real counts or uniformly. Measurement s of molecule i is

$$y_{is} = y_i + b_s + \sigma_e \left(\sqrt{\rho}\, u_i + \sqrt{1 - \rho}\, v_{is}\right), \tag{7.5}$$

where b_s is the source's offset, u_i is an error shared by every measurement of the molecule and v_{is} is an error of its own. The value of ρ, 0, 0.3 or 0.7, sets how much of the error the measurements share. The offsets have standard deviation 0 or $0.5\,\sigma_e$. The training label is the mean of the molecule's measurements. The methods remove

each source's offset, estimated by a mixed model with the source as a fixed effect and the molecule as a random intercept. They also weight each molecule by the inverse of its estimated error variance [280]. Two further arms use the true ρ and σ_e, which a user never has; hence they are bounds and not methods.

hERG IC_{50} carries its own known flag: how many times each molecule was measured, and how far apart those measurements are. A molecule is one canonical SMILES string as given by ChEMBL. Its count is its number of ChEMBL IC_{50} records, and its spread is the variance of their pIC_{50} values. Of the three methods that use the flag, the first weights each molecule by its count. The second weights it by the inverse of its estimated error variance, $1/(\sigma_u^2 + \tilde{s}_i^2/n_i)$ [280]. Here n_i is the number of measurements behind the label, and σ_u^2 is the error that remains however often a molecule is measured. Its spread $\tilde{s}_i^2$ is pulled toward the spread pooled over all molecules, as if the pool were three more measurements. We do this because a variance from two or three records is unreliable [281]. The third moves each label toward an out-of-fold LightGBM prediction, further for a less precise label. The share moved is $v_i/(v_i + \tau^2)$, where v_i is the molecule's spread over its count and τ^2 is the variance of the out-of-fold residual. Held-out molecules are graded only if they have three or more measurements.

Every molecule

One method moves every label, with no selection at all. Each label moves halfway toward the out-of-fold prediction of a LightGBM model fitted with a Huber loss [98], whose threshold is 1.4826 times the median absolute out-of-fold residual. The Huber loss keeps the outlying labels from pulling that prediction.

7.2.4 Noise-robust training

Five experiments test noise-robust training (Table 2.3). They change the downstream model, rerun the twelve published methods, use a robust loss, exchange molecules between two learners, or use repeat measurements.

Changing the model instead of the labels

A model can be fitted more carefully to noisy labels with no label changed, and that is the comparison point for every mitigation result. One experiment changes the downstream model in four ways. Retuning searches 20 random settings of the model plus its default, scores each on three inner folds split by scaffold family of the damaged training labels, and refits the best. Its control runs the same search on the clean labels

and fits the chosen setting to the damaged ones. No real user has that control, but it tells us whether a gain comes from tuning against the noise or from tuning at all. Seed averaging fits ten copies of LightGBM, XGBoost or the network with different random seeds, each tree model on 80% of the molecules and features, and averages their predictions. Stacking combines the dataset's two downstream models by non-negative least squares on their out-of-fold predictions [282]. The damage is each regression kind of Table 7.1, and Gaussian noise whose variance is a multiple, 0.1, 0.3 or 1.0, of a LightGBM model's out-of-fold mean squared error on the clean labels. This experiment ran on Buchwald–Hartwig, Caco-2, hERG K_i, LogD and QM9, though not under every damage kind on each.

The expanded published sweep

The twelve published methods of Chapter 4 are run again (Table 7.5). Every one uses the same network as the plain model, with three hidden layers of 256, 128 and 64 units and dropout 0.2. It is trained with the Adam optimizer at a learning rate of 10^{-3} for up to 200 epochs, with early stopping. The code is a new implementation of the same twelve, not a rerun of the earlier study. It is wider in four ways: it runs on many datasets instead of QM9 alone, under every damage kind instead of Gaussian noise alone, and on hybrids as well as single representations. It also scores each method against the plain network on the same damaged labels in the same feature space. Four of the twelve select molecules with a detector and delete them, so under Table 2.3 they are noise mitigation, and the table marks them so. Most were built for image classification, and several of our versions differ from their source, as the table states.

Robust losses

The network is also trained with a Huber loss [98], whose threshold is c times 1.4826 times the median absolute out-of-fold residual, with c of 0.5, 1, 1.5 or 2. Under the shift in 10% sized by model error, each model's own robust loss is tried at $c = 1.5$. That is a Huber loss for LightGBM and the network, a pseudo-Huber loss for XGBoost, and absolute-error splits for the random forest.

Two learners

Two learners, LightGBM and an SVM, each score every molecule by its out-of-fold error. For two rounds, each is refitted on the molecules the other finds easiest. Each then keeps its own easiest 80%, 90% or 95%, and the downstream model is fitted on

Method	Group	What it does here	Source, and how it differs
Small-loss selection	Mitigation	After 10 epochs, keeps the 70% of molecules with the smallest loss and retrains	Han et al. [151]; one network, not two
Confident learning	Mitigation	Three networks; deletes molecules with error above the 85th percentile and ensemble spread below the 25th, then retrains	Northcutt et al. [52]; in-sample, no per-class thresholds
Distance-based selection	Mitigation	Deletes the 15% with the largest neighbor-label gap	After Wilson [102]
Multistage cleaning	Mitigation	Three rounds, each deleting the 10% with the largest in-sample error	Adapted from Zhao et al. [283], which fits a mixture to loss histories and deletes once
Early learning	Robust training	Each epoch, molecules with above-median loss lose 5% of their weight	Adapted from Zhang et al. [284], which updates weights from a clean subset
Uncertainty curriculum	Robust training	After 20 epochs, trains on the lowest-loss 25%, then 50%, 75% and 100%	Self-paced learning [153]
DivideMix	Robust training	Two networks; a two-component mixture on each network's loss picks the molecules the other trains on	Li et al. [150], without its semi-supervised half
Meta-Weight-Net	Robust training	A small network learns each molecule's weight from its loss, tuned on a held-back set of noisy labels	Shu et al. [241], which needs clean labels for that set
MentorNet	Robust training	A small network weights each molecule from its loss, loss history and distance to its neighbors	After Jiang et al. [285]
Contrast-to-divide	Robust training	Down-weights molecules with high loss whose predictions are stable when the features are perturbed	Named after Zheltonozhskii et al. [286], which does not use perturbation
Heteroscedastic GP	Robust training	A GP's residuals train a network that predicts each molecule's noise; the plain network is then trained with weights $1/(1 + \text{noise})$	After Kersting et al. [154]
Evidential network	Robust training	The network predicts the parameters of a normal-inverse-gamma distribution	Amini et al. [287]

Table 7.5: The twelve published methods of the expanded sweep. All use the same network. "Mitigation" marks the four that select molecules with a detector and delete them. Multistage cleaning, early learning and contrast-to-divide are our adaptations of the named methods.

the molecules both keep. This follows the exchange of co-teaching [151], done between whole refits rather than inside one training loop. Its control deletes the same number of molecules at random, at three seeds.

Repeat measurements

On hERG IC_{50}, four methods change what the model sees rather than the labels. The first trains on one row per measurement instead of one row per molecule with the mean label. The second adds the log of each molecule's measurement count as an extra feature, and the third gives NGBoost the same feature. The fourth fits an XGBoost accelerated-failure-time model [288] to the interval between each molecule's lowest and highest measurement, instead of to one value.

7.2.5 Evaluation and controls

Each group has its own test, because each makes a different claim.

- **Noise detection.** Is the suspicion score high on the damaged molecules? This can only be read where some labels were damaged and others were not (Section 7.2.2). On QM9 with no noise added, every flag is a false flag.
- **Noise mitigation, selected by a detector or a flag.** The held-out score after the action, against the same model on the same features with no action. Where a control ran, the action must also beat the same action on molecules chosen at random.
- **Noise mitigation on every molecule.** Against no action. No random selection is possible, so QM9 with no noise added is the only check.
- **Noise-robust training.** Against the same model trained normally on the same labels. A gain on QM9 with no noise added comes from the training change alone, and we set it beside the gain under damage.

Five controls ran, each described with its experiment. Random deletion is the control for deleting flagged molecules and for the two learners, and random put-back for cliff put-back. Shuffled detector scores are the control for cleaning the labels as delivered, and shuffled lifts for the per-molecule lift. Settings tuned on clean labels are the control for retuning. Each seed of a control is reported on its own. The other methods have no control, which makes their gains weaker evidence.

QM9 is the clean-label control, and the most direct test in this chapter of whether a method removes noise. Its HOMO–LUMO gaps are computed rather than measured [109], so they carry no assay error. Every other dataset is measured, and its labels as delivered carry noise that nobody has marked. A method that removes noise should do nothing on QM9, or make it worse, because there is no noise to find. A gain on QM9 with no noise added comes from a change that would help on any labels, such as a better-trained model. The result we look for is a method that leaves QM9 unchanged and improves a measured dataset's labels as delivered. That is the strongest evidence we have that a method found real noise. A gain on the labels as delivered that QM9 matches is not such evidence. The shuffled-score control cannot make this distinction alone, because it asks whether the choice of molecules matters, not whether there is noise. We therefore set each method's result on QM9 beside its gains elsewhere (Section 7.3.2).

Where a clean score exists, we also report how much of the lost score a method wins back. Each model m is fitted to the clean, the damaged and the treated training labels, giving held-out scores s^m_{clean}, s^m_{damaged} and s^m_{treated} in the dataset's own metric. The recovery is

$$\text{Recovery}_m = 100 \times \frac{s^m_{\text{treated}} - s^m_{\text{damaged}}}{s^m_{\text{clean}} - s^m_{\text{damaged}}}\ \%, \tag{7.6}$$

which reads the same whether higher or lower values of the metric are better. It is left undefined when the gap between the clean and damaged score is 2% of the clean score or less. We report it for single representations only. A hybrid's features are re-ranked on the damaged labels, so its clean score is not the score of the same features, and the denominator would mix two changes.

7.3 Results

Three things hold for every result in this chapter. First, because of time and compute, most results are on the first outer fold only (Section 5.2.5), and we give the fold count wherever more exist. Second, where a method has several settings, such as a share to delete or a strength, we report its best setting on the held-out score; because the single representation and the hybrid are picked the same way, every reported gain is optimistic. Third, the network's score changes from one fit to the next. On QM9 its second hybrid scores between 0.348 and 0.362 RMSE on the same fold and labels, depending on which experiment fitted it, and 0.364 in one run of the published sweep (Table 7.33). LightGBM gives one score in every experiment. A network gain of up to

about 0.016 RMSE is inside that spread.

7.3.1 Overall effect of noise handling

Additional file 38 counts how often each group of methods improved the held-out score on the first outer fold. Mitigation helps most often under injected damage, and least often on the labels as delivered.

7.3.2 The clean-label test: where noise is removed

QM9 separates the two things a gain can mean (Section 7.2.5). A method that removes noise should leave QM9's computed labels alone and improve measured labels as delivered. A method that also helps QM9 is, at least in part, a better way to train. We apply this test to each kind of method here, before the sections that go through them one by one.

Deletion passes the test. On QM9 with no noise added it made every model worse, in every configuration (Table 7.11). Deleting 20% of the molecules flagged by the forest residual does the same on QM9's labels (Table 7.12). On SMAC colloidal stability and expression titer, the same deletion improved the labels as delivered and beat every shuffled control. These are the clearest cases in the chapter of a method finding noise that was already in the data. On the small-molecule labels as delivered, deletion found nothing to remove (Section 7.3.3).

Label smoothing and weighting pass the test only in part. With the network they improve QM9 with no noise added (Tables 7.10 and 7.12, and the QM9 row of Figure 7.3). LightGBM, which gives the same score on every fit, barely moves on QM9. On Caco-2 and LogD as delivered, the network gains about as much as it does on QM9, so those gains are not evidence of noise. The support vector regressor, which also gives the same score on every fit, gains almost nothing from any cleanup on these labels. Label smoothing on hERG K_i is the exception among the small molecules. Relative to its score, the network's first single representation gains several times what the same cleanup gives on QM9 (Table 7.12).

QM9 also shows the limit of the shuffled-score control. With the network, label smoothing on QM9's first single representation beats all three shuffles, although there is no noise to find (Table 7.12). The shuffles themselves move about as much as the network does between fits. Passing the shuffled control with the network is therefore weak evidence by itself, and we read it beside QM9.

Changing the model fails the test, so its gains are better training more than noise

handling. On QM9 with no noise added, stacking the two models improves every configuration, and averaging network seeds improves most of them (Table 7.30). On the labels as delivered of hERG K_i and LogD the same changes help by about as much. Under injected Gaussian noise, seed averaging on hERG K_i gains more than on the labels as delivered, in every configuration (Table 7.29). Part of its gain under damage does respond to the noise.

The twelve published methods pass the test in the other direction. On QM9 with no noise added each one made the network worse in the first run (Table 7.33). In a second run about half beat the plain network, by 0.001 to 0.021 RMSE, about the size of the network's own spread. Where one of them helps a measured dataset as delivered, the gain is therefore more likely noise found. The heteroscedastic GP on $\Delta\Delta G$ is the clearest case: it raises the best single representation's Spearman and makes QM9 worse (Tables 7.32 and 7.33).

Noise is removed most clearly, then, where deletion or a published method helps an antibody dataset as delivered and does nothing for QM9. On the small-molecule and reaction labels as delivered, three results clear the bar: label smoothing on hERG K_i and on hERG blocking, and deletion on Buchwald–Hartwig. The test rests on one computed property of one dataset, on the first outer fold, and several methods have no QM9 result (Section 7.4).

7.3.3 Changing a label against deleting a molecule

Under hERG's label flips every action helped, and deleting the flagged molecules helped most (Table 7.6). This holds for gradient-boosted trees on both the hybrid and the best single representation, with 10% of labels flipped along a confusion matrix. Deletion is the one action here with a control, and it passes it. With the neighbor-label gap as the detector, deleting takes the hybrid to 0.838 AUROC against 0.818 for the best of five random deletions of the same size. With the forest residual as the detector, deletion beats the best random deletion on all three folds that have results (Table 7.7).

Figure 7.2 shows how this depends on the share flagged, on one outer fold. Under both flip kinds, deleting the flagged molecules beat the best random deletion at every flagged share, on the hybrid and on the best single representation. The best random deletion is picked on the held-out score, so the control is optimistic. Under the confusion-matrix flips the hybrid started below the single representation and finished above it after every action. On Caco-2, censored EM lowered the network's RMSE at every clipped share, and deleting the clipped molecules raised it. At 40% clipped,

Action	Fold 0, mean of 3 draws		Fold 1, draw 0	
	First single	First hybrid	First single	First hybrid
No cleaning	0.806	0.803	0.8155	0.831
Act on the 10% the forest residual flags				
Delete	0.8253	**0.836**		
Label smoothing	0.8250	0.834		
Down-weight	0.811	0.814		
Delete the top 10% by each detector				
Cliff-aware gap	0.818	**0.838**	0.814	**0.833**
GP spread and residual	0.817	0.827	0.807	0.820
Variance-head network	0.797	0.813	0.795	0.815
NGBoost	0.804	0.807	0.804	0.829
Heteroscedastic GP	0.801	0.806	0.8147	0.823

Table 7.6: Actions and detectors under hERG label flips. hERG blocking with 10% of training labels flipped along a confusion matrix, XGBoost. One row is one action on the flagged molecules. Held-out AUROC, higher is better. Fold 1 was run for the detectors only, each at its best setting.

Configuration and detector	Share deleted	Fold	No cleaning	Delete flagged	Best random deletion
hERG, 10% confusion-matrix flips, XGBoost, first hybrid, AUROC					
Neighbor-label gap	10%	0	0.803	**0.838**	0.818
Forest residual	10%	0	0.803	0.836	0.813
Forest residual	10%	1	0.816	0.826	0.812
Forest residual	10%	2	0.819	0.835	0.828
LogD, 5% outliers, network, RMSE					
Second hybrid, forest residual	5%	0	0.585	**0.546**	0.558
First hybrid, forest residual	10%	0	0.588	0.569	0.565
Second hybrid, forest residual	5%	1	0.624	0.633	0.623
Buchwald–Hartwig, labels as delivered, LightGBM, RMSE					
First hybrid, neighbor-label gap	20%	0	20.1	**18.0**	19.9
Second single, neighbor-label gap	20%	0	20.2	18.3	19.9

Table 7.7: Deleting flagged molecules against random deletion. One row is one configuration, detector and outer fold. The random control deletes the same number of molecules at five seeds; the best of the five is shown. Scores are means over the noise draws. AUROC: higher is better. RMSE: lower is better; for Buchwald–Hartwig in percentage points of yield.

censored EM took the hybrid from 0.365 to 0.317 RMSE.

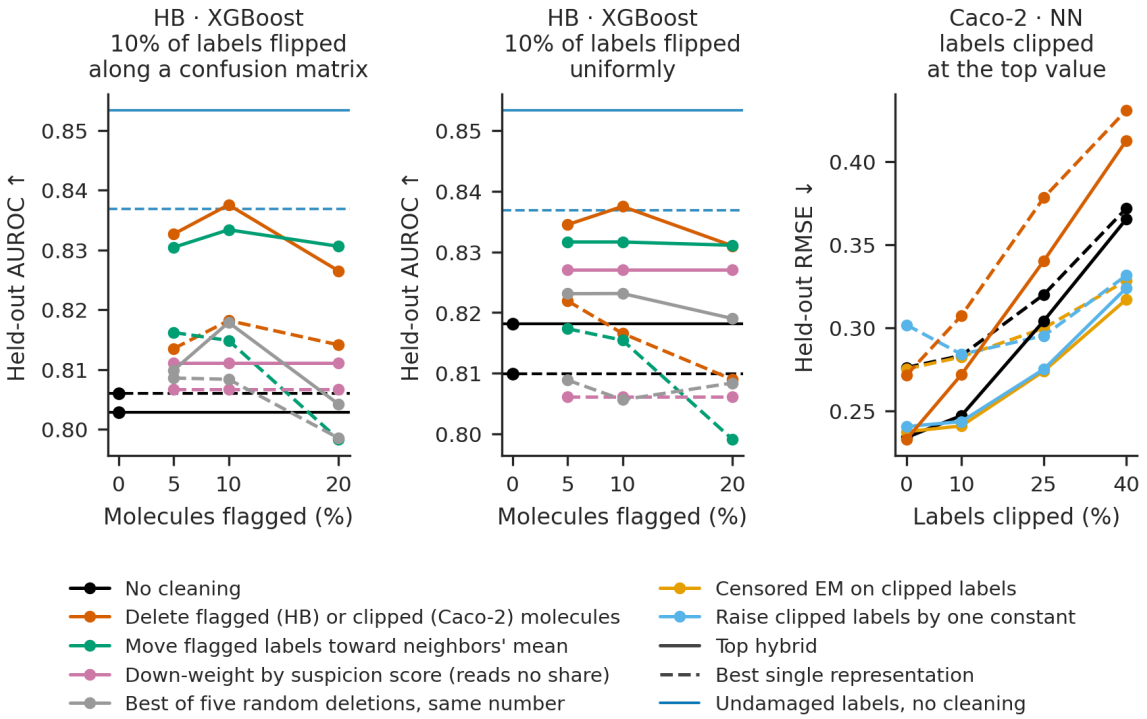


Figure 7.2: Left and middle: held-out AUROC (higher is better) of XGBoost on hERG blocking with 10% of training labels flipped, against the share of training molecules flagged by the neighbor-label gap. The blue line is the score on undamaged labels. Right: held-out RMSE (lower is better) of the network on Caco-2 against the share of training labels clipped. Solid lines are the top hybrid, dashed lines the best single representation. Outer fold 0. Dataset short names as in Table 5.8.

On a regression dataset the three actions differ less. On LogD with 5% outliers, every action improved the network on the first single representation, and the score-scaled shrink toward an out-of-fold prediction and deletion improved it most. On the first hybrid, down-weighting is best, and the shrink makes it slightly worse (Table 7.8). Deletion beats random deletion on the second hybrid but loses to it on the first hybrid. On the second fold the same deletion makes the second hybrid worse (Table 7.7).

Which detector selects the molecules to delete matters more than which action follows. We deleted the top 10% by each of five detectors under the same hERG flips. The cliff-aware gap helped the hybrid most, then the GP, while the variance-head network, NGBoost and the heteroscedastic GP helped it least (Table 7.6). On the single representation the last three make things worse. The three that predict a spread from the features alone never read a label (Section 7.2.2), so they cannot see a flipped one. On LogD with outliers the GP was best for the network's hybrid (Table 7.8). On a second fold of the hERG flips, none of the five improved the single representation,

and only the cliff-aware gap improved the hybrid.

Action	First single	First hybrid
No cleaning	0.627	0.588
Act on the 5% the forest residual flags		
Delete	0.603	0.572
Down-weight	0.615	**0.563**
Label smoothing	0.612	0.580
Score-scaled shrink	**0.601**	0.590
Separate run: delete the top 10% by each detector		
No cleaning	0.627	0.588
Cliff-aware gap	0.628	0.569
GP spread and residual	0.623	**0.558**
Variance-head network	0.636	0.584
NGBoost	0.657	0.580
Heteroscedastic GP	0.648	0.569

Table 7.8: Actions and detectors on LogD with 5% outliers. Network, outer fold 0, mean of 3 draws. One row is one action. Held-out RMSE, lower is better.

A second test of whether the gain comes from the detector uses sets of flagged molecules whose content we control. On hERG K_i with wide errors in 5% of molecules, we down-weighted the 10% of molecules the residual flags. This takes the network on the first single representation from 0.689 to 0.642 RMSE, and improves the hybrid by a similar amount (Table 7.9). Down-weighting the same number of molecules, none of them damaged, makes both worse, so the gain depends on which molecules the action reaches.

			10% with no damaged molecule	
Configuration	No cleaning	Residual's top 10%	Seed 0	Seed 1
First single	0.689	**0.642**	0.706	0.701
First hybrid	0.692	0.639	0.698	0.704

Table 7.9: Down-weighting flags of known content on hERG K_i with wide errors in 5% of molecules. Network, outer fold 0, mean of 3 draws. One row is one configuration. Held-out RMSE, lower is better.

Weighting keeps every molecule and every label, and it helped on hERG, LogD and Buchwald–Hartwig (Table 7.10). Under the hERG flips it helped the hybrid and barely moved the single representation. On QM9 with no noise added, neighbor weights took the network's second hybrid from 0.362 to 0.348 RMSE at their best setting, and

detector weights did about as well. LightGBM barely moved, so a network gain of about 0.01 RMSE from weighting is not evidence that noise was removed.

Configuration	Weighting	Unweighted	Weighted
hERG, 10% confusion-matrix flips, XGBoost, AUROC			
First single	Gap over neighbor spread	0.8060	0.8063
First hybrid	Gap over neighbor spread	0.803	**0.815**
LogD, 5% outliers, network, RMSE			
First single	Neighbor gap, best of four distances	0.627	0.591
First hybrid	Neighbor gap, best of four distances	0.588	**0.556**
Buchwald–Hartwig, Gaussian noise at 1.0, LightGBM, RMSE			
First hybrid	Near-identical pairs that disagree	22.0	**20.7**
QM9, no noise added, RMSE			
Network, second hybrid	Neighbor weights	0.362	**0.348**
Network, second hybrid	Detector weights		0.349
LightGBM, first hybrid	Neighbor weights	0.372	0.371
LightGBM, first hybrid	Detector weights		0.372

Table 7.10: Weighting flagged molecules. One row is one configuration and weighting, at its best setting. Outer fold 0. AUROC: higher is better. RMSE: lower is better; for Buchwald–Hartwig in percentage points of yield. The detector-weight run on QM9 stores no unweighted score.

Figure 7.3 shows, on the first outer fold only, how often each method improved on no cleaning. Figures 7.4 and 7.5 do the same for the known flag and for noise-robust training.

Additional file 39 counts, for each broad noise mitigation method, the conditions where it improved the best single representation, the hybrid, both or neither.

On the first outer fold, deleting never helped on clean labels. On QM9 with no noise added, the best deletion setting took the network's hybrid from 0.355 to 0.393 RMSE, and LightGBM's hybrid and every other configuration also got worse (Table 7.11). When deleting helps on a damaged dataset and beats random deletion, as it does under the hERG flips, the gain is noise removed.

The labels as delivered

On hERG K_i's labels as delivered, label smoothing passes the shuffled-score control with both detectors (Table 7.12). Label smoothing with the neighbor gap takes the network on the first single representation from 0.638 to 0.578 RMSE, while every shuffled run makes it slightly worse. With the forest residual, label smoothing also improves

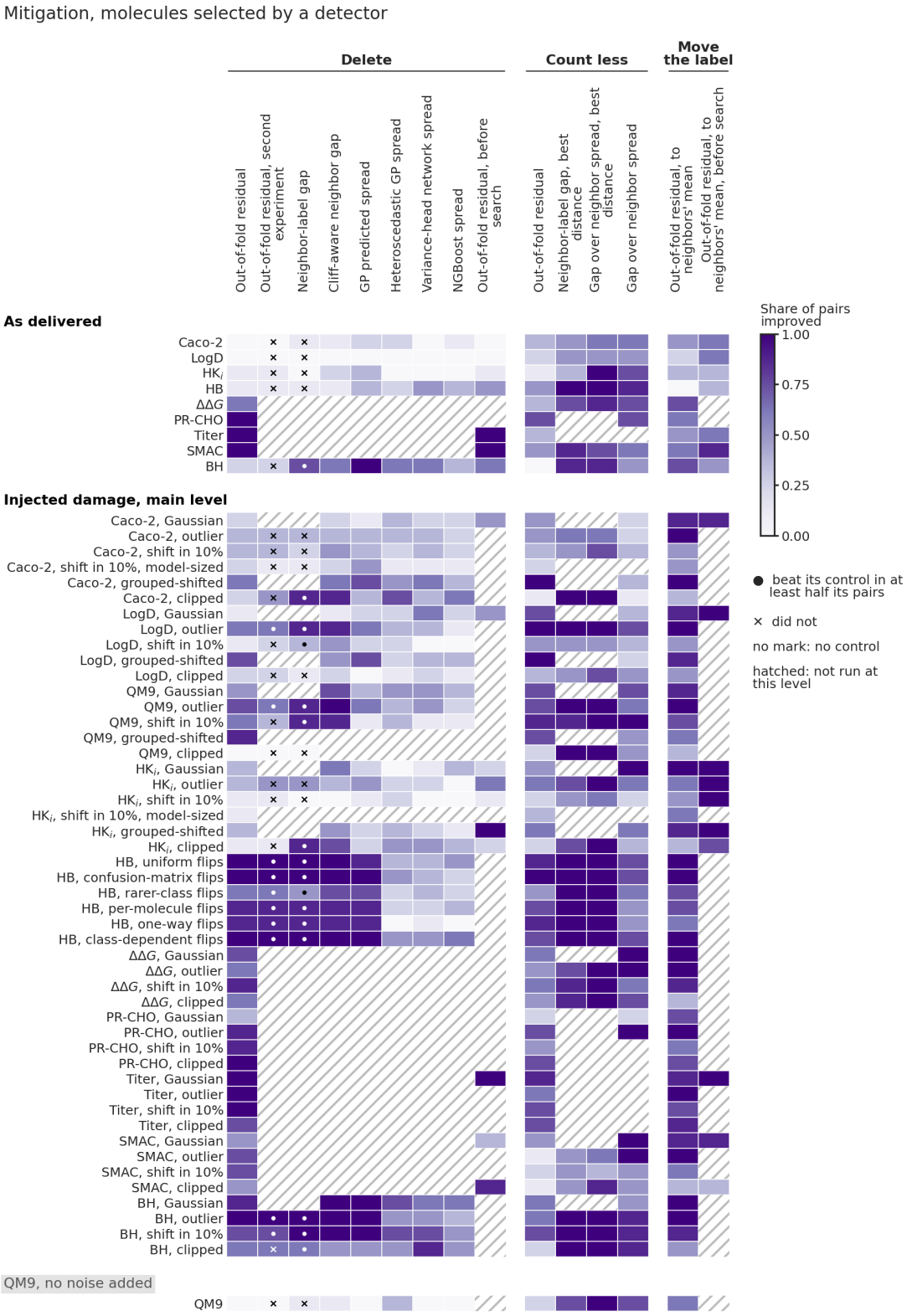


Figure 7.3: Noise mitigation with the molecules selected by a detector, first outer fold. Rows: labels as delivered, injected damage at the main level, and QM9 with no noise added. Columns: method, grouped by action. Shade: share of model-and-configuration pairs improved over no cleaning; darker is more. Dot: beat random deletion in at least half its pairs; cross: did not. Hatched: not run. Dataset short names as in Table 5.8.

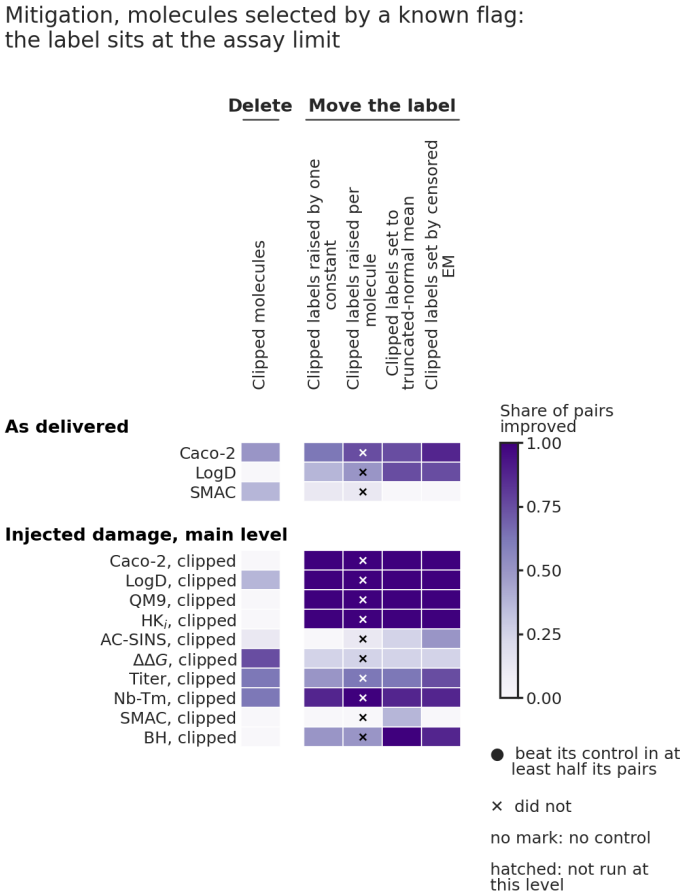


Figure 7.4: Noise mitigation with the molecules selected by a known flag, a label at the assay limit, first outer fold. Rows: labels as delivered and clipped labels at 25%. Columns: method, grouped by action. Shade: share of model-and-configuration pairs improved; darker is more. Cross: did not beat shuffled lifts in at least half its pairs. Dataset short names as in Table 5.8.

Model	Configuration	No deletion	Best deletion setting
Network	First single	0.396	0.430
Network	Second single	0.495	0.514
Network	First hybrid	**0.355**	**0.393**
Network	Second hybrid	0.362	0.388
LightGBM	First single	0.412	0.449
LightGBM	Second single	0.459	0.530
LightGBM	First hybrid	0.372	0.411
LightGBM	Second hybrid	0.376	0.412

Table 7.11: Deleting flagged molecules on QM9 with no noise added. One row is one model and configuration. Held-out RMSE, lower is better. Outer fold 0.

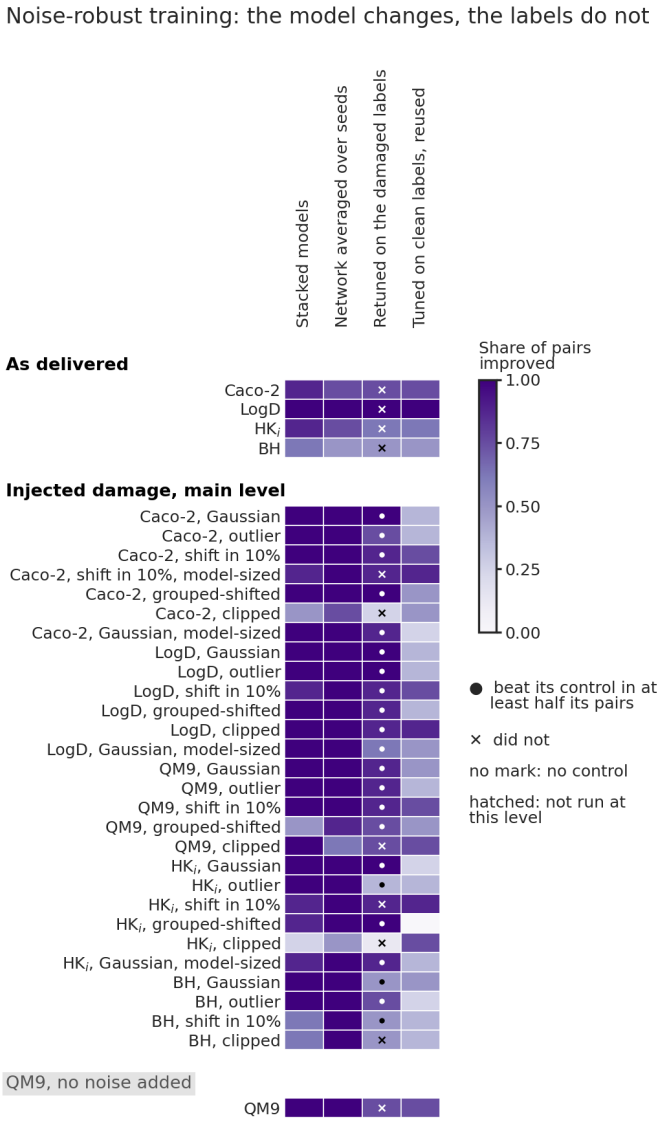


Figure 7.5: Noise-robust training, first outer fold. Rows: labels as delivered, injected damage at the main level, and QM9 with no noise added. Columns: change to the model. Shade: share of model-and-configuration pairs improved over the same model at default settings; darker is more. Dot: retuning beat tuning on clean labels in at least half its pairs; cross: did not. Dataset short names as in Table 5.8.

the hybrid by more than any of three shuffles. In a second experiment it improved the network's hybrid on each of folds 0, 1 and 2, while the second single representation got worse on fold 0 and better on the other two (Table 7.13). On hERG blocking, label smoothing beats all its shuffles with gradient-boosted trees on the first single representation and on the second hybrid. Caco-2 and LogD pass the control too, but by small amounts. With down-weighting on the network's second single representation, one shuffle wins back more than half of the gain on each.

Table 7.12: Cleaning the labels as delivered against the same cleanup driven by shuffled detector scores. One row is one model, configuration, cleanup and detector. QM9 has no noise to find. Outer fold 0.

Configuration, cleanup and detector	No cleaning	Cleaned	Shuffled, seed 0	Shuffled, seed 1	Shuffled, seed 2
hERG K_i, network, RMSE (lower is better)					
First single, label smoothing, neighbor-label gap	0.638	0.578	0.642	0.642	0.645
First hybrid, label smoothing, forest residual	0.599	0.578	0.600	0.588	0.596
hERG blocking, XGBoost, AUROC (higher is better)					
First single, label smoothing, neighbor-label gap	0.837	0.845	0.835	0.831	0.833
Second hybrid, label smoothing, neighbor-label gap	0.854	0.860	0.848	0.848	0.855
Caco-2, network, RMSE (lower is better)					
Second single, down-weight, forest residual	0.266	0.255	0.268	0.262	0.258
LogD, network, RMSE (lower is better)					
Second single, down-weight, neighbor-label gap	0.4212	0.414	0.436	0.417	0.4209
SMAC colloidal stability, Spearman (higher is better)					
SVR, first single, delete 20%, forest residual	0.274	0.403	0.221	0.234	0.278
SVR, second hybrid, delete 20%, forest residual	0.311	0.378	0.328	0.278	0.300
Network, first single, move toward out-of-fold, neighbor-label gap	0.065	0.319	0.133	0.073	0.064
Network, first hybrid, move toward out-of-fold, forest residual	0.227	0.442	0.220	0.217	0.223
$\Delta\Delta G$, network, Spearman (higher is better)					
First single, label smoothing, forest residual	0.341	0.484	0.277	0.352	0.460
First hybrid, label smoothing, forest residual	0.259	0.287	0.276	0.274	0.270
Expression titer, random forest, Spearman (higher is better)					
Second single, delete, forest residual	0.345	0.407	0.357	0.401	0.349
Second hybrid, delete, forest residual	0.224	0.317	0.190	0.271	0.294
QM9, no noise added, RMSE in eV (lower is better)					
Network, first single, label smoothing, neighbor-label gap	0.3999	0.3927	0.3996	0.3967	0.4005
Network, first hybrid, label smoothing, forest residual	0.3491	0.3487	0.3568	0.3502	0.3456

Configuration, cleanup and detector	No cleaning	Cleaned	Shuffled, seed 0	1	2
Network, first single, delete 20%, forest residual	0.400	0.461	0.433	0.429	0.412
LightGBM, first single, label smoothing, neighbor-label gap	0.412	0.414	0.417	0.418	0.417

Fold	First hybrid		Second single	
	No cleaning	Label smoothing	No cleaning	Label smoothing
0	0.599	**0.567**	0.589	0.596
1	0.550	**0.547**	0.610	0.600
2	0.550	**0.537**	0.645	0.629

Table 7.13: Label smoothing on hERG K_i's labels as delivered, by outer fold. Network. One row is one outer fold. Held-out RMSE, lower is better. First hybrid: forest residual, flag share 5%. Second single: the best detector setting on each fold.

Deleting does not pass its control on the small-molecule labels as delivered. On Caco-2, hERG K_i and LogD no deletion setting beat both no cleaning and every random deletion. Buchwald–Hartwig is different: deleting the 20% of reactions with the largest neighbor-label gap improves LightGBM on the hybrid and on the second single representation, and beats random deletion on both (Table 7.7). On the hybrid it scores 18.0 RMSE in percentage points of yield, against 19.9 for the best random deletion. Random deletion alone also lowered the RMSE a little, from 20.1 to 19.9 on the hybrid and from 20.2 to 19.9 on the second single representation.

Putting activity-cliff molecules back after a cleanup can be tested on hERG K_i only. On Caco-2, hERG and LogD more than half of the training molecules count as cliff molecules, which leaves too few other molecules to draw a random set of the same size. On hERG K_i's first outer fold, deleting the flagged molecules barely moves the network's hybrid. Putting the cliff molecules back takes it to 0.579 RMSE, against 0.595 with random molecules put back. Fold 2 agrees, and fold 1 does not (Table 7.14).

7.3.4 Detection of the suspect labels

This section uses injected damage alone (Section 7.2.2). The out-of-fold forest residual finds outliers, shifted labels and flipped classes well above chance on every dataset it ran on (Table 7.15). On Caco-2 under outlier noise its average precision is 0.338, against

Fold	No cleaning	Delete flagged	Cliff molecules put back	Random molecules put back
0	0.597	0.596	0.579	0.595
1	0.551	0.5681	0.5684	0.557
2	0.550	0.575	0.537	0.545

Table 7.14: Putting activity-cliff molecules back after deleting the 1% the neighbor-label gap flags, on hERG K_i's labels as delivered. Network on the first hybrid. One row is one outer fold. Held-out RMSE, lower is better.

a chance level of 0.051. Censoring is the exception on small molecules and reactions. At a clipped share of 0.25 the residual falls below chance on Buchwald–Hartwig, LogD and QM9, and is only a little above it on Caco-2 and hERG K_i. On SMAC colloidal stability it stays well above chance. A model fitted to clipped labels predicts values near the limit for the clipped molecules, which leaves their residuals small. Chapter 3 found the same in the models' own uncertainty, which was lowest on the clipped labels.

The cliff-aware gap does better than the residual when only some labels are shifted. Under the shift in 10%, its median neighbor setting reaches 0.269 on Caco-2 against 0.202 for the residual. Under outliers its best setting is ahead and its median setting behind, and under censoring it trails the residual on Caco-2, hERG K_i and LogD at every setting (Table 7.15). No detector is best everywhere: a neighbor detector is better at a shift that moves a few labels a long way, and a model residual is as good or better at outliers.

The space in which the neighbors are found changes detection little (Section 7.2.2). Taking the best setting in each family, the importance-weighted features were best on Buchwald–Hartwig, Caco-2, hERG K_i, LogD and SMAC colloidal stability (Table 7.16). On Caco-2 under outliers the four families reach average precisions close to each other, all far above chance. The hybrid's own features were the best family on the change in binding free energy ($\Delta\Delta G$) under outliers. They were also best on SMAC colloidal stability under censoring and on Caco-2 under the shift sized by model error, but on no dataset overall. Averaged over the detector settings instead of taking the best one, the hybrid's features come out ahead more often, so this ranking depends on how a detector's setting is chosen.

Scaling each feature by its importance before the neighbors are found raised detection under the shift in 10% on every dataset but hERG K_i, where it was unchanged, and lowered it under censoring on every dataset (Table 7.17). The largest gain was on Buchwald–Hartwig, 0.465 to 0.512 average precision, with smaller gains on LogD and SMAC colloidal stability, while under outliers scaling barely changed detection. Im-

Dataset	Damage	Chance	Forest residual	Cliff-aware, best setting	Cliff-aware, median setting
QM9	Clipped 25%	0.250	0.157	0.227	0.161
	Outliers	0.051	0.391	0.388	0.358
	Shift in 10%	0.101	0.611	0.712	0.590
Caco-2	Clipped 25%	0.250	0.306	0.292	0.253
	Outliers	0.051	**0.338**	0.347	0.294
	Shift in 10%	0.099	0.202	0.296	**0.269**
	Shift in 10%, model error	0.100	0.137	0.180	0.170
HK_i	Clipped 25%	0.244	0.264	0.248	0.212
	Outliers	0.056	0.370	0.379	0.356
	Shift in 10%	0.098	0.226	0.373	0.340
	Shift in 10%, model error	0.096	0.149	0.230	0.209
LogD	Clipped 25%	0.237	0.196	0.195	0.169
	Outliers	0.051	0.398	0.402	0.357
	Shift in 10%	0.099	0.339	0.473	0.439
HB	Uniform flips	0.093	0.397	0.493	0.406
	One-way flips	0.095	0.349	0.430	0.360
	Class-dependent flips	0.100	0.400	0.478	0.403
	Class-frequency flips	0.094	0.276	0.397	0.338
	Instance flips	0.106	0.403	0.494	0.411
	Confusion-matrix flips	0.100	0.412	0.498	0.415
BH	Clipped 25%	0.250	**0.193**	0.335	0.249
	Outliers	0.051	0.352	0.386	0.342
	Shift in 10%	0.099	0.285	0.765	0.497
SMAC	Clipped 25%	0.243	0.506	0.573	0.407
	Outliers	0.055	0.306	0.401	0.340
	Shift in 10%	0.095	0.233	0.470	0.373

Table 7.15: Detection by the out-of-fold forest residual and the cliff-aware gap. One row is one dataset and damage kind at its main level (25% clipped, 10% flipped, otherwise 1.0). Average precision of the damaged molecules, higher is better; chance is the share damaged. Outer fold 0. Dataset short names as in Table 5.8.

portance is computed against the damaged labels, so under censoring it likely rewards the features that predict the clipped values, and the clipped molecules then look like their neighbors.

A Gaussian process's own predicted spread plus its residual is the one model-uncertainty detector that could be graded. The other three predict a spread from the features alone. Every damage kind that can be graded picks its molecules at random, so these three have nothing to find (Section 7.2.2). The GP was slightly ahead of the cliff-aware gap under outliers on Caco-2 and LogD, and behind it under every

Dataset	Damage	Chance	Plain	Importance-weighted	Forest proximity	Hybrid's features
Caco-2	Clipped 25%	0.250	0.329	0.335	0.208	0.324
	Outliers	0.051	0.337	**0.352**	0.336	0.335
	Shift in 10%	0.099	0.346	0.374	0.316	0.367
	Shift in 10%, model error	0.100	0.227	0.226	0.213	**0.233**
HK_i	Clipped 25%	0.244	0.284	0.285	0.218	0.252
	Outliers	0.056	0.386	0.393	0.368	0.389
	Shift in 10%	0.098	0.411	0.434	0.370	0.425
LogD	Clipped 25%	0.237	0.223	0.234	0.194	0.211
	Outliers	0.051	0.397	0.407	0.400	0.395
	Shift in 10%	0.099	0.455	0.503	0.464	0.495
HB	Uniform flips	0.093	0.451	0.469	0.492	0.473
	One-way flips	0.095	0.403	0.414	0.437	0.402
	Class-dependent flips	0.100	0.451	0.474	0.466	0.470
	Class-frequency flips	0.094	0.359	0.400	0.393	0.397
	Instance flips	0.106	0.455	0.483	0.488	0.477
	Confusion-matrix flips	0.100	0.467	0.485	0.494	0.482
BH	Clipped 25%	0.250	0.355	0.374	0.225	0.324
	Outliers	0.051	0.389	0.411	0.404	0.391
	Shift in 10%	0.099	0.677	0.783	0.778	0.672
$\Delta\Delta G$	Clipped 25%	0.251	0.444	0.408	0.347	0.391
	Outliers	0.055	0.485	0.488	0.471	**0.507**
	Shift in 10%	0.098	0.357	0.329	0.327	0.328
SMAC	Clipped 25%	0.243	0.576	0.578	0.212	**0.613**
	Outliers	0.055	0.365	0.441	0.253	0.381
	Shift in 10%	0.095	0.494	0.633	0.394	0.525

Table 7.16: Detection by the neighbor detectors in four families of distance. One row is one dataset and damage kind at its main level. Each cell is the best average precision over the detector settings and spaces in that family; higher is better. Chance is the share damaged. Outer fold 0. Dataset short names as in Table 5.8.

shift and every hERG flip kind (Additional file 40). Detectors from outside cheminformatics did better than the residual where the residual is weakest. Under censoring the nearest-neighbor Shapley value reached 0.391 on Buchwald–Hartwig, against 0.195 for the residual in the same run. Table 7.15, from another run, gives the residual 0.193. The Shapley value was also ahead on SMAC colloidal stability. Under the shift in 10% the cleanlab score was well ahead of the residual on Buchwald–Hartwig. Each number is a mean over the detector's settings, and the Shapley value has many more settings than the residual.

Two more experiments graded the plain residual under damage sized by the model's own error. It ranked the shifted molecules above chance on every dataset there, for

	Clipped 25%		Outliers		Shift in 10%	
Dataset	Unscaled	Scaled	Unscaled	Scaled	Unscaled	Scaled
BH	0.253	0.237	0.305	0.312	**0.465**	**0.512**
Caco-2	0.256	0.253	0.258	0.259	0.268	0.271
HK_i	0.212	0.207	0.304	0.305	0.3259	0.3257
LogD	0.179	0.174	0.311	0.313	0.412	0.442
$\Delta\Delta G$	0.331	0.321	0.359	0.357	0.243	0.250
Nb-Tm	0.188	0.185	0.272	0.271	0.261	0.269
AC-SINS	0.450	0.415	0.375	0.373	0.236	0.241
SMAC	0.415	0.393	0.293	0.297	0.331	0.368
Titer	0.315	0.312	0.342	0.345	0.161	0.162

Table 7.17: Neighbor detection with and without scaling each feature by its importance. One row is one dataset; the damage is at its main level. Average precision of the damaged molecules, mean over matched settings; higher is better. Outer fold 0. Dataset short names as in Table 5.8.

example an average precision of 0.20 to 0.25 against a chance level of 0.106 on expression titer. Turning such a ranking into a decision is harder. Conformal p-values with a false-discovery-rate rule flagged no molecule at all at $q = 0.05$. At $q = 0.1$ they flagged none when the calibration set was drawn from the noisy training labels. Only with a clean calibration set, which a user would not have, did it flag any: 10 to 37 molecules, in three of 134 passes. Acting on those flags left the held-out score where it was in almost every pass.

The break-even precision explains when acting on a detector can work (Section 7.2.3). With wide errors in 5% of molecules, the residual's top 5% are mostly damaged molecules, and its precision is above the break-even on Caco-2 and hERG K_i (Table 7.18). Under the shift sized by model error, the residual's precision falls below the break-even for deletion. On Caco-2 the residual's top 5% has a precision of 0.171, and deleting them only pays off above 0.616. On hERG K_i under the plain shift in 10%, the precision of the top 10% is also far below the break-even for deletion. A detector that ranks the damaged labels above chance can still lead to a worse model: acting on its flags helps only above the break-even, and the break-even depends on the damage, the action and the model.

Figure 7.6 applies this test to the two learners. Each flags the training molecules with the largest loss, and we delete every molecule that either one flags, so slightly more than the flagged share is deleted. Under the shift in 10% on hERG K_i and Caco-2, the precision stayed below the break-even at every share, and deleting raised the RMSE every time. On hERG K_i the precision was at most 0.26, against a break-even of at

Dataset	Damage	Share flagged	Precision	Break-even precision			
				Delete	Down-weight	Relabel	Shrink
Caco-2	Wide errors in 5%	5%	0.627	0.275	0.239	0.134	
	Wide errors in 5%	10%	0.370	0.222	0.214	0.119	
	Wide errors in 5%	20%	0.206	0.095	0.134	0.091	
	Shift in 10%	5%	0.271	0.239	0.201	0.331	0.406
	Shift in 10%	10%	0.218	0.168	0.155	0.299	0.529
	Shift in 10%	20%	0.200	0.292	0.266	0.244	
	Shift in 10%, model error	5%	**0.171**	**0.616**	0.563	0.575	0.074
	Shift in 10%, model error	10%	0.144	0.405	0.242	0.366	0.290
	Shift in 10%, model error	20%	0.128	0.420	0.422	0.502	0.111
HK_i	Wide errors in 5%	5%	0.620	0.417	0.270	0.156	
	Wide errors in 5%	10%	0.355	0.268	0.223	0.184	
	Wide errors in 5%	20%	0.192	0.121	0.071	0.090	
	Shift in 10%	5%	0.294	0.484	0.441	0.313	0.204
	Shift in 10%	10%	**0.262**	**0.684**	0.213	0.105	0.106
	Shift in 10%	20%	0.224	0.450	0.298	0.402	0.319
	Shift in 10%, model error	5%	0.195	0.456	0.026	0.116	
	Shift in 10%, model error	10%	0.160	0.914	0.120	0.422	0.109
	Shift in 10%, model error	20%	0.141	0.393		0.154	0.064

Table 7.18: Precision of the out-of-fold residual's flags against the break-even precision of each action. One row is one dataset, damage kind at level 1.0 and flagged share. Precision is the share of flagged molecules that were damaged, mean over configurations and models; break-even is the median over configurations and models. An action helps when the precision is above its break-even. Empty: no break-even defined. Outer fold 0. Dataset short names as in Table 5.8.

least 0.32. LogD has no break-even precision. There, deleting at 5% under outliers with a precision of 0.36 lowered the RMSE from 0.587 to 0.557. hERG K_i and LogD have two of three noise draws.

7.3.5 Effect by noise type

Labels clipped at an assay limit

Clipping at an assay limit is the damage we could most often undo, and undoing it needs no detector. With a quarter of the training labels clipped (Figure 7.7), for every

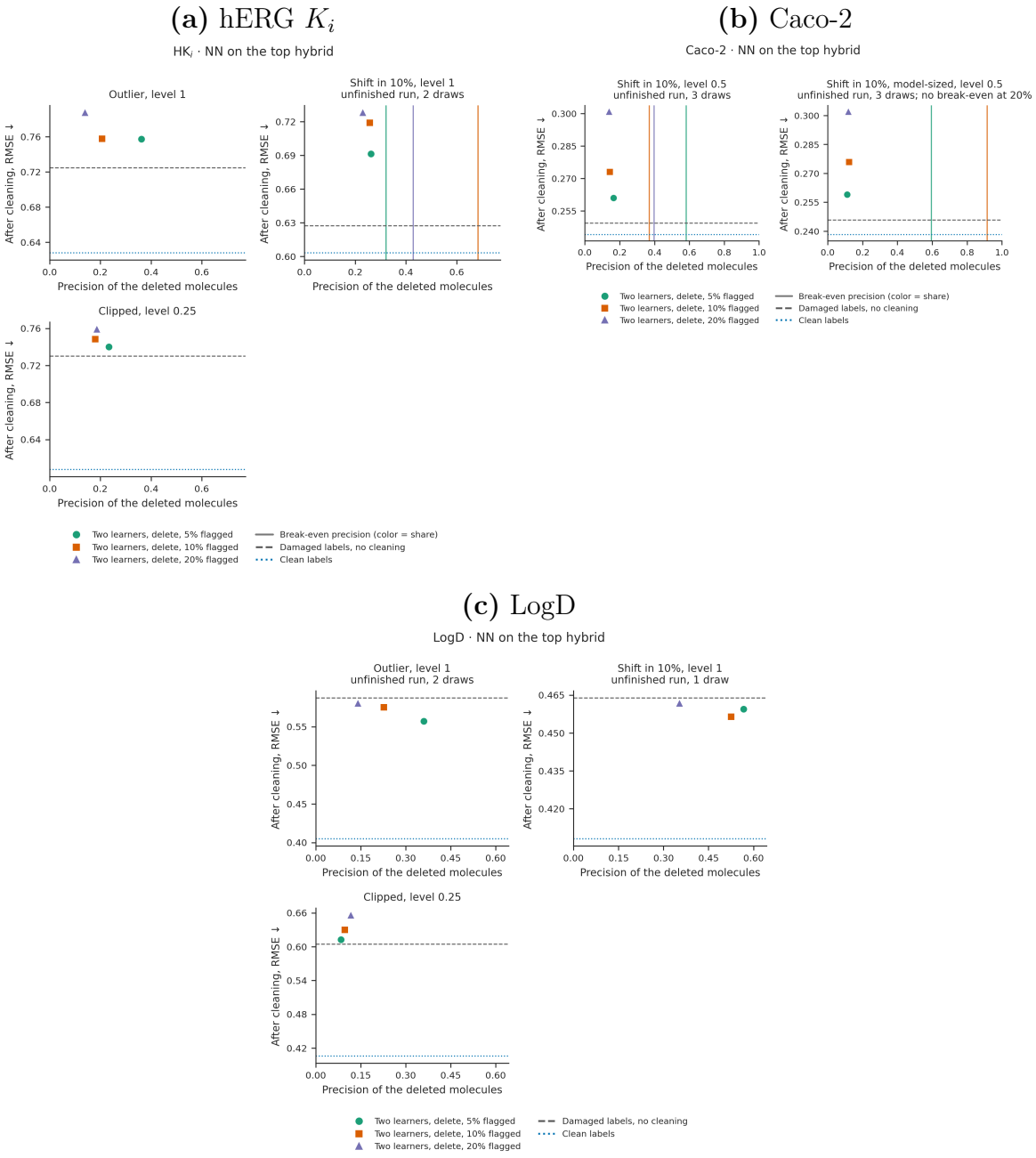


Figure 7.6: Held-out RMSE (lower is better) after deleting the molecules flagged by two learners, against the precision of the deleted molecules: the share whose label the damage moved. (a) hERG K_i, (b) Caco-2, (c) LogD. One panel per damage kind and one marker per flagged share. Vertical lines: break-even precision for the same share. Dashed: damaged labels, no cleaning. Dotted: clean labels. Network on the top hybrid, outer fold 0. Dataset short names as in Table 5.8.

model and configuration censored EM gives a lower held-out RMSE than training on the clipped labels, but not as low as training on the labels before clipping. On LogD, the network on the best single representation wins back about three quarters of the lost RMSE on outer fold 0 (Equation 7.6). QM9 shows the same pattern for the network and LightGBM, and Caco-2 at a smaller scale.

The gain holds on the further folds: on LogD folds 0 to 4, censored EM improves the network's hybrid over the clipped labels on every fold, and stays short of the score before clipping. It also beats the clipped labels on Caco-2 folds 1 and 2 (Table 7.19). In this run hERG K_i and QM9 have only fold 0; a separate five-fold run on QM9 is described at the end of this section.

Dataset	Fold	Before clipping	Clipped	Censored EM
LogD	0	0.404	0.613	0.480
	1	0.418	0.596	0.472
	2	0.411	0.572	0.470
	3	0.413	0.570	0.475
	4	0.398	0.515	0.437
Caco-2	1	0.261	0.322	0.291
	2	0.281	0.400	0.348

Table 7.19: Censored EM on further outer folds, with 25% of training labels clipped. Network on the first hybrid. One row is one outer fold. Held-out RMSE, lower is better. The run is separate from that of Figure 7.7. Dataset short names as in Table 5.8.

Which of the three moves works best depends on the dataset (Table 7.20; every model and configuration in Additional file 41). Censored EM gives the lowest RMSE in most configurations on LogD and QM9. The constant lift does so in most configurations on hERG K_i, where the network on the second single representation scores 0.666 RMSE after the constant lift and 0.704 after censored EM. The truncated-normal mean is behind both on LogD, hERG K_i and Caco-2. On Buchwald–Hartwig clipping hurt the random forest in every configuration and LightGBM in only some, and censored EM brought the random forest back to about its score before clipping (Additional file 41). On nanobody thermostability, scored by Spearman, the constant lift won back most of what clipping cost the network on the hybrid.

Deleting the clipped molecules is worse than leaving the labels clipped on Caco-2, hERG K_i and QM9 (Figure 7.7). On LogD it beats the clipped labels with the network on the hybrid, but stays far behind the lift. Without the clipped molecules, the top of the label range is missing from training.

The per-molecule lift tests whether the size of each lift matters. On LogD, the

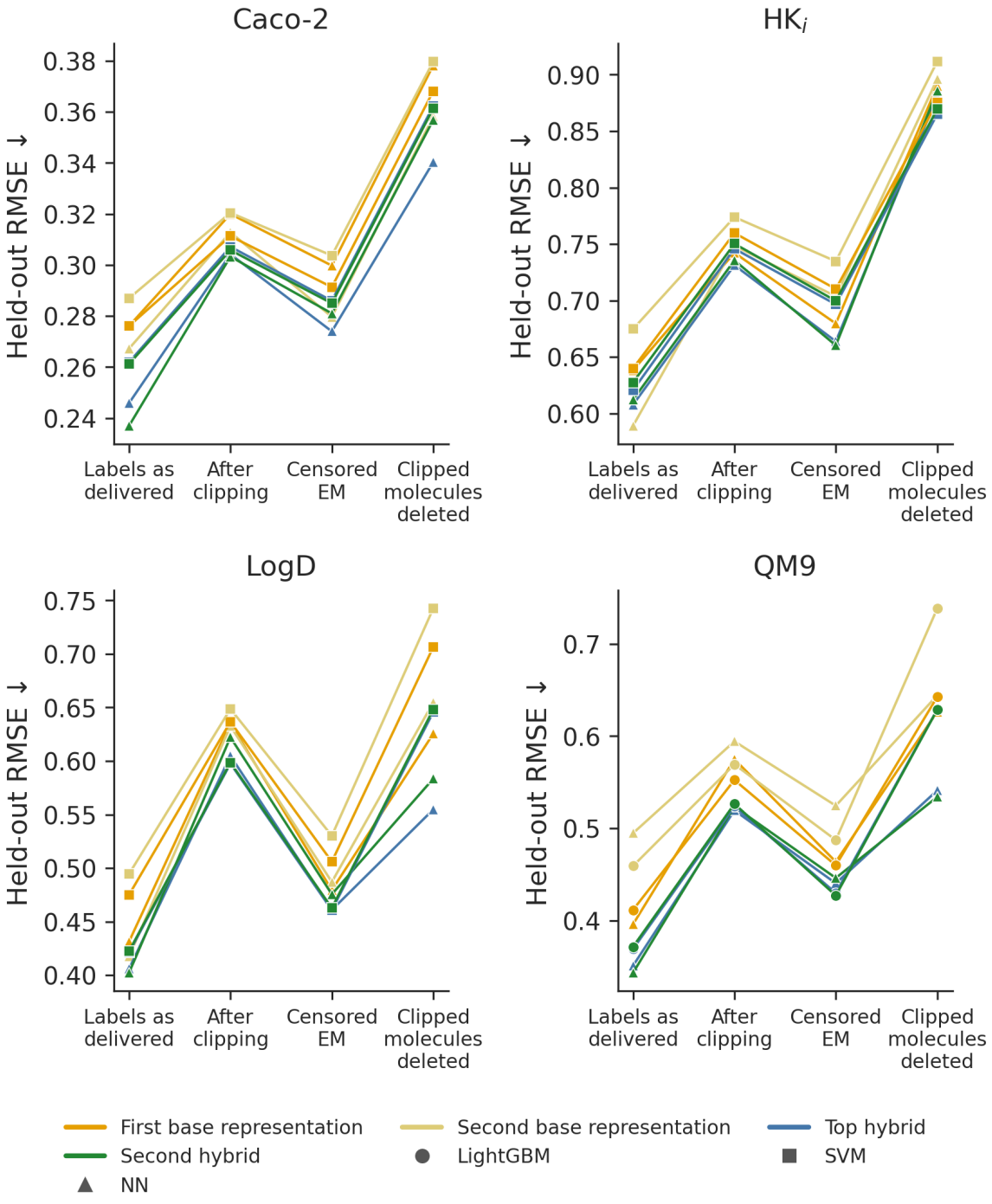


Figure 7.7: Held-out RMSE (lower is better) at four steps: the training labels before clipping, after a quarter of them are clipped at the assay limit, after censored EM raises the clipped labels, and with the clipped molecules deleted instead. One line per model and configuration; color is the configuration and marker shape the model. Outer fold 0. Dataset short names as in Table 5.8.

Dataset	Before clipping	Clipped	Constant lift	Truncated-normal	Censored EM
Network, RMSE in the label's units (lower is better)					
LogD	0.406	0.605	0.460	0.507	0.461
QM9	0.351	0.520	0.450	0.444	0.441
hERG K_i	0.608	0.731	0.668	0.693	0.664
Caco-2	0.246	0.304	0.275	0.288	0.274
Random forest, RMSE in yield points (lower is better)					
Buchwald–Hartwig	17.6	18.7	18.0	18.1	**17.6**
Network, Spearman (higher is better)					
Nanobody thermostability	0.749	0.610	**0.7164**	0.7156	0.699
SMAC colloidal stability	0.162	0.1469	0.141	0.1467	0.129
$\Delta\Delta G$	0.305	0.351	0.302	0.341	0.346

Table 7.20: Three ways to raise clipped training labels, with 25% of training labels clipped. One row is one dataset, on the first hybrid. Outer fold 0. Every model and configuration is in Additional file 41.

network on the hybrid scores 0.499 RMSE with each molecule's own lift, against 0.463 for the best of three shuffles. On LogD and Caco-2 every shuffle beats the molecule's own lift in every configuration, and on hERG K_i in the second single representation (Table 7.21). On QM9 the first hybrid's own lift beats one shuffle of three and loses to the other two. What matters is knowing which labels are clipped and raising them by about the right average amount, which is what the constant lift does.

Censoring did not need fixing on two antibody sets. On SMAC colloidal stability and $\Delta\Delta G$, clipping a quarter of the training labels raised the held-out Spearman in most configurations (Additional file 41). The network's first hybrid on SMAC, shown in Table 7.20, is an exception. On $\Delta\Delta G$, the support vector regressor on the first single representation goes 0.119 before clipping and 0.248 after. With nothing lost, censored EM had nothing to recover, and on both datasets it moved the score back down in most configurations.

In a separate run on all five outer folds, we also clipped a quarter of QM9's training labels, with a control: the same move applied to the same number of unclipped labels. Moving the clipped labels lowered the RMSE with both models in every feature space, and moving the unclipped labels raised it. With the network on the first hybrid, the mean RMSE over the five folds was 0.5072 before treatment and 0.3980 after censored EM. These differ from the fold-0 scores of Table 7.20.

Dataset	Configuration	Own lift	Shuffled seed 0	Shuffled seed 1	Shuffled seed 2
LogD	First single	0.527	0.505	0.507	0.507
	Second single	0.510	0.482	0.477	0.483
	First hybrid	**0.499**	0.471	0.467	**0.463**
	Second hybrid	0.509	0.484	0.482	0.483
HK_i	First single	0.691	0.681	0.708	0.687
	Second single	0.702	0.678	0.694	0.670
	First hybrid	0.677	0.666	0.696	0.680
	Second hybrid	0.667	0.708	0.692	0.669
Caco-2	First single	0.300	0.296	0.293	0.295
	Second single	0.295	0.277	0.280	0.282
	First hybrid	0.288	0.275	0.273	0.271
	Second hybrid	0.282	0.269	0.274	0.270
QM9	First single	0.494	0.492	0.492	0.481
	Second single	0.533	0.518	0.508	0.527
	First hybrid	0.448	0.450	0.440	0.446
	Second hybrid	0.458	0.464	0.446	0.442

Table 7.21: Each clipped label raised by its own estimated lift, against the same lifts dealt to the clipped molecules at random. Network, 25% of training labels clipped. One row is one dataset and configuration. Held-out RMSE, lower is better. Outer fold 0. Dataset short names as in Table 5.8.

A shared offset per scaffold group

Under grouped-shifted noise, subtracting each scaffold family's estimated offset worked on all three datasets it ran on, Caco-2, hERG K_i and LogD. On Caco-2 and LogD it improved the network on the hybrid on every damaged draw, and stayed short of the score on clean labels (Table 7.22). On the first Caco-2 draw the random-intercept model took it from 0.321 to 0.280 RMSE, and it was best on every Caco-2 draw and on the first two LogD draws. On hERG K_i neither of the two was best on every draw.

On the labels as delivered the same methods made every dataset worse (Table 7.23). On Caco-2 the network on the hybrid went from 0.233 to 0.267 RMSE after subtracting family means, and LogD got worse too. Because nothing had shifted a family, subtracting the estimated offsets removed real differences between families. The method assumes a shared offset per family and should be used only where there is reason to expect one.

Several sources per molecule

With simulated sources, removing each source's offset made little difference on small molecules. On hERG K_i the network on the hybrid scored 0.742 RMSE with the plain

Dataset	Draw	Damaged	Family mean	Random intercept	Clean labels
Caco-2	0	0.321	0.297	**0.280**	0.239
	1	0.344	0.298	0.294	0.242
	2	0.361	0.277	0.267	0.231
LogD	0	0.684	0.562	0.546	0.398
	1	0.690	0.606	0.573	0.402
	2	0.672	0.570	0.589	0.395
HK_i	0	0.804	0.691	0.693	0.610
	1	0.834	0.708	0.723	0.611
	2	0.743	0.719	0.666	0.591

Table 7.22: Removing a shared offset per scaffold family under grouped-shifted noise at level 1.0. Network on the first hybrid. One row is one dataset and noise draw. Held-out RMSE, lower is better. Outer fold 0. Dataset short names as in Table 5.8.

Dataset	As delivered	Family mean	Random intercept
Caco-2	**0.233**	**0.267**	0.244
LogD	0.398	0.487	0.437
HK_i	0.599	0.625	0.603

Table 7.23: Removing a shared offset per scaffold family from the labels as delivered. Network on the first hybrid. One row is one dataset. Held-out RMSE, lower is better. Outer fold 0. Dataset short names as in Table 5.8.

mean of the measurements and 0.748 after removing the offsets. Buchwald–Hartwig, where the sources' offsets were large against the label spread, is the one dataset where removing them helped in most settings. Even there the gain can be small, and the random forest on the hybrid stayed far from its score on clean labels.

Gaussian noise, outliers and shifts in a few labels

Moving every label halfway toward a Huber-fitted prediction, with no detector at all, helped the network on hERG K_i under each of the three kinds of damage it ran on. Under Gaussian noise at level 1.0 the second hybrid went from 0.718 to 0.681 RMSE. Under the scaffold-family shift the second hybrid improved too, and under outliers the first single representation did. All three gains held on each of the three draws (Table 7.24).

Damage and configuration	Draw	Damaged labels	Huber blend	Clean labels
Gaussian noise, second hybrid	0	0.693	0.660	0.612
	1	0.755	0.678	0.588
	2	0.708	0.705	0.622
Scaffold-family shift, second hybrid	0	0.776	0.700	0.613
	1	0.821	0.733	0.605
	2	0.739	0.684	0.595
Outliers, first single	0	0.753	0.727	0.638
	1	0.723	0.660	0.638
	2	0.748	0.707	0.638

Table 7.24: Moving every label halfway toward a Huber-fitted prediction on hERG K_i, damage at level 1.0. Network, outer fold 0. One row is one damage kind, configuration and noise draw. Held-out RMSE, lower is better.

7.3.6 Effect by dataset

On three antibody datasets, cleaning the labels as delivered passes the shuffled-score control, and on those datasets it gave the largest gains.

The antibody datasets

SMAC colloidal stability gives the strongest controlled result on any labels as delivered (Table 7.12). Deleting the 20% of antibodies the forest residual flags takes the support vector regressor on the first single representation from 0.274 to 0.403 Spearman, and the same deletion also helps its second hybrid. Moving the flagged labels toward an out-of-fold prediction helps the network on both the single representation and the hybrid. In all four cases, the same cleanup driven by shuffled detector scores gives a lower score on every shuffle. Fold 0 holds out only 53 antibodies, but the shuffles were fitted on the same fold and stayed low.

$\Delta\Delta G$ and expression titer also pass the shuffled control on their labels as delivered, in all four cases below (Table 7.12). On $\Delta\Delta G$, label smoothing takes the network's first single representation from 0.341 to 0.484 Spearman, and also helps its hybrid. On expression titer, deletion helps the random forest's second single representation and its second hybrid. On both datasets the cleaned single representation still scores above the cleaned hybrid.

Neighbor-based weights, which have no control, gave gains of the same size. On SMAC, weighting by agreement with the neighbors took the network on the first single representation from 0.065 to 0.209 Spearman, and also helped it on the hybrid (Table 7.25). The two antibody sets with the fewest noise results, CHO polyreactivity

and AC-SINS self-association, ran only part of the methods, and we do not draw a conclusion from them.

Configuration	Unweighted	Weighted
First single	**0.065**	**0.209**
First hybrid	0.227	0.370

Table 7.25: Weighting by agreement with the neighbors on SMAC colloidal stability's labels as delivered. Network, outer fold 0. One row is one configuration. Held-out Spearman, higher is better.

The held-out folds of these datasets are small, 44 to 135 antibodies, and several start from weak scores, so large gains are easier to see and weaker evidence. The shuffled controls are why we still count SMAC, $\Delta\Delta G$ and expression titer as real results.

Small molecules and the reaction set

On the small-molecule labels as delivered, two label smoothing gains pass their control and do not rest on the network's spread between fits. One is the network's on hERG K_i, the other XGBoost's on hERG blocking (Section 7.3.3). Buchwald–Hartwig is the one dataset where deleting flagged reactions from the labels as delivered beats random deletion.

Repeat measurements on hERG IC_{50}

hERG IC_{50} is the one dataset whose noise was measured rather than injected (Section 7.2). No noise is added here; each draw re-draws which measurements form each held-out molecule's label. Weighting each molecule by its number of measurements took the support vector regressor on the hybrid from 0.760 to 0.728 RMSE on the first draw, and helped on all three draws (Table 7.26). Weighting by the inverse of the estimated error variance also helped it, by less. The same count weights made the network worse. For the network, adding the log of the measurement count as a feature helped instead, and so did shrinking each label toward a prediction by its spread. NGBoost gained a little from the same extra feature, and the interval model did slightly worse than the point model. Training on one row per measurement instead of the mean barely changed the score for either model. Molecules flagged by ChEMBL curators were not tested as a detector.

Model and method	Draw 0	Draw 1	Draw 2
SVR			
No weighting	0.760	0.766	0.761
Weight by measurement count	**0.728**	**0.722**	**0.707**
Weight by inverse error variance	0.735	0.738	0.738
Network			
No weighting	0.712	0.701	0.699
Weight by measurement count	0.765	0.739	0.716
Log count as a feature	0.656	0.665	0.632
Shrink toward a prediction by spread	0.673	0.668	0.659
NGBoost			
No extra feature	0.786	0.823	0.811
Log count as a feature	0.763	0.805	0.797
XGBoost, accelerated failure time			
Point model	0.724	0.728	0.725
Interval model	0.736	0.747	0.730
Separate run, one fit: mean label against one row per measurement			
SVR, mean label		0.767	
SVR, one row per measurement		0.765	
Network, mean label		0.726	
Network, one row per measurement		0.734	

Table 7.26: Using the repeat measurements on hERG IC_{50}. First hybrid. One row is one model and method. Held-out RMSE on molecules with three or more measurements, lower is better. Outer fold 0.

7.3.7 Cleaning inside the feature search

Which of the four combinations does best changes from row to row (Section 7.2.3; Table 7.27). Cleaning both the search and the fit does best in about half the rows. On expression titer's labels as delivered, deleting the 20% of antibodies the forest residual flags takes the network on the hybrid from 0.155 to 0.562 Spearman this way. Cleaning only the fit, or only the search, also helps, by less. On SMAC colloidal stability with a quarter of the labels clipped, deleting the flagged 5% also does best when it reaches both, while cleaning the search alone does worse than no cleaning.

The largest single gain is on SMAC colloidal stability's labels as delivered (Table 7.27). There the network on the second hybrid goes from -0.014 to 0.560 Spearman when the labels are shrunk toward an out-of-fold prediction for the search alone. That starts from a score near zero, on one fold of 53 antibodies. Cleaning before the search can also go the other way. Deleting flagged antibodies before the search made the network's first hybrid on SMAC worse. On hERG K_i under Gaussian noise, deleting

before the search helped the second hybrid on the first noise draw and hurt it on the second. On hERG K_i's labels as delivered, the one dataset with further folds here, label smoothing before the search beat no cleaning on folds 1 and 2 and lost on fold 0. This experiment has no random-flag control and no QM9 result.

Dataset, labels, configuration and cleanup	No cleaning	Fit only	Search and fit	Search only
Spearman (higher is better)				
Titer, as delivered, first hybrid, delete 20%	0.155	0.390	**0.562**	0.423
SMAC, 25% clipped, first hybrid, delete 5%	0.237	0.416	**0.489**	0.221
SMAC, as delivered, second hybrid, shrink toward out-of-fold	−0.014	−0.021	0.484	**0.560**
SMAC, as delivered, first hybrid, delete 5%	0.227	0.241	0.085	0.107
hERG K_i, RMSE (lower is better)				
Gaussian 1.0, second hybrid, delete 5%, draw 0	0.718	0.749	0.649	0.676
Gaussian 1.0, second hybrid, delete 5%, draw 1	0.758	0.760	0.776	0.770
Gaussian 1.0, second hybrid, delete 5%, draw 2	0.723	0.715	0.713	0.714
As delivered, first hybrid, label smoothing, fold 0	0.599	0.593	0.607	0.613
As delivered, first hybrid, label smoothing, fold 1	0.551	0.546	0.529	0.540
As delivered, first hybrid, label smoothing, fold 2	0.550	0.538	0.5444	0.5438

Table 7.27: Cleaning the fit, the hybrid search, or both. Network; the flags are computed in a single representation. One row is one dataset, labels, configuration and cleanup. Outer fold 0 unless a fold is named. Dataset short names as in Table 5.8.

A separate experiment searched the hybrid on clean or damaged labels, then fitted both on labels with Gaussian noise at level 1.0 and scored them on held-out labels as delivered. On Caco-2 the network's second hybrid scores within a few thousandths of an RMSE between the two arms on every fold, and neither arm is ahead on all five (Table 7.28). On LogD with the support vector regressor the two arms agree to the third decimal on two of three folds. The largest loss was on hERG K_i with the network, 0.669 against 0.779 RMSE on the second fold. On SMAC colloidal stability the two arms swap places from fold to fold. For most datasets, the search itself is not where Gaussian noise does its damage.

Model	Searched on	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4
Caco-2, RMSE (lower is better)						
Network	clean labels	0.298	0.328	0.328	0.368	0.346
Network	damaged labels	0.303	0.329	0.334	0.365	0.340
SVR	clean labels	0.303	0.309	0.327	0.342	0.336
SVR	damaged labels	0.298	0.310	0.328	0.344	0.332
LogD, RMSE (lower is better)						
Network	clean labels	0.582	0.613	0.571		
Network	damaged labels	0.575	0.602	0.594		
SVR	clean labels	0.5726	0.579	0.5938		
SVR	damaged labels	0.5732	0.580	0.5942		
hERG K_i, RMSE (lower is better)						
Network	clean labels	0.718	**0.669**	0.699		
Network	damaged labels	0.733	**0.779**	0.708		
SVR	clean labels	0.675	0.620	0.651		
SVR	damaged labels	0.676	0.617	0.646		
SMAC colloidal stability, Spearman (higher is better)						
Network	clean labels	−0.045	0.255	0.057	0.110	0.041
Network	damaged labels	0.043	0.125	0.107	0.135	0.038
SVR	clean labels	0.078	0.131	−0.031	0.186	0.121
SVR	damaged labels	0.093	0.049	−0.012	0.116	0.091
Buchwald–Hartwig, RMSE in yield points (lower is better)						
LightGBM	clean labels	21.8	18.2	17.2		
LightGBM	damaged labels	21.6	17.8	17.1		
Random forest	clean labels	19.4	17.4	16.5		
Random forest	damaged labels	19.6	17.5	17.0		

Table 7.28: The second hybrid searched on clean labels or on labels with Gaussian noise at level 1.0, both fitted on the damaged labels and scored on the held-out labels as delivered. One row is one model and search labels. Mean over three noise draws (two for SVR on SMAC fold 3, the draws with both arms); empty cells were not run. Buchwald–Hartwig RMSE is in percentage points of yield.

7.3.8 Changing the model instead of the labels

On hERG K_i with Gaussian noise at level 1.0, averaging ten network seeds takes the network on the second hybrid from 0.718 to 0.656 RMSE (Table 7.29). Stacking the two downstream models and retuning the network on the damaged labels also help, by less, and none of the three reaches the score on clean labels. Seed averaging won on each of the three noise draws. These gains are as large as the best mitigation results on the same dataset, and they need no detector and change no label.

On the labels as delivered they help too, but not everywhere (Table 7.30). They help QM9 with no noise added by about as much, so on these labels their gains are

better training, not noise found (Section 7.3.2). On LogD, seed averaging improves the network on both its best single representation and its best hybrid. On hERG K_i the network's best single representation, the second, does not improve with any of the three, and retuning makes both hybrids worse. Stacking scores the same whichever model it is compared with, because it combines both. Against the support vector regressor it therefore compares two models with the weaker one alone, for example 0.495 to 0.408 RMSE on LogD's second single representation.

Under Gaussian noise on hERG K_i, retuning on the damaged labels mostly beats reusing settings tuned on clean labels (Table 7.29). On the network's second hybrid, the setting tuned on damaged labels scores 0.691 and the setting tuned on clean labels 0.703. On its first hybrid the clean-label settings are slightly ahead, 0.709 against 0.712. With a quarter of the labels clipped, the clean-label settings did better in every configuration. Retuning on the clipped labels also rarely beat the plain model.

Configuration	Plain	Seed averaging	Stacking	Retuned	Clean-label settings	Plain on clean labels
hERG K_i, Gaussian noise at 1.0, network						
First single	0.701	0.678	0.681	0.688	0.704	0.638
Second single	0.735	0.675	0.685	0.734	0.735	0.589
First hybrid	0.720	0.664	0.663	0.712	0.709	0.607
Second hybrid	0.718	**0.656**	0.671	**0.691**	**0.703**	0.608
hERG K_i, Gaussian noise at 1.0, SVR						
First single	0.699		0.681	0.682	0.724	0.640
Second single	0.708		0.685	0.689	0.759	0.675
First hybrid	0.668		0.663	0.660	0.702	0.616
Second hybrid	0.675		0.671	0.673	0.702	0.617
hERG K_i, 25% clipped, network						
First single	0.742	0.758	0.764	0.7634	0.7633	0.638
Second single	0.749	0.750	0.761	0.768	0.749	0.589
First hybrid	0.731	0.719	0.751	0.735	0.727	0.608
Second hybrid	0.736	0.724	0.748	0.746	0.728	0.612
hERG K_i, 25% clipped, SVR						
First single	0.760		0.764	0.769	0.749	0.640
Second single	0.774		0.761	0.766	0.765	0.675
First hybrid	0.746		0.751	0.749	0.727	0.621
Second hybrid	0.750		0.748	0.753	0.736	0.628

Table 7.29: Changing the model instead of the labels, on damaged hERG K_i labels. One row is one configuration. Held-out RMSE, lower is better. Retuned: settings tuned on the damaged labels. Stacking combines both models and has one score per configuration. Outer fold 0, means over three noise draws.

The other noise-robust training methods are in Additional file 42. A Huber loss

Configuration	Plain	Seed averaging	Stacking	Retuned
hERG K_i, network				
First single	0.638	0.618	0.632	0.628
Second single	0.589	0.596	0.606	0.589
First hybrid	0.597	0.596	0.582	0.623
Second hybrid	0.612	0.598	0.589	0.639
hERG K_i, SVR				
First single	0.640		0.632	0.618
Second single	0.675		0.606	0.640
First hybrid	0.607		0.582	0.573
Second hybrid	0.612		0.589	0.579
LogD, network				
First single	0.431	0.417	0.430	0.428
Second single	0.417	**0.405**	0.408	0.409
First hybrid	0.397	0.379	0.389	0.394
Second hybrid	0.394	**0.376**	0.387	0.390
LogD, SVR				
First single	0.475		0.430	0.421
Second single	0.495		0.408	0.474
First hybrid	0.421		0.389	0.387
Second hybrid	0.421		0.387	0.387
QM9, no noise added, network, RMSE in eV				
First single	0.400	0.391	0.375	0.389
Second single	0.487	0.463	0.410	0.487
First hybrid	0.3491	0.3493	0.331	0.356
Second hybrid	0.357	0.346	0.340	0.358
QM9, no noise added, LightGBM, RMSE in eV				
First single	0.412	0.403	0.375	0.403
Second single	0.459	0.458	0.410	0.455
First hybrid	0.372	0.371	0.331	0.3705
Second hybrid	0.376	0.370	0.340	0.370

Table 7.30: Changing the model instead of the labels, on the labels as delivered and on QM9 with no noise added. One row is one configuration. Held-out RMSE, lower is better. Retuned: settings tuned on these labels. Stacking combines both models and has one score per configuration. Outer fold 0.

for the network helped under outliers on hERG K_i, and made Gaussian noise worse at every threshold. Each model's own robust loss helped under the shift sized by model error, for example for the network on Caco-2, though not back to its score on clean labels. The two-learner selection beat its random-deletion control with tree models. On Buchwald–Hartwig under the shift in 10%, the random forest on the first single representation went from 20.3 to 18.6 RMSE in percentage points of yield and beat

random deletion at all three seeds. The tree models and the network ran on different datasets, though, so we cannot say whether the tree models or the datasets are the reason.

The expanded published sweep

Table 7.31 gives the expanded sweep by domain. Small-loss selection and multistage cleaning delete the molecules with the largest training loss. They rarely helped on small molecules, and more often on Buchwald–Hartwig and the antibodies, though Buchwald–Hartwig is a single dataset.

Method	Small molecules, of 31	Buchwald–Hartwig, of 5	Antibodies, of 25
Noise mitigation: selected by the network's loss or a neighbor gap, then deleted			
Small-loss selection	1	5	11
Confident learning	16	1	10
Distance-based selection	7	4	9
Multistage cleaning	0	2	4
Noise-robust training			
Early learning	11	2	10
Uncertainty curriculum	15	3	13
DivideMix	9	2	14
Meta-Weight-Net	10	3	9
MentorNet	14	4	12
Contrast-to-divide	10	2	12
Heteroscedastic GP[a]	14	2	12
Evidential network[a]	12	4	12

Table 7.31: Conditions where each method of the expanded sweep beat the plain network on both the best single representation and the best hybrid. A condition is one dataset, damage kind and level, or the labels as delivered. The network only; outer fold 0. [a]Regression only, so 24 small-molecule conditions.

In Chapter 4, the heteroscedastic GP was one of the methods that held up under Gaussian noise on QM9, with a retention of 79.0%. Here it is the only one of the twelve that helps on $\Delta\Delta G$'s labels as delivered, from 0.341 to 0.448 Spearman on the best single representation. It also helps the second hybrid, though the first hybrid drops (Table 7.32). Distance-based selection is the only one of the twelve that helps both sides on hERG with 10% of labels flipped one way only. Its AUROC gains are in the third decimal.

On QM9 with no noise added, every one of the twelve made the network worse on both the best single representation and the hybrid. DivideMix was the worst, 0.452

Method	Configuration	Plain network	With the method
$\Delta\Delta G$, labels as delivered, Spearman (higher is better)			
Heteroscedastic GP	First single	**0.341**	**0.448**
Heteroscedastic GP	Second hybrid	0.250	0.366
Heteroscedastic GP	First hybrid	0.305	0.288
hERG blocking, 10% of labels flipped one way, AUROC (higher is better)			
Distance-based selection	First single	0.815	0.817
Distance-based selection	First hybrid	0.821	0.825

Table 7.32: Two results of the expanded sweep. One row is one method and configuration. Outer fold 0.

RMSE on the hybrid against 0.355 for the plain network, and distance-based selection the least bad (Table 7.33). A second run of eleven of the methods inside another experiment scores the network's second hybrid, and there about half of them beat the plain network. The labels are the same in both runs; the difference is the network's own spread from one fit to the next, which we come back to in Section 7.3.9.

	First run		Second run
Method	First single	First hybrid	Second hybrid
Plain network	0.396	**0.355**	0.364
Small-loss selection	0.465	0.422	0.423
Confident learning	0.412	0.371	0.383
Distance-based selection	0.413	**0.368**	
Multistage cleaning	0.455	0.419	0.382
Early learning	0.420	0.370	0.356
Uncertainty curriculum	0.444	0.380	0.354
DivideMix	0.499	**0.452**	0.405
Meta-Weight-Net	0.426	0.380	0.353
MentorNet	0.425	0.392	0.343
Contrast-to-divide	0.419	0.393	0.350
Heteroscedastic GP	0.417	0.372	0.363
Evidential network	0.427	0.383	0.372

Table 7.33: The expanded sweep on QM9 with no noise added, in two runs of the same network. One row is one method. Held-out RMSE, lower is better. Outer fold 0. Empty: not run.

The method that does best on the hybrid is rarely the one that does best on both single representations. The three exceptions on the first hybrid are Caco-2 under the shift in 10% (distance-based selection), hERG under class-dependent flips (confident

learning) and hERG under instance-dependent flips (distance-based selection). A ranking of noise-handling methods made on one representation does not carry to another.

7.3.9 Effect by model and representation

Cleaning keeps the hybrid ahead on small molecules. Before any treatment the hybrid already beats the best single representation in almost every small-molecule condition, and after mitigation it still does. On LogD with outliers, for example, the network's cleaned hybrid scores 0.556 against 0.591 for the cleaned single representation (Table 7.10). Cleaning rarely changes which of the two wins, and when it does it more often moves the win to the hybrid. On antibodies the hybrid is ahead in about half the conditions, before and after cleaning. The reaction set sits between the two.

The closest test of the question this thesis started from, whether a better feature set makes noise handling work, is where the detector finds its neighbors. One experiment down-weights molecules by the neighbor gap and finds the neighbors either in the model's own single representation or in the hybrid, with everything else held fixed. Finding them in the hybrid helped the same model on the same single representation slightly more often. The largest differences go both ways. On LogD under a scaffold-family shift, the network's single representation goes from 0.705 to 0.663 RMSE with neighbors in the hybrid and gets worse with neighbors in its own space. On nanobody thermostability with outliers, the network barely moves with its own neighbors and gains with the hybrid's. On expression titer under Gaussian noise the reverse happens, and its own neighbors help more than the hybrid's (Table 7.34). These are the largest cases picked out of many, not typical ones, and this comparison has no control.

Dataset and damage	Configuration	No weighting	Own neighbors	Hybrid's neighbors
RMSE (lower is better)				
LogD, scaffold-family shift	First single	**0.705**	0.753	**0.663**
Spearman (higher is better)				
Nb-Tm, outliers	Second single	0.3336	0.3344	0.518
Titer, Gaussian noise	Second single	−0.050	0.150	−0.012

Table 7.34: Down-weighting by the neighbor gap with neighbors found in the model's own single representation or in the hybrid. Network, damage at level 1.0, one noise draw, outer fold 0. One row is one dataset, damage and configuration; the three largest differences. Dataset short names as in Table 5.8.

The network gains from cleaning more often than the support vector regressor on Caco-2, hERG K_i and LogD. Part of that may be the network's own spread between

fits, since a deterministic model with an unchanged score never counts as helped. On $\Delta\Delta G$ the support vector regressor gains more often, and on expression titer the random forest gains more often than the network.

On small molecules, a user who builds a hybrid loses nothing by cleaning afterwards. Whether a noise-handling method works depends on the feature space it runs in, as Chapter 4 found for distance.

7.4 Conclusions

On the first outer fold, we found noisy labels in QSAR data and, in several settings, corrected them. QM9, the one dataset with computed labels, gave the cleanest test of whether a method removes noise or trains a better model (Section 7.3.2). Deletion never helped QM9's clean labels, and the published methods helped it by at most about the network's own spread. Where they helped an antibody dataset as delivered, they found noise that was already in the data. Label smoothing and weighting improved QM9 as well, as did changes to the model. Their gains on measured labels as delivered are therefore largely better training, with label smoothing on hERG K_i the exception. Corrections worked best with a flag: raising labels clipped at an assay limit won back half to three quarters of the lost RMSE on the small-molecule datasets, with no detector. With a detector, deleting or relabeling flagged molecules helped under label flips and outliers. Cleanups driven by a detector passed random or shuffled controls under hERG's label flips. They also passed on the labels as delivered of hERG K_i, Buchwald–Hartwig and three antibody datasets, though QM9 shows that the shuffled control alone is weak evidence with the network. The hybrid stayed ahead of the single representation on small molecules in almost every condition, before and after treatment. Finding a detector's neighbors in the hybrid rather than in the model's own single representation helped slightly more often, with large differences in both directions. Which noise-handling method works best thus depends on the feature space it runs in.

Most results rest on the first outer fold; the further folds that exist mostly agree with it, and the text gives the exceptions. The results to confirm first are, in order, the cleaning of SMAC colloidal stability's labels as delivered and relabeling and cliff putback on hERG K_i, then cleaning under hERG's flips, the clipped-label corrections and the scaffold-family offsets. Cleaning before the search and moving every label toward a Huber-fitted prediction gave large gains but have no QM9 result with no noise added, so their gains cannot be split into noise removed and a better model. Chapter 8 uses

these results to answer the research questions of Chapter 1 and to set out how to choose a noise-handling method for a new dataset.

Chapter 8

Conclusions and Further Thoughts

This chapter answers the research questions of Chapter 1 on handling label noise in molecular data, then sets out how to choose a noise-handling method and what the thesis contributes.

8.1 Summary of findings

The first half of the thesis asked four questions about label noise, answered in Chapter 3. First, model family decided noise robustness more than the representation did. On QM9 under Gaussian noise, model family explained 47.2% of the variance in $\mathrm{AUC}_{\mathrm{norm}}$ and the representation far less, whereas the representation mattered more for predictive performance. Label noise corrupts the labels and leaves the features untouched, so this split is what one would expect. Second, random forests were the most consistent model under noise. LightGBM and XGBoost predicted well on clean labels and lost the most as noise rose, so a model's score on clean labels says little about how it holds up under noise. Third, the type of noise mattered more than the shape of its distribution. Systematic noise, shared within a scaffold family or clipped at an assay limit, did the most damage. A study that injects only Gaussian noise therefore misses it. Fourth, mean predicted uncertainty rose with noise, but outside censoring no model's uncertainty identified the corrupted labels. Under censoring, dividing a molecule's error by its uncertainty picked out the clipped labels better than the error alone.

Chapter 4 asked why published methods for learning with noisy labels did poorly in an earlier exploratory study. In that study's QM9 sweep, the configurations with the lowest retention all used a distance. The sweep added a distance to some methods designed without one, so poor adaptation may explain part of this. Within one dataset,

most features of a standard representation barely vary: on hERG K_i, 1,815 of the 2,048 ECFP4 features take one value in at least 95% of molecules. Constant features add nothing to a distance, but every other feature adds a term. When only a few features matter for the property, the rest dominate the distance. Any method that relies on such a distance inherits the problem, which the adaptive hybrid representation was built to repair.

The second half asked whether the representation can be adapted to the task. First, we found that the best set of features depends on the dataset and the downstream model, and a search can find it (Chapter 5). The adaptive hybrid representation beat the best single representation on every small-molecule dataset with both downstream models. The method carried over from small molecules to antibodies, although there the gain depended on the dataset. On the Buchwald–Hartwig reactions the hybrid gained nothing beyond the spread across folds. On two antibody datasets the best single representation was one built for this thesis, which reads only the regions or mutations that vary. Second, distance in a hybrid tracked the label better than in the single representation on all six small-molecule datasets, and predicting each label from its five nearest neighbors was more accurate (Chapter 6). On antibodies and reactions, distance tracked the label better too. However, prediction from neighbors was worse on several of these datasets, so the benefit of a hybrid has to be measured on each one.

Third, noisy labels could be found and corrected (Chapter 7). With a quarter of LogD's training labels clipped, censored EM removed most of the error that clipping added to the neural network. Cleaning guided by a detector beat random deletion or shuffled detector scores on hERG, hERG K_i, Buchwald–Hartwig and three antibody datasets. On small molecules the hybrid stayed ahead of the single representation in almost every condition, but not always with the network on hERG K_i.

Fourth, feature importance helped in building the hybrid, but using the importance scores again during training added little. The hybrid ranks features by importance within each representation, and at the same number of features it beat recursive feature elimination and random features at every budget on QM9, Caco-2, LogD and MDR1–MDCK efflux. On hERG K_i with the DNN, both beat the hybrid at 1,000 and 2,000 features. For finding noisy labels, the best feature space for the neighbor detectors depended on the dataset (Chapter 7).

8.2 Choosing a noise-handling method

No single method handled label noise on every dataset. Under clipping, the forest residual detector fell below chance on Buchwald–Hartwig, LogD and QM9, and acting on the known flag did best. Subtracting scaffold-family offsets made the model worse on Caco-2's labels as delivered, and deleting the molecules the network fits worst helped on the reactions but almost never on small molecules. Even the order of the methods changed between the hybrid and the single representations, so a method chosen on one representation may not be the best on another.

We recommend modeling the noise in a dataset before choosing a method. Data pooled from several laboratories can carry an offset per source, and an assay with a reporting limit gives clipped labels; both come with a flag that a correction can use. A method that assumes noise that is not there can do harm. Simple methods were often as good as complex ones: random forests were the most consistent model, and on hERG K_i one constant lift for every clipped label beat censored EM in most configurations.

8.3 Contributions

We make six contributions on label noise and three on the representation, in that order.

1. NoiseInject, an open-source Python package for benchmarking noise robustness and uncertainty in QSAR models (Section 2.6.1).

2. Noise conditions beyond Gaussian noise, including noise shared within scaffold families and labels clipped at an assay limit (Chapters 2 and 3).

3. A split of the variance in performance and in noise robustness between model family and representation (Chapter 3).

4. Published noise-handling methods adapted to QSAR data, for example censored EM with any downstream model, a random intercept per scaffold family, and the buddy check from meteorology as a detector of noisy labels (Chapter 7).

5. Three methods for which we found no earlier use: the cliff-aware gap, cleaning the labels before the hybrid search, and returning activity-cliff molecules after a cleanup (Chapter 7).

6. Tests of these methods on single representations and hybrids side by side, with three controls: QM9 with no noise added, random deletion and shuffled detector scores (Chapter 7).

7. The adaptive hybrid representation, which ranks features within each representation, splits a budget of features among them, and searches the selection settings for each dataset and model (Chapter 5). A newly published embedding is added to the search as one more representation.

8. Hybrids built for distance (Chapter 6), and hybrids built from data held at separate sites (Chapter 5).

9. Antibody representations built for this thesis: readouts of the complementarity-determining regions (CDRs) and of each region, mutation features, and the change in embedding from the parent antibody (Chapter 5).

8.4 Limitations

Most noise results rest on the first outer fold. QM9 is the only clean control, and most noise methods have no control. With no ground truth for the labels as delivered, a control can show that a gain beats chance but not that a label was corrected. The hybrid needs data, and most antibody datasets here hold a few hundred antibodies. We have not tested whether the best hybrid settings follow measurable properties of a dataset (Section 5.3.1).

8.5 Future work

The hybrid search needs only several feature matrices for the same samples and a supervised task. Gene-expression profiles and financial time series are natural next domains, since each has many competing representations of the same samples, and so are DNA promoter sequences. Beyond new domains, a hybrid could be the input to the model that scores molecules for a generative model. Starting the search from the settings found on a similar dataset may save compute, and a deployment across real sites would test the separate-site results of Chapter 5.

The noise methods need more clean controls: QM9 offers twelve computed properties, and only the HOMO–LUMO gap was used. In QSAR, a class label usually comes from cutting a continuous measurement at a threshold, so measurement error flips mostly the molecules near that threshold. Flips of that kind, on more classification datasets, are the next test for the detectors.

8.6 Concluding remarks

Label noise in molecular data can be handled. Which method handles it, however, depends on the prediction task and the data, as well as on the model and the kind of noise. No single model or representation was best on every dataset we tested, and neither was any noise-handling method. A dataset's noise should therefore be understood before a method is chosen.

The features matter as well: when the noise methods failed, we followed the failure down to the distance between molecules and then to the features that distance is computed over. The adaptive hybrid representation selects those features for each dataset and model. On small molecules it predicted better than any single representation and gave better neighbors. It stayed ahead, moreover, in almost every noise condition, but not always with the network on hERG K_i.

A new foundation model appears every few months in molecular and protein modeling, antibodies included. With the hybrid search, a researcher does not need to choose between their current representation and a new one. They add the new embedding to the search and rerun it, and the search decides whether the embedding contributes features and which (tested on LogD and hERG blocking in Chapter 5). A choice usually made by habit becomes one measured on each task.

Summary of Research Contributions

Research Papers

In preparation: Punt, A., Hanser, T., Werner, S., & Morris, G.M. (2026). NoiseInject: A Benchmarking Framework to Study Noise Robustness in QSAR Models. To be submitted to the *Journal of Cheminformatics*. Chapter 3.

In preparation: Punt, A., Morris, G.M., Werner, S., & Hanser, T. (2026). Adaptive hybrid representations: selecting features from several molecular representations for each dataset and model (working title). Chapters 5 and 6.

In preparation: Punt, A., Morris, G.M., Werner, S., & Hanser, T. (2026). Detecting, removing and training through label noise in molecular property prediction. Chapter 7.

Software

NoiseInject, a Python package for benchmarking noise robustness and uncertainty quantification: `https://github.com/adpunt/noise_inject`. Chapter 3.

KIRBy, the adaptive hybrid representation search: `https://github.com/adpunt/KIRBy`. Chapters 5 to 7.

Conference Presentations

Punt, A., Morris, G.M., Werner, S., & Hanser, T. (2024). Investigating the impact and mitigation of noise in QSAR models. Presented at the Autumn UK-QSAR 2024 Meeting.

Bibliography

[1] James P. Hughes, Stephen Rees, S. Barrett Kalindjian, and Karen L. Philpott. Principles of early drug discovery. *British Journal of Pharmacology*, 162(6):1239–1249, 2011.

[2] Duxin Sun, Wei Gao, Hongxiang Hu, and Simon Zhou. Why 90% of clinical drug development fails and how to improve it? *Acta Pharmaceutica Sinica B*, 12(7): 3049–3062, 2022.

[3] Bruno J. Neves, Rodolpho C. Braga, Cleber C. Melo-Filho, José Teófilo Moreira-Filho, Eugene N. Muratov, and Carolina Horta Andrade. QSAR-based virtual screening: Advances and applications in drug discovery. *Frontiers in Pharmacology*, 9:1275, 2018.

[4] María Virginia Sabando, Ignacio Ponzoni, Evangelos E. Milios, and Axel J. Soto. Using molecular embeddings in QSAR modeling: does it make a difference? *Briefings in Bioinformatics*, 23(1):bbab365, 2022.

[5] David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5):742–754, 2010.

[6] Laurianne David, Amol Thakkar, Rocío Mercado, and Ola Engkvist. Molecular representations in AI-driven drug discovery: a review and practical guide. *Journal of Cheminformatics*, 12(1):56, 2020.

[7] Markus Dablander, Thierry Hanser, Renaud Lambiotte, and Garrett M. Morris. Sort & Slice: a simple and superior alternative to hash-based folding for extended-connectivity fingerprints. *Journal of Cheminformatics*, 16(1):135, 2024.

[8] Peter Gedeck, Bernhard Rohde, and Christian Bartels. QSAR – how good is it in practice? Comparison of descriptor sets on an unbiased cross section of corporate data sets. *Journal of Chemical Information and Modeling*, 46(5):1924–1936, 2006.

[9] Akihiro Kishimoto, Hiroshi Kajino, Masataka Hirose, Junta Fuchiwaki, Indra Priyadarsini, Lisa Hamada, Hajime Shinohara, Daiju Nakano, and Seiji Takeda. MHG-GNN: Combination of molecular hypergraph grammar with graph neural network. arXiv:2309.16374, 2023.

[10] Scott S. Kolmar and Christopher M. Grulke. The effect of noise on the predictive limit of QSAR models. *Journal of Cheminformatics*, 13(1):92, 2021.

[11] Kjell Jorner, Tore Brinck, Per-Ola Norrby, and David Buttar. Machine learning meets mechanistic modelling for accurate prediction of experimental activation energies. *Chemical Science*, 12(3):1163–1175, 2021.

[12] Christian Kramer, Tuomo Kalliokoski, Peter Gedeck, and Anna Vulpetti. The experimental uncertainty of heterogeneous public K_{i} data. *Journal of Medicinal Chemistry*, 55(11):5165–5173, 2012.

[13] Tuomo Kalliokoski, Christian Kramer, Anna Vulpetti, and Peter Gedeck. Comparability of mixed IC50 data – a statistical analysis. *PLoS ONE*, 8(4):e61007, 2013.

[14] Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify: On the importance of chemical structure curation in cheminformatics and QSAR modeling research. *Journal of Chemical Information and Modeling*, 50(7):1189–1204, 2010.

[15] W. Patrick Walters. We need better benchmarks for machine learning in drug discovery. Practical Cheminformatics blog, `https://practicalcheminformatics.blogspot.com/2023/08/we-need-better-benchmarks-for-machine.html`, 2023. Accessed: 2026-04-12.

[16] Gregory A. Landrum and Sereina Riniker. Combining IC_{50} or K_i values from different sources is a source of significant noise. *Journal of Chemical Information and Modeling*, 64(5):1560–1567, 2024.

[17] Isidro Cortés-Ciriano, Andreas Bender, and Thérèse E. Malliavin. Comparing the influence of simulated experimental errors on 12 machine learning algorithms in bioactivity modeling using 12 diverse data sets. *Journal of Chemical Information and Modeling*, 55(7):1413–1425, 2015.

[18] Davide Bassani, Alessandro Brigo, and Andrea Andrews-Morger. Federated learning in computational toxicology: an industrial perspective on the Effiris hackathon. *Chemical Research in Toxicology*, 36(9):1503–1517, 2023.

[19] Thierry Hanser. Federated learning for molecular discovery. *Current Opinion in Structural Biology*, 79:102545, 2023.

[20] David Weininger. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988.

[21] Michael Backenköhler, Paula Linh Kramer, Joschka Groß, Gerrit Großmann, Roman Joeres, Azat Tagirdzhanov, Dominique Sydow, Hamza Ibrahim, Floriane Odje, Verena Wolf, et al. TeachOpenCADD goes deep learning: Open-source teaching platform exploring molecular DL applications. ChemRxiv, 2023.

[22] Alexandre V. Fassio, Laura Shub, Luca Ponzoni, Jessica McKinley, Matthew J. O'Meara, Rafaela S. Ferreira, Michael J. Keiser, and Raquel C. de Melo Minardi. Prioritizing virtual screening with interpretable interaction fingerprints. *Journal of Chemical Information and Modeling*, 62(18):4300–4318, 2022.

[23] Liangxu Xie, Lei Xu, Ren Kong, Shan Chang, and Xiaojun Xu. Improvement of prediction performance with conjoint molecular fingerprint in deep learning. *Frontiers in Pharmacology*, 11:606668, 2020.

[24] Isidro Cortés-Ciriano, Ctibor Škuta, Andreas Bender, and Daniel Svozil. QSAR-derived affinity fingerprints (part 2): modeling performance for potency prediction. *Journal of Cheminformatics*, 12(1):41, 2020.

[25] Sabrina Jaeger, Simone Fulle, and Samo Turk. Mol2vec: Unsupervised machine learning approach with chemical intuition. *Journal of Chemical Information and Modeling*, 58(1):27–35, 2018.

[26] Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. ChemBERTa: Large-scale self-supervised pretraining for molecular property prediction. arXiv:2010.09885, 2020.

[27] Benedek Fabian, Thomas Edlich, Héléna Gaspar, Marwin Segler, Joshua Meyers, Marco Fiscato, and Mohamed Ahmed. Molecular representation learning with language models and domain-relevant auxiliary tasks. arXiv:2011.13230, 2020.

[28] Walid Ahmad, Elana Simon, Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. ChemBERTa-2: Towards chemical foundation models. arXiv:2209.01712, 2022.

[29] Roberto Todeschini and Viviana Consonni. *Molecular Descriptors for Chemoinformatics*. Wiley-VCH, Weinheim, 2nd edition, 2009.

[30] Andrea Mauri, Viviana Consonni, Manuela Pavan, and Roberto Todeschini. DRAGON software: An easy approach to molecular descriptor calculations. *MATCH Communications in Mathematical and in Computer Chemistry*, 56(2): 237–248, 2006.

[31] Artem Cherkasov, Eugene N. Muratov, Denis Fourches, Alexandre Varnek, Igor I. Baskin, Mark Cronin, John Dearden, Paola Gramatica, Yvonne C. Martin, Roberto Todeschini, et al. QSAR modeling: Where have you been? Where are you going to? *Journal of Medicinal Chemistry*, 57(12):4977–5010, 2014.

[32] Greg Landrum. RDKit: Open-source cheminformatics. `https://www.rdkit.org`, 2023. Version 2023.09.1. Accessed: 2026-01-09.

[33] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3:1157–1182, 2003.

[34] Ian T. Jolliffe and Jorge Cadima. Principal component analysis: a review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 374(2065):20150202, 2016.

[35] Zhichun Guo, Kehan Guo, Bozhao Nan, Yijun Tian, Roshni G. Iyer, Yihong Ma, Olaf Wiest, Xiangliang Zhang, Wei Wang, Chuxu Zhang, et al. Graph-based molecular representation learning. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, pages 6638–6646, 2023.

[36] Sophia M. N. Hönig, Christian Lemmen, and Matthias Rarey. Small molecule superposition: A comprehensive overview on pose scoring of the latest methods. *WIREs Computational Molecular Science*, 13(2):e1640, 2023.

[37] Ge Yan, Huaijin Wu, and Junchi Yan. Quantum 3D graph learning with applications to molecule embedding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 39126–39137, 2023.

[38] Ziduo Yang, Weihe Zhong, Qiujie Lv, Tiejun Dong, and Calvin Yu-Chian Chen. Geometric interaction graph neural network for predicting protein–ligand binding affinities from 3D structures (GIGN). *The Journal of Physical Chemistry Letters*, 14(8):2020–2033, 2023.

[39] Wenjie Du, Xiaoting Yang, Di Wu, FenFen Ma, Baicheng Zhang, Chaochao Bao, Yaoyuan Huo, Jun Jiang, Xin Chen, and Yang Wang. Fusing 2D and 3D molecular graphs as unambiguous molecular descriptors for conformational and chiral stereoisomers. *Briefings in Bioinformatics*, 24(1):bbac560, 2023.

[40] Denis Kuzminykh, Daniil Polykovskiy, Artur Kadurin, Alexander Zhebrak, Ivan Baskov, Sergey Nikolenko, Rim Shayakhmetov, and Alex Zhavoronkov. 3D molecular representations based on the wave transform for convolutional neural networks. *Molecular Pharmaceutics*, 15(10):4378–4385, 2018.

[41] Giorgos Bouritsas, Fabrizio Frasca, Stefanos Zafeiriou, and Michael M. Bronstein. Improving graph neural network expressivity via subgraph isomorphism counting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(1):657–668, 2023.

[42] Jerret Ross, Brian Belgodere, Vijil Chenthamarakshan, Inkit Padhi, Youssef Mroueh, and Payel Das. Large-scale chemical language representations capture molecular structure and properties. *Nature Machine Intelligence*, 4(12):1256–1264, 2022.

[43] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth International Group, Belmont, CA, 1984.

[44] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.

[45] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[46] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[47] Junshui Ma, Robert P. Sheridan, Andy Liaw, George E. Dahl, and Vladimir Svetnik. Deep neural nets as a method for quantitative structure–activity relationships. *Journal of Chemical Information and Modeling*, 55(2):263–274, 2015.

[48] Igor I. Baskin, Vladimir A. Palyulin, and Nikolai S. Zefirov. Neural networks in building QSAR models. In David J. Livingstone, editor, *Artificial Neural Networks: Methods and Applications*, volume 458 of *Methods in Molecular Biology*, pages 133–154. Humana Press, Totowa, NJ, 2009.

[49] Alexios Koutsoukas, Keith J. Monaghan, Xiaoli Li, and Jun Huan. Deep-learning: investigating deep neural networks hyper-parameters and comparison of performance to shallow methods for modeling bioactivity data. *Journal of Cheminformatics*, 9(1):42, 2017.

[50] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations*, 2017. URL `https://openreview.net/forum?id=Sy8gdB9xx`.

[51] Hwanjun Song, Minseok Kim, Dongmin Park, Yooju Shin, and Jae-Gil Lee. Learning from noisy labels with deep neural networks: a survey. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11):8135–8153, 2023.

[52] Curtis G. Northcutt, Lu Jiang, and Isaac L. Chuang. Confident learning: estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021.

[53] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations*, 2018. URL `https://openreview.net/forum?id=r1Ddp1-Rb`.

[54] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2818–2826, 2016.

[55] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017. URL `https://openreview.net/forum?id=SJU4ayYgl`.

[56] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019. URL `https://openreview.net/forum?id=ryGs6iA5Km`.

[57] Markus Dablander, Thierry Hanser, Renaud Lambiotte, and Garrett M. Morris. Exploring QSAR models for activity-cliff prediction. *Journal of Cheminformatics*, 15(1):47, 2023.

[58] Derek van Tilborg, Alisa Alenicheva, and Francesca Grisoni. Exposing the limitations of molecular machine learning with activity cliffs. *Journal of Chemical Information and Modeling*, 62(23):5938–5951, 2022.

[59] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[60] Ismail Babajide Mustapha and Faisal Saeed. Bioactive molecule prediction using extreme gradient boosting. *Molecules*, 21(8):983, 2016.

[61] Hao Tian, Rajas Ketkar, and Peng Tao. ADMETboost: A web server for accurate ADMET prediction. *Journal of Molecular Modeling*, 28(12):408, 2022.

[62] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL `https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html`.

[63] Tony Duan, Anand Avati, Daisy Yi Ding, Khanh K. Thai, Sanjay Basu, Andrew Ng, and Alejandro Schuler. NGBoost: Natural gradient boosting for probabilistic prediction. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119, pages 2690–2700, 2020.

[64] Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995.

[65] Alex J. Smola and Bernhard Schölkopf. A tutorial on support vector regression. *Statistics and Computing*, 14(3):199–222, 2004.

[66] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.

[67] Chih-Chung Chang and Chih-Jen Lin. LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3):1–27, 2011.

[68] Ryan-Rhys Griffiths, Leo Klarner, Henry Moss, Aditya Ravuri, Sang Truong, Yuanqi Du, Samuel Stanton, Gary Tom, Bojana Rankovic, Arian Jamasb, et al. GAUCHE: A library for Gaussian processes in chemistry. In *Advances in Neural Information Processing Systems*, volume 36, pages 76923–76946, 2023.

[69] Liva Ralaivola, Sanjay J. Swamidass, Hiroto Saigo, and Pierre Baldi. Graph kernels for chemical informatics. *Neural Networks*, 18(8):1093–1110, 2005.

[70] Henry B. Moss and Ryan-Rhys Griffiths. Gaussian process molecule property prediction with FlowMO. arXiv:2010.01118, 2020.

[71] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, pages 6402–6413, 2017.

[72] Glenn Palmer, Siqi Du, Alexander Politowicz, Joshua Paul Emory, Xiyu Yang, Anupraas Gautam, Grishma Gupta, Zhelong Li, Ryan Jacobs, and Dane Morgan. Calibration after bootstrap for accurate uncertainty quantification in regression models. *npj Computational Materials*, 8(1):115, 2022.

[73] Mark A. Johnson and Gerald M. Maggiora, editors. *Concepts and Applications of Molecular Similarity*. John Wiley & Sons, New York, 1990.

[74] Gerald Maggiora, Martin Vogt, Dagmar Stumpfe, and Jürgen Bajorath. Molecular similarity in medicinal chemistry. *Journal of Medicinal Chemistry*, 57(8): 3186–3204, 2014.

[75] Dávid Bajusz, Anita Rácz, and Károly Héberger. Why is Tanimoto index an appropriate choice for fingerprint-based similarity calculations? *Journal of Cheminformatics*, 7(1):20, 2015.

[76] Gerald M. Maggiora. On outliers and activity cliffs—why QSAR often disappoints. *Journal of Chemical Information and Modeling*, 46(4):1535, 2006.

[77] Dagmar Stumpfe and Jürgen Bajorath. Exploring activity cliffs in medicinal chemistry. *Journal of Medicinal Chemistry*, 55(7):2932–2942, 2012.

[78] Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. When is "nearest neighbor" meaningful? In *Database Theory – ICDT'99*, volume 1540 of *Lecture Notes in Computer Science*, pages 217–235, 1999.

[79] Felix A. Kruger and John P. Overington. Global analysis of small molecule binding to related protein targets. *PLoS Computational Biology*, 8(1):e1002333, 2012.

[80] Esther Heid, Charles J. McGill, Florence H. Vermeire, and William H. Green. Characterizing uncertainty in machine learning for chemistry. *Journal of Chemical Information and Modeling*, 63(13):4012–4029, 2023.

[81] Ben Lambert, Chon Lok Lei, Martin Robinson, Michael Clerx, Richard Creswell, Sanmitra Ghosh, Simon Tavener, and David J. Gavaghan. Autocorrelated measurement processes and inference for ordinary differential equation models of biological systems. *Journal of the Royal Society Interface*, 20:20220725, 2023.

[82] Joe Bentz, Michael P. O'Connor, Dallas Bednarczyk, JoAnn Coleman, Caroline Lee, Johan Palm, Y. Anne Pak, Elke S. Perloff, Eric Reyner, Praveen Balimane, et al. Variability in P-glycoprotein inhibitory potency (IC50) using various in vitro experimental systems: Implications for universal digoxin drug-drug interaction risk assessment decision criteria. *Drug Metabolism and Disposition*, 41(7): 1347–1366, 2013.

[83] Emma Svensson, Hannah Rosa Friesacher, Susanne Winiwarter, Lewis Mervin, Adam Arany, and Ola Engkvist. Enhancing uncertainty quantification in drug discovery with censored regression labels. *Artificial Intelligence in the Life Sciences*, 7:100128, 2025.

[84] Vladimir Makarenkov, Pablo Zentilli, Dmytro Kevorkov, Andrei Gagarin, Nathalie Malo, and Robert Nadon. An efficient method for the detection and elimination of systematic error in high-throughput screening. *Bioinformatics*, 23 (13):1648–1657, 2007.

[85] James Inglese, Douglas S. Auld, Ajit Jadhav, Ronald L. Johnson, Anton Simeonov, Adam Yasgar, Wei Zheng, and Christopher P. Austin. Quantitative high-throughput screening: A titration-based approach that efficiently identifies biological activities in large chemical libraries. *Proceedings of the National Academy of Sciences*, 103(31):11473–11478, 2006.

[86] Ji-Hu Zhang, Thomas D. Y. Chung, and Kevin R. Oldenburg. A simple statistical parameter for use in evaluation and validation of high throughput screening assays. *Journal of Biomolecular Screening*, 4(2):67–73, 1999.

[87] Anton Simeonov, Ajit Jadhav, Craig J. Thomas, Yuhong Wang, Ruili Huang, Noel T. Southall, Paul Shinn, Jeremy Smith, Christopher P. Austin, Douglas S. Auld, et al. Fluorescence spectroscopic profiling of compound libraries. *Journal of Medicinal Chemistry*, 51(8):2363–2371, 2008.

[88] Christos Hatzis, Philippe L. Bedard, Nicolai J. Birkbak, Andrew H. Beck, Hugo J. W. L. Aerts, David F. Stern, Leming Shi, Robert Clarke, John Quackenbush, and Benjamin Haibe-Kains. Enhancing reproducibility in cancer drug screening: How do we move forward? *Cancer Research*, 74(15):4016–4023, 2014.

[89] Elena Postnikova, Yu Cong, Lisa Evans DeWald, Julie Dyall, Shuiqing Yu, Brit J. Hart, Huanying Zhou, Robin Gross, James Logue, Yingyun Cai, et al. Testing therapeutics in cell-based assays: Factors that influence the apparent potency of drugs. *PLOS ONE*, 13(3):e0194880, 2018.

[90] Anna Gaulton, Anne Hersey, Michał Nowotka, A. Patrícia Bento, Jon Chambers, David Mendez, Prudence Mutowo, Francis Atkinson, Louisa J. Bellis, Elena Cibrián-Uhalte, et al. The ChEMBL database in 2017. *Nucleic Acids Research*, 45(D1):D945–D954, 2017.

[91] Michael K. Gilson, Tiqing Liu, Michael Baitaluk, George Nicola, Linda Hwang, and Jenny Chong. BindingDB in 2015: A public database for medicinal chemistry, computational chemistry and systems pharmacology. *Nucleic Acids Research*, 44(D1):D1045–D1053, 2016.

[92] Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify II: A practical guide to chemogenomics data curation. *Journal of Chemical Information and Modeling*, 56(7):1243–1252, 2016.

[93] Douglas Young, Todd Martin, Raghuraman Venkatapathy, and Paul Harten. Are the chemical structures in your QSAR correct? *QSAR & Combinatorial Science*, 27(11-12):1337–1345, 2008.

[94] A. Patrícia Bento, Anne Hersey, Eloy Félix, Greg Landrum, Anna Gaulton, Francis Atkinson, Louisa J. Bellis, Marleen De Veij, and Andrew R. Leach. An open source chemical structure curation pipeline using RDKit. *Journal of Cheminformatics*, 12(1):51, 2020.

[95] Insha Ullah and A. H. Welsh. On the effect of noise on fitting linear regression models. *Computational Statistics & Data Analysis*, 224:108421, 2026.

[96] Alexander Camuto, Matthew Willetts, Umut Şimşekli, Stephen J. Roberts, and Chris C. Holmes. Explicit regularisation in Gaussian noise injections. In *Advances in Neural Information Processing Systems*, volume 33, pages 16603–16614, 2020.

[97] Guy W. Bemis and Mark A. Murcko. The properties of known drugs. 1. Molecular frameworks. *Journal of Medicinal Chemistry*, 39(15):2887–2893, 1996.

[98] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.

[99] Frank R. Hampel. Robust statistics: a brief introduction and overview. Research Report 94, Seminar für Statistik, Eidgenössische Technische Hochschule (ETH) Zürich, 2001.

[100] Alex Avdeef. Multi-lab intrinsic solubility measurement reproducibility in CheqSol and shake-flask methods. *ADMET and DMPK*, 7(3):210–219, 2019.

[101] David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996.

[102] Dennis L. Wilson. Asymptotic properties of nearest neighbor rules using edited data. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-2(3):408–421, 1972.

[103] Adelaide Punt. NoiseInject: A framework for label noise robustness testing. Version 0.3.0, Zenodo, `https://doi.org/10.5281/zenodo.20532710`, 2026. Accessed: 2026-10-10.

[104] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48, pages 1050–1059, 2016.

[105] Kevin Yang, Kyle Swanson, Wengong Jin, Connor Coley, Philipp Eiden, Hua Gao, Angel Guzman-Perez, Timothy Hopper, Brian Kelley, Miriam Mathea, et al. Analyzing learned molecular representations for property prediction. *Journal of Chemical Information and Modeling*, 59(8):3370–3388, 2019.

[106] Izhar Wallach and Abraham Heifets. Most ligand-based classification benchmarks reward memorization rather than generalization. *Journal of Chemical Information and Modeling*, 58(5):916–932, 2018.

[107] Gregory A. Landrum, Maximilian Beckers, Jessica Lanini, Nadine Schneider, Nikolaus Stiefl, and Sereina Riniker. SIMPD: an algorithm for generating simulated time splits for validating machine learning approaches. *Journal of Cheminformatics*, 15:119, 2023.

[108] Jianyuan Deng, Zhibo Yang, Hehe Wang, Iwao Ojima, Dimitris Samaras, and Fusheng Wang. A systematic study of key elements underlying molecular property prediction. *Nature Communications*, 14(1):6395, 2023.

[109] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, and O. Anatole von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1(1):140022, 2014.

[110] Open ADMET Consortium. OpenADMET-ExpansionRx challenge dataset. `https://huggingface.co/datasets/openadmet/openadmet-expansionrx-challenge-data`, 2026. Accessed: 2026-10-10.

[111] Barbara Zdrazil, Eloy Felix, Fiona Hunter, Emma J. Manners, James Blackshaw, Sybilla Corbett, Marleen de Veij, Harris Ioannidis, David Mendez Lopez, Juan F. Mosquera, et al. The ChEMBL database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods. *Nucleic Acids Research*, 52(D1):D1180–D1192, 2024.

[112] Vladimir Svetnik, Andy Liaw, Christopher Tong, J. Christopher Culberson, Robert P. Sheridan, and Bradley P. Feuston. Random forest: A classification and regression tool for compound classification and QSAR modeling. *Journal of Chemical Information and Computer Sciences*, 43(6):1947–1958, 2003.

[113] Nicolai Meinshausen. Quantile regression forests. *Journal of Machine Learning Research*, 7:983–999, 2006.

[114] Olga Obrezanova, Gábor Csányi, Joelle M. R. Gola, and Matthew D. Segall. Gaussian processes: a method for automatic QSAR modeling of ADME properties. *Journal of Chemical Information and Modeling*, 47(5):1847–1857, 2007.

[115] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, Cambridge, MA, 2005.

[116] Joaquin Quiñonero-Candela and Carl Edward Rasmussen. A unifying view of sparse approximate Gaussian process regression. *Journal of Machine Learning Research*, 6(65):1939–1959, 2005.

[117] Jacob R. Gardner, Geoff Pleiss, Kilian Q. Weinberger, David Bindel, and Andrew Gordon Wilson. GPyTorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/27e8e17134dd7083b050476733207ea1-Paper.pdf.

[118] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://papers.nips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[119] Alex Kendall and Yarin Gal. What uncertainties do we need in Bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/2650d6089a6d640c5e85b2b88265dc2b-Abstract.html.

[120] James Harrison, John Willes, and Jasper Snoek. Variational Bayesian last layers. In *International Conference on Learning Representations*, 2024.

[121] Jessica Lanini, Minh Tam Davide Huynh, Gaetano Scebba, Nadine Schneider, and Raquel Rodríguez-Pérez. UNIQUE: A framework for uncertainty quantification benchmarking. *Journal of Chemical Information and Modeling*, 64(22): 8379–8386, 2024.

[122] Iurie Caraus, Abdulaziz A. Alsuwailem, Robert Nadon, and Vladimir Makarenkov. Detecting and overcoming systematic bias in high-throughput screening technologies: a comprehensive review of practical issues and methodological solutions. *Briefings in Bioinformatics*, 16(6):974–986, 2015.

[123] Daniel Crusius, Flaviu Cipcigan, and Philip C. Biggin. Are we fitting data or noise? Analysing the predictive power of commonly used datasets in drug-, materials-, and molecular-discovery. *Faraday Discussions*, 256:304–321, 2025.

[124] Markus Orsi and Jean-Louis Reymond. One chiral fingerprint to find them all. *Journal of Cheminformatics*, 16(1):53, 2024.

[125] Lien Ai Nguyen, Hua He, and Chuong Pham-Huy. Chiral drugs: An overview. *International Journal of Biomedical Science*, 2(2):85–100, 2006.

[126] Alexander Golbraikh, Eugene Muratov, Denis Fourches, and Alexander Tropsha. Data set modelability by QSAR. *Journal of Chemical Information and Modeling*, 54(1):1–4, 2014.

[127] Robert P. Sheridan, Prabha Karnachi, Matthew Tudor, Yuting Xu, Andy Liaw, Falgun Shah, Alan C. Cheng, Elizabeth Joshi, Meir Glick, and Juan Alvarez. Experimental error, kurtosis, activity cliffs, and methodology: What limits the predictivity of quantitative structure–activity relationship models? *Journal of Chemical Information and Modeling*, 60(4):1969–1982, 2020.

[128] José L. Medina-Franco. Activity cliffs: Facts or artifacts? *Chemical Biology & Drug Design*, 81(5):553–556, 2013.

[129] Sereina Riniker and Gregory A. Landrum. Open-source platform to benchmark fingerprints for ligand-based virtual screening. *Journal of Cheminformatics*, 5 (1):26, 2013.

[130] Mateusz Praski, Jakub Adamczyk, and Wojciech Czech. Benchmarking pre-trained molecular embedding models for molecular representation learning. arXiv:2508.06199, 2025.

[131] Florian Huber and Julian Pollmann. Count your bits: fingerprint benchmarking to assess broad chemical space representation. *Journal of Cheminformatics*, 18 (1):83, 2026.

[132] Pawel Dabrowski-Tumanski, Bartosz Topolski, Dariusz Plewczynski, and Tomasz Jetka. The geometry of activity cliffs: Representation dependence and multi-scale characterization of activity landscapes. arXiv:2605.30831, 2026.

[133] Damien E. Coupry and Peter Pogány. Application of deep metric learning to molecular graph similarity. *Journal of Cheminformatics*, 14(1):11, 2022.

[134] Weifan Zheng and Alexander Tropsha. Novel variable selection quantitative structure–property relationship approach based on the k-nearest-neighbor principle. *Journal of Chemical Information and Computer Sciences*, 40(1):185–194, 2000.

[135] Peter Itskowitz and Alexander Tropsha. k nearest neighbors QSAR modeling as a variational problem: Theory and applications. *Journal of Chemical Information and Modeling*, 45(3):777–785, 2005.

[136] Prasanta Chandra Mahalanobis. On the generalized distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2(1):49–55, 1936.

[137] Cheng Fang, Ye Wang, Richard Grater, Sudarshan Kapadnis, Cheryl Black, Patrick Trapa, and Simone Sciabola. Prospective validation of machine learning algorithms for absorption, distribution, metabolism, and excretion prediction: An industrial perspective. *Journal of Chemical Information and Modeling*, 63 (11):3263–3274, 2023.

[138] Rajarshi Guha and John H. Van Drie. Structure–activity landscape index: Identifying and quantifying activity cliffs. *Journal of Chemical Information and Modeling*, 48(3):646–658, 2008.

[139] Kenneth López-Pérez, Juan F. Avellaneda-Tamayo, Lexin Chen, Edgar López-López, K. Eurídice Juárez-Mercado, José L. Medina-Franco, and Ramón Alain Miranda-Quintana. Molecular similarity: Theory, applications, and perspectives. *Artificial Intelligence Chemistry*, 2(2):100077, 2024.

[140] Peter Willett. Similarity-based virtual screening using 2D fingerprints. *Drug Discovery Today*, 11(23–24):1046–1053, 2006.

[141] Alice Capecchi, Daniel Probst, and Jean-Louis Reymond. One molecular fingerprint to rule them all: drugs, biomolecules, and the metabolome. *Journal of Cheminformatics*, 12(1):43, 2020.

[142] Noel M. O'Boyle and Roger A. Sayle. Comparing structural fingerprints using a literature-based similarity benchmark. *Journal of Cheminformatics*, 8(1):36, 2016.

[143] Charu C. Aggarwal, Alexander Hinneburg, and Daniel A. Keim. On the surprising behavior of distance metrics in high dimensional space. In *Database Theory — ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 420–434, 2001.

[144] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. Hubs in space: Popular nearest neighbors in high-dimensional data. *Journal of Machine Learning Research*, 11:2487–2531, 2010.

[145] Ye Hu, Norbert Furtmann, Dagmar Stumpfe, and Jürgen Bajorath. Comprehensive knowledge base of two- and three-dimensional activity cliffs for medicinal and computational chemistry. *F1000Research*, 4:168, 2015.

[146] David E. Patterson, Richard D. Cramer, Allan M. Ferguson, Robert D. Clark, and Laurence E. Weinberger. Neighborhood behavior: A useful concept for validation of "molecular diversity" descriptors. *Journal of Medicinal Chemistry*, 39(16): 3049–3059, 1996.

[147] Alexander Tropsha. Best practices for QSAR model development, validation, and exploitation. *Molecular Informatics*, 29(6-7):476–488, 2010.

[148] Darko Butina. Unsupervised data base clustering based on Daylight's fingerprint and Tanimoto similarity: A fast and automated way to cluster small and large data sets. *Journal of Chemical Information and Computer Sciences*, 39(4):747–750, 1999.

[149] Thierry Hanser, Ernst Ahlberg, Alexander Amberg, Lennart T. Anger, Chris Barber, Richard J. Brennan, Alessandro Brigo, Annie Delaunois, Susanne Glowienke, Nigel Greene, et al. Data-driven federated learning in drug discovery with knowledge distillation. *Nature Machine Intelligence*, 7(3):423–436, 2025.

[150] Junnan Li, Richard Socher, and Steven C. H. Hoi. DivideMix: Learning with noisy labels as semi-supervised learning. In *International Conference on Learning Representations*, 2020. URL `https://openreview.net/forum?id=HJgExaVtwr`.

[151] Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor Tsang, and Masashi Sugiyama. Co-teaching: Robust training of deep neural networks with extremely noisy labels. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[152] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 41–48, 2009.

[153] M. Pawan Kumar, Benjamin Packer, and Daphne Koller. Self-paced learning for latent variable models. In *Advances in Neural Information Processing Systems*, volume 23, pages 1189–1197, 2010.

[154] Kristian Kersting, Christian Plagemann, Patrick Pfaff, and Wolfram Burgard. Most likely heteroscedastic Gaussian process regression. In *Proceedings of the 24th International Conference on Machine Learning*, pages 393–400, 2007.

[155] Carla E. Brodley and Mark A. Friedl. Identifying mislabeled training data. *Journal of Artificial Intelligence Research*, 11:131–167, 1999.

[156] Benoît Frénay and Michel Verleysen. Classification in the presence of label noise: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 25(5): 845–869, 2014.

[157] Juan Martín, José A. Sáez, and Emilio Corchado. On the regressand noise problem: Model robustness and synergy with regression-adapted noise filters. *IEEE Access*, 9:145800–145816, 2021.

[158] Dejun Jiang, Zhenxing Wu, Chang-Yu Hsieh, Guangyong Chen, Ben Liao, Zhe Wang, Chao Shen, Dongsheng Cao, Jian Wu, and Tingjun Hou. Could graph neural networks learn better molecular representation for drug discovery? A comparison study of descriptor-based and graph-based models. *Journal of Cheminformatics*, 13(1):12, 2021.

[159] Andreas Mayr, Günter Klambauer, Thomas Unterthiner, Marvin Steijaert, Jörg K. Wegner, Hugo Ceulemans, Djork-Arné Clevert, and Sepp Hochreiter. Large-scale comparison of machine learning methods for drug target prediction on ChEMBL. *Chemical Science*, 9(24):5441–5451, 2018.

[160] Jessica Lanini, Gianluca Santarossa, Finton Sirockin, Richard Lewis, Nikolas Fechner, Hubert Misztela, Sarah Lewis, Krzysztof Maziarz, Megan Stanley, Marwin Segler, et al. PREFER: A new predictive modeling framework for molecular discovery. *Journal of Chemical Information and Modeling*, 63(15):4497–4504, 2023.

[161] Henry Kenlay, Frédéric A. Dreyer, Aleksandr Kovaltsuk, Dom Miketa, Douglas Pires, and Charlotte M. Deane. Large scale paired antibody language models. *PLOS Computational Biology*, 20(12):e1012646, 2024.

[162] Peter Neidlinger, Omar S. M. El Nahhas, Hannah Sophie Muti, Tim Lenz, Michael Hoffmeister, Hermann Brenner, Marko van Treeck, Rupert Langer, Bastian Dislich, Hans Michael Behrens, et al. Benchmarking foundation models as feature extractors for weakly supervised computational pathology. *Nature Biomedical Engineering*, 10(6):1113–1123, 2026.

[163] Gordon F. Hughes. On the mean accuracy of statistical pattern recognizers. *IEEE Transactions on Information Theory*, 14(1):55–63, 1968.

[164] Richard E. Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, Princeton, NJ, 1961.

[165] Sören Richard Stahlschmidt, Benjamin Ulfenborg, and Jane Synnergren. Multimodal deep learning for biomedical data fusion: a review. *Briefings in Bioinformatics*, 23(2):bbab569, 2022.

[166] Sandra Steyaert, Marija Pizurica, Divya Nagaraj, Priya Khandelwal, Tina Hernandez-Boussard, Andrew J. Gentles, and Olivier Gevaert. Multimodal data fusion for cancer biomarker discovery with deep learning. *Nature Machine Intelligence*, 5(4):351–362, 2023.

[167] Tongxin Wang, Wei Shao, Zhi Huang, Haixu Tang, Jie Zhang, Zhengming Ding, and Kun Huang. MOGONET integrates multi-omics data using graph convolutional networks allowing patient classification and biomarker identification. *Nature Communications*, 12(1):3445, 2021.

[168] Aurélien Beaude, Franck Augé, Farida Zehraoui, and Blaise Hanczar. CrossAttOmics: multiomics data integration with cross-attention. *Bioinformatics*, 41(6): btaf302, 2025.

[169] Mehmet Gönen and Ethem Alpaydın. Multiple kernel learning algorithms. *Journal of Machine Learning Research*, 12:2211–2268, 2011.

[170] Anna Cichonska, Tapio Pahikkala, Sandor Szedmak, Heli Julkunen, Antti Airola, Markus Heinonen, Tero Aittokallio, and Juho Rousu. Learning with multiple pairwise kernels for drug bioactivity prediction. *Bioinformatics*, 34(13):i509–i518, 2018.

[171] Oliver Laufkötter, Noé Sturm, Jürgen Bajorath, Hongming Chen, and Ola Engkvist. Combining structural and bioactivity-based fingerprints improves prediction performance and scaffold hopping capability. *Journal of Cheminformatics*, 11(1):54, 2019.

[172] Viktoriia Turkina, Melanie R. W. Messih, Etienne Kant, Jelle T. Gringhuis, Annemieke Petrignani, Garry Corthals, Jake W. O'Brien, and Saer Samanipour. Enhancing predictive modeling with molecular fingerprint fusion strategies. *Digital Discovery*, 5(5):2232–2242, 2026.

[173] Ian T. Jolliffe. A note on the use of principal components in regression. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 31(3):300–303, 1982.

[174] Svante Wold, Michael Sjöström, and Lennart Eriksson. PLS-regression: a basic tool of chemometrics. *Chemometrics and Intelligent Laboratory Systems*, 58(2): 109–130, 2001.

[175] Hanchuan Peng, Fuhui Long, and Chris Ding. Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(8):1226–1238, 2005.

[176] Ron Kohavi and George H. John. Wrappers for feature subset selection. *Artificial Intelligence*, 97(1–2):273–324, 1997.

[177] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996.

[178] Isabelle Guyon, Jason Weston, Stephen Barnhill, and Vladimir Vapnik. Gene selection for cancer classification using support vector machines. *Machine Learning*, 46(1–3):389–422, 2002.

[179] Cecile Valsecchi, Francesca Grisoni, Viviana Consonni, and Davide Ballabio. Consensus versus individual QSARs in classification: Comparison on a large-scale case study. *Journal of Chemical Information and Modeling*, 60(3):1215–1223, 2020.

[180] Ming Yuan and Yi Lin. Model selection and estimation in regression with grouped variables. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 68(1):49–67, 2006.

[181] Marcus Olivecrona, Thomas Blaschke, Ola Engkvist, and Hongming Chen. Molecular de-novo design through deep reinforcement learning. *Journal of Cheminformatics*, 9(1):48, 2017.

[182] Christophe Ambroise and Geoffrey J. McLachlan. Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10):6562–6566, 2002.

[183] Andrius Vabalas, Emma Gowen, Ellen Poliakoff, and Alexander J. Casson. Machine learning algorithm validation with a limited sample size. *PLOS ONE*, 14 (11):e0224365, 2019.

[184] Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7(1):91, 2006.

[185] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010.

[186] Jiayun Dong and Cynthia Rudin. Exploring the cloud of variable importance for the set of all good models. *Nature Machine Intelligence*, 2(12):810–824, 2020.

[187] Carolin Strobl, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8(1):25, 2007.

[188] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3145–3153, 2017.

[189] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1):56–67, 2020.

[190] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations*, 2015. URL `https://arxiv.org/abs/1412.6980`.

[191] Narine Kokhlikyan, Vivek Miglani, Miguel Martin, Edward Wang, Bilal Alsallakh, Jonathan Reynolds, Alexander Melnikov, Natalia Kliushkina, Carlos Araya, Siqi Yan, et al. Captum: A unified and generic model interpretability library for PyTorch. arXiv:2009.07896, 2020.

[192] Marco Ancona, Enea Ceolini, Cengiz Öztireli, and Markus Gross. Towards better understanding of gradient-based attribution methods for deep neural networks. In *International Conference on Learning Representations*, 2018. URL `https://openreview.net/forum?id=Sy21R9JAW`.

[193] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, and O. Anatole von Lilienfeld. QM9 dataset. Figshare, `https://doi.org/10.6084/m9.figshare.978904`, 2014. Accessed: 2026-06-02.

[194] Polaris contributors. polaris-hub/polaris. https://doi.org/10.5281/zenodo.13652587, 2025. Accessed: 2026-10-10.

[195] Patrick Koenig, Chingwei V. Lee, Benjamin T. Walters, Vasantharajan Janakiraman, Jeremy Stinson, Thomas W. Patapoff, and Germaine Fuh. Mutational landscape of antibody variable domains reveals a switch modulating the interdomain conformational dynamics and antigen binding. *Proceedings of the National Academy of Sciences*, 114(4):E486–E495, 2017.

[196] Michael Chungyoun, Jeffrey Ruffolo, and Jeffrey Gray. FLAb: Benchmarking deep learning methods for antibody fitness prediction. bioRxiv, 2024.

[197] Mario S. Valdés-Tresanco, Mario E. Valdés-Tresanco, Esteban Molina-Abad, and Ernesto Moreno. NbThermo: a new thermostability database for nanobodies. *Database*, 2023:baad021, 2023.

[198] David Prihoda, Jad Maamary, Andrew Waight, Veronica Juan, Laurence Fayadat-Dilman, Daniel Svozil, and Danny A. Bitton. BioPhi: A platform for antibody design, humanization, and humanness evaluation based on natural antibody repertoires and deep learning. *mAbs*, 14(1):2020203, 2022.

[199] Justina Jankauskaitė, Brian Jiménez-García, Justas Dapkūnas, Juan Fernández-Recio, and Iain H. Moal. SKEMPI 2.0: an updated benchmark of changes in protein–protein binding energy, kinetics and thermodynamics upon mutation. *Bioinformatics*, 35(3):462–469, 2019.

[200] Sarah Sirin, James R. Apgar, Eric M. Bennett, and Amy E. Keating. AB-Bind: Antibody binding mutational database for computational affinity predictions. *Protein Science*, 25(2):393–409, 2016.

[201] Ammar Arsiwala, Rebecca Bhatt, Lood van Niekerk, Porfirio Quintero-Cadena, Xiang Ao, Adam Rosenbaum, Aanal Bhatt, Alexander Smith, Yaoyu Yang, KC Anderson, et al. A high-throughput platform for biophysical antibody developability assessment to enable AI/ML model training. *mAbs*, 17(1):2593055, 2025.

[202] Lood van Niekerk, Joshua Moller, Seth Ritter, Porfirio Quintero-Cadena, Rich Cohen, Georgia Channing, Michael Chungyoun, Laura Rand, Alexander Smith, Aanal Bhatt, et al. 2025 Ginkgo Datapoints antibody developability competition

outcomes: limited model performance and a call for data standardization. *mAbs*, 18(1):2634216, 2026.

[203] Derek T. Ahneman, Jesús G. Estrada, Shishi Lin, Spencer D. Dreher, and Abigail G. Doyle. Predicting reaction performance in C–N cross-coupling using machine learning. *Science*, 360(6385):186–190, 2018.

[204] Luiz C. Vieira, Sophia Lin, and Claus O. Wilke. Intrinsic dataset features drive mutational effect prediction by protein language models. bioRxiv, 2026.

[205] Alexandre Varnek, Denis Fourches, Frank Hoonakker, and Vitaly P. Solov'ev. Substructural fragments: an universal language to encode reactions, molecular and supramolecular structures. *Journal of Computer-Aided Molecular Design*, 19 (9–10):693–703, 2005.

[206] Frank Hoonakker, Nicolas Lachiche, Alexandre Varnek, and Alain Wagner. A representation to apply usual data mining techniques to chemical reactions — illustration on the rate constant of S_N2 reactions in water. *International Journal on Artificial Intelligence Tools*, 20(2):253–270, 2011.

[207] Philippe Schwaller, Benjamin Hoover, Jean-Louis Reymond, Hendrik Strobelt, and Teodoro Laino. Extraction of organic chemistry grammar from unsupervised learning of chemical reactions. *Science Advances*, 7(15):eabe4166, 2021.

[208] Philippe Schwaller, Daniel Probst, Alain C. Vaucher, Vishnu H. Nair, David Kreutter, Teodoro Laino, and Jean-Louis Reymond. Mapping the space of chemical reactions using attention-based neural networks. *Nature Machine Intelligence*, 3(2):144–152, 2021.

[209] Tatsuya Sagawa and Ryosuke Kojima. ReactionT5: a pre-trained transformer model for accurate chemical reaction prediction with limited data. *Journal of Cheminformatics*, 17(1):126, 2025.

[210] Kangway V. Chuang and Michael J. Keiser. Comment on "Predicting reaction performance in C–N cross-coupling using machine learning". *Science*, 362(6416): eaat8603, 2018.

[211] Steven Henikoff and Jorja G. Henikoff. Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22):10915–10919, 1992.

[212] Marie-Paule Lefranc, Christelle Pommié, Manuel Ruiz, Véronique Giudicelli, Elodie Foulquier, Lisa Truong, Valérie Thouvenin-Contet, and Gérard Lefranc. IMGT unique numbering for immunoglobulin and T cell receptor variable domains and Ig superfamily V-like domains. *Developmental & Comparative Immunology*, 27(1):55–77, 2003.

[213] James Dunbar and Charlotte M. Deane. ANARCI: antigen receptor numbering and receptor classification. *Bioinformatics*, 32(2):298–300, 2016.

[214] Raymond E. Carhart, Dennis H. Smith, and R. Venkataraghavan. Atom pairs as molecular features in structure-activity studies: definition and applications. *Journal of Chemical Information and Computer Sciences*, 25(2):64–73, 1985.

[215] Ramaswamy Nilakantan, Norman Bauman, J. Scott Dixon, and R. Venkataraghavan. Topological torsion: a new molecular descriptor for SAR applications. Comparison with other descriptors. *Journal of Chemical Information and Computer Sciences*, 27(2):82–85, 1987.

[216] Joseph L. Durant, Burton A. Leland, Douglas R. Henry, and James G. Nourse. Reoptimization of MDL keys for use in drug discovery. *Journal of Chemical Information and Computer Sciences*, 42(6):1273–1280, 2002.

[217] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. Weisfeiler-Lehman graph kernels. *Journal of Machine Learning Research*, 12(77):2539–2561, 2011.

[218] Hirotomo Moriwaki, Yu-Shi Tian, Norihito Kawashita, and Tatsuya Takagi. Mordred: a molecular descriptor calculator. *Journal of Cheminformatics*, 10(1):4, 2018.

[219] Yu Rong, Yatao Bian, Tingyang Xu, Weiyang Xie, Ying Wei, Wenbing Huang, and Junzhou Huang. Self-supervised graph transformer on large-scale molecular data. In *Advances in Neural Information Processing Systems*, volume 33, pages 12559–12571, 2020.

[220] Shengchao Liu, Hanchen Wang, Weiyang Liu, Joan Lasenby, Hongyu Guo, and Jian Tang. Pre-training molecular graph representation with 3D geometry. In *International Conference on Learning Representations*, 2022. URL `https://openreview.net/forum?id=xQUe1pOKPam`.

[221] Eduardo Soares, Emilio Vital Brazil, Victor Shirasuna, Dmitry Zubarev, Renato Cerqueira, and Kristin Schmidt. An open-source family of large encoder-decoder foundation models for chemistry. *Communications Chemistry*, 8(1):193, 2025.

[222] Xiao-Chen Zhang, Cheng-Kun Wu, Jia-Cai Yi, Xiang-Xiang Zeng, Can-Qun Yang, Ai-Ping Lu, Ting-Jun Hou, and Dong-Sheng Cao. Pushing the boundaries of molecular property prediction for drug discovery with multitask learning BERT enhanced by SMILES enumeration. *Research*, 2022:0004, 2022.

[223] Atakan Yüksel, Erva Ulusoy, Atabey Ünlü, and Tunca Doğan. SELFormer: Molecular representation learning via SELFIES language models. *Machine Learning: Science and Technology*, 4(2):025035, 2023.

[224] Pedro J. Ballester and W. Graham Richards. Ultrafast shape recognition to search compound databases for similar molecular shapes. *Journal of Computational Chemistry*, 28(10):1711–1723, 2007.

[225] Adrian M. Schreyer and Tom Blundell. USRCAT: real-time ultrafast shape recognition with pharmacophoric constraints. *Journal of Cheminformatics*, 4(1):27, 2012.

[226] Matthias Rupp, Alexandre Tkatchenko, Klaus-Robert Müller, and O. Anatole von Lilienfeld. Fast and accurate modeling of molecular atomization energies with machine learning. *Physical Review Letters*, 108(5):058301, 2012.

[227] Yuyang Wang, Jianren Wang, Zhonglin Cao, and Amir Barati Farimani. Molecular contrastive learning of representations via graph neural networks. *Nature Machine Intelligence*, 4(3):279–287, 2022.

[228] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.

[229] Joshua Meier, Roshan Rao, Robert Verkuil, Jason Liu, Tom Sercu, and Alex Rives. Language models enable zero-shot prediction of the effects of mutations on protein function. In *Advances in Neural Information Processing Systems*, volume 34, pages 29287–29303, 2021.

[230] Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J. Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q. Tran, Jonathan Deaton, Marius Wiggert,

et al. Simulating 500 million years of evolution with a language model. *Science*, 387(6736):850–858, 2025.

[231] Salvatore Candido, Thomas Hayes, Alexander Derry, Roshan Rao, Zeming Lin, Robert Verkuil, Bryan Z. Wu, Jin Sub Lee, Elise S. Bruguera, Jehan A. Keval, et al. Language modeling materializes a world model of protein biology. bioRxiv, 2026.

[232] Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Ghalia Rehawi, Yu Wang, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, et al. ProtTrans: Toward understanding the language of life through self-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):7112–7127, 2022.

[233] Jin Su, Chenchen Han, Yuyang Zhou, Junjie Shan, Xibin Zhou, and Fajie Yuan. SaProt: Protein language modeling with structure-aware vocabulary. In *International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=6MRm3G4NiU`.

[234] Tobias H. Olsen, Iain H. Moal, and Charlotte M. Deane. Addressing the antibody germline bias and its effect on language models for improved antibody design. *Bioinformatics*, 40(11):btae618, 2024.

[235] Jeffrey A. Ruffolo, Jeffrey J. Gray, and Jeremias Sulam. Deciphering antibody affinity maturation with language models and weakly supervised learning. arXiv:2112.07782, 2021.

[236] Eunna Huh, Hyeonsu Lee, and Hyunjin Shin. Antibody foundational model: Ab-RoBERTa. arXiv:2506.13006, 2025.

[237] Brennan Abanades, Wing Ki Wong, Fergus Boyles, Guy Georges, Alexander Bujotzek, and Charlotte M. Deane. ImmuneBuilder: Deep-learning models for predicting the structures of immune proteins. *Communications Biology*, 6(1):575, 2023.

[238] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems*, volume 35, pages 507–520, 2022.

[239] Yoshua Bengio and Yves Grandvalet. No unbiased estimator of the variance of K-fold cross-validation. *Journal of Machine Learning Research*, 5:1089–1105, 2004.

[240] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction.* Springer, New York, second edition, 2009.

[241] Jun Shu, Qi Xie, Lixuan Yi, Qian Zhao, Sanping Zhou, Zongben Xu, and Deyu Meng. Meta-Weight-Net: Learning an explicit mapping for sample weighting. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[242] Sarah Nogueira, Konstantinos Sechidis, and Gavin Brown. On the stability of feature selection algorithms. *Journal of Machine Learning Research*, 18(174): 1–54, 2018.

[243] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[244] Ivan Olier, Noureddin Sadawi, G. Richard Bickerton, Joaquin Vanschoren, Crina Grosan, Larisa Soldatova, and Ross D. King. Meta-QSAR: a large-scale application of meta-learning to drug design and discovery. *Machine Learning*, 107(1): 285–311, 2018.

[245] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282, 2017.

[246] Wouter Heyndrickx, Lewis Mervin, Tobias Morawietz, Noé Sturm, Lukas Friedrich, Adam Zalewski, Anastasia Pentina, Lina Humbeck, Martijn Oldenhof, Ritsuya Niwayama, et al. MELLODDY: Cross-pharma federated learning at unprecedented scale unlocks benefits in QSAR without compromising proprietary information. *Journal of Chemical Information and Modeling*, 64(7):2331–2344, 2024.

[247] Pietro Cassarà, Alberto Gotta, and Lorenzo Valerio. Federated feature selection for cyber-physical systems of systems. *IEEE Transactions on Vehicular Technology*, 71(9):9937–9950, 2022.

[248] Ryan J. Tibshirani. Degrees of freedom and model search. *Statistica Sinica*, 25 (3):1265–1296, 2015.

[249] Sahibsingh A. Dudani. The distance-weighted k-nearest-neighbor rule. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-6(4):325–327, 1976.

[250] Donald Shepard. A two-dimensional interpolation function for irregularly-spaced data. In *Proceedings of the 1968 23rd ACM National Conference*, pages 517–524, 1968.

[251] Dietrich Wettschereck, David W. Aha, and Takao Mohri. A review and empirical evaluation of feature weighting methods for a class of lazy learning algorithms. *Artificial Intelligence Review*, 11(1–5):273–314, 1997.

[252] Arkaprava Banerjee and Kunal Roy. First report of q-RASAR modeling toward an approach of easy interpretability and efficient transferability. *Molecular Diversity*, 26(5):2847–2862, 2022.

[253] Robert P. Sheridan, Bradley P. Feuston, Vladimir N. Maiorov, and Simon K. Kearsley. Similarity to molecules in the training set is a good discriminator for prediction accuracy in QSAR. *Journal of Chemical Information and Computer Sciences*, 44(6):1912–1928, 2004.

[254] Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[255] Donald R. Jones, Matthias Schonlau, and William J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4):455–492, 1998.

[256] Alexander Kraskov, Harald Stögbauer, and Peter Grassberger. Estimating mutual information. *Physical Review E*, 69(6):066138, 2004.

[257] Brian C. Ross. Mutual information between discrete and continuous data sets. *PLoS ONE*, 9(2):e87357, 2014.

[258] Line Båserud, Cristian Lussana, Thomas N. Nipen, Ivar A. Seierstad, Louise Oram, and Trygve Aspelien. TITAN automatic spatial quality control of meteorological in-situ observations. *Advances in Science and Research*, 17:153–163, 2020.

[259] Mirosław Kordos and Marcin Blachnik. Instance selection with neural networks for regression problems. In *Artificial Neural Networks and Machine Learning – ICANN 2012*, volume 7553 of *Lecture Notes in Computer Science*, pages 263–270, 2012.

[260] Sanjay Chawla and Pei Sun. SLOM: a new measure for local spatial outliers. *Knowledge and Information Systems*, 9(4):412–429, 2006.

[261] Maximilian Seitzer, Arash Tavakoli, Dimitrije Antic, and Georg Martius. On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks. In *International Conference on Learning Representations*, 2022. URL `https://openreview.net/forum?id=aPOpXlnV1T`.

[262] Andy Liaw and Matthew Wiener. Classification and regression by randomForest. *R News*, 2(3):18–22, 2002.

[263] Amirata Ghorbani and James Zou. Data Shapley: Equitable valuation of data for machine learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97, pages 2242–2251, 2019.

[264] Ruoxi Jia, David Dao, Boxin Wang, Frances Ann Hubis, Nezihe Merve Gurel, Bo Li, Ce Zhang, Costas Spanos, and Dawn Song. Efficient task-specific data valuation for nearest neighbor algorithms. *Proceedings of the VLDB Endowment*, 12(11):1610–1623, 2019.

[265] Hang Zhou, Jonas Mueller, Mayank Kumar, Jane-Ling Wang, and Jing Lei. Detecting errors in a numerical response via any regression model. *Journal of Data-centric Machine Learning Research*, 1(6):1–25, 2024.

[266] Stephen Bates, Emmanuel Candès, Lihua Lei, Yaniv Romano, and Matteo Sesia. Testing for outliers with conformal p-values. *The Annals of Statistics*, 51(1): 149–178, 2023.

[267] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995.

[268] Linlin Zhao, Wenyi Wang, Alexander Sedykh, and Hao Zhu. Experimental errors in QSAR modeling sets: What we can do and what we cannot do. *ACS Omega*, 2(6):2805–2812, 2017.

[269] E. A. Nadaraya. On estimating regression. *Theory of Probability & Its Applications*, 9(1):141–142, 1964.

[270] Geoffrey S. Watson. Smooth regression analysis. *Sankhyā: The Indian Journal of Statistics, Series A*, 26:359–372, 1964.

[271] Scott Reed, Honglak Lee, Dragomir Anguelov, Christian Szegedy, Dumitru Erhan, and Andrew Rabinovich. Training deep neural networks on noisy labels with bootstrapping. In *International Conference on Learning Representations, Workshop Track*, 2015.

[272] W. James and Charles Stein. Estimation with quadratic loss. In *Proceedings of the Fourth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 361–379, Berkeley, CA, 1961. University of California Press.

[273] Miroslav Kubat and Stan Matwin. Addressing the curse of imbalanced training sets: one-sided selection. In *Proceedings of the Fourteenth International Conference on Machine Learning*, pages 179–186, 1997.

[274] Jerzy Stefanowski and Szymon Wilk. Selective pre-processing of imbalanced data for improving classification performance. In *Data Warehousing and Knowledge Discovery: 10th International Conference, DaWaK 2008*, volume 5182 of *Lecture Notes in Computer Science*, pages 283–292, 2008.

[275] Ahmad Abu Shanab, Taghi M. Khoshgoftaar, Randall Wald, and Jason Van Hulse. Evaluation of the importance of data pre-processing order when combining feature selection and data sampling. *International Journal of Business Intelligence and Data Mining*, 7(1/2):116–134, 2012.

[276] James Tobin. Estimation of relationships for limited dependent variables. *Econometrica*, 26(1):24–36, 1958.

[277] Jonathan Buckley and Ian James. Linear regression with censored data. *Biometrika*, 66(3):429–436, 1979.

[278] Ahlem Hajjem, François Bellavance, and Denis Larocque. Mixed-effects random forest for clustered data. *Journal of Statistical Computation and Simulation*, 84 (6):1313–1328, 2014.

[279] Fabio Sigrist. Gaussian process boosting. *Journal of Machine Learning Research*, 23(232):1–46, 2022.

[280] Larry V. Hedges and Ingram Olkin. *Statistical Methods for Meta-Analysis*. Academic Press, Orlando, FL, 1985.

[281] Gordon K. Smyth. Linear models and empirical Bayes methods for assessing differential expression in microarray experiments. *Statistical Applications in Genetics and Molecular Biology*, 3(1):1–25, 2004.

[282] Leo Breiman. Stacked regressions. *Machine Learning*, 24(1):49–64, 1996.

[283] Zixiao Zhao, Fatemeh H. Fard, and Jie JW Wu. Mantra: a framework for multi-stage adaptive noise treatment during training, 2025.

[284] Yiming Zhang, Chester Holtz, Gal Mishne, and Alex Cloninger. Revisiting meta-learning with noisy labels: Reweighting dynamics and theoretical guarantees, 2025.

[285] Lu Jiang, Zhengyuan Zhou, Thomas Leung, Li-Jia Li, and Li Fei-Fei. MentorNet: Learning data-driven curriculum for very deep neural networks on corrupted labels. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80, pages 2304–2313, 2018.

[286] Evgenii Zheltonozhskii, Chaim Baskin, Avi Mendelson, Alex M Bronstein, and Or Litany. Contrast to divide: Self-supervised pre-training for learning with noisy labels. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 387–397, 2022. doi: 10.1109/wacv51458.2022.00046.

[287] Alexander Amini, Wilko Schwarting, Ava Soleimany, and Daniela Rus. Deep evidential regression. In *Advances in Neural Information Processing Systems*, volume 33, pages 14927–14937, 2020.

[288] Avinash Barnwal, Hyunsu Cho, and Toby Hocking. Survival regression with accelerated failure time model in XGBoost. *Journal of Computational and Graphical Statistics*, 31(4):1292–1302, 2022.